Documentation de l'outil

Construction de tables d'expérience
Package `tablexp`, méthodes, vérifications et application

Une table d'expérience est l'estimation d'une transition d'un modèle multi-états

Établie le 06/11/2026 | Version 0.1 | Statut : Projet | Package `tablexp` 0.0.0.9000

Objet du document

Ce document est la documentation de l'**outil de construction de tables d'expérience** `outil_tables`. Il couvre le package R `tablexp`, qui porte toute la logique quantitative, les vérifications qui l'accompagnent (batteries de tests, contrôles du dépôt, plateforme de calcul) et, quand elle existera, l'application interactive qui l'exploite. Une *table d'expérience* est construite à partir des observations d'un portefeuille d'assurés : pour un risque donné, l'outil estime le taux d'une *transition* (vivant → décédé, en cours → racheté…) selon un ou plusieurs axes, le lisse ou le positionne sur une table de référence, le ferme aux âges extrêmes, le valide, et restitue la table avec tout ce qui permet de la rejouer et de la relire.

Les destinataires des tables et des rapports sont, par priorité, les hypothèses biométriques de la meilleure estimation (*best estimate*) Solvabilité II, que l'ACPR revoit en contrôle, puis la certification d'une table d'expérience par un actuaire indépendant agréé (`docs/exigences.md`, E1.2). Le document doit permettre à un relecteur externe, sans lire le code, de savoir ce qui a été calculé, avec quelles hypothèses, et de rejouer le calcul (E3.7). Les textes qui font foi sont recensés par le cahier des charges (`docs/exigences.md`, § 2.4) ; ils ne sont cités dans le présent document qu'une fois établis sur la source (issue #13).

Il est organisé en une partie par module du package (E3.1). La **partie I** présente l'outil : forme du package et graphe des modules, services transverses, reproductibilité, batteries de vérification et plateforme de calcul, contrôles du dépôt. La **partie II** décrit le point d'entrée unique et le plan de construction. Les parties suivantes suivent l'ordre des modules du moteur : données, exposition, taux bruts, lissage, positionnement, table prospective, fermeture, validation, rapport, puis le simulateur, qui produit les jeux de données sous une loi vraie connue. La dernière partie rassemble l'index des fonctions exportées, la traçabilité des exigences et les références. Chaque partie de module se termine par le tableau des fonctions du module, qui relie la description de la méthode à son implémentation ; un module qui n'existe pas encore a déjà sa partie, avec un tableau vide.

Le document ne porte aucun chiffre issu d'une exécution (E3.4) : les résultats d'une construction sont dans son rapport figé et sa fiche de construction, et les bilans des vérifications dans la sortie des batteries. Ses conventions de rédaction sont fixées par `docs/latex/CONVENTIONS.md` ; le vocabulaire est celui du glossaire du projet (`CONTEXT.md`).

Élément	Référence
Cahier des charges	`docs/exigences.md` (exigences E1.1 à E5.5)
Décisions d'architecture	`docs/adr/` (ADR 0002 à 0009, 0011 et 0012)
Moteur de calcul	package `tablexp` (`R/`), packages de priorité `base` seulement (ADR 0002)
Interface	`app/app.R`, `app/affichage.R` (à venir ; aucun calcul)
Version de R	4.3.3 ; `DESCRIPTION` déclare `Depends: R (>= 4.3.3)`
Plateforme de référence	conteneur `rocker/r-ver:4.3.3` épinglé par empreinte, BLAS et LAPACK de référence (ADR 0008 ; section 4.3)
Données	aucune donnée réelle : jeux simulés sous une loi vraie et une graine

Sommaire

Partie I — Présentation de l'outil 6

1 Architecture du package et graphe des modules 6
1.1 Principe de séparation . . . 6
1.2 Fichiers du package . . . 6
1.3 Graphe d'appels autorisés . . . 7

2 Services transverses : `R/outils.R` 10
2.1 Refus : une condition R typée . . . 10
2.2 Avertissements métier . . . 14
2.3 Fonctions du module . . . 14

3 Reproductibilité et aléa 18
3.1 Exigence et portée . . . 18
3.2 Générateur fixé . . . 18
3.3 Exécution sous graine locale . . . 19
3.4 Vérifications . . . 19

4 Batteries de vérification et plateforme de CI 20
4.1 Trois batteries . . . 20
4.2 Harnais . . . 21
4.3 Plateforme de référence . . . 24
4.4 Jobs de la CI . . . 26
4.5 Références de non-régression . . . 26

5 Contrôles du dépôt 34
5.1 Scripts de contrôle . . . 34
5.2 Absence de Java . . . 39

Partie II — Point d'entrée et plan de construction 41

6 Point d'entrée unique : `R/construire_table.R` 41
6.1 Plan de construction et validation . . . 41
6.2 Exécution . . . 43
6.3 Objet résultat . . . 44
6.4 Rejeu outillé . . . 44
6.5 Vérifications . . . 52
6.6 Fonctions du module . . . 53

Partie III — Module `donnees` 56

7 Données, rôles et contrôles : `R/donnees.R` 56
7.1 Conventions de temps . . . 56
7.2 Catalogue des rôles . . . 57

7.3 Contrôles, refus et exclusions . . . 58
7.4 Unité d'observation et fusion par tête . . . 61
7.5 Table de référence . . . 63
7.6 Vérifications . . . 63
7.7 Fonctions du module . . . 64

Partie IV — Module `exposition` 67

8 Exposition et événements : `R/exposition.R` 67
8.1 Recette d'exposition et conventions d'âge . . . 67
8.2 Troncature à gauche et censure à droite . . . 69
8.3 Découpage par année . . . 72
8.4 Vérifications . . . 72
8.5 Fonctions du module . . . 73

Partie V — Module `brut` 76

9 Taux bruts : `R/brut.R` 76
9.1 Étape `brut` du plan . . . 76
9.2 Estimateur de Poisson . . . 77
9.3 Estimateur binomial . . . 78
9.4 Estimateur produit-limite (`kaplan_meier`) . . . 80
9.5 Statuts des grandeurs restituées . . . 83
9.6 États des cellules et avertissements . . . 84
9.7 Résultat, paramètres et diagnostics . . . 85
9.8 Vérifications . . . 85
9.9 Constats de simulation : retrouver la loi vraie . . . 87
9.10 Fonctions du module . . . 91

Partie VI — Module `lissage` 93

10 Lissage : `R/lissage.R` 93
10.1 Fonctions du module . . . 93

Partie VII — Module `positionnement` 94

11 Positionnement sur une table de référence : `R/positionnement.R` 94
11.1 Fonctions du module . . . 94

Partie VIII — Module `prospective` 95

12 Tables prospectives : `R/prospective.R` 95
12.1 Fonctions du module . . . 95

Partie IX — Module `fermeture` 96

13 Fermeture aux âges extrêmes : `R/fermeture.R` 96
13.1 Fonctions du module . . . 96

Partie X — Module `validation` 97

14 Validation et suivi d'expérience : `R/validation.R` 97
14.1 Fonctions du module 97

Partie XI — Module `rapport` 98

15 Fiche de construction et empreintes : `R/rapport.R` 98
15.1 Fiche de construction 98
15.2 Empreinte des données 102
15.3 Empreintes du résultat et du code 109
15.4 Texte du plan et de la fiche 112
15.5 Rapport figé 116
15.6 Vérifications 122
15.7 Fonctions du module 125

Partie XII — Module `simulation` 134

16 Simulateur : `R/simulation.R` 134
16.1 Loi vraie et tables synthétiques 134
16.2 Portefeuille simulé 136
16.3 Défauts paramétrés 140
16.4 Constats de simulation 144
16.5 Vérifications 149
16.6 Fonctions du module 149

Partie XIII — Index et traçabilité 152

17 Index et traçabilité 152
17.1 Index des fonctions exportées 152
17.2 Traçabilité des exigences 153
17.3 Références 154

Partie I

Présentation de l'outil

Architecture du package et graphe des modules

1.1 Principe de séparation

L'outil sépare strictement le *calcul* de l'*affichage*. La règle est la suivante : si un résultat peut être calculé indépendamment de l'interface, son calcul figure dans le package `tablexp` (E4.3). Le moteur reçoit un `data.frame` et la déclaration des *rôles* de ses colonnes, jamais un fichier ; la lecture des fichiers, et les conversions de type qui l'accompagnent, relèvent d'une couche mince hors du moteur, qui n'existe pas encore (E4.4). Le moteur refuse un type incompatible, il ne le convertit pas.

Emplacement	Contenu	Interdit
`R/` (package `tablexp`)	toute la logique quantitative : rôles et validation des données, recettes d'exposition, estimation, lissage, positionnement, prospective, fermeture, validation, simulation, quantités des rapports et empreintes, écriture du rapport figé en octets HTML, sans calcul (section 15.5)	toute primitive Shiny (`input$`, `reactive`, `render*`) ; tout package hors priorité `base` sans ADR de méthode ; toute dépendance à Java
`app/app.R` (à venir)	déclaration des rôles, édition du plan, appel du point d'entrée sur clic du bouton « Relancer les calculs », affichage	tout calcul ; tout appel d'une fonction non exportée (`tablexp:::`)
`app/affichage.R` (à venir)	formatage, tableaux HTML de l'affichage interactif, tracés à partir des quantités du résultat ; le rapport figé n'y est pas construit	tout calcul
`tests/`	batteries, harnais, scripts de contrôle	—

L'application vit hors du package, sous `app/`, et non dans un fichier `R/affichage.R` : tout fichier de `R/` est installé dans l'espace de noms du package (section 1.2), si bien qu'un fichier d'affichage placé là entrerait dans le moteur et rendrait circulaire le contrôle de frontière (ADR 0007, point 1.3). Le dossier `app/` et son script de frontière n'existent pas encore : ils viennent avec la branche de l'application.

1.2 Fichiers du package

Le package `tablexp` est à la racine du dépôt (ADR 0007, point 1.1) :

— `DESCRIPTION` : `Package: tablexp`, `Depends: R (>= 4.3.3)` (décision M44), aucun champ `Imports`, `Suggests` ni `LinkingTo`, `Encoding: UTF-8`, `License: file LICENSE` ; le contrôle de la liste blanche (section 5) en fait une règle vérifiée ;

— `NAMESPACE`, écrit à la main : une directive `export` par fonction, sans `exportPattern` ; les fonctions exportées sont les points d'entrée et les fonctions que l'application appelle (contrôles de saisie à sens métier, constructeurs d'objets partagés) ou qu'un tiers appelle hors de l'application (rejeu outillé, rapport figé ; ADR 0012, points 2.8 et 3.10), recensées dans l'index (section 17.1) ; aucune méthode S3 n'est exportée ;

— `R/` : un fichier par module et deux fichiers transversaux (tableau 1) ;

— `.Rbuildignore` : exclut de l'archive du package `app/`, `docs/`, `tests/`, `.claude/`, `.github/` et les fichiers de pilotage du dépôt.

Aucun fichier `Rd` ni `roxygen2` : la documentation de référence est le présent document, tenu concordant avec le code par la batterie de concordance (section 4 ; E3.3).

`R/` ne contient que du code du package. Une installation depuis l'arbre de travail (`R CMD INSTALL <dossier>`), qui est celle des batteries, n'applique pas `.Rbuildignore` : tout fichier de `R/` est installé, même s'il y est exclu (constat : un fichier `R/b.R` exclu par `.Rbuildignore` est présent dans le package installé ; ADR 0007, mesures). Un fichier étranger au moteur ne peut donc pas être rangé dans `R/`.

Table 1 : Fichiers de `R/`, dans l'ordre des modules (ADR 0007, points 1.2 et 2.1).

Fichier	Contenu	État
`R/outils.R`	services transverses sans contenu métier : graine locale, refus et avertissements (section 2) ; n'appelle aucun module	présent
`R/donnees.R`	rôles, contrôles, refus et exclusions, unité d'observation, table de référence (section 7)	présent
`R/exposition.R`	recettes d'exposition, conventions d'âge, troncature et censure (section 8)	présent
`R/brut.R`	taux bruts, intervalles et statuts (section 9)	présent
`R/lissage.R`	lissages (section 10)	à créer
`R/positionnement.R`	positionnement sur une table de référence (section 11)	à créer
`R/prospective.R`	tables prospectives (section 12)	à créer
`R/fermeture.R`	fermeture aux âges extrêmes (section 13)	à créer
`R/validation.R`	validation et suivi d'expérience (section 14)	à créer
`R/rapport.R`	quantités des rapports, fiche de construction, empreintes, texte du plan et de la fiche, rapport figé en octets HTML, sans calcul (section 15)	présent
`R/simulation.R`	simulateur (section 16) ; hors de la chaîne du moteur	présent
`R/construire_table.R`	point d'entrée unique, validation du plan, orchestration seule, aucun calcul (section 6) ; rejeu outillé d'une fiche, qui contrôle puis appelle le point d'entrée sans exécuter d'étape (section 6.4)	présent

1.3 Graphe d'appels autorisés

Les modules sont ordonnés : `outils`, `donnees`, `exposition`, `brut`, `lissage`, `positionnement`, `prospective`, `fermeture`, `validation`, `rapport`. Un module ne peut appeler que des fonctions définies dans les modules situés **strictement avant lui** dans cet ordre (ADR 0007, point 2.1). Deux fichiers sont hors de la chaîne : `simulation`, qui ne peut appeler que `donnees` et `outils` et qu'**aucun module du moteur n'appelle**, et `construire_table`, qui peut appeler tous les modules **sauf** `simulation`. La figure 1 et le tableau 2 donnent ce graphe.

Deux raisons fondent cette règle. Un ordre sans cycle rend chaque module lisible et vérifiable avec ses seuls modules amont : une anomalie d'un module ne peut atteindre que les modules situés après lui. Et le simulateur, qui produit les jeux sur lesquels le moteur est testé, ne partage pas le code qu'il sert à tester : s'il pouvait appeler `exposition`, une erreur de la recette d'exposition se retrouverait à l'identique dans les données simulées et dans leur estimation, et la vérification ne la verrait pas (ADR 0007, « Options écartées »). Le simulateur reproduit le même modèle, pas le même code. Une seule exception à la lecture « en aval seulement » est assumée : `positionnement` peut appeler `lissage`, parce que la méthode 4 de positionnement est un lissage de la table du moment avec dérive de la référence (E2.12) ; le sens inverse est interdit, `lissage` n'appelant jamais `positionnement`.

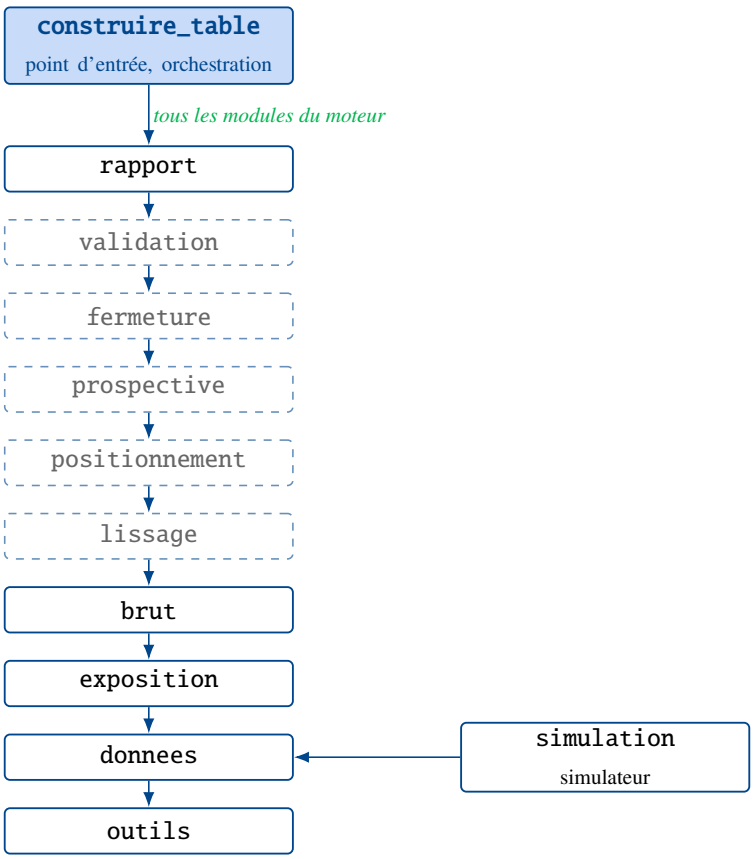


Figure 1 : Graphe d'appels autorisés des modules (ADR 0007, point 2.1). Une flèche de A vers B signifie que A peut appeler B et, la règle étant « strictement avant », tout module situé après B dans le sens des flèches. Blocs pleins : fichiers présents dans `R/` ; blocs en pointillé : modules prévus, fichiers absents. `simulation` n'est appelé par aucun module du moteur, ni par `construire_table` ; `lissage` n'appelle jamais `positionnement`.

Table 2 : Appels autorisés, par module appelant (ADR 0007, point 2.1 ; constante `APPELS_AUTORISES` de `tests/controles/graphe_modules.R`).

Module appelant	Peut appeler
`outils`	aucun module (fonctions des packages de priorité `base` seulement)
`donnees`	`outils`
`exposition`	`donnees`, `outils`
`brut`	`exposition` et les modules amont
`lissage`	`brut` et les modules amont
`positionnement`	`lissage` et les modules amont
`prospective`	`positionnement` et les modules amont

Module appelant	Peut appeler
`fermeture`	`prospective` et les modules amont
`validation`	`fermeture` et les modules amont
`rapport`	tous les modules ci-dessus
`simulation`	`donnees` et `outils` seulement
`construire_table`	tous les modules sauf `simulation`

Contrôle par script

Le graphe est vérifié par `tests/controles/graphe_modules.R`, en R de base (`codetools`, de priorité `recommended`, est exclu par l'ADR 0002), dans le job « Tests (R 4.3.3) » de la CI (ADR 0007, point 2.2) :

1. chaque fichier `R/<module>.R` est lu par `parse()` ; un fichier dont le nom n'est pas un module connu est un écart ;
2. les affectations de premier niveau (`nom <- ...` ou `nom = ...`, fonctions comme constantes) donnent la table « symbole → module » ; un symbole défini dans deux modules est un écart ;
3. pour chaque définition, les noms qu'elle emploie (`all.names()`), croisés avec cette table, donnent les arêtes entre modules ; toute arête absente du tableau 2 est un écart, imprimé avec le module appelant, le module appelé et les deux symboles ;
4. les appels dynamiques interdits (ci-dessous) sont cherchés dans tout l'arbre syntaxique.

Le script imprime toutes les arêtes trouvées et tous les écarts, puis sort avec le code 1 s'il en a trouvé au moins un. Ses limites sont écrites dans son en-tête : un nom local homonyme d'un symbole d'un autre module compte comme une arête (faux positif possible, jamais d'arête manquée par homonymie) ; un code de premier niveau qui n'est pas une affectation est un écart ; les appels vers les packages de base ne sont pas contrôlés ici, mais par la liste blanche de `DESCRIPTION`. Le script est lui-même testé sur des témoins (`tests/unitaires/test_controles.R`) : il échoue, entre autres, quand `exposition` appelle `brut`, quand `donnees` appelle `simulation`, quand `lissage` appelle `positionnement` et quand `construire_table` appelle `simulation`, et il passe sur un graphe conforme où `positionnement` appelle `lissage`.

Appels dynamiques interdits

Un appel dont la cible est calculée à l'exécution (`do.call("f", ...)`, `get("f")`) n'apparaît pas dans `all.names()` : il créerait une arête invisible pour le contrôle. Le script interdit donc dans `R/` (ADR 0007, point 2.3, liste élargie à la mise en œuvre et consignée dans l'annotation du 16/10/2026) :

— `do.call()` dont l'argument `what`, apparié par `match.call()` (position ou nom), n'est ni un symbole ni une fonction anonyme (chaîne, `paste0(...)...`) ; `do.call(f, args)` avec `f` un symbole désignant une fonction reste permis ;

— toute occurrence, appel ou fonction passée comme objet (`lapply(x, get)`), de `match.fun`, `get`, `get0`, `mget`, `getFromNamespace`, `getExportedValue`, `getFunction`, `asNamespace` et `getNamespace` ;

— `eval()` ou `evalq()` d'un `parse()`, `str2lang()`, `str2expression()`, `call()` ou `as.call()` ;

— l'opérateur `:::` et toute référence qualifiée `tablexp::` ;

— les formes qualifiées `base::`, `utils::` et `methods::` de ce qui précède, lues comme la forme nue.

Une fonction passée en argument puis appelée dans le corps de la fonction qui la reçoit crée une arête que le script attribue à l'appelant qui l'a passée : c'est voulu, la dépendance existe bien là. Les tests, seuls, appellent les fonctions non exportées par `tablexp:::` (ADR 0007, point 11.3).

Services transverses : R/outils.R

R/outils.R porte les services sans contenu métier partagés par tous les modules : l'exécution sous graine locale (section 3), la forme des refus et celle des avertissements (ADR 0007, point 1.2), ainsi que le contrôle de la forme exacte d'une valeur reçue, forme_exacte() (type de base et attributs admis, section 6.1 ; issue #23), et deux contrôles de forme qui s'y appuient, partagés par les modules : liste_aux_noms() (liste nue aux noms exacts) et chaine_unique() (chaîne unique sans attribut, ni NA ni vide ; issue #48), ainsi que la règle des noms complets, noms_complets() (noms présents, sans NA ni nom vide), écrite une seule fois pour le contrôle des rôles et pour celui du plan (issue #22), le seul point de passage des messages de refus, formuler() (section 2.1, « Règle de rédaction des messages » ; issue #127), et la règle unique de citation d'une valeur dans un message de refus, neutraliser_citation() (section 2.1 ; issue #94), avec le relevé des noms d'encodage bytes d'un objet reçu, noms_bytes() (issue #114), et les lectures de la classe et d'un champ par nom qui ne lèvent pas d'erreur R sur une classe ou un nom de cet encodage, a_classe(), classe_bytes() et lire_champ() (issue #128), et trois règles écrites une seule fois et partagées par plusieurs modules (issue #113) : deux autres contrôles de forme appuyés sur forme_exacte(), celui d'une graine, forme_graine(), et celui d'une date unique, date_unique(), et la conversion des chaînes en UTF-8 selon la locale, vers_utf8_locale(). Il n'appelle aucun module. Sans lui, la fonction de graine locale serait dupliquée dans le simulateur et dans le futur bootstrap par têtes, et chaque module inventerait sa forme de refus.

2.1 Refus : une condition R typée

Le moteur ne devine pas : quand une entrée ne permet pas de construire, il *refuse* (E2.3, E2.7). Un refus est une **condition R typée, jamais un objet retourné** (ADR 0007, point 6.4). refus_tablexp() construit la condition, de classe c("refus_tablexp", "error", "condition"), avec quatre champs :

— module : le module qui refuse ;

— motif : un code court stable, en minuscules ASCII, chiffres et soulignés, conforme au motif MOTIF_CODE_REFUS (^[a-z][a-z0-9_]*$) ;

— message : la phrase qui dit ce qui manque (E5.5), formée par formuler() (ci-dessous, « Règle de rédaction des messages ») ;

— details : une liste (rôle, colonne, étape, paramètre, décompte, premiers numéros de ligne).

refuser() la lève par stop() et ne rend jamais la main. Un argument mal formé de refus_tablexp() (module, motif ou message qui n'est pas une chaîne unique sans attribut, ni NA ni vide, au sens de chaine_unique(), chaîne nommée comprise ; motif non conforme ; détails qui ne sont pas une liste) n'est pas un refus mais une erreur de programmation du package, signalée par stop(). Les tests vérifient le motif, le décompte et les numéros par tryCatch() ; l'application attrapera la condition et affichera son message. Une condition est préférée à un objet portant ok = FALSE, forme d'outil_USP : un tel objet peut être lu par mégarde comme une construction partielle, une condition ne le peut pas. Les codes de refus de chaque module sont recensés dans la partie du module.

Règle de rédaction des messages (issue #127 ; ADR 0007, annotation du 05/11/2026, (c) à (f)). Tout message de refus du moteur, qu'il insère des valeurs ou non, dans tous les modules (outils compris) et dans les aides locales qui lèvent un refus, est formé par formuler(gabarit, ...), qui vaut

sprintf(texte_moteur(gabarit), ...) : le gabarit, littéral du moteur, est marqué UTF-8 *avant* l'insertion des valeurs (un % littéral s'y écrit %%). L'ordre importe. Un package installé sous la locale C garde ses littéraux non ASCII sans marque (section 4.2, « Locale de l'installation »), et, sous cette locale, sprintf(), paste() et paste0() traduisent un argument non marqué dès qu'un autre est marqué UTF-8, ce qu'est une valeur citée : les octets non ASCII du gabarit deviennent alors le texte ASCII « <c3><a9> », que plus rien ne distingue d'une saisie, et marquer le message après sa formation vient trop tard (constat consigné par l'ADR 0007, annotation du 05/11/2026, (a) et (b)). Pour la même raison, le marquage n'est pas fait dans refuser(), qui reçoit le message déjà formé ; une fonction refuser() qui recevrait le gabarit et les valeurs aurait changé le contrat de la condition, dont le champ message est reçu formé (option écartée, même annotation, (d)). Un argument inséré par %s est de l'une de quatre sortes : une valeur reçue (des données, des rôles, du plan, de la provenance, d'une fiche ou d'un texte lu : chaîne, nom, classe, chemin), citée par neutraliser_citation() ou par ses deux alias, citer() et sans_controle() (ci-dessous) ; un texte déjà formé selon la règle, qui n'est pas cité une seconde fois (résultat de formuler(), de liste_guillemets(), dont les guillemets sont marqués avant le collage pour la même raison, ou d'une aide de message du moteur) ; une constante ASCII ; le texte d'une chaîne constante ou d'une constante du moteur (affectation de premier niveau de R/, éventuellement indexée), marqué par texte_moteur() au moment de l'insertion, jamais une valeur reçue. Une constante du moteur non ASCII est de codage natif non marqué, si bien que, sous la locale C, sprintf() en écrirait les octets « <c3><a8> » : elle n'est insérée que sous cette quatrième forme (ainsi le libellé du type date de TYPES_ADMIS, « Date (valeurs entières finies) », dans le refus type_incompatible). Un nombre inséré par %d, %g ou %f n'est pas cité. **Garantie sur le champ message** : un message qui contient un caractère non ASCII est marqué UTF-8, a les mêmes octets sous les locales C et UTF-8, d'installation comme d'exécution, et tient sur une ligne ; un message entièrement ASCII n'est pas marqué, R ne marquant jamais une chaîne ASCII. **Statut** : la marque est mesurée (Encoding() sur des refus des modules outils, donnees, simulation, rapport et construire_table, paquets installés sous C et sous C.UTF-8, chacun exécuté en session C et en session C.UTF-8 : dans les quatre combinaisons, messages marqués UTF-8 et aux mêmes octets) ; l'identité des octets est un *constat* de tests/controles/locale_c.R (section 4.2) sur un refus par module, trois refus de lecture du texte du plan et deux refus type_incompatible sur un rôle date (colonne character, et classe formée d'une chaîne d'encodage bytes et de Date), non une propriété démontrée. **La garantie ne couvre pas une installation exécutée sous une locale latin-1** : paquet installé et exécuté sous fr_FR.ISO-8859-1 (locale générée par localedef dans un dossier de locales temporaire), le message du refus colonne_absente sort non marqué (Encoding() rend « unknown »), n'est pas de l'UTF-8 valide (validUTF8() rend FALSE) et porte l'octet e9 pour « é » : c'est du latin-1 natif, non marqué UTF-8, comme déjà au commit 3ed0301 (*constat* mesuré le 05/11/2026 ; la portée de la garantie est une question soumise au mainteneur). Les autres combinaisons mesurées donnent un message marqué UTF-8 (*constat*) : installation sous latin-1 exécutée sous C ou sous C.UTF-8, et installation sous C.UTF-8 exécutée sous latin-1 ; ce dernier cas donnait encore, au commit 3ed0301, « unknown » et l'octet e9, et la présente règle le corrige. **Contrôle** : tests/controles/messages_refus.R (tableau 7) lit les fichiers de R/ par parse(), sans charger tablexp, et vérifie que le gabarit de chaque appel de formuler() est une constante chaîne, ou un paste() ou paste0() de constantes, aux conversions en même nombre que les valeurs ; que chaque valeur insérée par %s est de l'une des quatre sortes ci-dessus (suivie à travers ses affectations locales), texte_moteur() n'étant admis que sur une chaîne constante ou une constante du moteur ; que le message de chaque refus (refuser(), refus_tablexp() et leurs relais locaux) est formé par formuler() ou par une aide de message, sans sprintf(), paste() ni paste0() ; qu'une exception figure dans une liste fermée, une ligne par site avec sa raison (texte produit par le moteur seul : constante, catalogue, nombre, type R), que chacune sert encore, et qu'aucune ne couvre une constante du moteur non ASCII non marquée : le script relève ces constantes (affectations de premier niveau de R/ dont l'expression contient une chaîne non ASCII hors d'un appel de texte_moteur()) et refuse toute exception

qui en nomme une, puisqu'elle doit être marquée à l'insertion. **Limites** : les messages d'avertissement (section 2.2) et les erreurs de programmation (`stop()`) sont hors de la règle ; le script ne contrôle pas le contenu d'une valeur citée, et reconnaît un relais ou une aide de message par son nom.

Citation d'une valeur dans un message (issue #94). Une valeur reçue (chaîne de l'utilisateur, champ lu dans une fiche ou un texte, classe, chemin, message d'une erreur) citée dans le `message` d'un refus passe par une règle unique, `neutraliser_citation()` (section 2.3), appliquée à chaque élément, aux sites suivants : `citer()` et `liste_guillemets()` (modules `donnees`, `exposition`, `brut`, `simulation` et contrôle du plan), `sans_controle()` (rejeu outillé), `valeur_message()` et `decrire_valeur()` (rapport figé), lecture du texte du plan et de la fiche (classe de l'objet reçu, première ligne, type, chemin et chemin attendu, colonne, valeur), `ecrire_fiche()` (nom de champ d'encodage `bytes`), `controler_rapport()` (classes de l'objet, nom de champ d'encodage `bytes`, message d'une erreur du recalcul de l'empreinte du résultat, chemin d'un élément non prévu) et `rejouer_fiche()` (classes de la fiche, nom de champ d'encodage `bytes`, noms et attributs de la forme lue). La valeur citée étant une valeur reçue, elle reste brute dans les `details` (classe, chemin, colonne, nom d'encodage `bytes`. . .), à trois précisions près : pour `fiche_non_conforme`, à l'étape 2 du rejeu, `details$lu` est la description de forme (section 6.4), citée par la même règle, et la valeur exacte est celle de la fiche reçue au chemin `details$champ` ; pour la lecture du texte du plan et de la fiche, la valeur reçue brute est dans les `details` (`classe`, `chemin`, `colonne`, `attendu`) ou, à défaut, dans le texte reçu à la ligne `details$ligne` ; pour `version_fiche_non_admise`, à l'étape 3 du rejeu, les `details` `tablexp`, `code` et `code_installe` sont la sortie de `texte_lu()` (section 6.4) : la chaîne elle-même si c'est une chaîne unique sans attribut, sinon « absente » ou « illisible (*type* de longueur *n*) », la version lue `details$lu` restant brute. Depuis l'issue #127, la règle vaut aussi pour tout autre refus qui cite une valeur reçue (règle de rédaction ci-dessus, contrôlée par `tests/controles/messages_refus.R`), dont `type_incompatible` du module `donnees` (rôle, nom de colonne et classe), `type_non_canonique` (rôle et classe) et `serialisation_impossible` (chemin, classes et noms d'attributs) du module `rapport`, où un caractère de contrôle ou un séparateur de ligne reçu apparaissait auparavant brut dans le `message`. La règle porte sur le texte des refus, jamais sur une valeur écrite ni empreintée. Par élément :

— `NA` reste `NA` ;

— une chaîne d'encodage `bytes` est remplacée par le repère « <chaîne d'encodage bytes> », sans réinterprétation de ses octets (`sprintf()` lèverait sur elle l'erreur R « translating strings with "bytes" encoding is not allowed » au lieu du refus ; issue #111) ;

— une chaîne marquée latin-1 est d'abord convertie par `enc2utf8()`, qui la lit selon la page de code CP1252 (aide `?Encoding` de R 4.3.3, « Details », pour R 3.5.0 et suivants ; constaté sous les locales C et C.UTF-8) : les octets 0x80 à 0x9F y deviennent des caractères imprimables (« € », « œ », « ’ ». . .), non des caractères de contrôle, et les cinq positions que CP1252 ne définit pas (0x81, 0x8D, 0x8F, 0x90, 0x9D) sont écrites <81>. . . par `enc2utf8()` elle-même ;

— dans une chaîne UTF-8 valide (marquée UTF-8, convertie, ou non marquée aux octets UTF-8 valides), chaque point de code des catégories Unicode Cc, Zl et Zp — U+0000 à U+001F, U+007F, U+0080 à U+009F, U+2028 et U+2029, soit 67 points de code, dont U+0000 qu'aucune chaîne R ne contient — s'écrit <U+XXXX> (quatre chiffres hexadécimaux majuscules) ; cet ensemble contient tous les séparateurs de ligne (LF, VT, FF, CR, U+001C à U+001E, NEL, U+2028, U+2029) ;

— une chaîne qui n'est pas de l'UTF-8 valide est citée octet par octet : un octet de 0x80 à 0xFF s'écrit <xx> (hexadécimal minuscule), un octet de 0x00 à 0x1F ou 0x7F <U+00XX>, les autres tels quels.

Les notations sont celles de `?iconv` (R 4.3.3) : <xx> pour `sub = "byte"`, <U+xxxx> pour `sub = "Unicode"`. Le travail porte sur les octets (`charToRaw()`, `utf8ToInt()`) : la sortie est la même sous les locales C et UTF-8 (constat des tests et de `tests/controles/locale_c.R`). Le résultat est marqué UTF-8 s'il contient un octet non ASCII. Un message de refus qui cite une valeur tient ainsi sur une ligne.

tests/unitaires/test_outils.R vérifie la règle et son prédicat sur onze caractères (VT, FF, LF, CR, tabulation, U+0001, U+007F, NEL, U+009F, U+2028, U+2029), sur l'ensemble des points de code de U+0001 à U+FFFF hors demi-codets (U+D800 à U+DFFF), dont il retrouve exactement les 66 de l'ensemble, U+0000 mis à part, sur des caractères de format et « é » cités tels quels, sur NA, une chaîne bytes, des octets qui ne sont pas de l'UTF-8 valide, une chaîne latin-1 et une chaîne native UTF-8, et son idempotence ; les sites qui l'appliquent sont testés avec leur module. **Limites acceptées** : les caractères de format (catégorie Cf : U+200B à U+200F, U+202A à U+202E, U+2066 à U+2069, U+FEFF) ne sont pas neutralisés, parce qu'ils ne coupent pas une ligne et sont légitimes dans certains noms (U+200C et U+200D) ; un contrôle bidirectionnel peut donc réordonner l'affichage d'une valeur citée ; le caractère « < » n'est pas échappé, si bien qu'un « <U+000A> » tapé tel quel se cite comme un LF. citation_a_neutraliser() est le prédicat de la règle (vrai pour un élément que la règle ne cite pas tel quel), et citer(), sans_controle() (rejeu) l'appliquent sous leur nom. L'écrivain du texte du plan et de la fiche (texte_objet()) remplace en outre par une espace les caractères de contrôle des lignes d'en-tête « # nom : valeur » qu'il écrit (fonction locale propre) : ce sont des octets écrits, non un message de refus, et ce remplacement reste hors de la règle.

Noms d'encodage bytes (issue #114). Une lecture par nom (x[["champ"]], x$champ) lève l'erreur R « translating strings with "bytes" encoding is not allowed » dès qu'elle rencontre un nom d'encodage bytes placé avant le nom cherché ou à sa place (constat sous les locales C et C.UTF-8). Les fonctions qui lisent par nom une fiche ou un objet reçu — rejouer_fiche(), ecrire_fiche() et le contrôle du rapport figé (controler_rapport()) — relèvent donc d'abord, par noms_bytes(), les noms de premier niveau de cet encodage, et refusent sous un motif nommé quelle que soit la position du nom, le nom étant cité par la règle ci-dessus (repère) et gardé brut dans details$lu : fiche_non_conforme au rejeu (section 6.4), chaine_codage_bytes à l'écriture de la fiche (section 15.4), objet_modifie au rapport figé (tableau 28). Les noms sont lus par attr(x, "names", exact = TRUE) et non par names(), fonction générique qui dispatche sur la classe et lève la même erreur sur une liste dont la classe est d'encodage bytes.

Classes et noms d'encodage bytes sous le premier niveau (issue #128). La même erreur R est levée, sur une classe d'encodage bytes, par les fonctions qui dispatchent sur la classe (inherits(), is.factor(), is.data.frame(), names(), length(), is.na(), dim()), et, sur un nom de cet encodage, par la lecture x[["champ"]] ; attr(), match(), %in%, Encoding(), unclass() et .subset2() ne la lèvent pas (constat sous les locales C et C.UTF-8, R 4.3.3). Trois fonctions de R/outils.R lisent donc sans dispatch ni traduction : a_classe() (une classe nommée par l'appelant figure-t-elle dans l'attribut class ?), classe_bytes() (l'attribut class a-t-il un élément d'encodage bytes ? l'appelant retire alors la classe par unclass() avant length(), names() ou is.na()) et lire_champ() (élément d'une liste trouvé par match() sur les noms lus par attr(), puis lu par position). Les contrôles des rôles, l'écrivain unique, l'écriture de la fiche, le contrôle du rapport figé et le rejeu outillé les emploient, et refusent sous un motif nommé, à toute position du nom : type_incompatible pour une colonne de rôle de classe bytes, ou de nom bytes et de type non conforme (section 7.3) ; type_non_canonique ou serialisation_impossible pour une valeur écrite de classe bytes, et chaine_codage_bytes pour un nom bytes d'une sous-liste écrite, rôle « *chemin*/@noms » (section 15.4) ; objet_modifie au rapport figé pour une fiche de classe bytes ou une sous-liste de nom ou de classe bytes dans le périmètre de l'empreinte du résultat, l'empreinte n'étant pas recalculable (tableau 28) ; au rejeu, fiche_non_conforme à l'étape 2 pour un champ que le rejeu lit lui-même ; pour roles et plan, les refus délégués de l'étape 6, tous nommés (un nom bytes dans plan : champ_inconnu ; dans plan/etapes : etape_inconnue ; dans plan/etapes/brut : parametre_inconnu ; dans plan/transition : transition_inconnue ; dans roles : role_inconnu ; une classe bytes sur plan/etapes : etapes_mal_formees) ; ailleurs, témoin « non déterminable » ou « différente » (section 6.4). Une classe d'encodage bytes est citée sous le repère de la règle ci-dessus. **Statut et limites** : pour l'écriture de la fiche, le rapport figé et le rejeu, *constat* des tests et de mesures

sous les locales C et C.UTF-8, aux niveaux 1 à 7 de la fiche, non une propriété démontrée pour toute profondeur ; pour le témoin de la fiche (règle V3, section 6.4), la tenue à toute profondeur est une *lecture* de la récursion de `champs_differents()`, bornée par la fiche rendue, avec un *constat* aux niveaux 2 à 7. Restent hors de cette règle, renvoyés à des issues : une *valeur* chaîne d'encodage `bytes` dans `versions` ou `plateforme` fait lever l'erreur R au rejeu (lecture des paires du témoin ; issue #142, constat) ; une colonne de rôle de nom `bytes` fait lever l'erreur R dans la préparation des données (`preparer_donnees()`, fonction interne), cas que les fonctions exportées n'atteignent pas : `construire_table()` et le rejeu refusent ce nom avant la première étape, au calcul des empreintes (`chaine_codage_bytes`, module `rapport` ; issue #140).

2.2 Avertissements métier

Un avertissement métier signale ce qui n'empêche pas la construction mais doit être lu : unité d'observation par défaut, exclusion appliquée, relevé d'incohérence non bloquant. Il est **inscrit dans l'objet résultat, jamais émis par `warning()`** (ADR 0007, point 6.5) : l'objet est la seule source du rapport figé et de l'application ; un `warning()` serait perdu par l'application, absent de la fiche et parasite dans les batteries. `avertissements_vides()` crée l'accumulateur, un `data.frame` de colonnes `etape`, `code`, `message` et `nombre` ; `nombre`, un dénombrement (lignes, cellules, têtes), est stocké en `integer`, positif ou nul (issue #24). `avertir()` lui ajoute une ligne, et refuse, par une erreur de programmation (`stop()`, sans conversion), un code qui n'est pas une chaîne unique sans attribut, ni `NA` ni vide, au sens de `chaine_unique()` (une chaîne nommée ou munie d'une classe est refusée ; issue #60), ou qui n'est pas conforme à `MOTIF_CODE_REFUS`, et un nombre qui n'est pas un nombre unique, fini, entier, positif ou nul et au plus égal à `.Machine$integer.max` ; sinon il le stocke par `as.integer()`. Chaque fonction d'étape rend ses avertissements ; le point d'entrée les rassemble dans le champ `avertissements` de l'objet et de la fiche. Les codes de chaque étape sont recensés dans la partie de son module (sections 7.4, 8 et 9.6).

2.3 Fonctions du module

Table 3 : Définitions de premier niveau de `R/outils.R`.

Nom	Statut	Entrées	Sortie et rôle
`GENERATEUR`	constante	—	générateur fixé (section 3.2) : Mersenne-Twister, inversion pour la loi normale, rejet pour `sample()`
`MOTIF_CODE_REFUS`	constante	—	motif d'un code de refus ou d'avertissement : `^[a-z][a-z0-9_]*$`
`refus_tablexp()`	interne	`module`, `motif`, `message`, `details` (liste, vide par défaut)	condition de classe `refus_tablexp`, sans la lever ; `stop()` si un argument est mal formé (section 2.1)
`refuser()`	interne	mêmes arguments	lève le refus ; ne rend jamais la main
`forme_exacte()`	interne	`x`, `types` (types de base admis), `attributs` (noms d'attributs admis, aucun par défaut)	`TRUE` si `typeof(x)` est pris dans `types` et si `x` ne porte aucun attribut hors de `attributs`, `FALSE` sinon, jamais `NA` ; un attribut admis n'est pas requis et sa valeur n'est pas contrôlée ; ne refuse pas, l'appelant refuse sous son propre motif (section 6.1 ; issue #23)

Nom	Statut	Entrées	Sortie et rôle
liste_aux_noms()	interne	x, noms (noms attendus)	TRUE si x est une liste nue (forme_exacte() de type list, seul attribut admis : les noms ; ni data.frame, ni classe) d'autant d'éléments que noms, aux mêmes noms, sans nom manquant ni doublé, comparés *sans tri* (anyNA(), anyDuplicated(), setequal()), FALSE sinon (liste sans noms comprise) ; sous la précondition de noms attendus ASCII, vraie de tous les appelants, jamais une erreur, et le résultat ne dépend pas de la locale, même pour un nom reçu non ASCII de codage indéterminé ou aux octets invalides ; hors de cette précondition, aucune de ces deux propriétés n'est garantie (le tri sort(method = "radix") de la version antérieure levait alors une erreur R ; issue #100) ; le contenu des éléments, lus par [[ ]], est contrôlé par l'appelant, qui refuse sous son propre motif (issue #48)
chaine_unique()	interne	v	TRUE si v est de forme exacte character, sans aucun attribut (noms compris), de longueur 1, ni NA ni vide, FALSE sinon, jamais NA ; ne refuse pas (issue #48)
noms_complets()	interne	noms (names(x))	TRUE si les noms sont présents (non NULL), sans NA et non vides, FALSE sinon, jamais NA ; prédicat sur les noms seuls : la forme de l'objet, les doublons et la liste vide sans noms relèvent de l'appelant ; seul endroit où s'écrit la règle, partagée par controler_roles() et liste_noms_complets() (issues #22, #60 et #62)
forme_graine()	interne	g	TRUE si g est de forme exacte integer ou double (ni noms, ni dimension, ni classe), de longueur 1, non NA, finie, entière et de valeur absolue au plus .Machine$integer.max, FALSE sinon, jamais NA ; ne refuse pas ; règle de sous_graine(), partagée avec le contrôle de la graine des défauts du simulateur (section 3.3 ; issues #43 et #113)

Nom	Statut	Entrées	Sortie et rôle
date_unique()	interne	x	TRUE si x est une date unique entière et finie, de forme exacte (classe Date seule, stockage double ou integer), FALSE sinon, jamais NA ; ne refuse pas ; règle partagée par les bornes de la période d'observation (modules donnees et simulation) et par les dates du portefeuille simulé (issues #58 et #113)
vers_utf8_locale()	interne	x (character)	chaînes converties en UTF-8 selon la locale de la session : enc2utf8() sur toutes sous une locale UTF-8 ou latin-1 (l10n_info()), sur les seules chaînes marquées latin-1 sinon, les autres gardant leurs octets ; même longueur, NA conservés ; ne refuse pas ; règle unique de l'empreinte des données (octets_chaines(), après ses deux premiers refus) et de l'ordre des segments (ADR 0007, point 9.3, annotation du 05/11/2026, (g) ; issues #106 et #113)
noms_bytes()	interne	x	positions (integer, jamais NA) des noms de premier niveau d'encodage bytes, integer(0) si x n'a pas de noms ou aucun de ce codage ; noms lus par attr(x, "names", exact = TRUE), non par names(), qui dispatche sur la classe et lève sur une classe d'encodage bytes ; contrôle préalable à toute lecture par nom d'un objet reçu (section 2.1 ; issue #114)
a_classe()	interne	x, classe	TRUE si classe figure dans l'attribut class de x, lu par attr(), FALSE sinon ; équivaut à inherits() pour une classe posée en attribut, sans lever d'erreur R sur une classe d'encodage bytes (section 2.1, « Classes et noms d'encodage bytes sous le premier niveau » ; issue #128)
classe_bytes()	interne	x	TRUE si un élément de l'attribut class de x est d'encodage bytes, FALSE sinon ; l'appelant retire alors la classe par unclass() avant toute fonction qui dispatche (issue #128)

Nom	Statut	Entrées	Sortie et rôle
lire_champ()	interne	x (liste), nom	premier élément de x de ce nom, NULL s'il est absent, comme x[[nom]], mais trouvé par match() sur les noms lus par attr() et lu par position (.subset2()) : aucune erreur R sur un nom ou une classe d'encodage bytes, à toute position (issue #128)
avertissements_vides()	interne	—	accumulateur vide des avertissements métier, colonne nombre en integer (section 2.2 ; issues #9 et #24)
avertir()	interne	accumulateur, etape, code, message, nombre	accumulateur augmenté d'une ligne, message marqué UTF-8 par texte_moteur(), sans warning() ; stop() si le code (chaîne unique au sens de chaine_unique(), conforme à MOTIF_CODE_REFUS) ou le nombre est mal formé (section 2.2 ; issues #9, #24, #50 et #60)
texte_moteur()	interne	x, chaîne ou liste (data.frame compris)	le même objet, chaînes et noms non ASCII du moteur marqués UTF-8 en toute locale d'installation et d'exécution ; réservée au texte du moteur, jamais aux données, rôles ou plan de l'utilisateur (section 4.2, « Locale de l'installation » ; issue #50)
formuler()	interne	gabarit (littéral du moteur), valeurs insérées	message de refus, sprintf() du gabarit marqué d'abord par texte_moteur() ; seul point de passage de tout message de refus, avec ou sans valeur ; valeurs reçues citées par l'appelant, textes formés et constantes ASCII seulement (section 2.1, « Règle de rédaction des messages » ; issue #127)
neutraliser_citation()	interne	x (converti par as.character() s'il n'est pas de type character)	vecteur de même longueur, sans noms, chaque élément cité selon la règle unique des valeurs citées dans un message de refus : NA conservé, repère pour une chaîne bytes, latin-1 converti, <U+XXXX> pour un caractère des catégories Cc, Zl et Zp, <xx> pour un octet hors UTF-8 valide ; marqué UTF-8 s'il est non ASCII (section 2.1, « Citation d'une valeur dans un message » ; issue #94)

Nom	Statut	Entrées	Sortie et rôle
`citation_a_neutraliser()`	interne	`x`, comme ci-dessus	vecteur logique, `TRUE` pour un élément que la règle ne cite pas tel quel (chaîne `bytes`, octets hors UTF-8 valide, caractère de l'ensemble neutralisé), `FALSE` pour `NA` et pour une chaîne citée telle quelle, conversion latin-1 et marque UTF-8 mises à part ; l'ensemble n'est défini qu'une fois, dans la règle (issue #94)
`sous_graine()`	interne	`graine` (entier explicite, sans défaut ni attribut), `expr`	valeur de `expr` évaluée sous la graine et `GENERATEUR`, état du générateur de l'appelant restauré ; refus `graine_invalide` si `forme_graine()` est faux (section 3.3)

Reproductibilité et aléa

3.1 Exigence et portée

À données, rôles, plan et graine identiques, le programme produit des résultats identiques **au bit près sur une même machine** ; toute simulation passe par une graine locale explicite sous un générateur fixé, consignée dans le résultat, l'état global du générateur étant restauré à la sortie ; aucun horodatage n'entre dans la partie du résultat comparée lors d'un rejeu (E4.6). La présente section décrit les deux mécanismes du socle : le générateur fixé et l'exécution sous graine locale. La consignation de la graine et du générateur dans la fiche de construction et le rejeu de la fiche sont décrits avec la fiche (section 15.1).

> **Statut.** L'identité au bit près est un *constat* des batteries (section 3.4), établi sur la machine qui les exécute ; ce n'est pas une propriété démontrée. L'identité entre deux machines n'est **pas garantie** : deux processeurs différents pourraient différer au dernier bit, même dans le même conteneur, si la bibliothèque mathématique du système choisit ses variantes de calcul selon le processeur (réserve de l'ADR 0008 : non observée sur trois modèles de processeur munis de FMA et d'AVX2, ouverte pour un exécuteur sans FMA ; annotation du 19/10/2026). C'est pourquoi les références de non-régression sont produites sur une plateforme unique et comparées à tolérance (sections 4.3 et 4.5).

3.2 Générateur fixé

Le générateur est la constante `GENERATEUR` de `R/outils.R` : `kind = "Mersenne-Twister"`, `normal.kind = "Inversion"`, `sample.kind = "Rejection"`. Ce sont les valeurs par défaut de R 4.3.3 (`RNGkind()` dans une session neuve), fixées en constante pour qu'un changement de défaut d'une version de R ne change pas les tirages en silence. Le générateur uniforme est le Mersenne-Twister de Matsumoto et Nishimura ; les lois normales s'obtiennent par inversion de la fonction de répartition ; la génération uniforme discrète (`sample()`) emploie la méthode par rejet, défaut de R depuis la version 3.6.0, la méthode antérieure par arrondi rendant `sample()` sensiblement non uniforme sur de grandes populations (documentation de R, `?Random`).

Référence. M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`).

Un changement de `GENERATEUR` change tout résultat simulé : c'est un changement de résultat, soumis au visa (ADR 0007, « Conséquences »).

3.3 Exécution sous graine locale

`sous_graine(graine, expr)` est la seule voie par laquelle le package tire un aléa (ADR 0007, point 8.1). Elle procède ainsi :

1. **contrôle de la graine** : un nombre unique, de type de base `integer` ou `double` et **sans aucun attribut** — ni noms, ni dimension, ni classe (`forme_exacte()`, section 2.3 ; issue #43) : une graine non canonique ne doit pas être consignée —, non manquant, fini, entier et de valeur absolue au plus `.Machine$integer.max`, règle écrite une seule fois, `forme_graine()`, et partagée avec le contrôle de la graine des défauts du simulateur (section 16.3 ; issue #113) ; sinon, et aussi quand la graine est omise, refus `graine_invalide` du module `outils`. Il n'y a jamais de graine par défaut : une graine par défaut serait une graine non déclarée ;
2. **sauvegarde de l'état de l'appelant** : `.Random.seed` de l'environnement global, ou son absence, et le réglage `RNGkind()` ;
3. **tirage** : `set.seed(graine, kind, normal.kind, sample.kind)` sous `GENERATEUR`, puis évaluation de `expr` ;
4. **restauration**, par `on.exit()`, donc aussi après une erreur dans `expr` : le réglage d'abord, parce que `RNGkind()` réécrit `.Random.seed`, puis `.Random.seed` de l'appelant, ou sa suppression s'il n'existait pas.

Les tirages ne dépendent donc que de la graine : ni de l'état du générateur de l'appelant, ni de son réglage, que `sous_graine()` laisse inchangés. Le simulateur (section 16) est, à cette date, la seule fonction du package qui tire un aléa ; il reçoit sa graine en argument et la consigne, avec `GENERATEUR`, dans le jeu simulé (et, pour des défauts paramétrés, une seconde graine déclarée, distincte, section 16.3).

Limite. La réserve de la méthode normale de Box-Muller d'un appelant, gardée par R hors de `.Random.seed`, n'est pas restaurable ; c'est sans effet sur le moteur, qui tire toujours sous `Inversion` (limite reprise d'`outil_USP`, ADR 0007, point 8.1).

Aucun aléa hors calcul. Les fichiers temporaires, employés par exemple pour calculer une empreinte par `tools::md5sum()`, sont créés par `tempfile()`, qui ne touche pas `.Random.seed` (constat : dans une session neuve, `.Random.seed` n'existe ni avant ni après `tempfile()` et `tools::md5sum()` ; ADR 0007, point 8.2).

3.4 Vérifications

Deux fichiers de tests portent ces propriétés (section 4) :

— `tests/unitaires/test_outils.R` : valeur de `GENERATEUR` ; mêmes tirages que `set.seed()` nu sous le générateur par défaut ; graine entière écrite en double admise ; appelant sous `L'Ecuyer-CMRG` : tirages de `GENERATEUR`, `.Random.seed` et `RNGkind()` de l'appelant restaurés ; absence de `.Random.seed` restaurée ; état restauré après une erreur dans `expr` ; refus `graine_invalide` des graines mal formées ;

— `tests/test_reproductibilite.R` : à graine égale, tirages identiques (`identical()`) dans un même processus, quel que soit l'état du générateur de l'appelant, et dans un autre processus R de la même machine ; à graines différentes, tirages différents. Les tirages comparés couvrent les trois réglages du

générateur (`runif()`, `rnorm()`, `sample.int()`) ; le processus fils les écrit en `%.17g`, dix-sept chiffres significatifs, que le processus parent relit : sur la plateforme de référence (glibc), la relecture au bit près est un constat de mesure, non une propriété démontrée (ADR 0011, points 2.4 et 3.2 ; section 15.2). La même batterie vérifie le simulateur et la construction (sections 16 et 15) et compare les plans de référence à leurs références de non-régression (section 4.5).

Ces vérifications sont des constats sur la machine qui exécute la batterie ; sur la CI, c'est la plateforme de référence (section 4.3).

Batteries de vérification et plateforme de CI

4.1 Trois batteries

Les vérifications du code passent par trois batteries en R de base, sans `testthat` (package du CRAN, hors priorité `base`), lancées depuis la racine du dépôt (ADR 0007, point 11.1) :

```
Rscript tests/test_unitaires.R
Rscript tests/test_reproductibilite.R
Rscript tests/concordance_doc_moteur.R --strict
```

La même liste, dans le même ordre, figure dans `CLAUDE.md` (« Commandes »), dans le workflow `.claude/workflows/circuit-technique.js` et dans le job « Tests (R 4.3.3) » de la CI ; un script le contrôle (section 5). Chaque batterie :

— **installe l'arbre de travail** dans une bibliothèque temporaire (`R CMD INSTALL --no-docs -l <bibliothèque> <racine>`, par `installer_tablexp()`, sous une locale UTF-8 : section 4.2), sans installer aucun autre package, puis charge `tablexp` depuis elle et échoue s'il est chargé d'ailleurs : c'est le code de l'arbre qui est testé, jamais un package installé auparavant ;

— affiche `R.version.string` puis, à la fin, une **ligne de bilan** : version de R, nombre de tests, d'échecs, d'échecs attendus et de succès inattendus ;

— **sort avec un code non nul** en cas d'échec ou de succès inattendu (section 4.2).

Une bibliothèque déjà installée, désignée par la variable d'environnement `TABLEXP_BIBLIOTHEQUE`, n'est réutilisée que si l'*empreinte de l'arbre source* inscrite à son installation est égale à celle de l'arbre courant ; sinon la batterie réinstalle et le dit. Cette empreinte est un MD5 (`tools::md5sum()`) de `DESCRIPTION`, de `NAMESPACE` et, s'ils existent, de tous les fichiers de `R/`, `inst/`, `data/` et `src/`, chemins relatifs compris, triés par octets (`empreinte_source()`). Elle empêche de tester un package périmé ; elle n'a aucun rôle dans les résultats.

`tests/test_unitaires.R` exécute chaque fichier `tests/unitaires/test_*.R`, triés par nom, chacun dans son propre environnement, où il est lu par `parse(encoding = "UTF-8")` puis évalué, si bien que ses littéraux non ASCII sont marqués UTF-8 quelle que soit la locale de la session (décision M54) ; `quit()`, `q()` et `bilan()` y sont neutralisés, une erreur au chargement d'un fichier est un échec, et un fichier sans aucune assertion aussi. **`tests/test_reproductibilite.R`** vérifie les identités de la section 3.4, puis celles du simulateur et de la construction, compare les plans de référence à leurs références de non-régression (section 4.5) et contrôle l'effet d'une permutation des lignes de l'entrée de référence (section 15.2, « Représentation canonique » ; issue #39), puis l'aller-retour du texte du plan et de la fiche et le rejeu à partir

de ce texte pour les six plans de référence (section 15.4). **tests/concordance_doc_moteur.R** confronte le présent document au code installé : toute fonction citée existe ; toute fonction exportée est citée, ou inscrite dans la liste d'attente `tests/concordance_attente.csv` avec renvoi à l'issue qui la documentera, où elle compte en échec attendu ; chaque catalogue recensé par le document (fonctions exportées, définitions de premier niveau de chaque fichier de `R/`, rôles, recettes d'exposition) est égal, nom pour nom, à sa source dans le code ; toute ligne de la liste d'attente désigne un export, un catalogue requis, une entrée d'un catalogue requis déjà écrit (type `entree`, de la forme `<catalogue>:<nom>`, pour une définition ajoutée à un module dont le tableau existe déjà) ou une valeur citée (type `fiche`, ci-dessous) et renvoie à une issue ; une entrée en attente absente du document compte en échec attendu, présente elle est un succès inattendu. Elle compare en outre au package les valeurs que le document cite (issue #70) : chaque constante de version (`VERSION_FICHE` et les quatre premières lignes `ENTETE_CANONIQUE`, `REPRESENTATION_RESULTAT`, `PREMIERE_LIGNE_PLAN`, `PREMIERE_LIGNE_FICHE`) est citée, et toute valeur citée est la valeur courante ; la liste des champs de la fiche (section 15.1) a les champs de `names(fiche)`, dans leur ordre, et chaque identifiant qu'elle cite est un nom de la fiche, un objet de `tablexp`, un estimateur ou un littéral d'une liste blanche du script ; les blocs de lecture sont ceux de `BLOCS_LECTURE_FICHE` (blocs, champs et ordre). Enfin, aucun nom de fonction n'est coupé par `\allowbreak` entre deux `\code{}` : le fragment final serait lu comme une fonction et le nom entier échapperait au contrôle d'existence (issue #84). Le format des citations et des catalogues qu'elle lit est fixé par `docs/latex/CONVENTIONS.md` (section 3) ; les tableaux de fonctions de chaque module en sont la principale application (E4.9). Sans `--strict`, elle rend compte des écarts sans échouer.

4.2 Harnais

Les trois batteries partagent `tests/harnais.R`, repris de la forme d'`outil_USP` (ADR 0007, point 11.2, et annotation du 16/10/2026, point d) :

— `verifier(nom, expr)` : succès si `expr` vaut `TRUE` ; une chaîne décrit l'écart ; toute autre valeur, ou une erreur, est un échec ;

— `echec_attendu(nom, issue, expr, motif, erreur_quelconque)` : **défaut connu**, codé en échec attendu avec renvoi obligatoire à l'issue, de la forme `#N` ; `expr` décrit le comportement *correct*. Tant qu'elle échoue, le test est « attendu » ; **si elle passe, c'est un succès inattendu et la batterie échoue** : le défaut semble corrigé, la marque est retirée et le test devient ordinaire. Trois natures d'échec attendu sont admises et décomptées : par valeur (`expr` rend `FALSE` ou une chaîne), par refus (un `refus_tablexp`, du motif donné s'il l'est) et, seulement si `erreur_quelconque = TRUE`, par une autre erreur ; toute autre issue est un échec, pour qu'un défaut connu ne masque pas une panne différente ;

— `proche(a, b, rel, abs)` : égalité numérique à tolérance déclarée, $|a - b| \leq abs + rel\,|b|$ élément par élément, sans tolérance par défaut ; rend un écart pour des valeurs non numériques, des longueurs différentes ou des valeurs manquantes à des positions différentes ; exige l'égalité exacte des valeurs infinies ;

— `leve_refus(expr, motif)` : vrai si `expr` lève un `refus_tablexp` du motif donné ;

— `sans_execution(x)` : retire de l'objet résultat le champ des informations d'exécution, seul champ qui change entre deux exécutions identiques, avant `identical()` ;

— `bilan()` : ligne de bilan et code de sortie. Il compte en outre un échec si un espace de noms hors priorité `base`, autre que `tablexp`, est chargé en fin de batterie (`espaces_hors_base()`) : l'image de la CI contient des packages `recommended`, et cette règle empêche un test de s'appuyer sur l'un d'eux à l'insu de la liste blanche (ADR 0008, « Conséquences »).

Locale de l'installation (décision M54 ; ADR 0007, annotation du 17/10/2026). Un package installé sous une locale qui n'est pas UTF-8 garde ses littéraux non ASCII non marqués, qu'une conver-

sion vers UTF-8 dénature dans une session de même locale (constat consigné par l'ADR 0007) ; les chaînes du moteur entrent dans la fiche et dans les références de non-régression. Si la session n'est pas en UTF-8, `installer_tablexp()` lance donc `R CMD INSTALL` sous la première locale UTF-8 disponible parmi `C.UTF-8`, `C.utf8` et `en_US.UTF-8` (`locale_installation()`), et échoue s'il n'y en a aucune ; la locale d'exécution des batteries reste libre. La disponibilité se sonde, elle ne se suppose pas (`locale_utf8_disponible()`) : une candidate est disponible si `Sys.setlocale()` l'accepte pour `LC_CTYPE`. Sous l'UCRT de Windows, cette acceptation ne prouve pas que la locale existe : `Sys.setlocale()` accepte un nom inventé (`xx_XX.UTF-8`, `yy_YY.utf8`, `en_XX.UTF-8`) et lui donne la culture invariante, que ni `l10n_info()`, ni les noms des mois, ni la casse, ni le tri ne distinguent de `en_US.UTF-8`, la candidate réelle de la liste sous Windows (`fr_FR.UTF-8`, par exemple, s'en distingue par ses mois, « janvier ») ; un nom erroné de la liste des candidates passerait donc inaperçu. Parmi les grandeurs mesurées, seules celles de `LC_MONETARY` (`Sys.localeconv()`) les distinguent ; le critère en retient le symbole monétaire international, champ `int_curr_symbol` : « XDR » pour la culture invariante, « USD » pour `en_US.UTF-8`, « EUR » pour `fr_FR.UTF-8`. La sonde refuse donc une candidate acceptée dont ce symbole égale celui d'un nom sûrement inventé (`LOCALE_INVENTEE`, `xx_XX.UTF-8`). Le critère n'est actif que si ce nom est lui-même accepté : la glibc le refuse, et le critère n'y joue pas, la sonde y étant déjà fiable (une locale non installée est refusée). Ce seul champ est comparé parce qu'il est en ASCII : le symbole local de la devise est rendu dans la page de code de `LC_CTYPE`, et différerait entre la session, sous sa page de code, et la candidate. `LC_CTYPE` et `LC_MONETARY` sont restaurées après la sonde. Ces propriétés sont testées dans `tests/unitaires/test_controles_portables.R` : sur toute plateforme, un nom inventé seul n'est pas disponible, et placé devant les candidates il ne change pas le résultat ; sous Windows, les noms inventés sont refusés, y compris depuis une session en page de code 1252, et le critère devient inactif quand le nom de référence est refusé. **Limite** : un nom à langue inventée et pays réel (`xx_US.UTF-8`, symbole « USD ») échappe au critère et serait tenu pour disponible. Sa portée est restreinte : `LOCALES_UTF8` ne contient que des noms fixés dans le code, dont aucun n'a cette forme ; sous l'UCRT, `C.UTF-8` et `C.utf8` sont refusés et `en_US.UTF-8` est réelle (constat de mesure) ; la limite ne jouerait que si un nom erroné de cette forme y était ajouté, et même alors un tel nom reçoit la page de code 65001, celle de l'UTF-8 (mesuré pour `xx_US.UTF-8`), tandis que `sonde_utf8()`, après le chargement (ci-dessous), reste le contrôle de ce qui compte. **Statut** : le comportement de l'UCRT est un *constat de mesure* sur un seul poste (Windows 11 Home, version 10.0.26300, R 4.3.3 ucrt, 06/11/2026) ; l'UCRT étant un composant du système, il peut changer avec la version de Windows ; il n'est pas tiré de la documentation de l'UCRT, qui n'a pas été consultée. Le critère et sa limite ont été retenus par le mainteneur le 06/11/2026 (issue #109 ; décision consignée dans le corps de la pull request #157, section « Décisions et termes proposés »). Après le chargement, une sonde (`sonde_utf8()`) vérifie qu'un littéral non ASCII du moteur, la constante `STATUT_EMPREINTE_RESULTAT`, est de l'UTF-8 valide et inchangé par la conversion ; sinon la batterie échoue. Limite admise : une installation hors UTF-8 sous une session UTF-8 passe la sonde, sans effet sur les résultats (constat de l'audit consigné par l'ADR 0007 : le plan de référence R1 aplati sous les deux locales donne le même fichier). La sonde ne protège que les batteries : une garde au chargement du package, pour tout usage, est l'objet de l'issue #25. Le **texte produit par le moteur** (messages d'avertissement, mentions, hypothèses et statuts de l'étape `brut`, unité d'exposition, hypothèse de censure, en-tête du texte du plan et de la fiche, gabarits des messages de refus, ces derniers par `formuler()`, section 2.1) est en outre marqué UTF-8 par `texte_moteur()`, quelle que soit la locale d'installation ou d'exécution : sans ce marquage, un package installé ou exécuté sous la locale C laissait des chaînes non ASCII du moteur sans codage déclaré, et `ecrire_fiche()` refusait la fiche (constat de la revue finale de l'issue #50). Mesuré pour R1 : même texte de fiche, octet pour octet, dans les quatre combinaisons d'installation et d'exécution sous C et C.UTF-8. Le marquage ne vaut que pour le texte du moteur : une chaîne non ASCII non marquée des données, des rôles ou du plan de l'utilisateur reste refusée (`chaine_codage_indetermine`, section 15.2) ; pour les données, les rôles et la provenance, le

refus a lieu avant la première étape (section 6.2 ; issue #106 pour les rôles et la provenance).

Contrôle sous la locale C (issue #66). Les batteries installant toujours `tablexp` sous une locale UTF-8, une régression propre à un package installé et exécuté sous la locale C leur échapperait ; des défauts de cette nature ont déjà été trouvés par lancement manuel (refus d'`ecrire_fiche()`, ci-dessus). Le contrôle `tests/controles/locale_c.R` (tableau 7) installe donc l'arbre de travail sous `LC_ALL=C` et sous la première locale UTF-8 disponible, exécute dans chaque cas, sous la locale de l'installation, les plans de référence R1 à R6 sur l'entrée de référence, écrit, relit et réécrit leur fiche et leur plan, écrit leur rapport figé et, pour R1, le bilan du rejeu outillé et cinq messages (deux refus du rejeu outillé et deux refus `objet_modifie` du rapport figé, dont trois citent des valeurs à caractères de contrôle ou non ASCII, et la citation d'une chaîne latin-1 ; issue #94), qui doivent être marqués UTF-8 et tenir sur une ligne, et dix-huit refus par module, six modules (`donnees`, `exposition`, `brut`, `simulation`, `rapport`, `construire_table`) citant chacun une valeur reçue non ASCII de codage natif non marqué, latin-1 et UTF-8, avec trois refus de la lecture du texte du plan (première ligne inconnue, type inconnu, valeur illisible) et deux refus `type_incompatible` sur un rôle date (colonne `character`, et classe formée d'une chaîne d'encodage `bytes` et de `Date` ; le libellé « entières » de `TYPES_ADMIS` y est exigé en octets UTF-8), soit vingt-trois messages, qui doivent en outre ne contenir aucun octet écrit « <xx> » et garder la valeur citée en octets UTF-8 (issue #127 ; section 2.1, « Règle de rédaction des messages »), puis compare octet à octet les textes obtenus sous les deux locales (tableau 7) ; le moindre écart le fait sortir avec un code non nul. Il est une étape bloquante du job « Tests (R 4.3.3) », distincte des trois batteries. **Statut** : l'identité des textes est un *constat* sur la machine qui exécute le contrôle (la plateforme de référence en CI) et sur ces six plans, non une propriété démontrée ; il ne couvre ni une locale latin-1 (contrôle suivant), ni Windows (issue #49), ni des données de l'utilisateur à chaînes non ASCII.

Locale latin-1 (issue #115). Sous une locale latin-1, `octets_chaines()` et l'ordre des segments (`segments_observations()`, section 8.2) suivent une branche propre : la session y est traitée comme une session UTF-8, toutes les chaînes passant par `enc2utf8()` ; depuis l'issue #113, la condition de cette conversion est écrite une seule fois, dans `vers_utf8_locale()`, tandis que la condition de locale connue du refus `chaine_codage_indetermine` d'`octets_chaines()` reste propre à ce refus. Aucune batterie ne s'exécute sous une telle locale, et `locale_c.R` ne compare que C et UTF-8 : sans contrôle dédié, une régression de cette branche, qui toucherait l'empreinte des données, l'ordre des segments, la fiche ou le rapport figé, passerait la CI. Le contrôle `tests/controles/locale_latin1.R` (tableau 7) génère la locale `fr_FR.ISO-8859-1` par `localedef` dans un dossier temporaire, sans droits particuliers, et **échoue**, sans jamais sauter, s'il ne peut pas la générer ; il installe l'arbre de travail sous cette locale et sous la première locale UTF-8 disponible, puis exécute dans chaque cas, sous la locale de l'installation et après avoir vérifié la locale effective, les plans de référence R1 à R6 et quatre jeux de segments non ASCII (valeurs natives non marquées, marquées latin-1, marquées UTF-8, et mêlées : « ÿ » natif et « € » UTF-8), construits à partir de leurs octets avec un marquage explicite ; il écrit la fiche, le plan et le rapport figé de chaque construction et, pour chaque jeu, l'ordre des segments. Les 34 textes (trois par plan de référence, quatre par jeu) sont comparés octet à octet entre les deux sessions, sans écart admis ; dans chaque session, l'ordre des segments est en outre comparé à un ordre de référence par octets UTF-8, calculé par le script hors du code testé. Il est l'étape « Locale latin-1 (installation et exécution) » du job « Tests (R 4.3.3) », qui installe le paquet Debian `locales` si les sources de locales manquent à l'image, hors de la liste des batteries. **Sensibilité**, *constat de mesure* (26/10/2026, sur deux copies de l'arbre de travail à `3df1b38`, chacune portant une seule mutation, contrôle lancé par `LC_ALL=C.UTF-8 Rscript tests/controles/locale_latin1.R`) : retirer la branche latin-1 de la condition de locale connue d'`octets_chaines()` fait échouer, dans la session latin-1, R1 à R6 et les quatre jeux de segments ; la retirer de celle de l'ordre des segments ne fait échouer que le jeu mêlé, seul à la détecter ; la session UTF-8 passe dans les deux cas, et le contrôle sort avec le code 1. Depuis

l'issue #113, la première mutation porte sur une condition inchangée (la condition de locale connue du refus `chaine_codage_indetermine`, propre à `octets_chaines()`) ; la seconde porterait désormais sur `vers_utf8_locale()`, que partagent la conversion de l'empreinte et l'ordre des segments, et toucherait donc les deux. Le constat, daté, n'a pas été refait sur le code actuel. **Statut** : l'identité des textes est un *constat* sur la machine qui exécute le contrôle et sur ces plans et ces jeux, non une propriété démontrée ; il ne couvre ni Windows (issue #49) ni la batterie unitaire exécutée sous une locale latin-1.

Batterie unitaire sous la locale C (issue #91). Le contrôle précédent ne rejoue que les six plans de référence : un défaut propre à un test unitaire sous la locale C (chaînes marquées UTF-8 mêlées à des chaînes non marquées, par exemple) lui échappe. L'étape « Locale C : tests unitaires » du job « Tests (R 4.3.3) », placée après l'étape « Locale C (installation et exécution) » et sous la même condition `${{ !cancelled() }}`, affiche d'abord la locale effective et l'indicateur UTF-8 de la session, puis relance `tests/test_unitaires.R` sous `env -u LANG LC_ALL=C` ; un échec de la batterie fait échouer l'étape. Comme pour toute batterie, `installer_tablexp()` installe alors `tablexp` sous la première locale UTF-8 disponible (ci-dessus) : l'étape couvre l'*exécution* des tests sous la locale C, non l'installation, que couvre `tests/controles/locale_c.R`. Son nom ne commence pas par « Batterie : » : elle n'est pas une quatrième batterie et reste hors de la liste des batteries de `CLAUDE.md` et du contrôle `tests/controles/identite_batteries.R` (décision M123). **Statut** : un succès est un *constat* sur la plateforme de référence, sous la locale C de son image ; il ne couvre ni une locale latin-1 ni Windows.

Une **garde de sortie** (`.Last()`) fait sortir avec le code 1 tout processus qui a chargé le harnais et se termine avant la ligne de bilan, même par `quit(status = 0)`. Le harnais est lui-même testé (`tests/unitaires/test_harnais.R`) : une assertion fausse donne un code de sortie non nul, et un échec attendu qui réussit fait échouer la batterie.

Chaque test nomme la nature de son attendu (ADR 0007, point 11.4 ; E2.20) : valeur calculée à la main (identité exacte, ou tolérance déclarée) ; *recoupement*, comparaison à la valeur que rend, dans la batterie, un appel direct d'une fonction d'un package de priorité `base`, dont la portée est limitée quand le moteur repose sur la même fonction : le test dit alors ce qu'il vérifie (la formule et son branchement, non la fonction de base) ; *contre-vérification*, valeur de référence produite hors CI par un package avancé et écrite en dur avec sa version et sa date ; *constat de simulation*, qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (`CONTEXT.md` ; E2.19).

4.3 Plateforme de référence

Les références de non-régression (section 4.5) sont produites et comparées sur une *plateforme de référence* unique (E4.7 ; ADR 0008) : le job de la CI s'exécute dans un **conteneur `rocker/r-ver:4.3.3` épinglé par empreinte**, dont R charge la **BLAS et la LAPACK de référence** (Netlib), en monothread. Identifiant fixé par l'annotation du 16/10/2026 de l'ADR 0008 (décision M47) :

> image `rocker/r-ver:4.3.3@sha256:`
> `732d15020af326da9e919c07f70ca32bf5d3e409220af32e0a4b6d0a89437309`
> suivie de la bascule `update-alternatives` vers les paquets `libblas3` et `liblapack3` 3.10.0-2ubuntu1 d'Ubuntu 22.04 ;
> `R version 4.3.3 (2024-02-29)` ;
> BLAS `/usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0` ;
> LAPACK `/usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0`, version 3.10.0 ;
> monothread (`OPENBLAS_NUM_THREADS` et `OMP_NUM_THREADS` valent 1).

Pourquoi un conteneur épinglé. Un libellé d'exécuteur (`ubuntu-24.04`) n'est pas une image figée : elle est mise à jour par GitHub, bibliothèques comprises, sans commit dans le dépôt. `outil_USP`, qui installait R sur l'image de l'exécuteur, a constaté une dérive entre deux exécutions de même R et de même image,

due à une BLAS optimisée (OpenBLAS) qui choisit ses noyaux selon le processeur (constat d'`outil_USP` rapporté par l'ADR 0008, « Contexte »). Une étiquette de conteneur peut elle aussi désigner une autre image après une reconstruction (hypothèse de l'ADR 0008, non vérifiée) ; l'empreinte (`@sha256:...`) désigne un contenu immuable. Le conteneur est une fonctionnalité native de GitHub Actions, et non une action tierce, ce que la politique des Actions du dépôt exige.

Pourquoi la bascule. L'image charge par défaut OpenBLAS (relevé de la première exécution, rapporté par l'annotation du 16/10/2026 de l'ADR 0008). Le mainteneur a retenu de basculer dans le job vers la Netlib déjà présente dans l'image, plutôt que de compiler R depuis la source (décision M47 ; le repli par compilation reste prévu par l'ADR 0008, point 10). L'étape « BLAS et LAPACK de référence » de `.github/workflows/ci.yml` pointe par `update-alternatives` les alternatives `libblas.so.3-x86_64-linux-gnu` et `liblapack.so.3-x86_64-linux-gnu` vers `libblas3` et `liblapack3`, puis affiche les cibles résolues et les versions des paquets. La plateforme de référence est l'image épinglée *suivie de cette bascule* : modifier l'étape, l'image ou les bibliothèques est un changement de plateforme, décidé par le mainteneur et, une fois des références existantes, accompagné d'un tableau avant / après de toutes les références (ADR 0008, point 9).

Script de plateforme. `tests/controles/plateforme.R` s'exécute en première étape de chaque job R qui calcule, avant tout calcul (ADR 0008, points 3 à 5) ; le job « Absence de Java », qui n'effectue aucun calcul numérique et ne produit aucun résultat, en est exempté, ainsi que des variables monothread (ADR 0008, annotation du 17/10/2026). En mode strict (`--exiger`, sur la CI), il **échoue** si : `R.version.string` n'est pas exactement `R_ATTENDU` (`R version 4.3.3 (2024-02-29)`) ; l'une des quatre lectures de la BLAS et de la LAPACK (`extSoftVersion()["BLAS"]`, `sessionInfo()$BLAS`, `sessionInfo()$LAPACK`, `La_library()`) est vide, désigne une BLAS optimisée (motif `MOTIF_OPTIMISE` : `openblas`, `mkl`, `atlas`, `blis`, `flexiblas`) ou sort des chemins de référence (`MOTIF_BLAS`, `MOTIF_LAPACK`) ; `La_version()` n'est pas `LA_VERSION_ATTENDUE` (3.10.0) ; `OPENBLAS_NUM_THREADS` ou `OMP_NUM_THREADS` ne vaut pas 1 ; `TABLEXP_IMAGE`, que le job renseigne avec la même chaîne que l'image du conteneur, n'est pas épinglée par empreinte (`MOTIF_IMAGE`). Sans `--exiger` (poste, session), il affiche les mêmes grandeurs et les écarts sans échouer : les batteries restent exécutables sur le poste Windows du mainteneur, dont la BLAS est celle de R. Dans les deux modes, il écrit l'*identifiant de plateforme* (image, `R.version.string`, BLAS, LAPACK et sa version, `sessionInfo()`) et consigne sans faire échouer le processeur et ses drapeaux (`fma`, `avx2`, `avx512f`), la version de la glibc, dont vient la `libm` (sortie de `getconf GNU_LIBC_VERSION`, à défaut première ligne de `ldd --version`, « inconnue » si aucune de ces sorties ne désigne la glibc ; issue #72), `ImageOS` et `ImageVersion` de l'hôte, `LC_COLLATE`, le fuseau horaire, la présence d'un exécutable `java` et `GITHUB_SHA`. Ces grandeurs consignées n'entrent pas dans l'identifiant de plateforme : elles ne déclenchent aucune bascule. Ce bloc est reporté dans le message de chaque commit de référence et dans l'en-tête de chaque fichier de référence (section 4.5), sous la clé « plateforme consignée » pour les grandeurs consignées ; la version de la glibc y figure depuis la régénération du 24/10/2026 (ligne « glibc (libm) »), à côté de `fiche$plateforme$bibliotheque_c`, consignée par la fiche elle-même (section 15.1).

> **Ce que la plateforme ne garantit pas.** Le modèle de processeur varie d'une exécution à l'autre sous le même libellé d'exécuteur (relevé de l'ADR 0008, annotation du 16/10/2026). La BLAS Netlib n'a pas de noyau choisi selon le processeur ; la réserve sur la bibliothèque mathématique du système (section 3.1) reste en revanche ouverte pour un exécuteur sans FMA : elle n'a pas été observée sur les trois modèles rencontrés, tous munis de FMA et d'AVX2 (ADR 0008, annotation du 19/10/2026). Les références sont donc comparées à tolérance, structure contrôlée à part (E4.7 ; section 4.5).

4.4 Jobs de la CI

`.github/workflows/ci.yml` définit les jobs suivants, exécutés sur chaque pull request, sur chaque push vers `main` et sur demande. Aucun job n'installe de package du CRAN ni ne configure de dépôt CRAN pour une installation : `tablexp` est installé depuis l'arbre de travail par chaque batterie.

Table 4 : Jobs de la CI (`.github/workflows/ci.yml`).

Job	Exécution	Contenu
« Contrôles du dépôt »	hôte `ubuntu-24.04`, sans R	réglages de Claude Code valides (JSON) ; fiches d'agents complètes ; fiches `-approfondi` à jour ; aucun fichier sensible versionné
« Tests (R 4.3.3) »	conteneur de la plateforme de référence	bascule vers la BLAS et la LAPACK de référence ; script de plateforme en mode strict ; les trois batteries ; liste blanche de `DESCRIPTION` ; ordre des modules ; identité des batteries et de l'image ; contrôle sous la locale C (installation et exécution), puis batterie unitaire relancée sous la locale C stricte (section 4.2) ; contrôle sous une locale latin-1 (installation et exécution) ; écriture « texte » des doubles recoupée par les juges glibc et Python (tableau 7), hors liste des batteries. Les étapes des batteries et des contrôles, sous la condition `${{ !cancelled() }}`, s'exécutent même si une étape antérieure échoue, pour que tous les écarts soient visibles ; le job reste en échec. L'étape du script de plateforme, sans condition, est sautée si la bascule vers la BLAS de référence échoue
« Absence de Java »	même conteneur, réseau requis	contrôle de l'absence de Java (section 5.2) et son cas témoin
« Documentation »	hôte `ubuntu-24.04`, sans R	installation des paquets TeX par `apt-get` (versions affichées), compilation du présent document selon `docs/latex/CONVENTIONS.md` (section 6 : passes jusqu'à stabilisation, cinq au plus ; échec sur une erreur ou un renvoi indéfini), PDF et journal publiés en artefact

Ces quatre jobs sont les contrôles requis que liste le ruleset de `main` (`.github/ruleset-main.json`, appliqué au dépôt par le mainteneur ; ADR 0008, point 6) : une pull request ne peut alors être fusionnée que s'ils sont verts. Le job « Documentation » ne produit aucun résultat ; le PDF versionné reste celui que commite la session principale, et la version des paquets TeX suit l'image de l'exécuteur. Les batteries et les scripts des jobs conteneurisés ne dépendent pas de git, absent de l'image ; un contrôle qui lit l'historique reste dans « Contrôles du dépôt », sur l'hôte (ADR 0008, point 11).

4.5 Références de non-régression

Une *référence de non-régression* fige, sur la plateforme de référence, ce que le moteur rend pour un *plan de référence* appliqué à l'*entrée de référence* ; toute modification ultérieure qui change une grandeur est détectée par la batterie de reproductibilité et appelle un tableau avant / après et un visa (E4.7 ; ADR 0009, accepté le 17/10/2026, décision M52, et ses annotations). L'outillage, en R de base, est sous `tests/reference/` : `outils_reference.R` (aplatissement, écriture, lecture, comparaison, tolérances, plans et entrée), `produire_references.R` (script du workflow de production) et `comparer_references.R` (comparaison de deux fichiers en ligne de commande).

Ce qui est figé

— **L'objet `construction` entier**, aplati, **moins** `execution` (jamais écrit), `fiche$versions` et `fiche$plateforme`, écrits dans l'en-tête du fichier et jamais comparés : une montée de version de `tablexp` n'est pas un changement de résultat. `fiche$version_fiche`, le plan et les rôles sont comparés : le plan de référence est lui-même une grandeur.

— **Les empreintes du résultat et du code** (`fiche/empreintes/resultat` et `fiche/empreintes/code`), **comparées par leur forme** (ADR 0009, seconde annotation du 20/10/2026 ; décision M69, point 4 ; issue #67). La première dépend de la plateforme (section 15.3) et la seconde change à chaque commit qui modifie `R/` : comparées en valeur, la première ferait échouer la comparaison sur tout autre exécuteur, et la seconde demanderait une régénération à chaque commit `moteur:`. L'aplatissement (`tests/reference/outils_reference.R`) remplace donc la valeur de ces deux chemins, et d'eux seuls (`CHEMINS_JETON_MD5`), par le jeton `"<md5>"` (`JETON_MD5`), **si et seulement si** c'est une chaîne de longueur 1 de la forme `^[0-9a-f]{32}$` ; toute autre valeur (`NA`, majuscules, autre longueur, autre type) est gardée telle quelle et relevée en écart contre le jeton de la référence, si bien qu'une empreinte redevenue `NA` est détectée. Le chemin, le type et la longueur restent comparés. La substitution se fait dans l'aplatissement, commun au producteur et à la batterie, et jamais par une règle générique sur la forme hexadécimale, qui masquerait deux grandeurs exactes et portables : `fiche/empreintes/donnees`, comparée en premier, et les lignes `@empreinte` des tables résumées. `statut_resultat_code`, `representation_resultat` et les autres champs de `fiche/empreintes` restent comparés exactement. Les valeurs réelles sont écrites dans l'en-tête, hors comparaison (ci-dessous). Le jeton ne vérifie que la forme : les propriétés des deux empreintes sont testées par les batteries (section 15.6). Options écartées : retirer les deux chemins de l'aplatissement, ce qui perdrait le contrôle de structure et la détection d'un `NA` ; les comparer en valeur. Le format reste `tablexp reference 1` : le jeton est une chaîne valide, sans grammaire nouvelle.

— **Les tables de lignes de l'étape `donnees`** (`lignes_conservees`, `numeros_conserves`, `lignes_exclues`, `observations`, `observation_des_lignes` ; constante `RESUMES_PAR_EMPREINTE`) sont **résumées** par leur nombre de lignes et l'empreinte MD5 de leur représentation canonique (colonnes écrites par `ecrire_valeurs()` en forme « texte », section 15.2 ; sans colonne double, ces tables s'écrivent de même dans les deux formes, si bien que le passage à la représentation canonique 2 a laissé leurs lignes `@empreinte` inchangées), non recopiées : elles ne contiennent que des entiers, des dates et des chaînes, sans arithmétique flottante, si bien que le résumé exact ne perd aucune détection, et la table « avant » se reconstitue en rejouant le plan sur le commit de référence. Les attributs autres que les noms, la classe et les noms de lignes n'y entrent pas.

— **Première grandeur** : l'empreinte des données (`fiche/empreintes/donnees`), qui identifie l'entrée.

Entrée de référence (décision M50, Q6 ; ADR 0009, point 4.1) : un jeu tiré par `simuler_portefeuille()` sous la loi vraie des tests (Gompertz-Makeham $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, facteurs H 1 et F 0,6 ; rachat constant 0,05), portefeuille uniforme de 10^5 contrats, proportion d'hommes 0,5, âges 20 à 75, origine 2005-01-01, période 2015-01-01 au 2024-12-31, graine 20261016 (constante `GRAINE_REFERENCE`), rôles `id_contrat`, `id_tete`, `sexe`, `date_naissance`, `date_entree`, `date_sortie`, `cause_sortie`. Le simulateur est ainsi figé pour cette graine : une modification qui change le jeu tiré change l'empreinte et met toutes les références en « entrée différente ».

Table 5 : Plans de référence (`plans_reference` de `tests/reference/outils_reference.R` ; tous sur la période de l'entrée, sans exclusion ; un identifiant n'est jamais réemployé).

Id	Description	Segm.	Délai	Convention	Par année	Estimateurs, niveau
R1	vivant → décédé, référence principale	`sexe`	0	`plus_fine`	non	Poisson, binomial ; 0,95
R2	sans segmentation, délai, niveau 0,99	aucune	90	`plus_fine`	non	Poisson, binomial ; 0,99
R3	convention forcée	`sexe`	0	`age_atteint_annee`	non	Poisson, binomial ; 0,95
R4	par année (surface)	`sexe`	0	`plus_fine`	oui	Poisson ; 0,95
R5	en cours → racheté, unité contrat	`sexe`	0	`plus_fine`	non	Poisson, binomial ; 0,95
R6	R1 avec l'estimateur produit-limite	`sexe`	0	`plus_fine`	non	Poisson, binomial, `kaplan_meier` ; 0,95

Format des fichiers

Un fichier texte par plan, `tests/reference/<id>.tsv`, en UTF-8, lignes terminées par LF, écrit octet par octet :

— un **en-tête** de lignes `# <clé> : <valeur>`, hors comparaison : version du format (`tablexp reference 1`), identifiant et description du plan, identifiant de plateforme écrit par `tests/controles/plateforme.R` (grandeurs contrôlées et grandeurs consignées), `fiche$versions` et `fiche$plateforme`, valeurs réelles des empreintes du résultat et du code (clés `fiche$empreintes/resultat` et `fiche$empreintes/code`, qui disent de quel résultat et de quel code la référence a été produite sur la plateforme de référence), commit calculé, exécution et adresse, mode, date UTC, empreinte des données en rappel ;

— un **corps** d'une ligne par vecteur atomique, `chemin<TAB>type<TAB>longueur<TAB>valeurs`, dans l'ordre de parcours de l'objet ; le chemin joint les noms par `/` (`etapes/brut/resultat/poisson/taux`) ;

— des **lignes de structure** : `@noms` de chaque liste et de chaque vecteur atomique nommé, `@colonnes` et `@lignes` de chaque `data.frame`, `@classe` de l'objet, `@lignes` et `@empreinte` d'une table résumée ;

— des **types** `double`, `integer`, `logical`, `character`, `Date`, `factor`, `NULL`, aux valeurs écrites par l'écrivain de la représentation canonique, `ecrire_valeurs()`, en forme « texte » (section 15.2) : doubles en `%.17g` (et non en octets, forme réservée à l'empreinte des données), dates sur quatre chiffres d'année, jeton `NaN` distinct de `NA`, relues par l'outillage, sans installer `tablexp`, à la condition que la réécriture de la valeur lue redonne le texte (issue #19), chaînes échappées entre guillemets, sous la même règle de codage, par **refus** et jamais par conversion ;

— des **refus d'écriture**, avec le chemin fautif, pour toute autre classe (`POSIXct`, `difftime`…), tout attribut autre que les noms, la classe et les noms de lignes, et des noms de lignes non automatiques : rien n'est converti ni ignoré en silence. Les niveaux d'un facteur ne sont pas écrits (limite sans portée : l'objet n'en contient aucun ; en ajouter demanderait une version 2 du format).

Lecture : en R de base, sans évaluation de code. La lecture des doubles dépend de la plateforme. *Hors Windows*, plateforme de référence comprise, le texte lu doit être égal à l'écriture `%.17g` de la valeur lue : la bibliothèque C du système arrondissant correctement, c'est une **preuve d'aller-retour**. *Sous Windows*, le texte doit seulement avoir la forme de `%.17g` : l'écriture du système n'y arrondit pas toujours correctement le dix-septième chiffre, et la lecture n'y rend pas toujours le double désigné (constats sur le

poste du mainteneur, ADR 0009, annotation du 18/10/2026) ; une comparaison exécutée sous Windows est donc un **constat de mesure**, non une preuve, et une référence n'est jamais produite hors de la CI.

Comparaison et tolérances

1. **Entrée, d'abord** : l'empreinte des données, exigée des deux côtés, est comparée en premier ; si elle diffère, le plan est en échec « entrée différente », et la comparaison se poursuit et s'imprime sous cette mention, ses écarts n'étant pas interprétables comme un tableau avant / après. L'impression nomme les causes possibles : le simulateur a changé pour cette graine (ADR 0009, point 4.2), ou la plateforme a produit un tirage différent (ADR 0008, hypothèse sur la bibliothèque mathématique du système) ; et, quand la représentation canonique (`fiche/empreintes/representation`) diffère entre la référence et l'objet, une troisième : la définition de la représentation a changé, l'entrée pouvant alors être inchangée, ce que seule prouve la garde « entrée différente » des transitions (« Règles durables », ci-dessous ; ADR 0009, troisième annotation du 19/10/2026, point (f)).
2. **Structure, exacte** : même ensemble de chemins, mêmes types et longueurs, mêmes positions des `NA` ; tout écart (chemin manquant, en trop, type, longueur) est structurel et fait échouer le plan.
3. **Valeurs, élément par élément** : deux éléments sont égaux s'ils sont identiques (`Inf`, 0, `NA` et `NaN` compris, sans confondre `NA` et `NaN` ; ± 0 confondus par la représentation canonique, qui écrit $x + 0$, soit `0` pour `-0`, et par l'égalité == de la comparaison, ce qui est voulu : aucune grandeur du moteur ne porte de sens par le signe d'un zéro ; ADR 0009, complément du 18/10/2026, point (e)) ; sinon, deux valeurs finies, non nulles, de même signe, sont égales à la tolérance τ de la grandeur si $|a-b| \leq \tau \max(|a|, |b|)$; tout autre cas est un écart. Cette règle, symétrique et relative, est propre aux références ; `proche()` reste celle des tests unitaires.

Les grandeurs de type entier, logique, chaîne, date et les lignes de structure sont exactes par leur type, sans clé de tolérance : c'est le cas du nombre des avertissements, stocké en `integer`, si bien qu'un retour au type `double` serait relevé deux fois, comme écart structurel de type et comme double sans classe (ADR 0009, quatrième annotation du 17/10/2026) ; c'est aussi le cas de la version de la fiche, de la graine, des colonnes entières de `kaplan_meier_sauts` (`numerateur`, `ensemble_a_risque`), de `kaplan_meier/instants_evenement`, de `denominateur_instants` (paramètres et fiche), des cinq diagnostics de `kaplan_meier` et, depuis l'issue #37, du nombre de cellules de l'étape `exposition` (`etapes/exposition/diagnostics/cellules`), toutes stockées en `integer`. Les treize clés exactes qui les nommaient dans `TOLERANCES` sont retirées : inertes, la tolérance n'étant cherchée que pour un double, elles auraient en outre masqué un retour au type `double` au contrôle de complétude des classes. Douze l'ont été sur avis d'`actuary` (ADR 0009, complément du 18/10/2026, point (f)) ; la treizième, la clé nue `cellules`, l'a été à son tour, sur avis d'`actuary`, parce qu'elle classait aussi `etapes/brut/diagnostics/cellules` et `etapes/brut/diagnostics/etats/cellules`, déjà `integer`, et aurait masqué leur retour au double (ADR 0009, seconde annotation du 19/10/2026, point (a)). Un double reçoit sa tolérance de la table `TOLERANCES` (tableau 6), cherchée par le **suffixe le plus long** du chemin : une clé qualifiée `<table>/<colonne>` l'emporte sur le nom nu, ce qui donne au `taux` de `kaplan_meier` une autre classe qu'au `taux` de Poisson ou du binomial sans changer le nom commun de la colonne. **Un double sans classe déclarée fait échouer la comparaison** : une grandeur nouvelle reçoit sa classe d'`actuary` avant d'entrer dans une référence, et toute modification de la table est annotée dans l'ADR 0009, datée, avec l'avis d'`actuary`.

Table 6 : Classes de tolérance des doubles (`TOLERANCES`, `tests/reference/outils_reference.R` ; avis d'`actuary`, ADR 0009, point 3.4, seconde annotation, complément du 18/10/2026, point (f), et seconde annotation du 19/10/2026, point (a)).

Classe	Grandeurs	Justification
exacte	`exposition_jours`, `niveau`, `delai_declaration`, `tolerance_effectif`	doubles à valeur entière ou déclarée : aucun bruit possible
10^{-14}	`exposition_centrale`, `complement_initial_cellule`, `complement_initial_fenetre`, `exposition_initiale`, `taux` ; `kaplan_meier_sauts/age_exact`, `kaplan_meier_sauts/variance_log_survie_cellule`	arithmétique IEEE de base, ordre des opérations fixé par le code
10^{-12}	les bornes exactes et asymptotiques, `probabilite` et ses quatre bornes ; `kaplan_meier/taux`, `kaplan_meier/variance`, `kaplan_meier_sauts/survie_cellule`	fonctions spéciales (`qgamma`, `qbeta`) et fonctions mathématiques de la bibliothèque du système (`exp`, `log`, `log1p`, `expm1`), dont la variante peut dépendre du processeur (sans écart mesuré sur trois modèles munis de FMA, non mesurée sans FMA ; ADR 0009, annotation du 19/10/2026, conséquence 3)

Les tolérances restent plusieurs ordres de grandeur sous le plus petit changement significatif estimé par `actuary` (un jour d'exposition dans la cellule la plus exposée, un événement de plus ; ADR 0009, point 3.4). **Impression**, pour chaque plan, dans la batterie comme dans le workflow : une ligne de bilan (grandeurs comparées, identiques, à écart sous tolérance, en écart, écarts structurels) ; l'écart relatif maximal observé par classe, toujours imprimé, même nul ; une ligne par grandeur dont l'écart n'est pas nul, avec sa position et les valeurs avant et après en `%.17g`. Ces lignes sont la matière du tableau avant / après ; `comparer_references.R` produit la même impression à partir de deux fichiers.

Batterie, production et commit

Dans la batterie (`tests/test_reproductibilite.R`, job « Tests (R 4.3.3) », donc sur la plateforme de référence) : le jeu est tiré une fois, chaque plan de référence est exécuté, aplati, écrit puis relu (aller-retour `identical()`), et comparé à son fichier ; un plan sans fichier est un échec attendu renvoyant à l'issue de création tant qu'il figure dans la liste `SANS_REFERENCE_17`, vide depuis la création des six fichiers ; hors de cette liste, un fichier manquant est un échec, de même qu'un fichier sans plan (référence orpheline). Des témoins, sur des copies temporaires et jamais sur `tests/reference/`, vérifient que la comparaison échoue quand elle le doit (valeur altérée au-delà de sa tolérance, grandeur retirée ou ajoutée, type ou longueur changés, empreinte altérée ou absente, double sans classe, classe ou attribut interdits), qu'une valeur altérée juste en deçà de sa tolérance passe, et que les chaînes non ASCII font l'aller-retour au bit près sous les locales C et UTF-8, l'aplatissement de R1 donnant les mêmes octets sous les deux. Pour le jeton (ADR 0009, seconde annotation du 20/10/2026, (i)), des témoins sur des copies de R6 vérifient que l'aplatissement porte le jeton aux deux seuls chemins et que l'aller-retour reste `identical()` ; qu'une empreinte du résultat ou du code `NA`, en majuscules ou de longueur 2 est gardée et relevée en écart ; qu'une empreinte des données altérée n'est pas remplacée et reste « entrée différente » ; qu'une ligne `@empreinte` altérée reste un écart ; que l'en-tête porte les deux valeurs réelles, et qu'en changer la valeur dans l'en-tête ne change pas la comparaison.

Production, par le workflow `.github/workflows/references.yml`, distinct de `ci.yml` : déclenché à la main seulement (`workflow_dispatch`), sur ordre du mainteneur, droits en lecture seule, sans secret,

dans le même conteneur épinglé que le job « Tests (R 4.3.3) », avec la même étape de bascule vers la BLAS et la LAPACK de référence et le script de plateforme en mode strict avant toute production (invariants contrôlés par `tests/controles/identite_batteries.R`, contrôle 4). Le script `produire_references.R` produit et compare toujours tous les plans ; le **mode** fixe ce qui est admis : `creation` (aucun fichier existant pour les plans visés, comparaison réussie pour les autres), `regeneration` (écarts admis sur les seuls plans visés), `bascule` (écarts admis sur tous, l'identifiant de plateforme de l'exécution devant différer de celui des en-têtes commités) ; un mode `releve`, refusé sur la CI, sert aux essais sur un poste, dont les fichiers ne sont jamais commités. Symétriquement, les modes `creation`, `regeneration` et `bascule` sont **refusés hors de la CI**, c'est-à-dire quand la variable d'environnement `GITHUB_ACTIONS` ne vaut pas `true` : le refus intervient avant le contrôle des autres options et avant toute écriture, si bien qu'aucun dossier de sortie n'est créé (ADR 0009, seconde annotation du 19/10/2026, point (d)). Cette garde protège contre l'erreur, **pas contre la malveillance** : la variable se pose à la main ; la garantie de fond reste l'appariement des MD5 des fichiers commités à ceux de l'artefact de l'exécution citée, vérifié par la session principale au commit puis en revue finale (ADR 0009, point 5.5). Le dossier de sortie ne peut être `tests/reference/` ni s'y trouver, quelle que soit l'écriture du chemin (séparateurs, casse, lecteur, chemins réseau, liens, noms courts). Le job ne commite rien : il publie un artefact (références, leurs MD5, identifiant de plateforme, exécution, écarts), conservé 90 jours. La session principale seule, après visa (`actuary` pour les taux bruts ; mainteneur dès qu'une table finale ou un verdict change), vérifie les MD5, copie les fichiers sans modification et les commite avec l'identifiant de plateforme, le tableau avant / après et le visa.

Références existantes : R1 à R6, créées le 18/10/2026 en mode `creation` (exécution 39140635363 du workflow, commit `1019d8e`, visa d'`actuary`). Les grandeurs des étapes `donnees` et `exposition` sont rattachées à l'état de la première branche (`f9ab269`) par une chaîne de mesures (différences de code vides, ou objets `identical()` hors `execution`, un changement de type sans changement de valeur, visé, étant admis ; ADR 0009, annotation du 18/10/2026). Les références R1 à R6 ont été **régénérées une fois**, le 19/10/2026, en mode `regeneration` sur tous les plans (exécution 39248539956 du workflow, calcul sur `99272a4`, commit `763da20`, visa d'`actuary`), pour le seul changement de type du nombre de cellules de l'étape `exposition`, passé de `double` à `integer` à valeur inchangée (issue #37) : le corps de chaque fichier ne change que de cette ligne, et toutes les autres grandeurs sont restées identiques au bit près (ADR 0009, seconde annotation du 19/10/2026, point (a)). Elles l'ont été **une seconde fois**, le 19/10/2026, en mode `regeneration` sur tous les plans (exécution 39315172327 du workflow, calcul sur `d18c269`, commit `50837f6`, visa d'`actuary`, sur ordre du mainteneur), pour les seuls changements de la fiche annoncés en transition par les issues #35 (représentation canonique et, par son en-tête, empreinte des données, l'entrée étant prouvée inchangée par la garde « entrée différente ») et #42 (version de la fiche, champs retirés) : toutes les autres grandeurs sont restées identiques au bit près. Elles l'ont été **une troisième fois**, le 20/10/2026, en mode `regeneration` sur tous les plans (exécution 39435673257 du workflow, calcul sur `5757c30`, commit `192180c`, visa d'`actuary`, sur ordre du mainteneur), régénération unique de la branche `claude/fiche-rapport`, pour les seuls changements de la fiche annoncés en transition par les issues #57 (`version_fiche` de 2 à 3, champ `nombre_lignes_exclues`), #64 (ordre des champs) et #67 (champ `representation_resultat`, empreintes du résultat et du code comparées par jeton, statut `statut_resultat_code`) : toutes les autres grandeurs sont restées identiques au bit près, et l'empreinte des données est inchangée. Elles l'ont été **une quatrième fois**, le 24/10/2026, en mode `regeneration` sur tous les plans (exécution 40111528681 du workflow, calcul sur `a96cf7f`, commit `26f7a29`, visa d'`actuary`, sur ordre du mainteneur), régénération unique de la branche `claude/rapport-fige`, pour le seul passage de `version_fiche` de 3 à 4 (issue #77, champ `fiche$plateforme$bibliotheque_c`, section 15.1) : `fiche/version_fiche` est la seule grandeur en écart ; l'empreinte du résultat, comparée par jeton, change en conséquence, ainsi que l'empreinte du code, `R/` ayant changé (en-tête `0be71b0b...` devenu `2cc03eab...`), et toutes les autres grandeurs sont restées identiques au bit près ; l'en-tête gagne

la ligne `fiche$plateforme/bibliotheque_c`, hors comparaison.

Ce que les références ne couvrent pas.

— La couverture est celle des plans : une recette ou une convention absente des plans n'est protégée qu'à partir du plan qui l'introduira. En particulier, l'estimateur `kaplan_meier` n'est figé que sous `dernier_anniversaire` (R6) ; sa branche `age_atteint_annee` n'a pas de référence (avis d'`actuary` au visa).

— L'entrée n'exerce pas la fusion des contrats par tête : elle est tirée en mode `uniforme`, une tête par contrat (section 16). La fusion n'est couverte que par les tests unitaires (section 7.6, et, sur des jeux à têtes multi-contrats, section 16.4).

— R6 comprend une cellule définie à $\hat{q}_x = 1$ avec un seul exposé à risque : la référence fige ce cas d'instabilité de l'estimateur produit-limite sous troncature (section 9.4), qui n'est pas une anomalie.

— Absence de bruit numérique sur la plateforme de référence : *constat* sur cinq exécutions de la CI sur la plateforme de référence, sur trois modèles de processeur (AMD EPYC 7763, 9V74 et 9V45, tous avec FMA et AVX2), où toutes les grandeurs des six plans sont identiques au bit près (ADR 0008, annotation du 19/10/2026) ; la réserve de l'ADR 0008 sur les variantes de la bibliothèque mathématique du système reste ouverte pour un exécuteur sans FMA, non observé.

— Le signe d'un zéro : la représentation canonique et la comparaison confondent 0 et `-0`, si bien qu'un passage de l'un à l'autre n'est pas détecté (limite admise ; aucune grandeur actuelle n'en dépend ; ADR 0009, complément du 18/10/2026, point (e)).

Règles durables (ADR 0009, point 9). Ajouter une grandeur à l'objet résultat est visible par la non-régression (écart structurel « en trop ») : les plans concernés sont régénérés, avec un tableau avant / après où les lignes nouvelles sont marquées « création », et la classe de tolérance d'un double nouveau est déclarée dans le même commit. Changer le format incrémente sa version et régénère tous les plans. Élargir une tolérance pour faire passer un écart est interdit sans tableau avant / après et visa. Un fichier de `tests/reference/` ne change que par un commit de la session principale qui cite une exécution du workflow et un visa. **Transition d'un changement de résultat annoncé** (ADR 0009, point 9.5, règle retenue par le mainteneur le 19/10/2026 ; mise en œuvre générique par l'issue #47, ADR 0009, troisième et quatrième annotations du 19/10/2026, cette dernière actant l'extension de la garde « entrée différente », décision M64) : quand un commit change un résultat avant la régénération des références, il déclare sa *transition* dans la liste unique `TRANSITIONS_EN_COURS` de `tests/test_reproductibilite.R`, vide hors transition. Une transition porte trois champs, et eux seuls : l'issue (de la forme `#N`), les plans de référence concernés et les changements admis (au moins un). Une issue peut porter plusieurs transitions, à condition que leurs plans soient deux à deux disjoints : un même retrait n'a pas toujours la même structure sur tous les plans (avant l'issue #42, `fiche/segmentation` était de longueur 0 dans R2 et de longueur 1 ailleurs), alors qu'une transition en déclare une seule. Chaque changement nomme un chemin et une **nature**, prise dans une liste fermée de cinq et déclarée exactement :

— **type** : conversion sans perte de `double` vers `integer`, ou l'inverse ; la référence est du type de départ déclaré ;

— **valeur** : valeurs numériques nouvelles d'une grandeur `double` ou `integer`, avec ou sans positions. Avec positions, la longueur est conservée et chaque position déclarée change ; sans positions, le vecteur déclaré remplace le vecteur entier. C'est ainsi, et seulement ainsi, que se déclare un changement de longueur : une nature « longueur » admettrait des valeurs quelconques. La longueur nouvelle est au moins 1 (valeurs non vides exigées) : le passage d'un vecteur à la longueur 0 n'est pas déclarable, et il est exclu jusqu'à décision contraire ;

— **valeur de chaîne** : de même pour une grandeur `character` (version, empreinte) ;

— **ajout** : un chemin absent de la référence, présent dans l'objet avec le type et la longueur déclarés ; ses valeurs, sans « avant », entrent dans le tableau avant / après de la régénération, sous visa ;

— **retrait** : un chemin présent dans la référence seule, avec le type et la longueur déclarés.

Un test ordinaire refuse une liste mal formée (champs, forme de l'issue, plans, natures et leurs champs, valeurs, positions). Pour chaque plan listé, les changements de toutes les transitions qui le portent **s'unissent** : deux déclarations identiques s'appliquent une fois ; deux valeurs différentes à une même position, un vecteur entier et une autre déclaration de valeurs sur le même chemin, deux conversions de type, un ajout ou un retrait combiné à une autre nature sur le même chemin sont des erreurs. Puis :

1. un **test ordinaire « seul écart »** aligne la référence commitée sur les changements déclarés, dans l'ordre type, valeur, valeur de chaîne, ajout, retrait, et exige que la comparaison ne relève alors aucun autre écart, aux tolérances près ; il exige aussi la réciproque : **tout changement déclaré est observé** sur la référence commitée telle qu'elle est lue (type égal au type de départ, valeurs différentes, chemin ajouté absent, chemin retiré présent), l'observation des valeurs se jugeant après la seule conversion de type, qui ne change aucune valeur ;
2. la **comparaison stricte** est un échec attendu, un seul par plan, rattaché à l'issue de la première transition de la liste qui porte le plan : une comparaison est un seul test, la régénération porte sur un plan entier et jamais sur une issue seule, et le harnais attend une issue par échec attendu. Le test des doubles sans tolérance déclarée n'est un échec attendu que si la référence commitée en porte ; un test ordinaire exige alors que ces doubles soient tous admis par une transition (type converti depuis `double`, ou retrait déclaré de type `double`) ;
3. **garde « entrée différente »** (point 3.2 de l'ADR 0009, étendu) : un changement de l'empreinte des données (`fiche/empreintes/donnees`) n'est admis qu'en valeur de chaîne sans positions, avec un changement de `fiche/empreintes/representation` dans l'union du plan, car seule une définition nouvelle de la représentation canonique change l'empreinte d'une entrée inchangée (section 15.2) ; et un test ordinaire de garde doit prouver que l'entrée est inchangée. Il calcule par le package le texte canonique de l'entrée de référence (données et rôles déclarés), vérifie que l'en-tête de sa première ligne (texte avant la première tabulation) est la représentation de l'objet, le remplace par la représentation de la référence, et exige que le MD5 obtenu soit égal à l'empreinte des données de la référence ; il exige aussi que les deux représentations diffèrent, et que l'objet ait été construit sur l'entrée qu'on lui passe : le MD5 du texte canonique de cette entrée, sous la représentation courante, doit être égal à l'empreinte des données de l'objet (réserve M2 de l'audit de l'issue #47, levée par l'issue #35 ; ADR 0009, troisième annotation du 19/10/2026, point (f) 4, et quatrième, point (a)). Lors de la transition de l'issue #35, la garde a prouvé l'entrée de référence inchangée : le texte de la représentation 2, sous l'en-tête de la représentation 1, redonne l'empreinte de la référence commitée, preuve refaite hors de R dans le commit de l'issue (ADR 0011, point 2.5). Sans cette garde, la transition absorberait le contrôle de l'entrée : l'entrée de référence aux lignes permutées, réellement différente, ne s'écarte des références de R1 et de R6 que par l'empreinte des données et celles des deux tables résumées, les autres écarts restant sous tolérance, si bien que déclarer ces trois chaînes suffit à faire passer le test « seul écart ». C'est un constat sur une permutation par plan : pour R1, la permutation tirée par `set.seed(1)` et `sample.int()` (ADR 0009, troisième annotation du 19/10/2026, point (f) 1) ; pour R6, la permutation tirée sous la graine 20261019 (`GRAINE_PERMUTATION`), dont un témoin de la batterie rejoue le constat et exige l'échec de la garde (section 15.2). Si la preuve échoue, aucune transition n'est possible : c'est un changement de l'entrée de référence (ADR 0009, point 4.2), qui régénère tous les plans, sous visa. **Limite de la garde** (ADR 0009, annotation du 20/10/2026) : elle ne remplace que l'en-tête de la première ligne et suppose donc qu'une représentation nouvelle ne change rien d'autre dans le texte de l'entrée de référence. C'était vrai pour l'issue #35, l'entrée n'ayant aucune colonne double, et pour l'issue #19, ses dates étant entre 1930 et 2024. Une représentation qui changerait l'écriture d'un type présent dans l'entrée (entier, chaîne, date) ou la forme de la première ligne ferait échouer la garde *à tort* : l'erreur est conservatrice, la garde ne pouvant conclure à tort à une entrée

inchangée (hors collision MD5), mais son diagnostic « l'entrée a changé » serait faux. La branche qui fera un tel changement choisira d'abord, par annotation de l'ADR 0009, sa voie : de préférence un écrivain figé de la représentation précédente, dans les tests, pendant la transition.

Le commit des références de la session principale retire la transition : un plan resté listé après la régénération fait échouer la batterie, le test « seul écart » relevant un changement déclaré non observé et la comparaison stricte devenant un succès inattendu. Une tolérance élargie ou un chemin admis « quel que soit l'écart » n'est pas une transition mais un changement de résultat déguisé ; la règle ne dispense ni du tableau avant / après, ni du visa. Elle a servi une première fois pour l'issue #37, sous une forme propre à cette issue (`A_REGENERER_37`, vidée par le commit des références, puis remplacée par la liste unique), puis pour les issues #35 et #42, dont les transitions ont été closes par la seconde régénération, puis pour les issues #57, #64 et #67, dont les transitions ont été closes ensemble par la troisième (« Références existantes », ci-dessus) : la liste est de nouveau vide. Chaque nature, l'union et ses conflits, l'observation, la forme de la liste, le rattachement de l'échec attendu, les doubles restants et la garde sont exercés par des témoins de la batterie, sur des objets en mémoire et jamais sur `tests/reference/`.

Contrôles du dépôt

5.1 Scripts de contrôle

Tout contrôle mécanique est un script, jamais une relecture (`CLAUDE.md`, « Workflows », principe 7). Les scripts sont en R de base, sous `tests/controles/`, et sortent avec un code non nul en cas d'écart. Chacun est testé sur des témoins dans `tests/unitaires/test_controles.R`, ou, pour les deux contrôles à juge Python, dans `tests/unitaires/test_controles_portables.R` (ci-dessous) : il doit échouer sur un cas fautif et passer sur le dépôt. Pour `ecriture_doubles.R` et `ecriture_doubles_texte.R`, dont l'exécution complète (10^6 doubles et plus) reste hors batterie, la batterie exerce des chemins d'échec bon marché (issue #119) :

— *juge absent* : chacun des deux contrôles, lancé sous un `PATH` réduit à quelques outils système, sans Python, sort avec le code 1 et nomme l'absence, avant toute construction de doubles ;

— *vrai défaut* : une copie d'`ecriture_doubles_texte.R`, placée dans une arborescence temporaire, porte trois mutations (constante MD5 de T2 altérée, juge Python faussé, qui écrit `%.16g` au lieu de `%.17g`, nombre d'égalités exactes réduit pour la durée) et s'exécute sur $n = 1\,000$ doubles ; elle doit sortir avec le code 1 en relevant exactement les deux écarts attendus (MD5 de T2, juge Python), le MD5 de T3, les égalités exactes (dont la tranche de T1, trois égalités attendues et trouvées), la table des puissances et, hors Windows, le juge glibc restant sans écart ; un quatrième défaut, ajouté à la même copie (nombre d'égalités exactes attendu dans T1 porté de 3 à 4), doit être relevé sur la ligne de cette tranche, que le contrôle termine par « ÉCART », ce qui prouve que le témoin sait voir une tranche fausse ; une mutation qui ne toucherait pas exactement une ligne du script fait échouer le témoin ;

— *chemin à espace* : une copie d'`ecriture_doubles.R` dont les fichiers temporaires sont placés sous un dossier dont le nom contient une espace passe, et la même copie privée de la protection des chemins (`shQuote()`) passée au juge échoue ; le témoin voit ainsi le défaut que l'issue #119 a corrigé dans ce script.

Ces témoins dépendent de l'environnement et se sautent avec un saut **annoncé** : le juge absent hors

POSIX (Windows), le vrai défaut sans interpréteur Python 3.8 ou plus, le chemin à espace sous Windows, où le contrôle refuse de s'exécuter, ou sans interpréteur. Sur la CI, la batterie unitaire s'exécute avant l'installation de `python3`, réservée à l'étape du contrôle `ecriture_doubles_texte.R` : seuls les témoins du juge absent y sont exercés (décision du mainteneur du 06/11/2026, le workflow restant inchangé, consignée dans le corps de la pull request #157, section « Décisions et termes proposés »). Le vrai défaut est exercé sur un poste muni d'un interpréteur, Windows compris (constat sur le poste du mainteneur, R 4.3.3 ucrt, Python 3.14.7, 06/11/2026) ; le chemin à espace ne l'est que sur un poste POSIX muni d'un interpréteur, donc ni sur la CI ni sous Windows. La batterie unitaire garde en outre un échantillon de l'écriture recoupée par `ecriture_doubles.R` et les témoins T1 à T3 de `ecriture_doubles_texte.R`, que la CI exécute en entier (section 15.6). Pour `locale_c.R`, dont l'exécution complète (deux installations, six plans) est une étape de la CI, la batterie ne rejoue que le chemin d'échec bon marché : une racine sans package sort en échec et nomme l'installation sous `LC_ALL=C`. Elle charge en outre le script sans le lancer (lu par `parse()` en UTF-8, puis évalué dans un environnement propre ; issue #81), vérifie que `LOCALES_UTF8`, dupliquée de `tests/harnais.R`, y est identique à celle du harnais et que `locale_utf8_disponible()` y rend la même locale que celle du harnais, cette sonde y étant une copie au code identique à celui du harnais (comparé par `deparse()`) et la constante `LOCALE_INVENTEE` égale à celle du harnais (issue #109 ; section 4.2), et teste la fonction `ecart_octets`, que le script charge de l'utilitaire commun (ci-dessous), qui rédige le message d'un écart de la comparaison octet à octet et que le chemin d'échec bon marché n'atteint pas, sur des attendus écrits à la main (textes identiques, premier octet différent à la deuxième ligne, longueurs inégales à préfixe commun, dans les deux sens). Pour `locale_latin1.R`, de même exécuté en entier par une étape de la CI, la batterie rejoue deux chemins d'échec bon marché (racine sans package : l'installation sous `fr_FR.ISO-8859-1` est nommée en échec ; `localedef` introuvable : échec nommé, jamais un saut), vérifie que le script sourcé ne fait que définir ses fonctions et constantes, avec `LOCALES_UTF8`, la sonde et `LOCALE_INVENTEE` identiques à celles du harnais, et teste le marquage de ses jeux de segments et son ordre de référence. Les deux chemins d'échec dépendent de l'environnement : ils se sautent, avec un saut **annoncé** (« [sauté] » et sa raison), hors POSIX (Windows), si la locale latin-1 ne peut pas y être générée, ou si les commandes ou le lien symbolique qu'ils emploient manquent ; la batterie unitaire reste ainsi exécutable sous Windows, tandis que le contrôle lui-même n'y saute jamais en silence. Pour `commits_empreinte_code.R`, qui n'est pas une règle du dépôt mais l'outil qui retrouve les commits d'une empreinte du code, la batterie exécute le mode `arbre` et les refus, et ne parcourt l'historique, sur des dépôts synthétiques, que si git est présent, avec un saut annoncé sinon (section 6.4, « Retrouver le code d'une fiche »).

Utilitaire commun des contrôles de locale (issue #117). L'orchestration « installer, puis exécuter sous un environnement donné », que `locale_c.R` et `locale_latin1.R` recopiaient avec de légères variantes, est écrite une seule fois dans `tests/controles/orchestration_locale.R` : pose des variables d'environnement le temps d'un appel et restauration ensuite, même après une erreur ; durée écoulée ; `R CMD INSTALL` de l'arbre de travail dans une bibliothèque temporaire et exécution du contrôle en sous-processus, chacun sous un vecteur d'environnement complet que fournit le contrôle ; message d'écart de la comparaison octet à octet (premier octet différent et sa ligne). Deux copies d'une même logique devaient rester synchronisées sans que rien ne le signale ; une correction de la restauration de l'environnement ou de la lecture du statut de sortie vaut désormais pour les deux contrôles. L'utilitaire ne lance rien et ne fait que définir des fonctions ; ce qui est propre à un contrôle (environnement de locale, jeux, comparaisons) reste dans le contrôle. Chaque contrôle le lit par `parse()` en UTF-8, quelle que soit la locale de la session (latin-1 comprise), et l'évalue dans son propre environnement ; il ne retient que le dossier qui contient aussi le contrôle, jamais un fichier homonyme placé à côté du script qui le lance, et s'arrête en nommant le contrôle si l'utilitaire y manque. Les fonctions de l'utilitaire, leur chargement par les deux contrôles (dont le texte ne doit redéfinir aucune d'elles) et le refus de l'homonyme sont testés dans `tests/unitaires/test_controles_portables.R`, et non dans `test_controles.R` : les tests

nouveaux d'une branche ouverte en parallèle d'une autre vont dans un fichier propre (ADR 0013, point 3).

Interpréteur Python des juges (issue #123). Le même utilitaire, chargé de la même façon, porte la recherche et l'appel de l'interpréteur Python des deux contrôles à juge indépendant, `ecriture_doubles.R` et `ecriture_doubles_texte.R`, qui cherchaient auparavant `python3` seul : l'installeur de Python pour Windows ne fournit que `python.exe` et le lanceur `py`, et le `python3` que le contrôle trouvait sur le poste du mainteneur était l'alias du Microsoft Store, qui n'est pas un interpréteur (constat rapporté par l'issue #123). Les candidats sont essayés dans l'ordre `python3`, `python`, puis `py -3` ; tout chemin situé sous un dossier `WindowsApps` est écarté sans être exécuté, qu'il s'agisse de l'alias du Store ou d'un Python installé depuis le Store (décision du mainteneur du 06/11/2026, consignée dans le corps de la pull request #157, section « Décisions et termes proposés »). Le nom est cherché dans le chemin tel que rendu par la recherche et dans sa forme longue : sous Windows, `Sys.which()` rend des chemins courts au format 8.3, où `WindowsApps` n'apparaît pas. Il l'est aussi dans la forme longue du dossier qui contient le fichier. Cette troisième forme repose sur une hypothèse de construction : le fichier d'un alias du Store serait un point d'analyse (`IO_REPARSE_TAG_APPEXECLINK`) dont la résolution pourrait échouer, auquel cas la forme courte serait rendue telle quelle, alors que son dossier, ordinaire, se résoudrait. Ni ce mécanisme ni cette forme n'ont été mesurés sur un vrai alias : c'est une précaution. Tout autre candidat est exécuté (option `-I`) et n'est retenu que si sa version est au moins 3.8, une exécution en échec ou qui dépasse 30 secondes l'écartant. Si aucun ne convient, le contrôle sort avec le code 1 et un message qui nomme les trois candidats essayés : l'absence de juge est un échec, jamais un saut. Le script du juge impose l'UTF-8 à ses sorties standard et d'erreur (l'option `-I` écarte la variable `PYTHONIOENCODING`, et Python écrirait sinon dans la page de code ANSI de Windows, accents abîmés) ; `ecriture_doubles_texte.R` capture cette sortie, la lit en UTF-8 et l'imprime lui-même. Seul l'affichage était en cause : les comptes et le verdict ne dépendent pas de ce codage. **Statut** : *constat de mesure* sur le poste du mainteneur (Windows 11 Home, version 10.0.26300, R 4.3.3 ucrt, Python 3.14.7, 06/11/2026) : pour `py -3`, le délai, réduit à 3 secondes pour la mesure, est respecté, et l'arrêt ne laisse aucun processus Python orphelin ; la sortie capturée du juge garde ses accents. La recherche elle-même (ordre, chemin `WindowsApps` écarté, version, délai, absence de candidat) est testée avec de faux interpréteurs hors Windows ; la recherche dans la forme longue du dossier l'est hors Windows seulement, par liens symboliques : un lien vers un dossier `WindowsApps` y donne accès à un lien vers un interpréteur situé ailleurs, si bien que ni le chemin donné ni la forme longue du fichier ne contiennent `WindowsApps` ; ce candidat n'est pas lancé, le même montage vers un dossier ordinaire l'est. Ce test exerce la condition, non l'échec de résolution d'un point d'analyse. Le filtre `WindowsApps` l'est aussi sous Windows, quand le volume fournit des noms courts 8.3 (sinon le test est sauté, et le saut est annoncé), par un faux interpréteur placé sous un dossier `WindowsApps` et désigné par son chemin court, qui n'est pas lancé, et par le même placé ailleurs, qui l'est (`test_controles_portables.R`).

Table 7 : Scripts de contrôle (`tests/controles/`).

Script	Règle contrôlée	Source
`liste_blanche_description.R`	`Depends` : exactement `R (>= 4.3.3)`, plus d'éventuels packages de priorité `base` ou admis ; `Imports` : packages de priorité `base` de R 4.3.3, ou admis ; `LinkingTo`, `Suggests`, `Enhances` : packages admis seulement. Un package est admis s'il figure, avec son champ et l'ADR de méthode qui l'introduit, dans la constante `ADMIS`, vide à cette date	ADR 0002, point 3 et A8 ; M44
`graphe_modules.R`	graphe d'appels autorisés et appels dynamiques interdits (section 1.3)	ADR 0007, points 2.1 à 2.3

Script	Règle contrôlée	Source
`identite_batteries.R`	mêmes batteries, dans le même ordre, dans `CLAUDE.md`, dans `circuit-technique.js` et dans les étapes « Batterie : » du job « Tests (R 4.3.3) » ; ce job ne porte ni `continue-on-error`, ni condition `if:` de job, et une étape de batterie n'a pas d'autre condition que `${{ !cancelled() }}` ; une seule chaîne d'image dans tous les workflows ; invariants du workflow `references.yml` (contrôle 4, section 4.5)	`CLAUDE.md` ; ADR 0008, points 1 et 8 ; ADR 0009, point 6
`plateforme.R`	plateforme de référence (section 4.3)	ADR 0008, points 3 à 5
`absence_java.R`	aucune dépendance à Java (section 5.2)	ADR 0002, A1 ; ADR 0008, point 7 ; issue #3
`locale_c.R`	`tablexp` installé dans deux bibliothèques temporaires, l'une sous `LC_ALL=C`, l'autre sous la première locale UTF-8 disponible, et exécuté dans un sous-processus sous la locale de son installation, qui vérifie d'abord la locale effective ; pour R1 à R6, fiche et plan écrits, relus `identical()` et réécrits à l'identique, rapport figé écrit, et, pour R1, bilan du rejeu outillé (chaînes non ASCII marquées UTF-8) et messages de refus (marqués UTF-8, sur une ligne ; issue #94) ; messages de refus par module, de la lecture du texte du plan et de deux refus `type_incompatible` sur un rôle date, vingt-trois en tout (marqués UTF-8, sur une ligne, sans octet écrit « <xx> » ; issue #127) ; puis ces textes, trois par plan, le bilan, les messages de R1 et les messages par module, soit vingt et un, comparés octet à octet entre C et UTF-8, sans écart admis (section 4.2) ; étape bloquante « Locale C (installation et exécution) » du job « Tests (R 4.3.3) », hors liste des batteries	ADR 0007, point 11 et annotation du 05/11/2026, (f) ; M54 ; issues #66 et #127
`messages_refus.R`	règle de rédaction des messages de refus (section 2.1) : fichiers de `R/` lus par `parse()`, sans charger `tablexp` ; gabarit de `formuler()` constant, aux conversions en même nombre que les valeurs ; valeur insérée par `%s` citée, formée, constante ASCII, ou chaîne constante ou constante du moteur marquée par `texte_moteur()` ; aucune exception sur une constante du moteur non ASCII non marquée (relevé par `constantes_moteur`, fonction du script) ; message de chaque refus formé par `formuler()` ou une aide de message ; exceptions en liste fermée, motivées, toutes employées ; décompte imprimé. Appelé par la batterie unitaire (`test_controles.R`), avec un témoin d'échec par règle ; pas d'étape propre de la CI	ADR 0007, point 6.4 et annotation du 05/11/2026, (c) et (f) ; M182 ; issue #127

Script	Règle contrôlée	Source
`locale_latin1.R`	`tablexp` installé dans deux bibliothèques temporaires, l'une sous `fr_FR.ISO-8859-1`, générée par `localedef` (échec, jamais de saut, si elle ne peut pas l'être), l'autre sous la première locale UTF-8 disponible, et exécuté sous la locale de son installation, vérifiée d'abord ; pour R1 à R6 et quatre jeux de segments non ASCII (natifs, latin-1, UTF-8, mêlés), fiche, plan et rapport figé écrits, ordre des segments écrit et comparé à un ordre par octets UTF-8, puis 34 textes comparés octet à octet entre latin-1 et UTF-8, sans écart admis (section 4.2) ; étape « Locale latin-1 (installation et exécution) » du job « Tests (R 4.3.3) », hors liste des batteries	ADR 0007, point 9.3 ; issues #106 et #115
`orchestration_locale.R`	aucune règle : utilitaire commun, chargé par `locale_c.R` et `locale_latin1.R` (environnement posé le temps d'un appel, installation et exécution sous un environnement donné, durée, premier octet différent) et par les deux contrôles à juge Python (recherche de l'interpréteur, appel, message d'absence), qui ne fait que définir des fonctions ; testé dans `test_controles_portables.R`	issues #117 et #123
`ecriture_doubles.R`	écriture des doubles de l'empreinte en octets IEEE 754 identique, ligne à ligne, à celle d'un juge indépendant (Python, `struct.pack`) sur 10^6 doubles tirés sous graine et des cas limites ; **hors CI** (Python 3.8 ou plus requis, cherché parmi `python3`, `python` et `py -3` ; code 1 s'il n'est pas trouvé), non exécuté sous Windows (section 15.2) ; témoins d'échec (juge absent, chemin à espace) dans la batterie unitaire	ADR 0011, points 2.1 et 2.4 ; issues #35, #119 et #123
`ecriture_doubles_texte.R`	écriture des doubles en forme « texte » (`%.17g` calculé par `tablexp`) identique, ligne à ligne, à celle de deux juges, la glibc (`sprintf`, sauté sous Windows) et Python (`format`), sur plus de 10^6 doubles construits par leurs octets ou par des opérations sous graine, sans lecture d'un décimal (T1, T2, T3, un échantillon de 10^6 doubles, des égalités exactes tirées, leurs voisins et leurs opposés, des non-finis à charge utile) ; nombre d'égalités exactes exigé dans chaque partie ; MD5 des textes de T2 et de T3 égaux aux constantes de la batterie unitaire ; table des puissances égale aux entiers exacts de Python ; code 1 si aucun interpréteur Python 3.8 ou plus n'est trouvé (`python3`, `python`, `py -3`) ; sortie du juge lue en UTF-8. Exécution complète **hors batterie**, témoins d'échec (juge absent, vrai défaut) dans la batterie unitaire ; étape « Écriture %.17g des doubles (juges glibc et Python) » du job « Tests (R 4.3.3) », hors liste des batteries (section 15.2)	ADR 0011, annotation du 26/10/2026 ; M161 ; issues #104, #119 et #123

Script	Règle contrôlée	Source
`commits_empreinte_code.R`	commits dont les `R/*.R` ont une empreinte du code donnée, recalculée sur les blobs de l'historique git sans charger `tablexp` (mode `historique`) ; empreinte de la racine (mode `arbre`, égale à `EMPREINTE_CODE` en batterie) ; concordance des en-têtes des références (mode `references`) ; codes de sortie 0 à 3 (trouvé, refus, aucun commit ou écart, non déterminable). **Hors CI** pour l'historique, lancé à la main sur un clone complet (section 6.4)	ADR 0011, annotation du 27/10/2026, (e) ; M126, M166 ; issue #89

La liste des packages de priorité `base` est écrite en dur dans `liste_blanche_description.R` pour R 4.3.3 (`PACKAGES_BASE_R433`) : le verdict ne dépend pas de la machine ; le script signale, sans échouer, un écart avec les packages de priorité `base` effectivement installés.

5.2 Absence de Java

Règle. Aucun code du projet ne dépend de Java, ni directement ni indirectement : ni le moteur, ni l'application, ni la contre-vérification (ADR 0002, annotation du 16/10/2026, A1, décision M40). Java n'est pas utilisable sur le poste de travail du mainteneur ; c'est pour cette raison que le package `ELT`, qui dépend de `xlsx` et donc de Java, est exclu de la contre-vérification. Sa source ne peut être exécutée qu'hors du dépôt, sans Java, en appoint, sans qu'aucun code en soit copié (ADR 0002, A5 et troisième annotation).

Contrôle. `tests/controles/absence_java.R` (issue #3) contrôle trois ensembles de packages :

— le *moteur* : les packages déclarés dans les champs `Depends`, `Imports` et `LinkingTo` de `DESCRIPTION`, hors R ; aucun à cette date ;

— l'*application* : `shiny`, `htmltools`, et `openxlsx` à titre facultatif ;

— la *contre-vérification* : les packages `survival`, `mgcv`, `WH`, `StMoMo`, `demography`, `MortalityLaws`, `lifecontingencies`, `survPen`, `MortCast`, `locfit`, et `flexsurv` à titre facultatif ; seul `survival` a, à cette date, produit des valeurs versionnées (version 3.5.8, sous R 4.3.3, section 9.8).

Les deux derniers ensembles sont lus dans une liste unique, le fichier `packages_hors_moteur.csv` du dossier `tests/controles/` (colonnes `ensemble`, `package`, `statut`, `source`), qui est la référence ; la liste ci-dessus en est la transcription à la date du document. Le script lit les métadonnées d'un **instantané daté** du CRAN chez Posit Package Manager, par `available.packages()`, sans rien installer ; la date est la constante `INSTANTANE` du script (2026-10-15, un jour ouvré : selon la documentation de Posit citée par l'ADR 0008, aucun instantané n'est publié les week-ends et jours fériés), qui ne change que par un commit `repo:`. Il calcule, pour chaque package, la fermeture récursive de ses dépendances `Depends`, `Imports` et `LinkingTo`, par la fonction `tools::package_dependencies()` appelée avec `recursive = TRUE`, et **échoue** (code 1) :

— si un package de `PACKAGES_JAVA` (`rJava`, `xlsx`, `xlsxjars`) figure dans un ensemble ou dans la fermeture de l'un de ses packages ; la chaîne de dépendances la plus courte qui y mène est affichée ;

— si un package listé, ou une dépendance récursive hors priorité `base`, manque aux métadonnées : un package introuvable n'a pas de dépendances connues, il ne passe pas en silence ;

— si la liste est mal formée (en-tête, ensembles et statuts connus, noms valides, sans doublon, ensembles non vides).

Des métadonnées illisibles (panne réseau, adresse fausse) donnent le code 2 : aucun verdict n'est rendu, l'échec du job n'est jamais masqué et une relance le lève. Seul le filtre `duplicates` d'`available.packages()` est appliqué : le filtre par version de R écarterait les packages qui exigent un R plus récent que celui qui exécute le contrôle, alors que la contre-vérification peut tourner sur un autre poste muni d'un R récent, et ces packages seraient alors signalés absents (faux échec) ; le filtre par système d'exploitation est écarté aussi, le poste étant sous Windows et la CI sous Linux.

Cas témoin. Le job « Absence de Java » exécute aussi le script avec `--ajouter ELT` : il doit rendre le code 1 (un verdict, et non le code 2 d'une panne) et afficher la chaîne de `ELT` à `xlsx`. Les tests du script s'exécutent sans réseau, sur des dépôts locaux témoins (`tests/unitaires/test_controles.R`).

Limite. La détection porte sur les seuls noms de `PACKAGES_JAVA`. Le champ `SystemRequirements`, qui pourrait déclarer Java, n'est pas renseigné dans les métadonnées `PACKAGES` du CRAN (relevé consigné dans l'en-tête du script) : un package qui exigerait Java sans dépendre de l'un de ces noms échapperait au contrôle. Étendre la détection relève du mainteneur.

Partie II

Point d'entrée et plan de construction

Point d'entrée unique : R/construire_table.R

Le package a un point d'entrée unique, `construire_table(donnees, roles, plan, provenance = NULL)`, qui exécute un *plan de construction* d'un bloc et retourne un objet complet (ADR 0004 ; ADR 0005 ; ADR 0007, point 3.1). `R/construire_table.R` se limite à l'orchestration : il valide le plan, appelle une fonction d'étape par module dans l'ordre du plan et assemble l'objet, sans aucun calcul. Le quatrième argument, `provenance`, facultatif, est la voie par laquelle la couche de lecture transmet le nom du fichier lu, son empreinte et l'algorithme de celle-ci ; `NULL` signifie « données construites en mémoire, aucun fichier », ce que la fiche écrit (section 15.1).

6.1 Plan de construction et validation

Le plan est une liste R nommée, construite en mémoire par l'appelant, et **validée entièrement avant toute exécution, sans aucun défaut implicite** (ADR 0005, point 3 ; ADR 0007, point 5) : un paramètre absent est un refus, jamais une valeur supposée. Il porte six champs de premier niveau (constante `CHAMPS_PLAN` ; décisions M48 et M49) :

— `transition` : `list(depart = , arrivee = )`, deux noms d'états ; dans cette branche, `vivant` → `deces` ou `en_cours` → `rachat` ; les autres causes de sortie sont des sorties concurrentes ;

— `periode` : `list(debut = , fin = )`, deux dates, fin incluse ;

— `delai_declaration` : entier de jours ≥ 0, requis (zéro explicite admis) ;

— `segmentation` : vecteur de rôles pris dans `sexe` et `segment`, sans doublon, requis, vide explicite admis (E2.9). Elle est au premier niveau, et non parmi les paramètres de l'étape `exposition`, parce que l'étape `donnees`, qui précède, en a besoin pour ses contrôles (rôle requis, `segment_manquant`, `tete_segment_incoherent`) : une étape antérieure ne doit pas dépendre d'un paramètre d'une étape postérieure, et un plan réduit à l'étape `donnees` doit rester défini sans défaut (décision M49) ;

— `graine` : entier, requis dès qu'une étape du plan tire un aléa, interdit sinon. Aucune étape de cette branche ne tire d'aléa : la graine y est donc refusée, et la fiche écrit « aucune étape aléatoire » ;

— `etapes` : liste nommée ordonnée ; chaque nom est une étape, au plus une fois, dans l'ordre des modules, et chaque valeur la liste nommée des paramètres de l'étape, tous requis.

`transition`, `periode`, `delai_declaration` et `segmentation` sont déclarés une seule fois et transmis par le point d'entrée, en arguments nommés, à chaque fonction d'étape qui les emploie ; **une fonction d'étape ne reçoit jamais le plan entier** (ADR 0007, annotation du 17/10/2026, point c). Deux étapes ne peuvent

donc pas lire deux périodes ou deux segmentations différentes, et le chemin d'un champ dans le plan n'est connu que du point d'entrée et du validateur.

Les étapes de la branche et leurs paramètres sont des constantes du package (ETAPES, PARAMETRES_ETAPES), que le validateur applique :

Étape	Fonction d'étape	Préalable	Paramètres (tous requis)
donnees	preparer_donnees()	—	exclusions : vecteur de codes de motif sans doublon, vide explicite admis (section 7.3)
exposition	calculer_exposition()	donnees	convention : plus_fine, dernier_anniversaire ou age_atteint_annee ; par_annee : TRUE ou FALSE (section 8.1)
brut	calculer_taux_bruts()	donnees, exposition	estimateurs : sous-ensemble non vide, sans doublon ni NA, de poisson, binomial et kaplan_meier, restitué dans cet ordre ; binomial et kaplan_meier refusés sous par_annee = TRUE ; niveau : niveau de confiance, nombre double unique, fini, $0 <$ niveau < 1 (section 9.1)

Aucune des trois ne tire d'aléa. La recette d'exposition et l'unité d'observation ne sont pas des paramètres : le moteur les déduit des rôles (sections 8.1 et 7.4) ; seule une convention d'âge plus grossière peut être forcée.

Règle commune de forme (issue #23) : chaque paramètre d'étape a exactement le type de base que déclare la colonne type de PARAMETRES_ETAPES (character pour exclusions, convention et estimateurs, logical pour par_annee, double pour niveau) et **ne porte aucun attribut** : ni noms, ni dimension, ni classe (un facteur, une matrice ou un vecteur nommé sont refusés), sinon refus sous le motif propre au paramètre. Il en va de même des champs du plan : transition et periode sont des listes nues, dont le seul attribut admis est celui des noms, exactement autant d'éléments que de noms attendus (un data.frame est refusé) ; les deux bornes de la période n'ont d'autre attribut que la classe Date, stockées en double ou en integer ; delai_declaration est un integer ou un double sans attribut ; segmentation un vecteur character sans attribut, vide compris. La forme est contrôlée par forme_exacte(), directement ou par liste_aux_noms() et chaine_unique() (section 2.3), sous le motif existant de chaque paramètre : aucun motif n'est propre à la forme. Hors du plan, la même règle s'applique notamment à la graine reçue par sous_graine() (refus graine_invalide, section 3.3), aux rôles reçus par controler_roles() (seul attribut admis : les noms ; refus roles_mal_formes, section 7.3 ; issue #62), au mode du portefeuille simulé (refus portefeuille_invalide, section 16.2 ; issue #60), au portefeuille simulé lui-même (liste nue) et à ses champs numériques et de date (même refus ; issue #78), aux bornes de la période du simulateur (même forme que celles du plan ; refus periode_invalide) et aux facteurs de la loi vraie (double ou integer, seul attribut les noms ; refus loi_invalide, section 16.1 ; issue #58) et, à l'appel direct de la fonction d'étape calculer_taux_bruts() (argument controler à sa valeur par défaut, TRUE ; section 6.2), à la segmentation, contrôlée dans sa forme et dans son catalogue par le même contrôle que dans le plan, controler_segmentation() (refus segmentation_invalide du module brut, section 9.1 ; issues #43 et #59). Pour chaque paramètre, elle l'est avant sa valeur, si bien qu'un vecteur nommé de codes inconnus est refusé sous exclusions_invalides ou estimateurs_invalides, et non sous exclusion_inconnue ou estimateur_inconnu.

valider_plan(plan), exportée parce que l'application l'appellera comme contrôle de saisie (E4.3), rend le plan inchangé, ou refuse. Elle porte les codes *structurels* du plan, module construire_table : plan_mal_forme, champ_inconnu, champ_manquant, etapes_mal_formees, aucune_etape,

etape_en_double, etape_inconnue, etapes_hors_ordre, etape_prealable_manquante, graine_inattendue, graine_manquante, parametres_mal_formes, parametre_inconnu et parametre_manquant. Les préalables d'une étape forment une liste (colonne prealable de ETAPES) ; chacun est contrôlé, et le refus etape_prealable_manquante ne cite que les préalables absents, au singulier ou au pluriel selon leur nombre. Son message et ses champs etape et prealable de details rassemblent les étapes fautives d'une part, les préalables absents d'autre part ; le champ prealables_par_etape, liste nommée par étape fautive dans l'ordre du plan, donne pour chacune ses propres préalables absents, dans l'ordre de ETAPES, si bien que le refus dit quel préalable manque à quelle étape (issue #27). Le plan, son champ etapes et la liste des paramètres de chaque étape sont des listes nommées **nues**, dont le seul attribut admis est celui des noms : un data.frame, un objet de classe ou une liste munie d'un autre attribut est refusé sous plan_mal_forme, etapes_mal_formees ou parametres_mal_formes (issue #23). Leurs noms sont présents, non vides et non manquants (liste_noms_complets(), qui leur applique la règle noms_complets() du module outils, section 2.3) ; un nom en double est refusé sous plan_mal_forme ou parametres_mal_formes (liste_nommee()), mais, dans etapes, sous etape_en_double, qui nomme les étapes répétées, et non sous etapes_mal_formees (issue #60). Elle ne réimplémente pas les contrôles de valeur : elle appelle ceux des modules (controler_parametres_donnees(), controler_parametres_exposition(), controler_parametres_brut()), dont les codes et le module sont ainsi identiques, quelle que soit la fonction qui détecte la condition (ADR 0007, annotation du 17/10/2026, point d). Dans une construction, ces contrôles ne sont faits que là : les fonctions d'étape ne les refont pas (section 6.2).

Exemple de plan (vivant → décédé, par sexe, sans exclusion) :

```
plan <- list(transition = list(depart = "vivant", arrivee = "deces"),
             periode = list(debut = as.Date("2015-01-01"),
                            fin = as.Date("2024-12-31")),
             delai_declaration = 0L, segmentation = "sexe",
             etapes = list(donnees = list(exclusions = character(0)),
                           exposition = list(convention = "plus_fine",
                                             par_annee = FALSE),
                           brut = list(estimateurs = c("poisson", "binomial"),
                                       niveau = 0.95)))
res <- construire_table(donnees, roles, plan)
```

6.2 Exécution

construire_table() procède dans cet ordre : validation du plan, puis sa mise sous forme canonique, une seule fois (canoniser_plan(), section 15.1) ; les étapes et la fiche ne voient que le plan canonique ; contrôle des rôles (controler_roles()) ; déduction de la recette et, si le plan comprend l'étape exposition, de la convention retenue ; contrôle de la provenance ; contrôle du codage des chaînes que la fiche écrira hors des données, valeurs des rôles (noms de colonnes) et champs de la provenance, par l'écrivain de l'empreinte (octets_chaines(), refus chaine_codage_bytes, chaine_non_utf8 et chaine_codage_indetermine ; issue #106) ; calcul de l'empreinte des données reçues (section 15.2). Tout refus possible avant le calcul a donc lieu avant la première étape ; un nom de colonne ou un chemin non ASCII de codage indéterminé, sous la locale C, n'est plus refusé après tous les calculs, à l'écriture de la fiche, en erreur de programmation. Chaque contrôle n'y est exécuté qu'une fois (issue #22) : construire_table() appelle chaque fonction d'étape avec son argument interne controler = FALSE, sous lequel elle ne refait aucun de ces contrôles (ses paramètres et, selon l'étape, les rôles, la recette, la convention et la segmentation), et reprend les valeurs qui en dérivent par des fonctions qui ne contrôlent rien (valeurs_parametres_donnees(), recette_branche(), convention_retenue()). Appelée directement, avec la valeur par défaut controler = TRUE, une fonction d'étape fait elle-même ses contrôles,

chacun une fois, et en lève les refus ; seule la valeur FALSE les omet. Les étapes s'exécutent ensuite dans l'ordre du plan, chacune par sa fonction d'étape, qui rend ses paramètres effectifs, son résultat, ses diagnostics et ses avertissements ; l'étape exposition lit la table des observations de l'étape donnees ; l'étape brut lit la table d'exposition et les paramètres effectifs de l'étape exposition (découpage par année, convention retenue, unité de l'exposition) et, pour l'estimateur kaplan_meier, les fenêtres des observations calculées par l'étape exposition, avec la transition, la période et le délai du plan. Ces fenêtres sont la seule donnée qu'une étape transmet à une autre hors de l'objet résultat (issue #38 ; ADR 0007, seconde annotation du 19/10/2026) : calculer_exposition() les rend dans un élément fenetres à côté de son résultat (section 8.2) ; construire_table() ne les conserve que si le plan retient l'estimateur kaplan_meier, et les abandonne sinon dès la fin de l'étape exposition ; conservées, elle les garde dans une variable locale, les passe à calculer_taux_bruts() par son argument observations, puis les libère après l'étape brut. Seuls les paramètres effectifs, le résultat et les diagnostics de chaque étape entrent dans etapes ; les fenêtres n'entrent ni dans l'objet résultat ni dans la fiche, qui sont ceux de la section 6.3. L'étape exposition précède toujours l'étape brut, préalable que valider_plan() contrôle ; dans une construction, l'étape brut ne lit donc pas la table des observations de l'étape donnees elle-même, mais ses fenêtres. L'objet assemblé, construire_table() calcule l'empreinte du résultat sur cet objet (empreinte_resultat(), section 15.3), la range dans fiche$empreintes$resultat, puis ajoute execution, hors du périmètre de l'empreinte.

6.3 Objet résultat

L'objet retourné est une liste nommée de classe S3 construction (ADR 0007, point 6.1), lue par $ et jamais recalculée par ses lecteurs :

— table : la table finale ; NULL dans cette branche, dont les plans s'arrêtent aux taux bruts ;

— etapes : une entrée par étape exécutée, dans l'ordre, avec parametres (paramètres effectifs, après résolution par le moteur), resultat et diagnostics ;

— avertissements : le data.frame (etape, code, message, nombre, ce dernier en integer) des avertissements de toutes les étapes, repris tel quel dans la fiche (section 2.2) ;

— fiche : la fiche de construction (section 15.1) ;

— execution : horodatage de début, durée, version de R, plateforme, système, paramètres régionaux et packages attachés ; **hors de toute comparaison** (rejeu, reproductibilité, non-régression) : c'est le seul champ qui change entre deux exécutions identiques.

Ajouter un champ est compatible ; renommer ou retirer un champ change le contrat partagé « objet résultat » (ADR 0007, point 6.3). C'est à ce titre que le retrait des quatre copies de premier niveau de la fiche (section 15.1) a été décidé par le mainteneur (décision M64, question Q2 ; issue #42).

6.4 Rejeu outillé

Le *rejeu* est l'exécution de construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance) sur les données qu'une fiche de construction identifie, pour retrouver la construction qu'elle décrit (E3.7). La note de lecture d'E3.7 du 19/10/2026 (décision M64 ; ADR 0011, point 12) en fait un critère en quatre temps : le relecteur (1) vérifie version_fiche, la représentation canonique et l'empreinte des données de la fiche ; (2) exécute la construction sur le plan relu du texte de la fiche ; (3) compare les grandeurs à tolérance par classes (section 4.5) sur une autre machine, au bit près sur la même (E4.6) ; (4) n'emploie l'empreinte du résultat que comme témoin sur la même machine (section 15.3). Le *rejeu outillé*, fonction exportée rejouer_fiche(donnees, fiche) de R/construire_table.R (issue #86 ; ADR 0012, point 3, précisé par sa seconde annotation du

24/10/2026), outille les temps (1), (2) et (4) — pour le temps (2), la relecture du texte de la fiche reste à l'appelant, par `lire_fiche()` —, et le cas « même machine » du temps (3) par le seul témoin de l'empreinte du résultat, qui ne localise pas un écart ; la comparaison à tolérance par classes sur une autre machine reste hors de l'outil (ci-dessous, « Limites »).

Une fonction de contrôle, non un second point d'entrée (ADR 0012, points 3.2 et 3.3). `rejouer_fiche()` n'exécute aucune étape et n'appelle aucune fonction d'étape : elle contrôle la fiche et les données, puis appelle `construire_table()` **exactement une fois**, avec les rôles, le plan et la provenance **lus dans la fiche**, sans en recevoir d'autres ; `construire_table()` reste la seule fonction qui exécute un plan, et sa signature est inchangée. Elle vit dans `R/construire_table.R` parce que le module `rapport`, qui porte la fiche et ses empreintes, ne peut pas appeler `construire_table()` (section 1.3) ; un troisième fichier transversal, écarté, aurait changé la forme du package pour une seule fonction. Elle reçoit un `data.frame`, jamais un fichier (E4.4) : la fiche est relue de ses octets par `lire_fiche()` chez l'appelant, ou prise dans `x$fiche`.

Étapes, dans cet ordre ; le rejeu s'arrête au premier refus, sans appeler `construire_table()` :

1. **nature des arguments**, la fiche d'abord : un objet `construction` ou `rejeu` reçu à la place de la fiche, une liste dont un nom de premier niveau, à toute position, est d'encodage `bytes` (`noms_bytes()` ; section 2.1, « Noms d'encodage `bytes` » ; issue #114), ou une fiche qui n'est pas une liste nommée aux noms uniques, sans autre attribut que ses noms (`liste_nommee()`), est refusé sous `fiche_non_conforme` ; puis des données qui ne sont pas un `data.frame` sont refusées par `controler_roles()`, sous le motif `donnees_mal_formees` du module `donnees`, avec le message même qu'en donnerait `construire_table()` ;
2. **forme des champs que le rejeu lit lui-même** : `version_fiche`, `integer` de longueur 1, non `NA`, sans attribut (forme exacte : un `double` relu d'un texte édité est refusé) ; `empreintes`, liste nommée aux noms uniques, sans autre attribut que ses noms et sans nom d'encodage `bytes`, et, si elle est une liste nommée aux noms uniques, ses éléments `algorithme`, `representation` et `donnees`, lus par `lire_champ()`, chacun chaîne unique, non vide, non `NA`, sans attribut (`chaine_unique()`) ; noms `roles`, `plan` et `provenance` présents (`provenance` peut valoir `NULL`). Une classe d'encodage `bytes` sur l'un de ces champs est un attribut en défaut comme un autre, et un nom de cet encodage dans `empreintes` un défaut de forme de ce champ : ils sont refusés ici, avec les autres défauts, et non par une erreur R (ADR 0012, point 3.4 ; décision M202 ; issue #128). **Tous** les champs en défaut sont cités dans un seul refus `fiche_non_conforme` ; aucun n'est complété ni converti. Le contenu de `roles`, `plan` et `provenance` est laissé à l'étape 6 ;
3. **version admise** : `version_fiche` appartient à `VERSIONS_FICHE_REJEU`, sinon refus `version_fiche_non_admise`. Le contrôle précède ceux du contenu, parce que la version donne le sens des autres champs ;
4. **algorithme de l'empreinte des données** : `empreintes$algorithme` égal à `ALGORITHME_EMPREINTE`, sinon `algorithme_empreinte_inconnu` ;
5. **représentation canonique** : `empreintes$representation` égale à `ENTETE_CANONIQUE`, sinon `representation_non_admise` : un recalcul sous une autre définition échouerait toujours, sous un motif trompeur ;
6. **entrées de la construction** : la séquence de contrôles de `construire_table()` (section 6.2), refaite en entier, dans son ordre, sur les champs de la fiche : `valider_plan()`, `canoniser_plan()`, `controler_roles()` sur la segmentation canonique, `deduire_recette()`, `resoudre_convention()` si l'étape `exposition` est au plan, `controler_provenance()`. Ces refus gardent leur module, leur motif et leur message (`colonne_absente`, `type_incompatible`, `plan_mal_forme`, `convention_invalide`, `provenance_invalide`…). La séquence est dupliquée, non factorisée, pour ne pas modifier

construire_table() ; un test vérifie qu'elle donne le même refus par les deux chemins. Elle précède l'empreinte des données, qui, calculée la première, nommerait mal la cause : une colonne de rôle absente donnerait type_non_canonique (module rapport) et une date en character une empreinte discordante, au lieu de colonne_absente et de type_incompatible (ADR 0012, « Contexte », mesures d'actuary) ;

7. **empreinte des données** : recalculée par empreinte_donnees(donnees, fiche$roles) et comparée à fiche$empreintes$donnees, sinon empreinte_donnees_discordante. Elle est donc calculée deux fois par rejeu, ici et dans construire_table(), qui ne reçoit pas d'empreinte : la lui transmettre aurait changé la signature du point d'entrée ;
8. **construction** : construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance), une fois. Si l'empreinte des données de l'objet rendu diffère de celle de l'étape 7, le rejeu s'arrête sur une erreur de programmation (garde-fou, non un refus) ;
9. **témoins**, consignés dans le bilan, jamais motifs de refus (ci-dessous).

Versions admises : la version courante seule (ADR 0012, point 3.5). VERSIONS_FICHE_REJEU vaut VERSION_FICHE, écrite comme une expression et non comme un littéral (un test exige leur identité) ; elle est rangée dans R/rapport.R, à côté de VERSION_FICHE, parce qu'une constante de premier niveau de R/construire_table.R ne peut pas lire une constante de R/rapport.R, évalué après lui à l'installation (section 15.1, « Ordre des valeurs des champs ensembles ») ; rejouer_fiche() la lit à l'appel. Trois raisons : aucune conversion silencieuse d'une fiche d'une autre version (ADR 0011, point 10 (d)) ; une telle fiche donnerait de toute façon un témoin « différente », même sur la même machine, version_fiche étant dans le périmètre de l'empreinte du résultat ; aucune fiche d'une version antérieure n'a été livrée hors des tests. Admettre une version antérieure demandera de montrer, version par version, que les champs lus par le rejeu ont la même structure et le même sens. Une fiche de version 3, relue sans refus par lire_fiche(), qui ne contrôle pas la version, est ainsi refusée par rejouer_fiche().

Textes des refus. Les cinq motifs propres au rejeu sont levés sous le module construire_table, comme les refus de valider_plan() (tableau 8) ; les refus délégués (étape 1 pour les données, étape 6) gardent leur module. Chaque message cite la valeur lue et la valeur attendue. La version de tablexp et les empreintes du code y sont écrites par texte_lu() : « absente » si la valeur manque, elle-même si c'est une chaîne unique sans attribut (NA écrit « NA »), « illisible (*type* de longueur *n*) » sinon ; elles sont rangées sous cette forme dans les details de version_fiche_non_admise (tablexp, code, code_installe), la version lue y restant brute (details$lu). Toute valeur lue dans la fiche et insérée dans un message (classes de l'objet reçu, valeurs et noms lus) est citée selon la règle unique des refus (section 2.1, « Citation d'une valeur dans un message » : caractères de contrôle et séparateurs de ligne écrits <U+XXXX>, octets hors UTF-8 valide <xx>, chaîne bytes sous un repère ; sans_controle(), issue #94) ; les details et le bilan gardent la valeur brute. Le message est marqué UTF-8 sous toute locale (gabarits passés par formuler(), section 2.1, « Règle de rédaction des messages »). Une valeur d'encodage bytes donne ainsi le refus nommé de l'étape, et non l'erreur R « translating strings with "bytes" encoding is not allowed » : empreintes$algorithme d'encodage bytes est refusé sous algorithme_empreinte_inconnu, et une fiche dont une classe est d'encodage bytes sous fiche_non_conforme (classes lues par %in%, là où inherits() lève cette erreur, constat sous les locales C et C.UTF-8). Les formes lues de l'étape 2 sont écrites par decrire_forme() : « champ absent », ou « *type* de longueur *n* », complété au besoin par « , NA », « , vide », « , attribut(s) *noms* » (les noms d'une liste ne sont pas cités comme attribut), « , noms en double : *noms* », « , noms incomplets » et « , nom(s) d'encodage bytes (n° *k* : *repère*) », où *k* est la position du premier de ces noms (issue #128) ; les noms d'attributs et les noms en double y sont cités un à un par la règle unique, si bien qu'un nom bytes n'efface pas les autres. Une valeur de classe d'encodage bytes est décrite par son type et sa longueur sans sa classe, ses attributs, class compris, étant relevés avant (length(), is.na() et names() lèveraient l'erreur R sur elle). Pour ces formes, details$lu est ce texte descriptif, et non la

valeur brute : la valeur exacte est celle de la fiche reçue au chemin details$champ. Dans le tableau, un crochet désigne une valeur insérée.

Table 8 : Refus propres au rejeu outillé (module construire_table).

Motif et étape	Message	details
fiche_non_conforme, étape 1 (objet construction)	« Rejeu : objet ‘construction‘ reçu à la place de sa fiche ; passer x$fiche, ou la fiche relue de son texte par lire_fiche(). »	champ « (fiche) », lu « construction », attendu « liste nommée »
fiche_non_conforme, étape 1 (objet rejeu)	« Rejeu : objet ‘rejeu‘ reçu à la place d'une fiche ; passer r$construction$fiche. »	idem, lu « rejeu »
fiche_non_conforme, étape 1 (nom d'encodage bytes)	« Rejeu : la fiche porte un nom de champ d'encodage « bytes » (champ n° [position du premier] : [nom cité]), non convertible en UTF-8 sans deviner ; aucun champ du premier niveau n'est lu par nom. Une fiche produite par le moteur ou relue par lire_fiche() n'en porte pas. »	champ « (fiche) », lu les noms bytes bruts, attendu « noms de champ sans encodage bytes »
fiche_non_conforme, étape 1 (autre objet)	« Rejeu : la fiche doit être une liste nommée aux noms uniques, sans autre attribut que ses noms ; reçu : objet de classe [classes] (type [type]). Le rejeu ne lit aucun fichier : relire la fiche de ses octets par lire_fiche(readBin(fichier, "raw", file.size(fichier))), ou passer x$fiche. »	idem, lu les classes
fiche_non_conforme, étape 2	« Fiche non conforme au rejeu : « [champ] » : [forme lue] au lieu de [forme attendue] ; … Le rejeu ne lit que ces champs avant construire_table() ; aucun n'est complété ni converti. » Formes attendues : « integer de longueur 1, non NA, sans attribut » (version_fiche) ; « liste nommée aux noms uniques, sans autre attribut que ses noms » (empreintes) ; « chaîne unique, non vide, non NA, sans attribut » (ses trois éléments) ; « champ présent » (roles, plan) ; « champ présent (NULL admis) » (provenance)	champ, lu, attendu : un élément par champ en défaut, dans l'ordre de l'étape
version_fiche_non_admise, étape 3	« Version de fiche non admise au rejeu : version_fiche [version lue] lue, version(s) admise(s) par ce code : [VERSIONS_FICHE_REJEU]. Fiche produite par tablexp [version de la fiche], empreinte du code [empreinte du code de la fiche] ; code installé : tablexp [version installée], empreinte du code [EMPREINTE_CODE]. Aucune conversion de fiche n'est faite : rejouer cette fiche avec le code dont l'empreinte est celle de la fiche. » La version de la fiche est « absente » si versions manque ou est une liste sans élément tablexp	lu, admises, tablexp, code, code_installe
algorithme_empreinte_inconnu, étape 4	« Algorithme d'empreinte « [lu] » lu dans la fiche, « [ALGORITHME_EMPREINTE] » attendu : ce code ne calcule l'empreinte des données que par cet algorithme ; aucune comparaison n'est possible. »	lu, attendu
representation_non_admise, étape 5	« Représentation canonique « [lue] » lue dans la fiche, « [ENTETE_CANONIQUE] » attendue : ce code calcule l'empreinte des données sous cette seule définition, et une empreinte d'une autre définition ne lui est jamais égale ; aucune conversion n'est faite. »	lu, attendu

Motif et étape	Message	details
empreinte_donnees_discordante, étape 7	« Empreinte des données discordante : [empreinte de la fiche] lue dans la fiche, [empreinte recalculée] recalculée sur les données reçues ([nombre] lignes). L'empreinte porte sur la représentation « [ENTETE_CANONIQUE] » des colonnes des rôles de la fiche ([colonnes, entre guillemets]), lignes dans l'ordre reçu : un autre ordre des lignes, une valeur ou un type de colonne différents suffisent à la changer ; les colonnes hors rôles n'y entrent pas. »	lu, attendu, lignes, colonnes

Le message de version_fiche_non_admise cite l'empreinte du code, et non la seule version du package, parce que celle-ci, déclarée dans DESCRIPTION, est immobile (0.0.0.9000) et ne permet pas de retrouver le code d'une fiche : elle ne s'incrémente qu'aux jalons de livraison (décision M126).

Retrouver le code d'une fiche (issue #89 ; ADR 0011, annotation du 27/10/2026, (e) ; décisions M126 et M166). On passe de fiche$empreintes$code aux commits qui l'ont produite en **recalculant l'empreinte du code dans l'historique git**, par le script tests/controles/commits_empreinte_code.R, en R de base, hors du package (le moteur ne dépend pas de git, et un package installé n'a pas d'historique) et sans charger tablexp : son recalcul est indépendant de empreinte_sources() et la recoupe. Aucun registre « empreinte → commit » n'est tenu, et la version du package n'est pas incrémentée à chaque fusion. Le script applique la définition et la garde de l'empreinte du code (section 15.3, « Empreinte du code ») : fichiers de premier niveau de R/ dont le nom finit par .R, ni cachés ni dossiers, triés par octets ; MD5 de chaque fichier ; relevé « <md5>, deux espaces, R/<fichier> », une ligne par fichier, LF ; MD5 du relevé ; pas d'empreinte, jamais devinée, pour un commit sans DESCRIPTION de « Package : tablexp » ou sans R/rapport.R. Dans l'historique, il calcule sur les *blobs* (git cat-file), non sur un arbre extrait, et lit les noms de R/ en octets (git ls-tree -z), si bien que le résultat ne dépend pas de l'arbre de travail. Trois modes, lancés depuis la racine du dépôt :

— historique <empreinte> : l'empreinte, 32 caractères hexadécimaux minuscules, est refusée sous une autre forme avant tout appel à git ; le script parcourt les commits de toutes les références locales (git rev-list --all), sauf refs/stash, un code remisé n'étant pas commité, et liste, du plus ancien au plus récent, ceux dont R/*.R a cette empreinte (SHA complet, date, sujet) ;

— arbre : empreinte des R/*.R de la racine, sans git ; la batterie unitaire exige son égalité à EMPREINTE_CODE ;

— references : hors batteries et hors CI, pour chaque tests/reference/*.tsv, vérifie que le commit de l'en-tête # commit est dans la classe de l'empreinte de l'en-tête # fiche$empreintes/code.

Codes de sortie distincts, rien n'étant deviné : **0**, commits trouvés, empreinte rendue ou références concordantes ; **1**, refus nommé seulement (argument, git absent, racine hors d'un dépôt, sources introuvables, aucun .tsv) ; **2**, conclusion négative : aucun commit sur un historique complet (le code n'était pas commité, ou sa branche est absente du clone), ou, en mode references, écart (commit absent d'un historique complet, ambigu, empreinte différente de l'en-tête, en-tête inexploitable ; un écart sur un commit présent d'un clone superficiel est concluant aussi) ; **3**, non déterminable : aucun commit, ou commit d'un .tsv absent, sur un clone superficiel, ou, en mode historique, aucun commit alors qu'un commit lu n'a pas d'empreinte, ou, en mode references, empreinte différente parce que le commit de l'en-tête lui-même n'a pas d'empreinte ; un commit n'a pas d'empreinte quand une entrée de R/ nommée *.R n'est pas un fichier ordinaire (lien symbolique, sous-module, dossier) : ces commits sont nommés. En mode references, l'écart (2) l'emporte sur le non déterminable (3). **Limites** :

— une empreinte désigne une **classe de commits** : tout commit qui ne touche pas R/*.R garde l'empreinte de son parent. La classe identifie le code R, non le package entier : deux commits d'une classe peuvent

différer par DESCRIPTION, NAMESPACE, inst/ ou un fichier caché ; qu'aucun résultat n'en dépende est une **hypothèse**, non mesurée, et le relecteur lit ces écarts par git diff entre deux commits de la classe ;

— sur un **clone superficiel**, une liste trouvée est valable mais peut être incomplète, et une absence n'est jamais concluante (le script le dit) ; le contrôle de l'historique se lance donc à la main, sur un clone complet (git fetch --unshallow au besoin) : le job « Tests (R 4.3.3) » tourne sans git, et les autres jobs de la CI reçoivent un historique de profondeur 1 (*hypothèse*, d'après le défaut d'actions/checkout, non relu ; ADR 0011, annotation du 27/10/2026, (e) 4) ; .github/workflows/ci.yml n'est pas modifié (M166) ;

— un commit perdu (branche supprimée sans fusion, historique réécrit) rend sa fiche introuvable : le script le dit, il ne le répare pas. La règle de fusion par commit de fusion, jamais par squash ni rebase, garde accessibles depuis main les commits d'une branche fusionnée ;

— le MD5 d'un blob est celui du fichier extrait sous la règle eol=lf de .gitattributes ; un arbre extrait avec d'autres fins de ligne aurait une autre empreinte (limite déjà connue de la commande des tiers).

tests/unitaires/test_controles.R vérifie en batterie le mode arbre (égalité à EMPREINTE_CODE, fichiers .r et cachés ignorés, sources introuvables refusées) et les refus sans git ; le parcours de l'historique, sur des dépôts synthétiques créés sous tempfile() (ordre, garde, classe, codes 0, 2 et 3, clone de profondeur 1, blob partagé avec DESCRIPTION, lien symbolique, code remisé, noms que git cite entre guillemets, mode references), n'est exécuté que si git est présent : il est sauté, avec un saut **annoncé** (« [sauté] »), dans le job « Tests (R 4.3.3) ».

Objet rendu et bilan de rejeu (ADR 0012, points 3.1 et 3.6). rejouer_fiche() rend une liste nommée de classe S3 rejeu à deux éléments : construction, l'objet rendu par construire_table(), **sans modification ni attribut** — il reste identical() hors execution à un original, et le périmètre de l'empreinte du résultat n'est pas touché —, et bilan, un data.frame de treize lignes et cinq colonnes character, sans NA (controle, attendu, lu, verdict, statut) : sept contrôles, puis six témoins (tableau 9). La colonne lu porte ce que reçoit le rejeu (la fiche, et les données pour la première ligne), la colonne attendu ce que le code installé exige ou obtient. Le bilan **n'entre pas dans l'objet** : un champ ou un attribut de plus ferait que l'objet rejoué ne serait plus identical() à un original, changerait le périmètre de l'empreinte du résultat, et empreinte_resultat() s'arrête sur une erreur de programmation devant tout attribut inconnu. Un bilan n'existe que pour un rejeu abouti : ses sept lignes de contrôle valent donc toujours « conforme » ; elles disent ce qui a été contrôlé, sur quelles valeurs. Un refus est une condition R (section 2.1) et ne rend aucun bilan.

Table 9 : Bilan de rejeu : une ligne par contrôle, puis par témoin.

controle	attendu / lu	verdict	statut
nature_arguments	« fiche : liste nommée ; donnees : data.frame » / « fiche : liste nommée de [n] champs ; donnees : classe [classes], [n] lignes, [m] colonnes »	conforme	contrôle de forme
forme_fiche	formes exigées à l'étape 2 / « champs lus de la forme attendue »	conforme	contrôle de forme
version_fiche	VERSIONS_FICHE_REJEU / version_fiche de la fiche	conforme	comparaison exacte
algorithme_empreinte	ALGORITHME_EMPREINTE / empreintes$algorithme	conforme	comparaison exacte
representation	ENTETE_CANONIQUE / empreintes$representation	conforme	comparaison exacte

controle	attendu / lu	verdict	statut
entrees_construction	« valider_plan(), controler_roles(), recette et convention, controler_provenance() : aucun refus » / « étapes : [étapes du plan] ; rôles : [rôles de la fiche] ; recette : [recette déduite] ; provenance : [fichier, ou NULL] »	conforme	contrôles délégués aux modules
empreinte_donnees	empreinte recalculée / empreintes$donnees	conforme	exacte ; sans rôle de sécurité
temoin_code	EMPREINTE_CODE / empreintes$code	V1	exacte ; sans rôle de sécurité
temoin_versions	versions de la fiche rendue / de la fiche reçue, en paires « nom=valeur » jointes par « ; »	V2	comparaison exacte
temoin_plateforme	idem pour plateforme	V2	comparaison exacte
temoin_fiche	« fiche rendue par construire_table(), hors versions, plateforme, empreintes/resultat, empreintes/code, empreintes/statut_resultat_code » / « fiche reçue, même périmètre »	V3	comparaison exacte
temoin_empreinte_resultat	empreinte du résultat de l'objet rendu / empreintes$resultat de la fiche	V1	témoin, même machine seulement
temoin_empreinte_fichier	« non recalculée par le moteur (couche de lecture, E4.4) » / empreintes$fichier	non vérifiée	reprise de la fiche, non vérifiée par le moteur

Les libellés forment des ensembles fermés, au féminin singulier (ADR 0012, seconde annotation du 24/10/2026, (a) 4). Verdicts : « conforme », « identique », « différente », « différente : *chemins* », « non déterminable », « non vérifiée ». Statuts : « contrôle de forme », « comparaison exacte », « contrôles délégués aux modules », « exacte ; sans rôle de sécurité », « témoin, même machine seulement », « reprise de la fiche, non vérifiée par le moteur ». Les verdicts des témoins suivent trois règles :

— **V1** (verdict_chaines(), empreintes du code et du résultat) : « non déterminable » si l'un des deux côtés n'est pas une chaîne unique, non vide, non NA, sans attribut (chaine_unique()) — deux NA ne prouvent pas une identité —, sinon « identique » ou « différente » ;

— **V2** (verdict_liste(), versions et plateforme) : « non déterminable » si le côté de la fiche reçue n'est pas une liste sans classe aux noms complets, ou porte un nom d'encodage bytes (une classe de cet encodage étant déjà une classe), la colonne lu citant alors ce nom sous le repère de la règle unique, ou « illisible (list de longueur *n*) » pour une classe bytes : ces champs sont des témoins, jamais motifs de refus (ADR 0012, point 3.6 ; décision M202 ; issue #128) ; sinon la règle V3 sur ces deux listes, champ par champ, les noms étant lus dans la fiche et dans l'objet rendu (le champ bibliotheque_c y entre ainsi sans retouche) ;

— **V3** (verdict_chemins(), fiche) : « identique » si les deux côtés sont identical(), sinon « différente : » suivi des chemins qui diffèrent, joints par « , ». Les chemins sont ceux de champs_differents() : deux listes nommées sans classe, aux noms complets et uniques, sont parcourues sur l'union de leurs noms (ordre de la fiche rendue, puis noms propres à la fiche reçue) ; un nom présent d'un seul côté est cité « (absent de la fiche reçue) » ou « (absent de la fiche rendue) » ; toute autre valeur est une feuille, citée par son chemin, dont une liste qui porte un nom d'encodage bytes, à toute profondeur (*lecture* de la récursion de champs_differents(), bornée par la fiche rendue ; *constat* aux niveaux 2 à 7), citée « différente » à son chemin, ou à celui de son plus proche ancêtre qui est une feuille de la fiche rendue, sans que ses noms soient lus (issue #128) ; un nœud qui diffère sans qu'aucun de ses enfants diffère est cité « (ordre des champs ou attributs) ». Pour le témoin de la fiche, les deux côtés sont d'abord privés de versions, plateforme, empreintes/resultat, empreintes/code et

empreintes/statut_resultat_code (perimetre_temoin_fiche()).

Ce que dit chaque ligne (E2.20). Les lignes 1 et 2 sont des contrôles de forme, exacts ; les lignes 3 à 5 comparent exactement des libellés (version, nom de l'algorithme, en-tête de la représentation), non des empreintes ; la ligne 6 dit seulement que les contrôles des modules n'ont rien refusé, sans rien ajouter à ce qu'ils vérifient ; la ligne 7 est l'égalité exacte de deux MD5 : l'empreinte des données est portable (exacte sous glibc, non mesurée ailleurs ; section 15.2) et, comme toute empreinte du projet, sans rôle de sécurité : elle identifie un contenu, elle ne le protège pas contre une modification délibérée. Les témoins **signalent et ne refusent jamais** :

— **code** : égalité exacte de l'empreinte du code de la fiche et de celle du code installé (section 15.3, « Empreinte du code »), qui identifie le texte source et non le comportement numérique ; rejouer une fiche avec un code corrigé est un usage légitime (analyse de sensibilité d'un certificateur), d'où un signal et non un refus ; « non déterminable » si l'une des deux vaut NA (sources introuvables à l'installation) ;

— **versions** et **plateforme** : comparaison exacte des chaînes ; un écart dit que l'environnement diffère, sans juger de son effet ;

— **fiche** : un écart révèle un code au comportement différent sur ce que la fiche consigne (recette, convention, fin effective, hypothèses, décomptes d'exclusion, avertissements) ; la fiche ne portant ni exposition ni taux, un écart de dernier bit des grandeurs calculées n'y apparaît que par l'empreinte du résultat. La stabilité de ce témoin d'une machine à l'autre n'est pas mesurée directement ; elle a un *constat* indirect sur deux machines Linux x86_64, par les références de non-régression (glibc 2.35 et processeur AMD EPYC 7763 ; glibc 2.39 et processeur Intel Xeon : R1 à R6 identiques sur tout le périmètre du témoin), et n'est pas mesurée hors glibc ;

— **empreinte du résultat** : témoin d'identité sur la même machine seulement (section 15.3) ; « différente » quand les témoins du code, des versions, de la plateforme et de la fiche sont « identique » est un **signal d'écart à E4.6** si la machine est la même, ce que le bilan ne peut pas établir seul : la fiche ne consigne pas le processeur ; sinon c'est un écart possible et attendu (les empreintes du résultat de R4 et R5 diffèrent entre la plateforme de référence et un hôte hors de celle-ci, par un ulp d'une grandeur, section 15.3), et le critère reste la comparaison à tolérance par classes, hors de l'outil ;

— **empreinte du fichier** : recopiée de la provenance par la couche de lecture et non recalculée par le moteur, qui ne lit aucun fichier : toujours « non vérifiée ».

Limites (ADR 0012, point 3.9, et seconde annotation du 24/10/2026, (a) 7 et 8 ; constats de tests/unitaires/test_rejeu.R).

— **L'identité de l'objet ne vaut qu'à colonnes identiques.** Une colonne hors rôles ajoutée aux données ou retirée ne change pas l'empreinte des données, qui ne porte que sur les colonnes des rôles : le rejeu passe, les témoins du code, des versions, de la plateforme, de la fiche et de l'empreinte du résultat valent « identique » (celui du fichier, toujours « non vérifiée »), mais l'objet rendu n'est pas identical() à l'original, qui ne diffère que par etapes/donnees/resultat/lignes_conservees, copie de l'entrée hors du périmètre de l'empreinte du résultat (décision M71).

— **Un plan modifié dans la fiche ne se voit que par ses échos.** La fiche ne porte pas d'empreinte d'elle-même (voulu) : le plan de la fiche reçue est celui que la construction emploie, si bien que la fiche rendue porte le même plan. Une modification du plan n'apparaît au témoin de la fiche que par ce qu'elle change ailleurs dans la fiche (un niveau modifié se lit dans hypotheses_brut/niveau), et à l'empreinte du résultat sur la même machine ; une modification sans écho dans la fiche passerait inaperçue de ce témoin.

— **Lignes permutées** : refusées (empreinte des données discordante) ; le refus est fondé, l'exposition

et les taux n'étant pas identical() sous un autre ordre de sommation (ADR 0012, « Contexte », mesure d'actuary).

— **Colonne de rôle renommée** : refusée (colonne_absente) si les rôles de la fiche ne sont pas adaptés ; adaptés, le rejeu passe, le témoin de la fiche vaut « identique » et celui de l'empreinte du résultat « différente », les rôles étant dans son périmètre.

— **Provenance** : une fiche sans provenance et la même avec provenance se rejouent toutes deux, avec des empreintes du résultat distinctes, fiche$provenance étant dans le périmètre.

— **construire_table() appelée directement ne contrôle pas la version** : une fiche de version non admise s'y exécute sans refus, et lire_fiche() ne la contrôle pas non plus. Seuls le rejeu outillé et l'écriture du rapport figé (section 15.5) appliquent ce contrôle.

— **Le temps (3) n'est pas outillé** : la comparaison à tolérance par classes sur une autre machine demande un artefact de valeurs — le rapport figé, qui les écrit en %.17g, en est un (section 15.5) — et relèverait d'une issue à part, si le besoin est confirmé (ADR 0012, « Conséquences »).

Les textes fixes et composés du bilan sont du texte du moteur, marqué UTF-8 (texte_moteur()) ; les comparaisons portent sur les valeurs brutes. Le bilan du rejeu de R1 est identique octet à octet sous la locale C et sous une locale UTF-8 (constat du contrôle tests/controles/locale_c.R, section 5.1).

6.5 Vérifications

tests/unitaires/test_construire_table.R vérifie la liste des exports, chaque refus de valider_plan() (noms exacts des champs de transition et de période compris), la règle commune de forme (pour chaque paramètre d'étape et chaque champ du plan, une valeur valide munie de noms, d'une dimension ou d'un autre attribut est refusée sous le motif du paramètre, porté par le module qui le contrôle ; issue #23), la cohérence entre ETAPES et les fonctions d'étape appelées, la structure de l'objet et de la fiche, et des identités exactes sur un portefeuille simulé (section 8.4). Il vérifie aussi que les contrôles ne sont pas répétés (issue #22) : le nombre d'exécutions de chaque contrôle, compté par trace() sur l'espace de noms installé, vaut un par construction (plan avec ou sans kaplan_meier, plan réduit à l'étape donnees ; aucun contrôle n'est exécuté au-delà d'un refus de valider_plan()), et un par contrôle que fait chaque fonction d'étape appelée directement, taux_kaplan_meier() n'en faisant aucun ; les refus de l'appel direct gardent leur module et leur motif ; seule la valeur FALSE de controler omet les contrôles ; sur des entrées valides, chaque fonction d'étape rend sous controler = FALSE un objet identical() à celui de l'appel avec contrôle, et chaque valeur dérivée sans contrôle est égale à celle que rend le contrôle correspondant ; une segmentation dupliquée passée à taux_kaplan_meier() arrête le calcul par une erreur de programmation, non par un refus. tests/test_reproductibilite.R vérifie que deux constructions identiques sont identical() hors execution, dans le même processus et dans un autre, et que le rejeu de la fiche redonne le même objet.

Rejeu outillé (section 6.4 ; issue #86). tests/unitaires/test_rejeu.R vérifie le contrat (fonction exportée, arguments donnees et fiche, VERSIONS_FICHE_REJEU identical() à VERSION_FICHE) ; la structure d'un rejeu réussi (classe, colonnes, lignes, libellés et valeurs du bilan ; objet rendu identical() hors execution à l'original ; données et fiche inchangées ; même rejeu avec x$fiche qu'avec la fiche relue de son texte) ; chaque refus, avec son module, son motif, ses details et les valeurs lue et attendue de son message ; l'ordre des contrôles, sur des fiches qui en violent deux à la fois ; pour chaque refus délégué, l'égalité du module, du motif, du message et des details avec ceux de construire_table() appelée directement sur les champs de la fiche ; le nombre d'appels de construire_table(), compté par trace() sur l'espace de noms installé : aucun sur un refus, un sur un rejeu réussi ; les verdicts des témoins sur des fiches modifiées ; les cas adverses de la section 6.4, « Limites », testés tels que constatés ; la citation des valeurs lues dans les messages (VT, FF, NEL et U+2028 écrits <U+XXXX>, message sur une ligne et

marqué UTF-8) et le refus nommé, au lieu d'une erreur R, d'un algorithme ou d'une classe d'encodage `bytes` (issue #94), et d'un nom d'encodage `bytes` au premier niveau de la fiche, en première, quatrième et dernière positions (issue #114) ; pour l'issue #128, un nom d'encodage `bytes` dans `empreintes` (chacune de ses positions) ou une classe de cet encodage sur un champ lu par le rejeu (refus `fiche_non_conforme` à l'étape 2, avec un autre défaut de forme dans le même refus), un nom ou une classe `bytes` dans `versions` et `plateforme` (rejeu rendu, témoin « non déterminable »), un nom `bytes` aux deuxième et troisième niveaux d'autres champs (témoin de la fiche « différente » à son chemin), des noms en double dont un `bytes`, cités un à un, et une colonne de rôle de nom `bytes` (refus `chaine_codage_bytes` au lieu d'une erreur R). `tests/test_reproductibilite.R` rejoue, pour les six plans de référence, la fiche écrite puis relue : objet `identical()` hors `execution` à l'objet de référence, témoins du code, des versions, de la plateforme, de la fiche et de l'empreinte du résultat « identique », empreinte du fichier « non vérifiée ». `tests/controles/locale_c.R` compare les octets du bilan du rejeu de R1, et ceux de messages de refus du rejeu, sous les deux locales.

6.6 Fonctions du module

Table 10 : Définitions de premier niveau de `R/construire_table.R`.

Nom	Statut	Entrées	Sortie et rôle
`CHAMPS_PLAN`	constante	—	les six champs de premier niveau du plan
`CHAMPS_PLAN_REQUIS`	constante	—	les cinq champs toujours requis (tous sauf `graine`)
`ETAPES`	constante	—	étapes : module, fonction d'étape, préalables (colonne liste), tirage d'un aléa
`PARAMETRES_ETAPES`	constante	—	paramètres de chaque étape : nom, type de base exigé (règle commune de forme, section 6.1), domaine
`liste_noms_complets()`	interne	objet	`TRUE` pour une liste nue (seul attribut admis : les noms, par `forme_exacte()`) aux noms complets au sens de `noms_complets()` (module `outils` : présents, non vides et non manquants), ou vide (sans noms admis) ; doublons non contrôlés ; partagée par `liste_nommee()` et par le contrôle du champ `etapes` de `valider_plan()`, qui admet les doublons pour que le refus `etape_en_double` les nomme (issues #22, #23 et #60)
`liste_nommee()`	interne	objet	`TRUE` pour une liste nue (seul attribut admis : les noms) aux noms uniques, non vides et non manquants, ou vide ; `liste_noms_complets()` sans doublon (issues #23 et #60)
`valider_plan()`	exportée	plan	plan inchangé, ou refus (section 6.1)

Nom	Statut	Entrées	Sortie et rôle
ordres_canoniques()	interne	—	vecteurs d'ordre des trois champs ensembles du plan, passés à canoniser_plan() : segmentation (ROLES_SEGMENTATION), etapes/donnees/exclusions (c(MOTIFS_LIGNES, MOTIFS_TETE)), etapes/brut/estimateurs (ESTIMATEURS_BRUT) ; fonction et non constante, construite à l'appel (section 15.1)
construire_table()	exportée	données, rôles, plan, provenance	objet construction (section 6.3) ; empreinte du résultat posée avant execution
texte_lu()	interne	valeur lue	« absente » (NULL), la chaîne (chaîne unique sans attribut, NA écrit « NA ») ou « illisible (type de longueur n) », longueur lue sans la classe si celle-ci est d'encodage bytes (issue #128), pour un message ou le bilan du rejeu
sans_controle()	interne	texte	valeur citée selon la règle unique des refus (section 2.1 : <U+XXXX>, <xx>, repère bytes, marque UTF-8 ; issue #94) ; messages de refus du rejeu seulement
decrire_forme()	interne	conteneur, nom	forme lue d'un champ pour fiche_non_conforme : « champ absent », ou type et longueur complétés de NA, vide, attributs, noms en double, noms incomplets, nom(s) d'encodage bytes ; valeur de classe bytes décrite sans sa classe (section 6.4 ; issue #128)
paires_lues()	interne	liste	paires « nom=valeur » jointes par « ; », noms cités par la règle unique (repère pour un nom bytes), ou texte_lu() d'une autre valeur, liste de classe bytes comprise ; colonnes du bilan pour versions et plateforme (issue #128)
champs_differents()	interne	fiche rendue, fiche reçue, racine	chemins des feuilles qui diffèrent, par descente sur l'union des noms ; une liste à nom d'encodage bytes est une feuille (issue #128) ; absent d'un côté, ou « (ordre des champs ou attributs) » ; vide si identical()
verdict_chemins()	interne	fiche rendue, fiche reçue, racine	verdict V3 : « identique » ou « différente : » et les chemins
verdict_chaines()	interne	attendu, lu	verdict V1 : « non déterminable » si l'un n'est pas une chaîne unique, sinon « identique » ou « différente »

Nom	Statut	Entrées	Sortie et rôle
verdict_liste()	interne	liste rendue, liste reçue, racine	verdict V2 : « non déterminable » si la liste reçue n'est pas une liste sans classe aux noms complets ou porte un nom d'encodage bytes, sinon verdict_chemins()
perimetre_temoin_fiche()	interne	fiche	fiche privée de versions, plateforme et des trois dernières empreintes, périmètre du témoin de la fiche
rejouer_fiche()	exportée	données, fiche	objet de classe rejeu (construction, bilan) ; contrôles, puis construire_table() une fois ; refus nommés du module construire_table (section 6.4)

PARTIE III

Module donnees

Données, rôles et contrôles : R/donnees.R

Le module donnees reçoit la base d'observation, un data.frame, et la déclaration des rôles de ses colonnes. Il contrôle les rôles, évalue les motifs d'invalidité de chaque ligne, applique les règles d'exclusion déclarées par le plan, déduit l'unité d'observation et fusionne les intervalles d'observation d'une même tête (E2.2, E2.4, E2.7). Il ne modifie jamais les données : une ligne invalide arrête la construction, sauf règle d'exclusion déclarée, et rien n'est corrigé (E2.7). Sa fonction d'étape est preparer_donnees() ; il porte aussi le catalogue des rôles (catalogue_roles()) et la représentation commune d'une table de référence (table_reference()), toutes deux exportées. Le contenu de fond (catalogue, motifs, règles de fusion) vient de la spécification d'actuary du 16/10/2026, versée dans l'issue #11 (parties A.0 et A.1).

7.1 Conventions de temps

Trois conventions de temps sont communes à toutes les recettes (E2.3) ; elles sont appliquées par donnees et par exposition (section 8), qui ajoute l'unité de l'exposition et la fenêtre d'observation.

— **Une date est un jour** (classe Date, sans heure ni fuseau). Un intervalle d'observation est l'intervalle d'instants semi-ouvert $[\text{entrée}, \text{sortie})$: date_entree est l'instant où l'observation commence, date_sortie celui où elle cesse (jour du décès ou du rachat, ou lendemain du dernier jour couvert pour une fin de garantie). Un événement daté du jour s survient à l'instant s : la tête est exposée jusqu'à s, jour s exclu. Une ligne en_cours est ouverte à droite. *Choix de conception* : il donne des décomptes entiers et additifs, au prix d'une sous-estimation de l'exposition d'un demi-jour par événement en moyenne.

— **Âge au dernier anniversaire.** L'âge au jour t d'une tête née le jour n est le nombre d'années révolues

$$\text{age}(n,t) = \text{an}(t) - \text{an}(n) - \mathbb{1}\{(\text{mois}(t), \text{jour}(t)) < (\text{mois}(n), \text{jour}(n))\},$$

comparaison lexicographique, appliquée sans cas particulier (age_revolu()). Pour une naissance le 29 février, l'anniversaire tombe donc le 1er mars des années non bissextiles : le 28 février d'une telle année, $(2, 28) < (2, 29)$ et l'année n'est pas révolue. Une année d'âge dure ainsi 365 ou 366 jours. Convention de calcul, conséquence de la formule (spécification, A.0, T2), validée par actuary le 17/10/2026, d'impact négligeable (un jour d'exposition déplacé entre deux âges, pour les seules naissances du 29 février) ; aucune source juridique n'est établie à son sujet. Elle est appliquée par trois arithmétiques de dates écrites indépendamment (modules donnees et exposition, simulateur), dont l'accord est vérifié (section 8.4).

— **Période d'observation et délai de déclaration.** La période [début, fin] est déclarée par le plan, fin incluse, jamais déduite des données (E2.5). Le délai de déclaration est un nombre entier de jours ≥ 0, sans arithmétique de mois. La fin effective est fin $-$ délai et la borne d'instant B = fin effective $+ 1$: l'observation s'arrête à l'instant B. Une période dont le début est postérieur à la fin effective est refusée (`periode_vide`).

7.2 Catalogue des rôles

Le moteur ne lit une colonne que par son *rôle*, déclaré par l'utilisateur dans un catalogue fermé (E2.2 ; ADR 0003, point 2). Les rôles sont transmis à `construire_table()` sous la forme d'un vecteur de chaînes nommé : noms = rôles du catalogue, valeurs = noms de colonnes de la base (ADR 0007, point 4.1). Les colonnes sans rôle sont ignorées. `catalogue_roles()`, exportée pour que l'application propose la déclaration des rôles, rend le catalogue sous la forme d'un `data.frame` de colonnes `role`, `famille`, `type_admis`, `usage` et `recettes` (tableau 11).

La colonne *usage* décrit l'emploi du rôle par la seule recette d'exposition disponible, `lignes_dates_completes` (section 8.1) :

— `requis` : rôle sans lequel aucune recette ne s'applique (refus `aucune_recette`) ;

— `facultatif` : rôle lu pour des contrôles (unicité du contrat, cohérence de l'année de naissance) ou, pour `id_tete`, pour l'unité d'observation ;

— `segmentation` : rôle requis si la segmentation du plan l'emploie (refus `role_requis_manquant`), contrôlé s'il est seulement déclaré, avec l'avertissement `role_non_employe` ;

— `non_pris_en_charge` : rôle qu'aucune recette disponible n'emploie et dont l'omission changerait le sens du résultat (pondération, agrégats, états, date de survenance distincte de la sortie, date d'inventaire, âge fourni) ; sa déclaration est **refusée** (`role_non_pris_en_charge`) plutôt qu'ignorée en silence. Le rôle `montant` est de ceux-là : la pondération par les montants attend la spécification de ses intervalles (E2.8).

Les types sont contrôlés sans conversion (E4.4) : une colonne de dates doit être de classe `Date`, exactement, à valeurs entières et finies ; une colonne `POSIXct`, une chaîne ou un nombre sont refusés (`type_incompatible`), la conversion revenant à la couche de lecture. Au contrôle de type, une valeur manquante, stockée `NA` ou `NaN`, est admise (les motifs de lignes en décident ensuite), et les deux restent distinctes dans l'empreinte (section 15.2). Une date de type conforme doit en outre appartenir au **domaine** [0001-01-01, 9999-12-31] : hors de lui, aucune écriture ISO 8601 à quatre chiffres d'année n'existe, et la date est refusée (`date_hors_domaine`, tous les rôles en cause cités avec leurs lignes) avant tout calcul, empreinte comprise (`dates_hors_domaine()` ; ADR 0011, point 4 ; décision M64, Q7 ; issue #19). Les années 1 à 999 sont admises. Le motif `age_aberrant` mesure l'âge à la plus petite des dates de sortie et de fin de période : une ligne née avant l'an 1000 et observée dans une période postérieure à l'an 1129 a au moins 130 ans à cette date et est relevée ; une ligne entièrement antérieure à la période ne l'est pas, mais elle n'a aucune intersection avec la période. Les modalités des rôles catégoriels sont comparées après conversion en chaîne, en respectant la casse : `H` et `F` pour `sexe` ; `deces`, `rachat`, `terme`, `autre` et `en_cours` pour `cause_sortie` (constantes `MODALITES_SEXE` et `MODALITES_CAUSE`). Un recodage (par exemple `1/2` vers `H/F`) relève de la couche de lecture.

Table 11 : Catalogue fermé des rôles (`catalogue_roles()` ; constantes `CATALOGUE_ROLES` et `TYPES_ADMIS`). La colonne `recettes` du catalogue vaut `lignes_dates_completes` pour tout rôle pris en charge, vide sinon.

Rôle	Famille	Types R admis	Usage
`id_contrat`	identification	`character`, `integer`, `double` entier fini	facultatif
`id_tete`	identification	`character`, `integer`, `double` entier fini	facultatif
`sexe`	état civil	`character`, `factor`	segmentation
`date_naissance`	état civil	`Date` (valeurs entières finies)	requis
`annee_naissance`	état civil	`integer`, `double` entier fini	facultatif
`age_fourni`	état civil	`integer`, `double` entier fini ≥ 0	non pris en charge
`date_age_fourni`	état civil	`Date` (valeurs entières finies)	non pris en charge
`date_entree`	observation	`Date` (valeurs entières finies)	requis
`date_sortie`	observation	`Date` (valeurs entières finies ; manquante admise pour une ligne `en_cours`)	requis
`cause_sortie`	observation	`character`, `factor`	requis
`date_evenement`	observation	`Date` (valeurs entières finies)	non pris en charge
`date_inventaire`	observation	`Date` (valeurs entières finies)	non pris en charge
`age`	agrégats	`integer`, `double` entier fini	non pris en charge
`annee`	agrégats	`integer`, `double` entier fini	non pris en charge
`exposition`	agrégats	`double` fini ≥ 0	non pris en charge
`evenements`	agrégats	`double` fini ≥ 0	non pris en charge
`montant`	pondération	`double` fini > 0	non pris en charge
`segment`	segmentation	`character`, `factor`, `integer`	segmentation
`etat_depart`	multi-états	`character`, `factor`	non pris en charge
`etat_arrivee`	multi-états	`character`, `factor`	non pris en charge
`date_changement`	multi-états	`Date` (valeurs entières finies)	non pris en charge

7.3 Contrôles, refus et exclusions

Refus au niveau des rôles et des paramètres

`controler_roles()` contrôle la déclaration des rôles dans cet ordre, et chaque refus nomme tous les rôles et colonnes en cause : `donnees_mal_formees` (l'entrée n'est pas un `data.frame`) ; `roles_mal_formes` (les rôles ne sont pas un vecteur de chaînes nommé, de forme exacte `character` sans autre attribut que ses noms — une classe, `noquote` ou `AsIs` comprises, une dimension ou un attribut quelconque sont refusés, par `forme_exacte()` —, sans valeur ni nom manquant ni nom vide ; `details$classe` et `details$attributs` donnent la classe et les noms des attributs reçus, qui désignent la cause quand elle tient à une classe, une dimension ou un attribut, mais non quand elle tient à une valeur ou à un nom manquant ou vide ; issue #62) ; `role_inconnu` ; `role_declare_deux_fois` ; `role_non_pris_en_charge` ; `colonne_absente` ; `colonne_ambigue` (le nom de colonne figure plusieurs fois dans la base : lire la première sans le dire serait deviner) ; `colonne_declaree_deux_fois` (une colonne pour deux rôles) ; `type_incompatible` (une colonne de classe d'encodage `bytes` y est refusée comme toute classe non admise, et une colonne de nom `bytes` y est jugée sur son type, nom et classe cités sous le repère, sans erreur R ; un tel nom est refusé plus loin par le module `rapport`, `chaine_codage_bytes` ; section 2.1 ; issue #128) ; `date_hors_domaine` (une ou plusieurs colonnes de date portent une date hors de [0001-01-01, 9999-12-31] ; le refus nomme tous les rôles et colonnes en cause et, dans `details$lignes`, les lignes de chacun, comme `ligne_invalide`) ; `aucune_recette` (un rôle requis manque ; le message nomme la recette et les rôles absents) ; `role_requis_manquant` (un rôle employé par la segmentation n'est pas

déclaré).

controler_parametres_donnees() contrôle les champs du plan que l'étape reçoit, les cinq premiers motifs ci-dessous par controler_transition_periode(), qui contrôle la transition, la période et le délai et que la fonction d'étape brut appelle aussi pour kaplan_meier en appel direct (section 9.1), les suivants par elle-même : transition_inconnue (liste qui n'est pas nue — un data.frame ou une liste munie d'un autre attribut que ses noms —, de champs autres qu'exactement depart et arrivee, ou d'un élément de plus, même nommé NA ; état qui n'est pas une chaîne unique sans attribut, ni NA ni vide, refusé dès ce contrôle de forme ; ou transition non prise en charge : seules vivant → décédé et en cours → racheté le sont, constante TRANSITIONS) ; periode_invalide (liste qui n'est pas nue, de champs autres qu'exactement debut et fin, ou d'un élément de plus ; bornes manquantes, non Date, non entières ou munies d'un autre attribut que la classe Date) ; date_hors_domaine (borne de la période hors de [0001-01-01, 9999-12-31], même domaine que les données ; details$bornes nomme les bornes en cause) ; delai_invalide (délai qui n'est pas un nombre unique, integer ou double, sans attribut, entier et ≥ 0) ; periode_vide ; segmentation_invalide (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, pris dans sexe et segment, vide explicite admis ; contrôle controler_segmentation(), partagé avec la fonction d'étape calculer_taux_bruts() en appel direct, section 9.1) ; exclusions_invalides (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, vide explicite admis) ; exclusion_inconnue (code absent de la liste fermée des motifs). Les deux derniers sont contrôlés dans cet ordre : exclusions_invalides l'emporte, si bien qu'un vecteur nommé ou en double de codes inconnus est refusé sous ce motif et non sous exclusion_inconnue. Les noms de champs sont lus exactement, jamais par appariement partiel. Le même contrôle est appelé par valider_plan(), avant toute étape : un code désigne une condition, quelle que soit la fonction qui la détecte, et le refus porte le module donnees (ADR 0007, annotation du 17/10/2026, point d). Dans une construction, il n'est appelé que là, une fois : preparer_donnees(), appelée avec controler = FALSE, ne contrôle ni ces paramètres ni les rôles, déjà contrôlés, et reprend la transition retenue, les bornes de la période et la borne B par valeurs_parametres_donnees() ; appelée directement, elle appelle ce contrôle, puis controler_roles() (section 6.2).

Motifs d'invalidité des lignes

Les motifs forment une liste fermée (E2.7), évaluée par evaluer_motifs() (tableau 12).

— **Exhaustivité** : tous les motifs de ligne sont évalués sur toutes les lignes, sans court-circuit ; un motif de comparaison n'est pas levé si l'une de ses dates est manquante (le motif de la valeur manquante l'est) ;

— **motifs de tête** : évalués, quand id_tete est déclaré, sur les lignes exemptes de motif de ligne (lignes *propres*), ils marquent les lignes propres de la tête, et elles seules : une ligne déjà porteuse d'un motif de ligne n'est pas comptée de nouveau sous le motif de tête. Les lignes conservées n'en dépendent pas, puisqu'une telle ligne est de toute façon refusée ou exclue au titre de son motif de ligne ; seul le décompte du motif de tête, dans le refus et dans les diagnostics, l'omet (constat : une tête de trois lignes dont la première porte annee_naissance_incoherente et dont les naissances diffèrent donne tete_naissance_incoherente sur les lignes 2 et 3 seulement, et le refus annonce deux lignes pour ce motif) ; le rapport est ainsi complet en un seul passage ;

— **décompte** : une ligne peut porter plusieurs motifs ; le décompte par motif compte les lignes porteuses, et le nombre de lignes distinctes est donné à part.

Table 12 : Motifs d'invalidité, dans l'ordre de restitution (constantes MOTIFS_LIGNES et MOTIFS_TETE). Ce sont aussi les codes admis comme règles d'exclusion.

Code	Condition
Motifs de ligne	
id_contrat_manquant	id_contrat déclaré et manquant
id_tete_manquant	id_tete déclaré et manquant
naissance_manquante	date de naissance manquante
entree_manquante	date d'entrée manquante
sortie_manquante	date de sortie manquante et cause autre que en_cours
cause_manquante	cause de sortie manquante
cause_hors_modalites	cause hors des cinq modalités
sexe_manquant	sexe déclaré et manquant
sexe_hors_modalites	sexe déclaré, hors de H et F
segment_manquant	segment déclaré et manquant
naissance_apres_entree	naissance postérieure à l'entrée (l'égalité est admise)
sortie_avant_entree	sortie antérieure à l'entrée (l'égalité est admise : durée nulle)
en_cours_avant_fin	cause en_cours et date de sortie renseignée antérieure à la fin déclarée : la base ne couvre pas la période déclarée. Une ligne en_cours dont la date de sortie renseignée précède l'entrée relève de sortie_avant_entree et, si sa sortie précède la fin déclarée (toujours le cas quand son entrée ne la dépasse pas), de ce motif-ci : elle se contredit, le moteur ne choisit pas entre la cause et les dates et refuse (E2.7) ; elle s'écarte par une exclusion déclarée de chacun des motifs qu'elle porte (exclure le seul sortie_avant_entree laisse le refus sous en_cours_avant_fin si elle le porte ; issue #20)
annee_naissance_incoherente	annee_naissance déclarée et différente de l'année de la date de naissance
age_aberrant	âge au dernier anniversaire au moins égal à AGE_ABERRANT (130) à la plus petite des dates de sortie et de fin de période (à la fin si la sortie manque) ; constante de catalogue, *choix de conception* : la borne laisse passer toute durée de vie humaine connue, elle n'est pas un paramètre du plan
contrat_duplique	id_contrat déclaré et présent sur plusieurs lignes (toutes les lignes du groupe sont listées)
Motifs de tête (si id_tete est déclaré)	
tete_naissance_incoherente	dates de naissance différentes entre lignes d'une même tête
tete_sexe_incoherent	sexes différents au sein d'une tête (sexe déclaré)
tete_segment_incoherent	segments différents au sein d'une tête, si la segmentation emploie segment
tete_observee_apres_deces	le premier décès s_d de la tête est antérieur à B et une ligne de la tête a une fin d'observation postérieure à s_d (lecture B de E2.4, ci-dessous)

Refus par défaut et exclusions déclarées

Une ligne qui porte un motif **non déclaré** dans les règles d'exclusion du plan arrête la construction (refus ligne_invalide), même si elle porte aussi un motif déclaré. Le message nomme, pour chaque motif en cause, le nombre de lignes et leurs dix premiers numéros (NB_LIGNES_CITEES, numéros 1 à n des lignes du data.frame reçu), puis le total des lignes distinctes ; la condition porte la liste complète (E2.7, E5.5).

Le plan peut déclarer des *règles d'exclusion*, paramètre exclusions de l'étape donnees : un vecteur de codes de motif sans doublon, vide explicite admis. Une ligne est exclue si l'un de ses motifs est déclaré. Les exclusions sont comptées et listées par motif dans le résultat de l'étape (resultat$exclusions,

`resultat$lignes_exclues`) et chacune donne un avertissement `exclusion` ; une ligne exclue sous plusieurs motifs est comptée sous chacun, et le nombre de lignes distinctes exclues est donné à part (`resultat$nombre_lignes_exclues`, repris par la fiche, section 15.1). Si toutes les lignes sont exclues, la construction est refusée (`aucune_ligne_conservee`) ; une base sans ligne l'est aussi (`base_vide`). Le diagnostic `diagnostics$motifs` restitue, pour chaque motif, sa nature (ligne ou tête), son décompte et s'il est exclu.

Les lignes conservées sont celles de l'entrée : `donnees` lui-même s'il n'y a aucune exclusion, sinon le sous-ensemble de ses lignes, sans aucune modification de valeur ; les quantités dérivées (observations fusionnées) sont rangées à part. Une réserve technique : l'extraction des lignes conservées par l'opérateur d'indexation de R retire des colonnes les attributs autres que la classe et les niveaux d'un facteur (par exemple `tzone`, `label` ; constat sur R 4.3.3) ; les valeurs et les classes ne changent pas, et le moteur ne lit pas ces attributs.

7.4 Unité d'observation et fusion par tête

Unité d'observation, déduite des rôles

L'unité d'observation n'est pas un paramètre du plan : le moteur la déduit des rôles et de la transition, comme il déduit la recette (décision M48 ; ADR 0007, annotation du 17/10/2026, point a), et l'inscrit dans les paramètres effectifs de l'étape et dans la fiche :

— **tête**, si l'état de départ de la transition est un état de la personne (vivant → décédé) et que `id_tete` est déclaré ;

— **contrat** sinon : transition dont l'état de départ est un état du contrat (en cours → racheté, décision M46), ou transition de tête sans `id_tete`. Dans ce second cas, l'avertissement `unite_contrat` est inscrit dans l'objet et repris dans la fiche : « une tête titulaire de plusieurs contrats est comptée plusieurs fois, ce qui surestime l'exposition et peut compter plusieurs fois un même décès » (cas F4 des tests : deux décès comptés au lieu d'un).

Un paramètre d'unité serait soit redondant avec les rôles, soit en contradiction avec eux : une tête déclarée « contrat » alors que `id_tete` est présent compterait plusieurs fois un même décès. Pour le rachat, l'unité est le contrat ; l'identifiant de tête sert alors aux contrôles de cohérence et au futur rééchantillonnage par têtes (E2.4, E2.14) : il est gardé dans la table des observations.

Fusion des intervalles d'une tête

En unité tête, les intervalles $[\text{entrée}, \text{fin})$ des lignes d'une même tête (fin $= +\infty$ pour une ligne `en_cours`) sont triés par entrée et fusionnés quand entrée suivante $\leq$ fin courante, ce qui couvre le chevauchement et la contiguïté exacte (`fusionner_intervalles()`). **Aucune tolérance « à un jour près »** : un écart d'un jour reste un trou non exposé, et la tête est de nouveau tronquée à gauche à sa réentrée ; combler l'écart inventerait une exposition. Un tel écart trahit souvent une date de fin inclusive dans la base : il est relevé et décompté à part, non corrigé (*choix de conception*). La cause de fin d'une composante fusionnée est `deces` si une ligne de décès se termine à cette fin, `en_cours` si la fin est $+\infty$, `censure` sinon ; le décès n'est compté qu'une fois.

Le calcul de la fusion est vectorisé par un maximum cumulé décalé par tête, exact en `double` tant que (nombre de têtes) $\times$ (largeur du décalage) reste inférieur à 2^{52}, la largeur valant $\max - \min + 2$ jours sur les dates d'entrée et de sortie (`fusionner_intervalles()`). Au-delà, le moteur refuse (`dates_hors_plage`, dont le message cite l'étendue et le nombre de têtes) au lieu de fusionner inexactement. Depuis le refus des dates hors du domaine (issue #19), qui compte 3 652 059 jours, la largeur est au plus de 3 652 060 : par

construire_table(), le refus n'est atteint qu'au-delà d'environ $1{,}23 \cdot 10^9$ têtes ($2^{52}/3\,652\,060$), sous le plus grand entier de R ; la garde est conservée (avis d'actuary), une étendue plus grande restant possible par un appel direct de la fonction de fusion. C'est une limite d'implémentation, non un motif de ligne : elle n'est pas excluable.

Observation après le décès : lecture B de E2.4

Une tête dont un contrat est observé après son décès a une trajectoire indéterminée, et ses lignes sont refusées (E2.4). Le décès doit cependant être connu : le mainteneur a retenu une *lecture B* (décision M48) : le motif tete_observee_apres_deces n'est levé que si le premier décès s_d de la tête est **antérieur à la borne** B de la période effective et qu'une fin d'observation d'une ligne de la tête le dépasse. Une ligne en_cours compte pour $+\infty$, quelle que soit sa date de sortie, et une sortie manquante aussi (deces_par_tete()). Deux lignes closes par le décès le même jour sont cohérentes.

Si $s_d \geq B$, le décès est hors de la période effective : aucune ligne n'est refusée, mais le relevé observations_apres_deces_hors_periode compte ces têtes et donne un avertissement du même code. Ce relevé est calculé **sur les seules lignes conservées** : une tête écartée par une exclusion déclarée n'y est pas comptée, une tête dont une ligne seulement est exclue l'est sur ses lignes conservées. Il est calculé dès que id_tete est déclaré, **quelle que soit l'unité**, donc aussi en unité contrat pour le rachat (contrôle par identifiant de tête, E2.4, M46, M48) ; sans id_tete, il n'est pas calculable et vaut NA.

Relevés et avertissements

L'étape restitue dans diagnostics$releves, sans bloquer, le nombre de têtes et de têtes multi-contrats, de chevauchements, de contiguïtés, d'intervalles disjoints, d'écarts d'un jour, de fenêtres vides (observations sans intersection avec la période), d'observations, et le relevé après décès hors période. Les relevés de fusion valent NA sans id_tete. Ses avertissements sont : unite_contrat ; role_non_employe ; exclusion ; chevauchements, disjoints et ecarts_un_jour, en unité tête seulement ; fenetres_vides et observations_apres_deces_hors_periode, dans toutes les unités. Les contiguïtés sont décomptées sans avertissement : une contiguïté exacte est fusionnée sans perte.

Le résultat de l'étape (resultat) contient les lignes conservées et leurs numéros, les exclusions, l'unité retenue, la table des observations (une ligne par composante fusionnée ou par contrat : identifiant, numéro de la première ligne, nombre de lignes, date de naissance, sexe s'il est déclaré, segment s'il est employé, début, fin, cause) et, pour chaque ligne conservée, le numéro de son observation. Les paramètres effectifs (parametres) sont le code interne de la transition (vivant_decede ou en_cours_rachete), l'unité d'observation, la segmentation, les exclusions et la fin effective.

Copie des colonnes dans la table des observations (issue #67 ; décisions M71, point 4, et M72 ; ADR 0011, septième annotation du 20/10/2026, (e), et huitième). La table des observations est une quantité dérivée : les colonnes qu'elle recopie de l'entrée (identifiants de contrat et de tête, date de naissance, sexe, segment) y sont **normalisées**, l'entrée et sa copie lignes_conservees restant intactes, si bien que rien n'est corrigé dans les données (E2.7). Une chaîne, un entier ou une date est recopié par sous-indiçage (première ligne de chaque composante en unité tête, chaque ligne conservée en unité contrat), dans les deux unités, ce qui garde valeurs, type et classe et retire les attributs de colonne de l'entrée (par exemple label). Un facteur sexe ou segment est reconstruit par la fonction locale categorie de preparer_donnees() : codes sous-indicés, niveaux tels quels (ordre et niveaux inutilisés compris), classe ramenée à factor, ou à c("ordered", "factor") s'il est ordonné ; contrasts, sous-classe et tout autre attribut sont retirés. La table des observations porte des noms de lignes automatiques, même si une colonne d'identifiant porte l'attribut names (par exemple dans un data.frame construit par list2DF()), dont data.frame() tirerait sinon ses noms de lignes (row.names = NULL dans les deux unités). Motif : l'empreinte du résultat écrit

la table des observations et arrête le calcul, par une erreur de programmation, sur tout attribut ou toute classe hors de sa grammaire (section 15.3) ; sans cette normalisation, `construire_table()` échouait sur des entrées valides : attribut `label` sur `id_contrat`, `id_tete` ou `date_naissance` en unité contrat, où ces colonnes étaient recopiées entières, ou facteur muni de `contrasts` ou de sous-classe, que le contrôle des types des rôles admet (section 7.2). Option écartée : refuser ces facteurs à la validation, ce qui resserrerait le contrat d'entrée pour une métadonnée sans effet sur le résultat. Pour les entrées déjà acceptées, la table des observations est inchangée : celles des plans R1 à R6 sont restées identiques à la régénération des références qui a suivi (section 4.5).

7.5 Table de référence

Toute table de référence — loi vraie synthétique du simulateur, table embarquée, table importée — est lue par le moteur sous une seule forme : la classe S3 `table_reference`, sur un `data.frame` en format long (ADR 0007, point 7.1, et annotation du 17/10/2026, point b), que construit et contrôle `table_reference(x, nom, statut, source, parametres, forme_origine)`, exportée pour l'import de tables par l'application :

— colonnes `age` (entier ≥ 0), `sexe` (`H`, `F`, ou `ensemble` pour une table sans distinction de sexe, modalité qui ne coexiste jamais avec `H` ou `F`, dans la table ou au sein d'une même année), `qx` (double fini dans $[0, 1]$) et, facultative, `annee` (entier), pour les tables prospectives de la v2 ;

— âges sans doublon, croissants dans l'ordre reçu et contigus, au sein de chaque sexe (et de chaque année) ;

— attributs `nom`, `statut` (`synthetique`, `embarquee` ou `importee`), `source` (arrêté, URL, fichier de provenance), `forme_origine` (`qx` ou `lx`, défaut `qx`), chacun une chaîne unique de forme exacte — sans attribut, noms compris, ni `NA` ni vide (`chaine_unique()`, section 2.3 ; une chaîne nommée est refusée sous le motif du champ ; issue #48) —, et `parametres` : une liste non vide est **requise** pour une table synthétique, qui n'a d'autre provenance que sa loi, et **interdite** pour une table embarquée ou importée, qu'elle ferait passer pour dérivée d'une loi.

Le constructeur refuse au lieu de corriger, avec un code par contrôle : `table_nom_manquant`, `table_statut_inconnu`, `table_source_manquante`, `table_parametres_manquants`, `table_parametres_inattendus`, `table_forme_inconnue`, `table_mal_formee`, `table_colonne_manquante`, `table_colonne_inconnue`, `table_vide`, `table_colonne_mal_typee`, `table_sexe_hors_modalites`, `table_sexes_melanges`, `table_qx_non_fini`, `table_qx_hors_bornes`, `table_age_double`, `table_ages_non_croissants` et `table_ages_non_contigus`.

7.6 Vérifications

`tests/unitaires/test_donnees.R` vérifie le catalogue, chaque refus de rôle et de paramètre, chaque motif et les règles d'évaluation, les exclusions (lignes conservées identiques à l'entrée), la lecture B de E2.4 au jour près de la borne B (délais 0 et 30 jours) et le relevé après décès hors période (lignes conservées, deux unités), la table de référence et ses refus. Les cas de fusion F1 à F7 de la spécification (chevauchement, contiguïté, écart d'un jour, décès sur deux contrats, observation après décès, deux décès, naissances discordantes) ont des attendus calculés à la main ; la fusion est vérifiée en outre contre un second algorithme, une boucle naïve écrite dans le test. Le domaine des dates est vérifié à ses bornes : 0001-01-01 et 9999-12-31 admis, la veille et le lendemain (années 0 et 10 000) refusés (`date_hors_domaine`), pour les colonnes de date comme pour les bornes de la période ; la garde `dates_hors_plage` l'est à l'étendue maximale du domaine, de part et d'autre du nombre de têtes limite.

7.7 Fonctions du module

Table 13 : Définitions de premier niveau de R/donnees.R.

Nom	Statut	Entrées	Sortie et rôle
CATALOGUE_ROLES	constante	—	catalogue fermé des rôles (rôle, famille, code de type, usage)
TYPES_ADMIS	constante	—	libellé des types R admis par code de type
RECETTE_BRANCHE	constante	—	seule recette d'exposition disponible : lignes_dates_completes
MODALITES_SEXE	constante	—	H, F
MODALITES_CAUSE	constante	—	deces, rachat, terme, autre, en_cours
ROLES_SEGMENTATION	constante	—	sexe, segment
AGE_ABERRANT	constante	—	130 (motif age_aberrant)
dates_hors_domaine()	interne	jours depuis 1970-01-01	vrai hors de $[-719162, 2932896]$, soit hors de [0001-01-01, 9999-12-31], infinis compris, faux pour NA et NaN ; fonde le refus date_hors_domaine
TRANSITIONS	constante	—	transitions prises en charge : départ, arrivée, code interne, porteur de l'état de départ (tête ou contrat)
MOTIFS_LIGNES	constante	—	motifs de ligne (tableau 12)
MOTIFS_TETE	constante	—	motifs de tête (tableau 12)
NB_LIGNES_CITEES	constante	—	10 numéros de ligne cités par motif dans un message de refus
catalogue_roles()	exportée	—	catalogue des rôles (tableau 11)
composantes_date()	interne	jours depuis 1970-01-01	année, mois, jour
age_revolu()	interne	naissance, date	âge au dernier anniversaire (section 7.1)
entiers_finis()	interne	vecteur	valeurs non manquantes entières et finies
type_conforme()	interne	colonne, code de type	conformité au type du catalogue, sans conversion ; dimension lue par attr() et facteur par a_classe(), une colonne de classe d'encodage bytes étant non conforme sans erreur R (issue #128)
citer()	interne	vecteur (converti par as.character() s'il n'est pas de type character)	chaînes de l'utilisateur citées dans un message, par neutraliser_citation(), règle unique des refus (section 2.1) : vecteur de même longueur, sans noms ni paste, NA conservé, chaîne bytes remplacée par le repère « <chaîne d'encodage bytes> » (sprintf() lèverait sur elle au lieu du refus), caractères de contrôle et séparateurs de ligne écrits <U+XXXX>, octets hors UTF-8 valide <xx>, latin-1 converti, marque UTF-8 (issues #111 et #94)

Nom	Statut	Entrées	Sortie et rôle
liste_guillemets()	interne	chaînes	liste entre guillemets, jointe par « , », pour les messages, chaque chaîne passée d'abord par citer() ; guillemets marqués par texte_moteur() avant le collage, si bien que le résultat est un texte formé, admis tel quel par formuler() sans nouvelle citation (section 2.1, « Règle de rédaction des messages » ; issues #111 et #127)
message_aucune_recette()	interne	rôles requis, manquants	message du refus aucune_recette, partagé avec exposition
controler_roles()	interne	données, rôles, segmentation	refus au niveau des rôles, dont roles_mal_formes (forme exacte : seul attribut admis, les noms ; issue #62), type_incompatible (colonnes lues par lire_champ() : atteint, au lieu d'une erreur R, pour une colonne de classe d'encodage bytes, ou de nom bytes et de type non conforme ; issue #128) et date_hors_domaine ; rend les rôles inchangés
controler_segmentation()	interne	segmentation, étape appelante	seul endroit où s'écrivent la forme et le catalogue de la segmentation : vecteur character sans attribut, sans NA ni doublon, dont chaque clé est prise dans ROLES_SEGMENTATION, vide explicite admis ; sinon refus segmentation_invalide au nom de l'étape appelante (donnees ou brut) ; rend NULL ; appelée par le contrôle des paramètres de l'étape donnees et, en appel direct (controler = TRUE), par la fonction d'étape brut (issues #22 et #59)
ligne_transition()	interne	transition (list(depart, arrivee), de forme déjà contrôlée)	ligne de TRANSITIONS de la transition, zéro ligne si elle n'est pas prise en charge ; seul endroit où s'écrit la recherche (issue #22)
valeurs_parametres_donnees()	interne	transition, période, délai, déjà contrôlés	transition retenue (ligne_transition()), début, fin, borne B (jours depuis 1970-01-01) ; ne contrôle rien ; employée par les fonctions d'étape appelées avec controler = FALSE et par taux_kaplan_meier() (issue #22)

Nom	Statut	Entrées	Sortie et rôle
controler_transition_periode()	interne	transition, période, délai	refus de la transition, de la période (dont date_hors_domaine pour ses bornes), du délai et de la période vide, dans cet ordre ; rend la transition retenue, le début, la fin et la borne B, calculés par la fonction sans contrôle de la ligne précédente ; appelée par le contrôle des paramètres de l'étape donnees et, pour kaplan_meier en appel direct, par la fonction d'étape brut (issue #22)
controler_parametres_donnees()	interne	transition, période, délai, segmentation, exclusions	refus de la transition, de la période et du délai par le contrôle de la ligne précédente, puis de la segmentation et des exclusions ; transition retenue, début, fin, borne B
groupes_heterogenes()	interne	groupes, valeurs	groupes où la valeur n'est pas constante
extremes_par_groupe()	interne	groupes, valeurs	minimum et maximum par groupe
deces_par_tete()	interne	têtes, sorties, causes	premier décès de chaque tête et observation au-delà
evaluer_motifs()	interne	données, rôles, période, borne, segmentation	numéros des lignes porteuses de chaque motif
fusionner_intervalles()	interne	têtes, entrées, fins	composantes fusionnées et relevés ; refus dates_hors_plage
preparer_donnees()	interne	données, rôles, transition, période, délai, segmentation, exclusions, controler (TRUE par défaut)	fonction d'étape donnees : paramètres effectifs, résultat, diagnostics, avertissements ; avec controler = FALSE, réservé à construire_table(), ni les paramètres ni les rôles ne sont recontrôlés (issue #22)
STATUTS_TABLE	constante	—	synthetique, embarquee, importee
SEXES_TABLE	constante	—	H, F, ensemble
FORMES_ORIGINE	constante	—	qx, lx
table_reference()	exportée	table, nom, statut, source, paramètres, forme d'origine	objet table_reference contrôlé (section 7.5)

Partie IV

Module exposition

Exposition et événements : R/exposition.R

Le module exposition calcule, pour la transition étudiée, l'exposition au risque et le nombre d'événements par cellule (segment, âge et, sur demande, année civile), en traitant explicitement la troncature à gauche et la censure à droite (E2.3, E2.5, E2.6 ; ADR 0003, points 3, 6 et 8). Sa fonction d'étape est calculer_exposition() ; elle reçoit la table des observations de l'étape donnees (composantes fusionnées ou contrats, section 7.4), les rôles, les champs du plan et ses deux paramètres. Le contenu de fond vient de la spécification d'actuary (issue #11, parties A.0 et A.3) et de son erratum du 17/10/2026. Les fenêtres et les événements, les segments des observations et la position d'un jour dans sa cellule sont calculés par trois fonctions (fenetres_observations(), segments_observations(), position_dans_cellule()) que l'estimateur kaplan_meier du module brut emploie aussi (section 9.4) : les deux modules lisent ainsi les mêmes fenêtres, sans seconde implémentation. Dans une construction, les fenêtres et les segments ne sont calculés qu'une fois : l'étape exposition les rend à côté de son résultat, et construire_table() les transmet à l'étape brut, si le plan retient l'estimateur kaplan_meier, sans les inscrire dans l'objet résultat (section 8.2, « Fenêtres transmises à l'étape brut » ; issue #38). Leur extraction de calculer_exposition() n'a changé aucun résultat (constat : objets identical() hors execution avant et après l'extraction, sur les cas de test et sur des plans sans kaplan_meier).

8.1 Recette d'exposition et conventions d'âge

Recette déduite des rôles

La *recette d'exposition* n'est pas un paramètre du plan : le moteur retient la plus fine que les rôles permettent, la nomme dans les paramètres effectifs et dans la fiche, et **refuse si aucune ne s'applique**, sans jamais deviner (E2.3 ; ADR 0003, point 3 ; ADR 0007, point 5.3). Une seule recette existe dans cette branche (tableau 14) ; les recettes « agrégats » et « stocks successifs » (E2.1) viendront avec les branches qui les créent. deduire_recette() retient la recette dont tous les rôles requis sont déclarés, ou refuse (aucune_recette, le message nommant la recette et les rôles absents) ; dans construire_table(), le même manque est détecté plus tôt par le contrôle des rôles, avec le même code et le module donnees. construire_table() appelle deduire_recette() une fois, avant toute étape ; la fonction d'étape, appelée avec controler = FALSE (section 6.2), ne la rappelle pas et lit la recette de la branche, sa convention native et ses rôles requis par recette_branche(), qui ne contrôle rien ; appelée directement, elle appelle deduire_recette(). Cette lecture sans contrôle vaut tant que la branche n'a qu'une recette.

Table 14 : Recettes d'exposition (constantes RECETTE_BRANCHE et CONVENTION_NATIVE).

Recette	Rôles requis	Convention native
lignes_dates_completes	date_naissance, date_entree, date_sortie, cause_sortie (lignes individuelles, dates complètes)	âge au dernier anniversaire (dernier_anniversaire)

Conventions d'âge

Les conventions d'âge sont ordonnées de la plus fine à la plus grossière (ORDRE_CONVENTIONS) : âge au dernier anniversaire (dernier_anniversaire), âge atteint dans l'année (age_atteint_annee), âge fourni (age_fourni). Le paramètre convention de l'étape vaut plus_fine, qui retient la convention native de la recette, ou désigne une convention :

— une convention **plus grossière** que la native est un *forçage*, admis : il est inscrit dans les paramètres effectifs et dans la fiche (convention demandée, native, retenue, forcée ou non) et donne l'avertissement convention_forcee (E2.3) ;

— une convention **plus fine** que la native est refusée (convention_plus_fine) : les rôles ne la permettent pas ;

— age_fourni, non mise en œuvre (ses rôles ne sont pas pris en charge), est refusée (convention_non_mise_en_oeuvre) ; toute autre valeur aussi, de même qu'une convention qui n'est pas une chaîne unique sans attribut ni NA (convention_invalide) ; par_annee doit valoir TRUE ou FALSE, valeur logical unique sans attribut (par_annee_invalide). Ces deux contrôles appliquent la règle commune de forme du plan (section 6.1 ; issue #23).

Aux refus des paramètres s'ajoute un refus de capacité, trop_de_cellules, levé à la construction de la table quand elle compterait plus de .Machine$integer.max cellules (issue #37), ou, plus tôt, quand le décompte de présence des âges compterait plus de .Machine$integer.max couples (segment, âge) (issue #45) (section 8.2, « Paramètres effectifs, diagnostics, avertissements »).
Chaque convention définit ses cellules d'âge et l'unité de l'exposition (UNITES_EXPOSITION, inscrite dans la fiche) :

— dernier_anniversaire : la cellule d'âge x d'une tête est $[\text{anniv}(x), \text{anniv}(x+1))$, l'âge étant celui de la section 7.1 (29 février compris) ; l'unité est l'**année d'âge** : un morceau de j jours dans la cellule x vaut j/L_x, avec $L_x = \text{anniv}(x+1) - \text{anniv}(x)$, soit 365 ou 366 jours ;

— age_atteint_annee : l'âge au jour t est $\text{an}(t) - \text{an}(n)$, la cellule est l'année civile $[1^{\text{er}}$ janvier, 1er janvier suivant), et l'unité la fraction d'année civile ($j/365$ ou $j/366$).

Pourquoi l'année d'âge. Avec cette unité, une année d'âge entière vaut exactement 1 ; c'est la seule pour laquelle l'hypothèse « force constante par âge entier » donne exactement $q_x = 1 - e^{-\mu_x}$ pour toute année d'âge (résultat exact). Avec une division par 365,25, une année d'âge pèserait $365/365{,}25$ ou $366/365{,}25$ selon les années, et la correspondance ne serait plus exacte. L'unité est un *choix de conception* de la spécification (A.0, T3), validé par actuary le 17/10/2026 (validation de fin de branche).

Statut des valeurs. L'exposition est produite en jours, entiers exacts stockés en double : l'exposition totale en jours d'un portefeuille de 10^6 lignes sur 30 ans (environ $1{,}1 \cdot 10^{10}$) dépasse .Machine$integer.max ; résultat exact. L'exposition en unité d'exposition est une somme de quotients en virgule flottante : approximation numérique, vérifiée à 10^{-12} près en relatif. L'arithmétique des dates du module (années bissextiles, anniversaires, 1er janvier) est entière et vectorisée, écrite indépendamment de celle du simulateur ; elle est recoupée par as.Date() et comparée à age_revolu() du module donnees, écrite indépendamment, dans les tests. Trois implémentations de la convention du 29 février (section 7.1) sont **volontairement** conservées (décision M89, issue #22, point 3) : celle du module

donnees (composantes_date(), enveloppe de as.POSIXlt, sur laquelle repose age_revolu()), celle-ci, et les jours civils du simulateur (jours_civils(), jour_anniversaire(), age_anniversaire(), section 16), qu'aucun module du moteur n'appelle (ADR 0007, point 2.1). Le motif de ne pas les unifier, qui relève de la présente rédaction et non de la décision M89, est double : elles se contrôlent l'une l'autre par comparaison directe pour la décomposition d'un jour, les jours civils et les âges, et par l'intermédiaire de as.Date(), oracle commun, pour les anniversaires et les bornes du domaine ; et le simulateur, qui produit les jeux sur lesquels le moteur est vérifié, ne partage pas avec lui une erreur de calendrier que la comparaison du moteur à la loi vraie des jeux simulés ne révélerait pas. Leur accord sur le domaine [0001-01-01, 9999-12-31] est vérifié, avec as.Date() pour oracle des anniversaires et des bornes (section 8.4) : exhaustivement pour la décomposition d'un jour en année, mois et jour et pour les jours civils, partiellement pour les anniversaires et les âges.

8.2 Troncature à gauche et censure à droite

Fenêtre d'observation et décompte

Pour une observation d'entrée e et de fin s ($s = +\infty$ si elle est en cours), la période effective étant $[D, B)$, avec D le début de la période et B la borne de la section 7.1 :

— la **fenêtre** est $[a, b)$, avec $a = \max(e, D)$ et $b = \min(s, B)$, vide si $a \geq b$: une entrée antérieure à D est tronquée à gauche à D, et l'observation est censurée à droite à B (fin de période et délai de déclaration) ;

— l'**exposition centrale** de la cellule (g, x) est

$$E^{c}_{g,x} = \sum_{i \in g} \big| [a_i, b_i) \cap [\mathrm{anniv}_i(x), \mathrm{anniv}_i(x+1)) \big|,$$

en jours, et en unité d'exposition par division de chaque morceau par la longueur de sa cellule ;

— l'**événement** de la transition étudiée est compté si et seulement si la cause est la cible et $D \leq s < B$; il est attribué à la cellule du jour s. Un décès le jour d'un anniversaire est donc attribué à l'âge $x + 1$, où l'exposition de la tête est nulle (cas H8 des tests), et un événement le jour de l'entrée est compté avec une exposition nulle (cas H11) ; le relevé evenement_sans_exposition compte les cellules à exposition nulle qui comptent un événement.

Sorties concurrentes

Une sortie dans la fenêtre dont la cause n'est ni la cible ni en_cours est une *sortie concurrente* : rachat ou décès pour l'autre transition, terme, autre cause, composante de tête close sans décès. Elle est traitée comme une **censure à droite à son instant**, et comptée dans la colonne sorties_concurrentes de sa cellule, censures programmées comprises. Ce traitement suppose une **censure non informative** (risques concurrents indépendants) : l'hypothèse est nommée dans les paramètres effectifs et dans la fiche (HYPOTHESE_CENSURE ; E2.6).

Compléments de l'exposition initiale

L'exposition initiale n'est pas choisie dans cette branche : pour chaque événement compté au jour s dans une cellule qui se termine en $\mathrm{fin}(x)$, deux compléments, en unité d'exposition, sont restitués :

— complement_initial_cellule : $c_1 = \mathrm{fin}(x) - s$, jusqu'à la fin de la cellule (anniversaire suivant sous dernier_anniversaire, 1er janvier suivant sous age_atteint_annee), convention actuarielle classique ;

— `complement_initial_fenetre` : $c_2 = \min(\text{fin}(x), B) - s$, jusqu'à la fin de la fenêtre, la censure par la période étant connue d'avance.

D'où deux expositions initiales, $E^{i,\text{cellule}} = E^c + \sum c_1$ et $E^{i,\text{fenêtre}} = E^c + \sum c_2$. Aucune n'est sans biais indépendamment de l'hypothèse de répartition dans l'année. *Résultat exact* : pour une cohorte entrée à l'âge exact x, sans sortie concurrente, censurée par la période à un instant $c \sim \mathcal{U}(0,1)$ indépendant de la durée de vie, la limite presque sûre de $D/E^{i,\text{fenêtre}}$ quand le nombre de têtes croît (rapport des espérances) vaut $2\big(1-(1-e^{-\mu})/\mu\big)$ sous force constante, soit environ $0{,}05043$ pour $q = 0{,}05$, et exactement q sous l'hypothèse de répartition uniforme ; dans le même schéma, la limite de $D/E^{i,\text{cellule}}$ vaut exactement q sous l'hypothèse ${}_{1-t}q_{x+t} = (1-t)\,q_x$ (calcul direct). À effectif fini, le rapport a un biais que ce résultat ne couvre pas. Le choix de l'exposition initiale revient donc à l'estimateur binomial, qui doit nommer son hypothèse de répartition : le module `brut` retient $E^{i,\text{cellule}}$, sans paramètre, et la colonne `complement_initial_fenetre` reste un relevé de l'étape `exposition`, que le binomial n'emploie pas (preuve et hypothèse : section 9.3). Dans les deux définitions, une sortie concurrente reste à son instant réel : elle est traitée comme une censure, et la portée de l'estimateur binomial sous censure est décrite avec lui.

Table d'exposition

`resultat$exposition` a une ligne par cellule : clés de segmentation, `age` et, avec `par_annee`, `annee` ; colonnes `exposition_jours`, `exposition_centrale` (unité nommée dans `parametres$unite_exposition`), `complement_initial_cellule` et `complement_initial_fenetre` (sans `par_annee` seulement), `evenements` et `sorties_concurrentes`. Par segment, la table couvre la plage **contiguë** des âges dont les cellules ont une exposition non nulle, un événement ou une sortie concurrente ; les zéros sont explicites. Inclure les sorties concurrentes dans la plage garde exacte l'identité « somme de `sorties_concurrentes` = nombre de sorties concurrentes dans la fenêtre » ; la plage de validité de la table finale se fixera à la fermeture (erratum A.3, point 1).

Clés de segmentation. Leur ordre et leur type ne dépendent ni des niveaux d'un facteur ni de la classe reçue : `sexe`, et `segment` reçu en chaîne ou en facteur, sont rendus en chaînes et triés dans l'**ordre des octets UTF-8 de la représentation que hache l'empreinte des données** (section 15.2), quel que soit le marquage d'encodage des chaînes reçues (UTF-8, latin-1 ou non marqué) ; un `segment` entier est rendu en entier et trié par valeur. L'empreinte des données ignore les niveaux d'un facteur et ne dépend pas du marquage : sans cette règle, deux entrées de même empreinte pourraient donner des tables ordonnées différemment (erratum A.3, point 4). Le tri porte sur une copie des valeurs distinctes marquée `bytes`, comparée octet à octet sans traduction, après la conversion en UTF-8 que fait l'écriture de l'empreinte (`octets_chaines()`), par la même fonction, écrite une seule fois, `vers_utf8_locale()` (module `outils` ; issue #113) : dans une session de locale UTF-8 ou latin-1, toutes les chaînes sont converties par `enc2utf8()` ; sous une autre locale (la locale C), seules les chaînes marquées latin-1 le sont, une chaîne non marquée gardant ses octets ; les valeurs rendues gardent leur encodage d'origine. Le tri ne refuse rien : une valeur que l'empreinte refuse est refusée avant la première étape (section 6.2). Ainsi, sur les segments « ÿ » (marqué latin-1), « € » (marqué UTF-8) et « a », l'ordre est « a », « ÿ », « € » (octets `61`, `c3 bf`, `e2 82 ac`), sous la locale C comme sous C.UTF-8. **Défaut corrigé** (issue #106) : jusque-là, le tri de R (`sort(method = "radix")`) rangeait une chaîne latin-1 sur son octet natif (`ff` pour « ÿ », d'où l'ordre « a », « € », « ÿ », contraire à la règle) et levait une erreur R sur une valeur non ASCII non marquée, forme que donne `read.csv()` d'un fichier UTF-8 ; seul l'ordre des segments a changé, non leurs valeurs ni l'empreinte des données. La branche de la locale latin-1 est exercée en CI par le contrôle `tests/controles/locale_latin1.R` (issue #115 ; section 4.2, « Locale latin-1 ») : l'ordre des segments non ASCII y est le même sous `fr_FR.ISO-8859-1` et sous une locale UTF-8, et égal à l'ordre des octets UTF-8 (*constat*). *Cas limite d'une fonction interne* : sous la locale

C, un « é » non marqué et un « é » marqué latin-1 sont deux valeurs distinctes pour `unique()` et `match()`, donc deux segments si l'on appelle `segments_observations()` directement ; le cas n'est pas atteignable par `construire_table()`, qui refuse la chaîne non marquée sous cette locale avant toute étape (`chaine_codage_indetermine`). `tests/unitaires/test_exposition.R` vérifie, sous la locale C et sous une locale UTF-8, que l'ordre des segments est celui des octets que hache l'empreinte des données (corps des lignes écrites par `octets_chaines()`, rangés par octets hors du code de l'ordre des segments), sur des valeurs ASCII, marquées UTF-8 et latin-1 mêlées, et, sous UTF-8, natives non marquées, et que ces octets sont ceux de `vers_utf8_locale()` (issue #113 ; *constat* sur ces valeurs).

Paramètres effectifs, diagnostics, avertissements. Les paramètres effectifs sont la recette, la convention (demandée, native, retenue, forcée), l'unité d'exposition, `par_annee`, le code de la transition, la segmentation, la période, le délai, la fin effective et l'hypothèse de censure. Les diagnostics donnent le nombre d'observations et d'observations exposées, l'exposition totale en jours, le nombre d'événements et de sorties concurrentes, le nombre de cellules, stocké en `integer` (issue #37), et le nombre de cellules à exposition nulle qui comptent un événement. Le nombre de cellules est calculé par `nombre_cellules()` : somme, sur les segments, du nombre d'âges de la plage par le nombre d'années (une seule sans `par_annee`), formée en `double` ; s'il dépasse `.Machine$integer.max` ($2^{31} - 1$), l'étape **refuse** sous le motif `trop_de_cellules` (refus de capacité : segmentation trop fine ou plage d'âges et d'années trop étendue), avant toute allocation par cellule, au lieu de laisser `tabulate()` échouer sur une erreur R. Le même motif est levé plus tôt, par la même fonction, avant le décompte de présence des âges (issue #45) : ce décompte, fait par `tabulate()`, porte sur une matrice qui croise les segments et la plage globale des âges présents (du plus petit au plus grand âge présent, tous segments confondus), soit le produit du nombre de segments par cette étendue, en couples (segment, âge) ; au-delà de `.Machine$integer.max`, l'étape **refuse**, avec un message qui compte des couples (segment, âge) et non des cellules, et un décompte inscrit dans `details` sous le nom `couples_presence` au lieu de `cellules`. Ce produit n'est pas un nombre de cellules mais en est un majorant quand il n'y a pas de découpage par année : sans `par_annee`, c'est donc la garde de présence qui refuse, et le refus sur le nombre de cellules n'est atteignable qu'avec `par_annee = TRUE`. Ce majorant peut être lâche : la garde de présence peut refuser alors que la table elle-même tiendrait sous la limite, par exemple quand les plages d'âges des segments sont disjointes, la plage globale couvrant alors, pour chaque segment, les âges de tous les autres. Une base de cette taille étant hors de portée d'un test, la garde est vérifiée par appel direct de `nombre_cellules()` sous ses deux libellés, et son placement avant le `tabulate()` de présence par une copie instrumentée de `calculer_exposition()` (`tests/unitaires/test_exposition.R`). Avertissements : `convention_forcee`, `evenement_sans_exposition` et `aucune_exposition` (aucune exposition ni aucun événement dans la période).

Fenêtres transmises à l'étape `brut` (issue #38 ; ADR 0007, seconde annotation du 19/10/2026). Outre ses paramètres effectifs, son résultat, ses diagnostics et ses avertissements, `calculer_exposition()` rend un cinquième élément, `fenetres`, objet de classe `fenetres_exposition`. Il contient les fenêtres des observations et leurs événements (`composantes`, `calendrier`, `a`, `b`, `evenement`, issus de `fenetres_observations()`), le segment de chaque observation et la table des clés de segmentation (`g`, `segments`, issus de `segments_observations()`), les paramètres dont ils dépendent : début D (`debut`) et borne B (`borne`) de la période, code de la transition (`transition`) et segmentation (`segmentation`), et la convention retenue par l'étape (`convention`). La convention n'entre pas dans le calcul des fenêtres ni des segments (`fenetres_observations()` et `segments_observations()` ne la prennent pas) : elle est jointe pour que l'étape `brut` vérifie qu'elle emploie la même convention que l'étape `exposition` qui a produit la table d'exposition. Ce sont les valeurs mêmes qui ont servi à construire la table d'exposition, non recalculées. Cet élément est **interne à l'exécution** : `construire_table()` ne le conserve que si le plan retient l'estimateur `kaplan_meier` ; elle le passe alors à l'étape `brut`, puis le libère ; il n'entre ni dans

etapes de l'objet résultat ni dans la fiche (section 6.2). L'estimateur kaplan_meier le reprend après un contrôle d'égalité exacte de ces cinq paramètres (section 9.4, « Données »). Ainsi, les deux étapes lisent les fenêtres d'une seule source, calculées une seule fois. Les positions des jours dans leur cellule ne sont pas transmises : seul kaplan_meier les emploie et il les calcule lui-même. Les calculer ici coûterait à toute construction, y compris sans cet estimateur, et ferait dépendre le module exposition des besoins d'un estimateur du module brut.

8.3 Découpage par année

Avec par_annee = TRUE, chaque morceau d'exposition est en outre découpé au 1er janvier, et un événement est attribué à l'année civile de son jour. La grille est **rectangulaire** : elle croise la plage d'âges du segment et toutes les années civiles de l'année du début de la période à celle de la fin effective, avec des zéros explicites ; la première et la dernière année peuvent être partielles (erratum A.3, point 2). La somme sur les années redonne la table par âge seul, exactement en jours. Les compléments de l'exposition initiale ne sont pas produits dans ce découpage.

8.4 Vérifications

Les cas construits à la main de la spécification (H1 à H13 pour une ligne, F1 à F4 pour la fusion) sont vérifiés par tests/unitaires/test_exposition.R, en jours exactement et en unité d'exposition à 10^{-12} près. Quelques-uns illustrent les règles ci-dessus (période du 2015-01-01 au 2024-12-31, délai nul, vivant → décédé) :

Table 15 : Exemples de cas calculés à la main (spécification, A.4 ; valeurs reproduites par le moteur sur R 4.3.3).

Cas	Ligne	Exposition par âge (jours / L_x)	Événement
H2	née 1945-10-15 ; 2016-05-20 → 2019-02-10, décès	70 : 148/366 ; 71 : 365/365 ; 72 : 365/365 ; 73 : 118/365 ; total 996 jours	à 73 ans ; $c_1 = c_2 = 247/365$
H6	née 1964-02-29 ; 2021-01-01 → en cours	56 : 59/366 ; 57 à 59 : 1 chacun ; 60 : 307/366 ; total 1 461 jours, 4 années d'âge	aucun
H8	née 1955-04-12 ; 2019-09-01 → 2020-04-12, décès	64 : 224/366 ; 65 : 0	à 65 ans, exposition nulle à 65 ; $c_1 = 365/365$
H11	née 1980-01-01 ; 2018-06-15 → 2018-06-15, décès	aucune	à 38 ans ; $c_1 = 200/365$
H12	H2 avec age_atteint_annee	71 : 226/366 ; 72 : 1 ; 73 : 1 ; 74 : 40/365	à 74 ans ; $c_1 = 325/365$

Le cas F4 (décès porté par deux contrats d'une même tête) donne un décès en unité tête et deux en unité contrat, avec l'avertissement unite_contrat. Sur un portefeuille simulé, tests/unitaires/test_construire_table.R vérifie des *identités exactes* : exposition totale en jours égale à la somme des $\max(0, b - a)$, égalité entière ; décès comptés égaux aux décès simulés tels que $D \leq s \leq$ fin effective ; même identité pour les rachats, décès censurés ; décès par sexe et par âge égaux aux décès simulés par âge au dernier anniversaire ; avec par_annee, somme sur les années égale à l'exposition par âge seul, en jours. La segmentation est vérifiée avec des facteurs aux niveaux permutés, en chaîne et en entier.

Accord des trois arithmétiques de dates (décision M89, issue #22, point 3 ; section 8.1, « Statut des valeurs »). tests/unitaires/test_exposition.R compare les implémentations des modules donnees,

exposition et du simulateur, sans les modifier, avec as.Date() pour oracle externe des anniversaires et des bornes :

— **décomposition**, exhaustive : pour chaque jour du 0001-01-01 au 9999-12-31, composantes_date() et composantes_jour() donnent la même année, le même mois et le même jour ; les jours voisins des bornes tombent dans les années 0 et 10 000 ;

— **jours civils**, exhaustifs : pour chaque jour du domaine, jours_civils() appliquée aux composantes rend le jour de départ, annee_du_jour() et annee_calendrier() rendent son année, et premier_janvier() égale jours_civils() au 1er janvier de chacune des années 1 à 10 001 ;

— **anniversaires**, partiels : anniversaire_age() et jour_anniversaire() égalent l'oracle (le 29 février d'une année commune remplacé par le 1er mars, convention T2) pour toutes les naissances du domaine aux âges 0, 1, 3, 4, 100 et 130, et pour les naissances des 28 février, 29 février et 1er mars de chaque année bissextile à tous les âges de 0 à 130, tant que l'anniversaire reste dans le domaine ;

— **âges au dernier anniversaire**, partiels : les trois fonctions d'âge, age_revolu(), age_anniversaire() et age_dernier_anniversaire(), concordent sur un tirage de couples (naissance, jour) sous graine locale (sous_graine()), naissances du 29 février comprises, et sur un balayage des 28 février, 29 février et 1er mars postérieurs aux naissances du 29 février des années 4, 1964 et 2000 ;

— **témoin écrit à la main** : pour une naissance le 29 février de l'an 4, anniversaires et âges aux années séculaires communes (le 1er mars) et bissextiles (le 29 février), les anniversaires par les deux fonctions qui les calculent (anniversaire_age() et jour_anniversaire()), les âges par les trois fonctions d'âge.

L'accord exhaustif couvre tous les jours du domaine (vérification par énumération, sur la plateforme où la batterie s'exécute) ; l'accord partiel n'est constaté que sur les cas énumérés.

8.5 Fonctions du module

Table 16 : Définitions de premier niveau de R/exposition.R.

Nom	Statut	Entrées	Sortie et rôle
ORDRE_CONVENTIONS	constante	—	conventions d'âge, de la plus fine à la plus grossière
CONVENTIONS_MISES_EN_OEUVRE	constante	—	dernier_anniversaire, age_atteint_annee
VALEURS_CONVENTION	constante	—	valeurs admises du paramètre convention
CONVENTION_NATIVE	constante	—	convention native de chaque recette
UNITES_EXPOSITION	constante	—	unité de l'exposition par convention
HYPOTHESE_CENSURE	constante	—	hypothèse de censure non informative, inscrite dans la fiche
recette_branche()	interne	—	recette de la branche, convention native, rôles requis, sans contrôle des rôles ; employée par deduire_recette() et par calculer_exposition() appelée avec controler = FALSE (issue #22)
deduire_recette()	interne	rôles	recette, convention native, rôles requis (recette_branche()) ; refus aucune_recette

Nom	Statut	Entrées	Sortie et rôle
controler_parametres_exposition()	interne	convention, par_annee	refus des valeurs
convention_retenue()	interne	convention demandée, native	convention demandée, native, retenue (la native pour plus_fine, la demandée sinon) et forçage, sans contrôle ; employée par resoudre_convention() et par calculer_exposition() appelée avec controler = FALSE (issue #22)
resoudre_convention()	interne	convention demandée, native	convention retenue et forçage (convention_retenue()) ; refus convention_plus_fine
an_bissextile()	interne	années	bissextilité grégorienne
bissextiles_anterieures()	interne	années	nombre d'années bissextiles antérieures
premier_janvier()	interne	années	jour du 1er janvier
JOURS_AVANT_MOIS	constante	—	jours écoulés avant chaque mois, année commune
annee_du_jour()	interne	jours	année civile
composantes_jour()	interne	jours	année, mois, jour, quantième commun
calendrier()	interne	années extrêmes	table calendaire
indice_annee()	interne	table, années	indice dans la table calendaire
annee_calendrier()	interne	table, jours	année civile, par la table
anniversaire_age()	interne	table, naissances, âges	jour de l'anniversaire
age_dernier_anniversaire()	interne	table, naissances, jours	âge au dernier anniversaire
debut_cellule()	interne	table, naissances, âges, convention	début de la cellule d'âge
cellule_du_jour()	interne	table, naissances, jours, convention	cellule d'âge du jour
extraire_composantes()	interne	composantes, indices	sous-ensemble
decouper_cellules()	interne	fenêtres, convention	morceaux contenus chacun dans une cellule
decouper_annees()	interne	morceaux	morceaux découpés au 1er janvier
somme_par_cellule()	interne	cellules, valeurs	sommes par cellule, zéros explicites
nombre_cellules()	interne	nombres d'âges par segment, nombre d'années ; libellé et nom du décompte, facultatifs	nombre de cellules de la table, en integer ; refus trop_de_cellules au-delà de .Machine$integer.max (issue #37) ; appelée aussi, avant le décompte de présence des âges, sur le produit du nombre de segments par l'étendue des âges, avec son propre libellé et le décompte couples_presence (issue #45)
fenetres_observations()	interne	observations, début de la période, borne B, cause cible	fenêtres $[a, b)$, événements et sorties concurrentes (section 8.2) ; composantes des naissances, table calendaire

Nom	Statut	Entrées	Sortie et rôle
segments_observations()	interne	observations, segmentation	segment de chaque observation et table des clés, triées sans dépendre des niveaux d'un facteur ni du marquage d'encodage (ordre des octets UTF-8 de l'empreinte des données)
position_dans_cellule()	interne	table calendaire, composantes, jours, convention	cellule x du jour, rang j dans la cellule, longueur L de la cellule, début de la suivante
calculer_exposition()	interne	observations, rôles, transition, période, délai, segmentation, convention, par_annee, controler (TRUE par défaut)	fonction d'étape exposition : paramètres effectifs, résultat, diagnostics, avertissements, et fenêtres transmises à l'étape brut (objet fenetres_exposition, hors de l'objet résultat ; issue #38) ; avec controler = FALSE, réservé à construire_table(), ni les paramètres, ni la recette, ni la convention ne sont recontrôlés (issue #22)

Partie V

Module brut

Taux bruts : R/brut.R

Le module brut estime, par cellule de la table d'exposition (segment, âge et, sur demande, année civile), le taux de la transition étudiée et ses intervalles de confiance exacts et asymptotiques, chacun restitué avec son statut (E2.12, E2.14, E2.20). Sa fonction d'étape est calculer_taux_bruts() ; elle reçoit la table d'exposition et les paramètres effectifs de l'étape exposition, jamais le plan. Le module n'appelle que des modules situés avant lui (donnees, exposition, outils ; section 1.3) et ne tire aucun aléa. Le contenu de fond vient de la spécification d'actuary du 17/10/2026 (issue #14, parties A.1 à A.7) et des décisions du mainteneur sur ses questions A.8 (décision M51) ; pour l'estimateur kaplan_meier, de la spécification d'actuary du 17/10/2026 (issue #15, parties A.1 à A.12) et des décisions M52 (b) et M53.

Appellations et références. Les sources primaires des estimateurs et des intervalles de la présente partie ne sont pas encore établies (issue #6 ; décision M50, Q4). Chaque méthode est donc décrite par sa formule, sa dérivation et son statut, **sans attribution d'auteur** : référence à établir, issue #6. Les textes inscrits par le code dans la fiche emploient, pour les intervalles asymptotiques, l'appellation d'usage « Wald », dont la référence est elle aussi à établir. L'estimateur produit-limite porte dans le plan le nom kaplan_meier, celui que lui donne le cahier des charges (E2.12) : la source de cette appellation, celle de sa formule de variance et celle de ses propriétés asymptotiques sous troncature à gauche sont à établir (issue #6).

9.1 Étape brut du plan

Notations, par cellule de la table etapes$exposition$resultat$exposition (section 8.2) : D le nombre d'événements (evenements) ; E^c l'exposition centrale (exposition_centrale, dans l'unité de la fiche : année d'âge, ou fraction d'année civile sous age_atteint_annee) ; $E^i = E^c + \sum c_1$ l'exposition initiale « cellule », où c_1 est le complément complement_initial_cellule de chaque événement (section 8.2) ; $\alpha = 1 -$ niveau et $z = \Phi^{-1}(1 - \alpha/2)$. Les intervalles sont à queues égales, $\alpha/2$ de chaque côté, quel que soit D.

Paramètres, tous requis, sans défaut (ADR 0007, point 5 ; constante PARAMETRES_ETAPES), soumis à la règle commune de forme (type de base exact, aucun attribut ; section 6.1, issue #23) : estimateurs, vecteur character, sous-ensemble non vide, sans doublon ni NA, de ESTIMATEURS_BRUT ; les résultats sont toujours rendus dans l'ordre de cette constante, quel que soit l'ordre déclaré ; niveau, nombre double unique, fini, strictement compris entre 0 et 1, partagé par tous les estimateurs. L'étape a pour préalables les étapes donnees et exposition (constante ETAPES ; refus etape_prealable_manquante de valider_plan() sinon) : l'estimateur kaplan_meier lit les fenêtres des observations de la première,

calculées par la seconde et transmises par construire_table() (section 6.2), tous les estimateurs la table de la seconde.

Refus du module brut, levés par controler_parametres_brut(), que valider_plan() appelle avant toute étape et que la fonction d'étape appelle en appel direct seulement (controler = TRUE, valeur par défaut ; construire_table() l'appelle avec controler = FALSE, section 6.2) : estimateurs_invalides (pas un vecteur character, attribut — noms, dimension ou classe —, vide, NA ou doublon) ; estimateur_inconnu, contrôlé après la forme : un vecteur nommé de noms inconnus relève du motif précédent ; niveau_invalide (autre type que double, attribut, longueur différente de 1, valeur non finie, ≤ 0 ou ≥ 1) ; binomial_par_annee : sous par_annee = TRUE, l'exposition initiale n'est pas produite (section 8.3), et une cellule âge $\times$ année ne couvre pas une année d'âge entière, si bien que q_x n'y est pas défini ; kaplan_meier_par_annee, pour la même raison de couverture (un estimateur de période sur une cellule âge $\times$ année relèverait d'une autre spécification). Si le binomial et kaplan_meier sont tous deux déclarés sous par_annee, binomial_par_annee est levé en premier. En appel direct, la fonction d'étape contrôle en outre, en tête et avant controler_parametres_brut(), la forme et le catalogue de la segmentation qu'elle reçoit, par controler_segmentation() du module donnees : segmentation_invalide, module brut (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, dont chaque clé est prise dans ROLES_SEGMENTATION, soit sexe et segment ; vide explicite admis ; issues #43 et #59). Une clé hors de ce catalogue est donc refusée à l'appel direct, qu'elle désigne une colonne de la table d'exposition ("age", "evenements") ou non ("region"), de même que la chaîne vide, qui n'en fait pas partie : sans ce contrôle, une clé hors du catalogue présente dans la table serait employée comme clé de segmentation sans avoir été déclarée comme rôle ("age" dupliquerait la clé d'âge dans la table résultat), et une clé absente de la table ne serait signalée que par une erreur de programmation sur les colonnes manquantes. Sur le chemin Kaplan-Meier, la fonction d'étape contrôle ensuite, en appel direct, la transition, la période et le délai par controler_transition_periode() du module donnees (refus de ce module, section 7.3), sans recontrôler la segmentation ni contrôler d'exclusions ; l'estimateur taux_kaplan_meier() ne contrôle rien lui-même (section 9.4). Une segmentation invalide y est donc refusée sous le module brut, une transition inconnue sous le module donnees. Le même code est porté par le module qui détecte la condition (ADR 0007, annotation du 17/10/2026, point d) : dans le plan, la segmentation est déjà refusée sous ce code par controler_parametres_donnees(), module donnees, que valider_plan() appelle et qui emploie le même contrôle (section 7.3) ; la forme et le catalogue de la segmentation ne sont écrits qu'à un endroit, et le message du refus est le même dans les deux modules. Ce contrôle-ci ne sert qu'à l'appel direct de la fonction d'étape, et une segmentation invalide y est signalée avant un estimateur inconnu.

Quantiles supérieurs. Ils sont calculés par la queue supérieure (qgamma(alpha / 2, D + 1, lower.tail = FALSE), de même pour qbeta() et qnorm()) plutôt que par le quantile $1 - \alpha/2$: pour un niveau très proche de 1 (par exemple $1 - 2^{-53}$), $1 - \alpha/2$ vaut exactement 1 en double et le quantile serait infini, alors que la queue supérieure garde une borne finie (spécification, A.3 ; cas couvert par les tests). Aux niveaux usuels, les deux écritures ne diffèrent que par l'arrondi (approximation numérique).

9.2 Estimateur de Poisson

— **Objet** : l'intensité μ de la transition dans la cellule, sur l'axe de l'âge (et de l'année sous par_annee) ; estimateur poisson.

— **Hypothèses** : force de la transition constante dans la cellule ; sorties concurrentes non informatives (HYPOTHESE_CENSURE, section 8.2) ; troncature à gauche traitée par la fenêtre d'observation.

— **Estimateur** : sous ces hypothèses, la log-vraisemblance de μ vaut exactement $D \log \mu - \mu E^c$; son

maximum est

$$\hat{\mu} = D/E^c.$$

— **Probabilité associée** : $\hat{q}^P = 1 - e^{-\hat{\mu}}$, calculée par `-expm1(-taux)`, sous l'hypothèse « force constante sur la cellule entière, égale au taux de la cellule » ; les bornes sont transformées par la même fonction croissante, si bien que la couverture est conservée exactement. Sa portée dépend de la cellule (`PORTEE_PROBABILITE`) : sous `dernier_anniversaire`, l'unité étant l'année d'âge, c'est le q_x exact sous force constante par âge entier (section 8.1) ; sous `age_atteint_annee`, c'est la probabilité sur l'année civile de la cellule ; sous `par_annee`, c'est le q_x de l'année t à force $\mu_{x,t}$.

— **Biais dû à la datation au jour** (spécification de #16, A.1 ; résultat exact, preuve écrite). L'exposition court jusqu'au jour de la sortie exclu (section 7.1) : il manque en moyenne un demi-jour d'exposition par sortie, quelle qu'en soit la cause. Dans une cellule de L jours où la transition a la force constante μ_d et les sorties concurrentes la force μ_r, avec $m = \mu_d + \mu_r$, une tête à risque au début d'un jour subit la transition ce jour-là avec la probabilité $h_d = (\mu_d/m)(1 - e^{-m/L})$, constante dans la cellule ; il en résulte $E[D - \mu^* E^c] = 0$ exactement, avec

$$\mu^* = \frac{\mu_d}{m} L \left(e^{m/L} - 1\right), \qquad \frac{\mu^*}{\mu_d} - 1 \approx \frac{m}{2L}.$$

$\hat{\mu}$ estime donc μ^*, et non μ_d : l'écart relatif, de l'ordre de 10^{-5} à $5 \cdot 10^{-4}$ pour les forces des constats de simulation (section 9.9), est négligeable devant l'erreur d'estimation, mais non nul. Il vaut pour des données réelles datées au jour comme pour le simulateur. Le binomial sur une cellule observée entière ($E^i = n$) et l'estimateur produit-limite sans sortie concurrente n'en sont pas affectés : leur cible exacte est q_x. Avec une sortie concurrente, la cible de l'estimateur produit-limite sur la grille des jours, $q^* = 1 - (1 - h_d)^L$, s'écarte de $1 - e^{-\mu_d}$ d'un écart relatif du même ordre (constats, section 9.9).

— **Intervalle exact** : $\left[G_D^{-1}(\alpha/2),\ G_{D+1}^{-1}(1-\alpha/2)\right]/E^c$, où G_k est la loi gamma de forme k et d'échelle 1 ; borne inférieure 0 si $D = 0$, borne supérieure alors $-\ln(\alpha/2)/E^c$.

— **Intervalle asymptotique** : normal sur l'échelle de $\log \mu$, $\mathrm{Var}(\log \hat{\mu}) \approx 1/D$, soit $\hat{\mu}\, e^{\mp z/\sqrt{D}}$; non défini à $D = 0$.

— **Calcul** : `taux_poisson()`, bornes exactes par `qgamma()`.

— **Statut** : $\hat{\mu}$, résultat exact (maximum de vraisemblance du modèle), valeur en arithmétique flottante ; $\hat{q}^P$, résultat exact (image du maximum de vraisemblance par $q = 1 - e^{-\mu}$) ; intervalle exact, exact sous la loi supposée $D \sim \mathcal{P}(\mu E^c)$, E^c tenue pour fixe (E2.14) : couverture au moins égale au niveau pour tout μ ; sous le modèle de survie, où E^c est aléatoire, sa couverture n'est qu'asymptotique ; bornes par approximation numérique (`qgamma`) ; intervalle asymptotique, asymptotique ($\mu E^c \to \infty$).

— **Références** : à établir (issue #6).

9.3 Estimateur binomial

— **Objet** : la probabilité q de la transition dans la cellule, sur l'axe de l'âge ; estimateur `binomial`, refusé sous `par_annee`.

— **Hypothèse de répartition dans la cellule** : ${}_{1-t}q_{x+t} = (1-t)\, q_x$ pour $0 \le t \le 1$, ce qui revient à interpoler linéairement $1/S$ dans la cellule ; censures (sorties concurrentes, fin de période, délai de déclaration) indépendantes de l'instant de l'événement.

— **Estimateur** : $\hat{q}^B = D/E^i$, avec l'exposition initiale « cellule » (`EXPOSITION_INITIALE_BRUT`), fixée sans paramètre (décision M51, question 1).

— **Pourquoi la définition « cellule »** (spécification, A.2 ; preuve écrite). Soit une tête présente dans la cellule à la fraction u et censurée en c ($u < c \leq 1$, c fixe ou indépendant de l'instant T de l'événement), et $\pi = P(T \in (u, c) \mid T > u)$. *Définition « fenêtre »* : le complément court jusqu'à c, donc $E^{i,\text{fenêtre}} = c - u$ quelle que soit la loi ; l'égalité $E[D] = q\,E[E^i]$ exige $\pi = q(c-u)$ pour tout (u, c) ; avec $u = 0$, cela impose la répartition uniforme $P(T \leq c) = qc$, sous laquelle $\pi = q(c-u)/(1-qu) \neq q(c-u)$ dès que $u > 0$: **aucune hypothèse de répartition ne rend cette définition exacte dès qu'une tête entre en cours de cellule**. *Définition « cellule »* : le complément court jusqu'à la fin de la cellule, donc $E[E^{i,\text{cellule}}] = (c-u)(1-\pi)+(1-u)\pi$, et l'égalité $\pi = q\,E[E^i]$ équivaut à $\pi = q(c-u)/(1-q(1-c))$, qui est exactement la valeur de π sous l'hypothèse ci-dessus. En sommant sur les têtes, $E[D] = q\,E[E^i]$ **pour toute troncature à gauche et toute censure indépendante dans la cellule** ; avec $u = 0$, cette hypothèse est la seule loi dans la cellule qui rende l'égalité vraie pour tout c. Sous une autre loi, en particulier la force constante du simulateur, l'identité ne tient plus dès qu'une entrée ou une censure tombe dans la cellule ; elle tient pour une cellule observée entière, sans entrée ni censure, où E^i est le nombre de têtes. Exemple exact (spécification de #16, A.1, point 3 ; section 9.9) : pour n têtes entrées à l'âge exact x, sans sortie concurrente, censurées par la période à la fraction c de la cellule, sous la force constante μ, $D \sim \mathcal{B}(n, \pi_c)$ avec $\pi_c = 1 - e^{-\mu c}$, et D/E^i tend vers $\pi_c/\big(c(1-\pi_c)+\pi_c\big)$ pour la définition « cellule » et vers π_c/c pour la définition « fenêtre » ; avec $q = 0{,}182091641$ et $c = 183/365$, ces limites s'écartent de q de $-4{,}13\ \%$ et de $+5{,}01\ \%$.

— **Intervalle exact** : $\big[B^{-1}_{D,\,E^i-D+1}(\alpha/2),\ B^{-1}_{D+1,\,E^i-D}(1-\alpha/2)\big]$, où $B_{a,b}$ est la loi bêta ; borne inférieure 0 si $D = 0$, borne supérieure 1 si $D = E^i$. Si E^i n'est pas entier, aucune loi binomiale ne lui correspond : la formule est prolongée continûment (décision M51, question 4).

— **Intervalle asymptotique** : normal sur l'échelle de logit q, $\operatorname{logit}^{-1}\big(\operatorname{logit}\hat{q} \mp z\sqrt{1/D + 1/(E^i - D)}\big)$; non défini à $D = 0$ et à $D = E^i$.

— **Tolérance sur l'effectif** (`TOLERANCE_EFFECTIF`, 10^{-9}, absolue) : si $|E^i - D| \leq 10^{-9}$, le calcul prend $E^i := D$, la colonne `exposition_initiale` gardant la somme calculée. Raison : E^i est une somme de fractions j/L, $L \in \{365, 366\}$, dont l'arrondi flottant peut placer une valeur rationnellement égale à D juste en dessous, ce qui la ferait déclarer à tort « effectif inférieur » ; le plus petit écart rationnel non nul entre E^i et un entier vaut $1/(365 \times 366) = 1/133\,590 \approx 7{,}5 \cdot 10^{-6}$, si bien que la tolérance ne peut absorber un écart réel (approximation numérique).

— **Calcul** : `taux_binomial()`, bornes exactes par `qbeta()`.

— **Statut** : $\hat{q}^B$, estimateur des moments : $E[D] = q\,E[E^i]$ exactement sous l'hypothèse de répartition, convergent, non sans biais à effectif fini (c'est un rapport) ; intervalle exact, exact sous la loi supposée $D \sim \mathcal{B}(E^i, q)$, E^i tenue pour fixe, **pour une cellule observée entière**, sans entrée ni censure dans la cellule (alors E^i est le nombre de têtes, entier) ; intervalle asymptotique, asymptotique ($E^i q(1-q) \to \infty$) sous la même condition ; bornes par approximation numérique (`qbeta`). **Avec une entrée ou une censure dans la cellule**, D ne suit pas $\mathcal{B}(E^i, q)$, même si E^i est entier : seule l'égalité $E[D] = q\,E[E^i]$ subsiste, et la **couverture des deux intervalles n'est pas établie** ; si E^i n'est pas entier, l'intervalle exact est en outre un prolongement continu de la formule exacte. Exemple (calcul exact de la loi de D, sans simulation : résultat exact sur un cas, bornes par `qbeta`, approximation numérique) : n têtes entrées à la fraction $u = 0{,}5$ de la cellule et observées jusqu'à sa fin, sous l'hypothèse de répartition avec $q = 0{,}2$, si bien que $E^i = n/2$ est entier et $D \sim \mathcal{B}(n, 0{,}1)$; pour un niveau de 0,95, la couverture vraie de l'intervalle exact vaut 0,9561, 0,9423, 0,9381 et 0,9364 pour $n = 200$, $1\,000$, 10^4 et 10^5, celle de l'intervalle logit 0,9245, 0,9352, 0,9357 et 0,9357 : l'intervalle « exact » peut couvrir moins que le niveau. L'écart ne disparaît pas quand l'effectif croît : $\operatorname{Var}(D)$ vaut $n \cdot 0{,}1 \cdot 0{,}9$, au lieu de $E^i q(1-q) = n \cdot 0{,}08$ supposé par les deux intervalles, et les deux couvertures

tendent, quand $n \to \infty$, vers $2\,\Phi\big(\Phi^{-1}(0{,}975)/\sqrt{1{,}125}\big) - 1 = 0{,}9354$, où $1{,}125 = 0{,}09/0{,}08$ est le rapport de ces deux variances (résultat asymptotique).

— **Références** : à établir (issue #6), y compris pour l'appellation usuelle de l'hypothèse de répartition, nommée ici par sa formule.

Choix des échelles asymptotiques (décision M51, question 2). La couverture exacte des intervalles, calculée par sommation sur la loi du décompte (spécification, A.3 ; résultat exact calculé numériquement, sans simulation), est meilleure sur l'échelle du lien canonique, $\log \mu$ pour Poisson et $\operatorname{logit} q$ pour le binomial, que sur l'échelle naturelle dès environ cinq événements attendus ; en deçà, les deux formes asymptotiques sont mauvaises, à cause du poids de $D = 0$, et c'est l'intervalle exact qui fait foi. Ces échelles gardent en outre les bornes dans l'espace des paramètres, sans troncature à 0 ou à 1.

9.4 Estimateur produit-limite (`kaplan_meier`)

— **Objet** : la probabilité q_x de la transition dans la cellule d'âge x, estimée **sans hypothèse de répartition dans la cellule**, à partir des observations individuelles de l'étape `donnees` (composantes fusionnées par tête, ou contrats ; section 7.4) ; estimateur `kaplan_meier`, refusé sous `par_annee`. Sous `age_atteint_annee`, admis (décision M52 (b), question 3), q_x est la probabilité de la transition dans l'année civile $\mathrm{an}(n) + x$ pour une tête présente au 1er janvier, même portée que le binomial sous cette convention.

— **Hypothèses** : troncature à gauche non informative (aucun effet de sélection à l'entrée) ; censure non informative (`HYPOTHESE_CENSURE`) ; unités d'observation indépendantes.

— **Échelle de temps** : celle des cellules de la convention retenue, en années d'âge (section 8.1) : à l'instant t (un jour), $\tau(t) = x + j/L$, avec x la cellule du jour, j le nombre de jours écoulés depuis le début de la cellule et $L \in \{365, 366\}$ sa longueur (`position_dans_cellule()`). Sous `dernier_anniversaire`, un âge entier est l'anniversaire de chaque tête, et un événement est dans la cellule x si et seulement si $x \le \tau(s) < x + 1$: c'est exactement l'attribution de l'étape `exposition`. L'échelle « jours depuis la naissance » est écartée : un âge entier n'y correspond pas au même instant pour toutes les têtes, et rendre q_x commun demanderait de diviser par 365,25, ce que l'unité de l'exposition a écarté. Les instants sont comparés exactement par la clé entière $K = 133\,590\,x + j \cdot 133\,590/L$ (`DENOMINATEUR_INSTANTS` $= 365 \times 366$) : deux instants distincts diffèrent d'au moins $1/133\,590$ an, et aucune tolérance n'intervient.

— **Données** : fenêtres et événements identiques à ceux de l'étape `exposition` (`fenetres_observations()`, section 8.2). Dans une construction, ce sont ceux-là mêmes : l'étape `exposition` les transmet, avec les segments des observations, dans un objet `fenetres_exposition` (« Fenêtres transmises à l'étape `brut` », section 8.2), que `taux_kaplan_meier()` reprend tel quel après avoir contrôlé à l'égalité exacte (`identical()`) les cinq paramètres qu'il porte, comparés à ceux de l'étape `brut` : début et borne de la période, transition et segmentation, dont dépendent les fenêtres et les segments ; convention retenue, qui n'entre pas dans leur calcul et dont le contrôle vérifie la cohérence avec la convention de l'étape `exposition` ; un écart est une erreur de programmation. `taux_kaplan_meier()` ne lève aucun refus : la transition, la période, le délai et la segmentation qu'il reçoit ont été contrôlés par `valider_plan()` dans une construction, par la fonction d'étape en appel direct (section 9.1), et il en reprend les valeurs par `valeurs_parametres_donnees()`, qui ne contrôle rien (issue #22). Les observations elles-mêmes ne sont pas contrôlées : seul `construire_table()` transmet cet objet, produit par `calculer_exposition()` à partir des observations de l'étape `donnees` de la même exécution. En appel direct de la fonction d'étape sur la table des observations, les fenêtres et les segments sont recalculés par les mêmes fonctions (`fenetres_observations()`, `segments_observations()`) ; les deux voies donnent des résultats `identical()` (constat de test,

section 9.8). Les positions des jours dans leur cellule (`position_dans_cellule()`) sont calculées ici, dans les deux cas. Une observation est retenue si sa fenêtre $[a, b)$ est non vide, ou si elle porte un événement ($a \le s = b$). Sorties concurrentes, fin de période et délai de déclaration sont des censures à leur instant. Une tête dont deux composantes sont disjointes quitte l'ensemble à risque pendant l'écart et y revient ; elle n'y est jamais deux fois.

— **Ensemble à risque fermé** (`ENSEMBLE_A_RISQUE_KM`, `"ferme"` ; décision M52 (b), question 1) : $r(t) = \#\{i : \tau(a_i) \le t \le \tau(b_i)\}$, si bien qu'à un même instant l'ordre est **entrées, puis événements, puis censures**. Raisons : un événement du jour s survient à l'instant s et la tête y est présente, ce qui est le seul ordre possible pour une tête qui entre et subit l'événement le même jour ; une tête censurée en c a la même exposition $[a, c)$ qu'une tête dont l'événement survient en c, et appartient donc à l'ensemble à risque de cet instant. Conséquences (identités exactes, vérifiées) : $d_j \le r_j$; la somme des d_j d'une cellule égale le nombre d'événements de la table d'exposition ; un événement à l'instant d'entrée est compté. La convention « prévisible » $\{a < t \le b\}$, écartée, exclurait ces derniers événements ; aucune source ne tranche entre les deux (*choix de conception*, mesuré sur un cas construit qui les départage, ci-dessous).

— **Estimateur** : avec t_j les instants d'événement de $[x, x+1)$, d_j le nombre d'événements et r_j l'effectif à risque à t_j,

$$\hat{P}_x = \prod_j \Big(1 - \frac{d_j}{r_j}\Big), \qquad \hat{q}_x = 1 - \hat{P}_x,$$

calculé par `-expm1(sum(log1p(-d / r)))`, plus précis que `1 - prod(1 - d / r)` pour les petits q (spécification, A.3 ; approximation numérique). Si $d_j = r_j$ à un instant, $\hat{q}_x = 1$. $\hat{q}_x$ est $1 - S((x+1)^-)/S(x^-)$, limites à gauche : l'événement du jour de l'anniversaire appartient à $x + 1$.

— **Justification** (spécification, A.3 ; preuve écrite). Avec des risques discrets h_j aux instants d'événement, une observation contribue, conditionnellement à sa présence en a_i, $1 - h_j$ à chaque instant où elle est à risque sans l'événement et h_j à son événement ; la vraisemblance $\prod_j h_j^{d_j}(1-h_j)^{r_j - d_j}$ est maximale en $h_j = d_j/r_j$. Aux instants où personne n'est à risque, elle ne dépend pas du risque : q_x n'y est **pas identifié**.

— **Cellule définie** : la couverture d'une cellule est l'union des intervalles $[\tau(a_i), \tau(b_i))$ non vides ; soit t_0 le premier instant de la cellule où $d = r$, ou $x + 1$ s'il n'y en a pas. La cellule est définie (`defini`) si $[x, t_0)$ est entièrement couvert ; après t_0, la survie est nulle et $\hat{q}_x = 1$ quel que soit le risque, qui n'est pas identifié. Sinon, l'état est `ensemble_a_risque_vide` (couverture incomplète : taux, variance et bornes `NA`) ou, sans aucune couverture ni événement, `exposition_nulle` (section 9.6). Un produit calculé sur la seule partie couverte sous-estimerait q_x (décision M52 (b), question 4).

— **Variance** : $\Sigma_x = \sum_j d_j / \big(r_j(r_j - d_j)\big)$, calculée en `double` (le produit d'entiers $r_j(r_j - d_j)$ déborde) ; $\widehat{\mathrm{Var}}(\hat{q}_x) = \hat{P}_x^2\, \Sigma_x$, par la méthode delta sur $\log \hat{P}_x$; restituée pour $0 < \hat{q}_x < 1$, `NA` à $\hat{q}_x \in \{0, 1\}$.

— **Intervalle asymptotique** : normal sur l'échelle de $\mathrm{logit}\, q$, $\mathrm{Var}(\mathrm{logit}\, \hat{q}_x) \approx \Sigma_x / \hat{q}_x^2$, soit

$$\mathrm{logit}^{-1}\big(\mathrm{logit}\, \hat{q}_x \mp z\sqrt{\Sigma_x}/\hat{q}_x\big)\ ;$$

`NA` à $\hat{q}_x \in \{0, 1\}$.

— **Aucun intervalle exact** (décision M52 (b), question 5) : sous troncature et censure, D ne suit aucune loi binomiale, et la colonne n'existe pas.

— **Identité exacte avec le binomial** (spécification, A.10 ; preuve écrite). Si toute observation dont la fenêtre fermée rencontre $[x, x+1)$ vérifie $a_i \le x$ et, soit $b_i \ge x + 1$, soit un événement en $b_i \in [x, x+1)$ (ni entrée ni censure dans la cellule, y compris à l'instant x), alors, avec n le nombre de ces observations, $r_{j+1} = r_j - d_j$, d'où $\hat{q}_x^{\mathrm{KM}} = D/n = \hat{q}_x^{B}$ $(E^i = n)$; par télescopage,

$\Sigma_x = 1/(n-D) - 1/n$, d'où $\Sigma_x/\hat{q}_x^2 = 1/D + 1/(n-D)$: les bornes asymptotiques sont celles du binomial.

— **Petit ensemble à risque** : sous troncature, une cellule peut être définie avec un ensemble à risque très réduit ; une seule unité à risque qui subit l'événement donne $\hat{q}_x = 1$. C'est l'instabilité de l'estimateur quand l'ensemble à risque est petit : le diagnostic `cellules_survie_nulle` compte ces cellules, et la colonne `ensemble_a_risque` de la table des sauts donne l'effectif à chaque instant.

— **Pas de survie cumulée par segment** (décision M52 (b), question 2) : le moteur restitue la survie dans la cellule (table des sauts) et q_x, non le produit sur tous les âges d'un segment ; sous troncature, ce produit tombe à zéro au premier âge où l'ensemble à risque s'épuise par un événement, et y reste (mesure de la spécification sur le cas composite C). Le chaînage des q_x en une fonction de survie relève de l'assemblage de la table.

— **Calcul** : `taux_kaplan_meier()` ; $r(t) = \#\{K_a \leq t\} - \#\{K_b < t\}$ par `findInterval()`, couverture par `cummax()`, en $O(n \log n)$. La table d'exposition est appariée **par clé** (segment, âge) : chaque instant d'événement est rattaché, par `match()`, à la ligne dont la cellule commence au même point de l'échelle des clés, sans hypothèse sur l'ordre des lignes ; les instants étant triés et les cellules étant des intervalles disjoints de cette échelle, les instants d'une même cellule forment une plage contiguë, sur laquelle se font les sommes cumulées de $\log(1 - d_j/r_j)$ et de Σ. Gardes-fous, erreurs de programmation et non refus : la segmentation ne doit pas porter de clé en double (sans quoi la table aurait une colonne parasite ; issue #22), garde-fou vérifié en premier ; la période et la transition doivent être celles de l'étape `exposition` ; les fenêtres transmises doivent porter les mêmes début et borne de période, transition, segmentation et convention (ci-dessus, « Données ») ; la clé d'un instant doit rester un entier exact en `double` ($< 2^{53}$) ; chaque clé (segment, âge) ne doit figurer qu'une fois dans la table d'exposition, sous son propre message (« clé (segment, âge) en double dans la table d'exposition »), contrôlé avant le suivant ; chaque instant d'événement doit tomber dans une cellule de la table d'exposition, et chaque segment de la table d'exposition figurer parmi les segments des observations (transmis ou recalculés), ces deux contrôles partageant le message « instant d'événement hors des cellules de la table d'exposition » ; les instants d'une cellule doivent être contigus ; enfin, la somme des d_j par cellule doit être `identical()` aux événements de la table d'exposition.

— **Statut** : $\hat{q}_x$, maximum de vraisemblance non paramétrique (risques discrets aux instants d'événement), résultat exact, valeur en arithmétique flottante ; non sans biais à effectif fini ; variance, asymptotique (ensemble à risque $\to \infty$ sur toute la cellule) ; intervalle, asymptotique, sous indépendance des unités d'observation. Sous l'unité contrat (rachat, ou décès sans `id_tete`), les contrats d'une même tête sont supposés indépendants : une dépendance positive rend l'intervalle trop étroit, ce que le relevé `tetes_multi_contrats` de l'étape `donnees` permet d'apprécier. La validité asymptotique de la variance sous troncature à gauche est une hypothèse, sans source établie.

— **Références** : à établir (issue #6) : appellation, formule de variance, résultat asymptotique sous troncature à gauche.

Cas construit qui départage les conventions d'ex aequo (cas K de la spécification, A.11 ; neuf têtes, attendus calculés à la main et vérifiés par les tests). Il réunit, à un même instant, l'entrée d'une tête et son décès ; l'entrée d'une tête et le décès d'une autre ; le décès d'une tête, la censure d'une autre et l'entrée d'une troisième : sous l'ensemble fermé, $\hat{q}_{69} = 1/6$ et $\hat{q}_{70} = 29/45$, avec $\Sigma_{70} = 3/10$; la convention prévisible et le binomial donnent d'autres valeurs de $\hat{q}_{70}$, si bien qu'une erreur de convention ne passerait pas inaperçue.

9.5 Statuts des grandeurs restituées

Le statut de chaque grandeur est restitué par le moteur, dans les paramètres de l'étape et dans la fiche (`statuts`, constante `STATUTS_BRUT` : colonnes `estimateur`, `grandeur`, `intervalle`, `statut`, `loi_supposee`, `calcul`), pour les seuls estimateurs retenus. Le tableau 17 en donne le contenu, complété pour les intervalles du binomial (note a) ; la mention « E^c fixe » ou « E^i fixe » est réservée aux intervalles exacts.

Table 17 : Statuts restitués par l'étape `brut` (constante `STATUTS_BRUT`).

Estimateur, grandeur	Intervalle	Statut	Loi supposée
`poisson`, taux	sans objet (estimation ponctuelle)	résultat exact : maximum de vraisemblance du modèle à force constante dans la cellule	force constante dans la cellule (vraisemblance de survie, proportionnelle à celle de Poisson)
`poisson`, taux	exact	exact sous la loi supposée : couverture $\geq$ niveau pour tout μ, E^c tenue pour fixe ; sous le modèle de survie (E^c aléatoire), couverture seulement asymptotique	$D \sim \mathcal{P}(\mu E^c)$, E^c fixe
`poisson`, taux	asymptotique	asymptotique ($\mu E^c \to \infty$) ; non défini à $D = 0$	$D \sim \mathcal{P}(\mu E^c)$
`poisson`, probabilité	sans objet	résultat exact : image du maximum de vraisemblance par $q = 1 - e^{-\mu}$, force constante sur la cellule entière	force constante dans la cellule
`poisson`, probabilité	exact	exact sous la loi supposée (transformation croissante des bornes exactes du taux : couverture identique)	$D \sim \mathcal{P}(\mu E^c)$, E^c fixe
`poisson`, probabilité	asymptotique	asymptotique (transformation croissante des bornes asymptotiques du taux : même régime) ; non défini à $D = 0$	$D \sim \mathcal{P}(\mu E^c)$
`binomial`, taux	sans objet	estimateur des moments : $E[D] = q\,E[E^i]$ exactement sous l'hypothèse de répartition ; convergent ; non sans biais à effectif fini	${}_{1-t}q_{x+t} = (1-t)q_x$, censure indépendante
`binomial`, taux	exact	exact sous la loi supposée pour une cellule observée entière (sans entrée ni censure, E^i entier) ; avec une entrée ou une censure dans la cellule, couverture non établie, même si E^i est entier (prolongement continu de la formule exacte si E^i n'est pas entier) [a]	$D \sim \mathcal{B}(E^i, q)$, E^i fixe
`binomial`, taux	asymptotique	asymptotique ($E^i q(1-q) \to \infty$) pour une cellule observée entière ; avec une entrée ou une censure dans la cellule, couverture non établie [a] ; non défini à $D = 0$ et à $D = E^i$	$D \sim \mathcal{B}(E^i, q)$
`kaplan_meier`, taux	sans objet	résultat exact : maximum de vraisemblance non paramétrique (risques discrets aux instants d'événement) ; non sans biais à effectif fini	troncature à gauche et censure non informatives, unités indépendantes ; aucune hypothèse de répartition dans la cellule
`kaplan_meier`, variance	sans objet	asymptotique (méthode delta sur $\log \hat{P}$, ensemble à risque $\to \infty$ sur toute la cellule) ; non définie à $q = 0$ et $q = 1$	idem

Estimateur, grandeur	Intervalle	Statut	Loi supposée
kaplan_meier, taux	asymptotique	asymptotique (ensemble à risque $\to \infty$ sur toute la cellule), sous indépendance des unités d'observation ; non défini à $q = 0$ et $q = 1$; sous l'unité contrat, contrats d'une même tête supposés indépendants (une dépendance positive rend l'intervalle trop étroit ; relevé tetes_multi_contrats)	idem
kaplan_meier, taux	exact	aucun : non restitué (sous troncature et censure, D ne suit aucune loi binomiale)	sans objet

[a] Statut documenté plus complet que le libellé inscrit dans la fiche : pour les deux intervalles du binomial, la colonne statut de STATUTS_BRUT (R/brut.R) ne pose pas encore la condition de cellule observée entière (intervalle exact : « exact sous la loi supposée si E^i est entier », suivi du seul cas non entier ; intervalle asymptotique : « asymptotique », avec son régime). Le statut qui fait foi est celui de ce tableau et de la section 9.3 (entrée « Statut ») ; la mise à jour du texte du code, donc de la fiche et des références de non-régression, est renvoyée à l'issue #31 (décision M55 du mainteneur, 18/10/2026). Aucune valeur calculée n'en dépend.
Les bornes exactes sont calculées numériquement par qgamma() ou qbeta() : leur valeur est une approximation numérique de la borne exacte, ce que la colonne calcul de la fiche écrit.

9.6 États des cellules et avertissements

Une cellule où l'estimateur n'est pas défini reçoit NA pour le taux et ses bornes, un *état* et, si elle compte un événement, un avertissement ; **jamais un refus** (décision M51, question 3) : un décès survenu le jour d'un anniversaire, au bord de la plage d'âges, ne doit pas bloquer la construction. Les états possibles sont ceux de ETATS_BRUT :

— defini, pour tous les estimateurs ;

— exposition_nulle : exposition nulle et aucun événement ; zéro explicite de la table d'exposition, compté dans les diagnostics, sans avertissement ;

— evenement_sans_exposition (Poisson) : $E^c = 0$ et $D > 0$; la vraisemblance croît sans borne et le maximum est infini (cas H8, H10b et H11 de l'étape exposition). Avertissement poisson_evenement_sans_exposition, de nombre le nombre de ces cellules, le message donnant aussi le nombre d'événements ;

— effectif_inferieur_aux_evenements (binomial) : $0 < E^i < D$, la tolérance appliquée ; le modèle binomial ne s'applique pas. C'est le cas d'une tête entrée dans la cellule puis touchée par l'événement, qui apporte moins d'une unité d'exposition initiale. Avertissement binomial_effectif_inferieur_aux_evenements, de nombre le nombre de ces cellules. Côté binomial, $E^i = 0$ avec $D > 0$ est impossible : chaque événement apporte un complément d'au moins un jour ;

— ensemble_a_risque_vide (kaplan_meier) : couverture incomplète de la cellule avant le premier instant où $d = r$ (section 9.4) ; taux, variance et bornes NA. L'avertissement kaplan_meier_ensemble_a_risque_vide n'est émis que pour les cellules de cet état **qui portent au moins un événement** ; son nombre est le nombre de ces cellules, et le message donne la somme de leurs événements (décision M53). Toutes les cellules de cet état restent comptées dans diagnostics$etats. Motif de la restriction : les cellules mal couvertes sont, en pratique, celles de l'âge minimal d'entrée, sans événement, et l'avertissement apparaîtrait sinon à presque chaque construction (mesure d'actuary sur des portefeuilles simulés).

Dans une cellule définie, les bornes asymptotiques valent en outre NA à $D = 0$ et, pour le binomial, à $D = E^i$ ($\hat{q} = 1$; borne exacte inférieure $(\alpha/2)^{1/D}$, borne supérieure 1) ; pour kaplan_meier, la variance et les bornes valent NA à $\hat{q}_x \in \{0, 1\}$.

9.7 Résultat, paramètres et diagnostics

etapes$brut$resultat contient une table par estimateur retenu, dans l'ordre de ESTIMATEURS_BRUT, aux **mêmes lignes et dans le même ordre** que la table d'exposition (zéro ligne si elle n'en a aucune) :

— poisson : clés de segmentation, age et, sous par_annee, annee ; evenements et exposition_centrale, identiques aux colonnes de l'exposition ; taux, borne_inf_exacte, borne_sup_exacte, borne_inf_asymptotique, borne_sup_asymptotique ; probabilite et ses quatre bornes, de même nom préfixé (probabilite_borne_inf_exacte…) ; etat ;

— binomial : clés, age, evenements, exposition_initiale ($E^c + \sum c_1$, avant tolérance), taux, les quatre bornes, etat ;

— kaplan_meier : clés, age, evenements et instants_evenement (entiers), taux ($\hat{q}_x$), variance, borne_inf_asymptotique, borne_sup_asymptotique, etat ;

— kaplan_meier_sauts, si kaplan_meier est retenu : une ligne par segment et par instant d'événement, triée par segment puis par instant : clés, age, numerateur ($j \cdot 133\,590/L$, entier), age_exact ($x + j/L$, quotient correctement arrondi du rationnel), ensemble_a_risque (r_j), evenements (d_j), survie_cellule (produit dans la cellule jusqu'à cet instant inclus) et variance_log_survie_cellule (Σ cumulée dans la cellule, infinie à partir d'un instant où $d = r$) ; ces deux dernières valent NA aux sauts des cellules non définies.

Paramètres effectifs : estimateurs (ordre canonique), niveau, exposition_initiale ("cellule"), tolerance_effectif, convention (retenue), unite_exposition, hypotheses (texte par estimateur retenu et, si Poisson est retenu, pour la probabilité, complété de sa portée ; constantes HYPOTHESES_BRUT et PORTEE_PROBABILITE) et statuts (tableau 17) ; si kaplan_meier est retenu, ensemble_a_risque ("ferme") et denominateur_instants (133 590, entier). **Diagnostics** : nombre de cellules, somme des événements, nombre de cellules sans événement, nombre de cellules où $D = E^i > 0$ à la tolérance près (NA si le binomial n'est pas retenu), et le tableau etats (estimateur, état, nombre de cellules) ; si kaplan_meier est retenu, la liste kaplan_meier, en fin de diagnostics, de cinq entiers : instants (nombre d'instants d'événement), evenements, evenements_a_l_instant_d_entree (événements dont l'instant coïncide avec le début d'observation retenu, $\max$(entrée, début de la période) : ceux qu'exclurait l'ensemble à risque prévisible), cellules_survie_nulle (cellules définies où $\hat{q}_x = 1$) et ensemble_a_risque_maximal (plus grand r_j, 0 sans événement). Les champs de kaplan_meier n'apparaissent que si l'estimateur est retenu : un plan qui ne le retient pas garde l'objet antérieur à son ajout. La fiche reçoit le champ hypotheses_brut (section 15.1).

9.8 Vérifications

tests/unitaires/test_brut.R vérifie, au niveau 0,95, avec z écrit en dur :

— des *formes closes*, indépendantes de qgamma(), qbeta() et pgamma() : Poisson à $D = 0$ (borne $-\ln(\alpha/2)/E$) et à $D = 1$ (borne supérieure u/E, u racine de $(1 + u)e^{-u} = \alpha/2$ obtenue par la méthode de Newton) ; binomial à E^i réel, $D = 0$ ($1 - (\alpha/2)^{1/n}$), $D = 1$ et $D = n$; séries explicites (Poisson $D \leq 5$, binomial $n \in \{2, 5, 12\}$) ;

— les cas d'une ligne et de fusion de l'étape exposition (H, F) et un cas composite à onze têtes (C), aux attendus calculés à la main : taux, bornes, états, avertissements, diagnostics ;

— chaque refus, depuis valider_plan() et depuis la fonction d'étape, avec le même code ; le niveau $1 - 2^{-53}$, admis, à bornes finies ;

— des identités exactes : evenements et exposition_centrale identical() aux colonnes de l'exposition, clés et ordre compris ; $\hat{\mu}E^c = D$ à 10^{-14} près ; exposition_initiale $= E^c +$ complement_initial_cellule ;

— l'*invariance* : un plan sans étape brut donne un objet identical() à celui du plan avec brut, privé de ses seuls ajouts (etapes$brut, fiche$hypotheses_brut, avertissements de l'étape et plan).

Portée du recoupement (CONTEXT.md). Les bornes exactes sont recoupées par stats::poisson.test() et, pour un E^i entier seulement, par stats::binom.test(), qui reposent sur les mêmes fonctions qgamma() et qbeta() que le moteur : le recoupement vérifie la formule, les branchements à $D = 0$ et $D = E^i$, la division par l'exposition et le partage $\alpha/2$, **pas** les fonctions spéciales ; ce sont les formes closes qui en font une vérification indépendante. Les bornes asymptotiques et la probabilité ne sont recoupées par aucune fonction de stats : leurs attendus sont calculés à la main.

tests/unitaires/test_brut_kaplan_meier.R vérifie l'estimateur kaplan_meier sur les cas H et F (une tête), le cas K des ex aequo, un cas Z (événement, censure et unité en cours au même instant x, censure exactement au début de la cellule) et le cas composite C : états, taux, variances et bornes calculés à la main, en formes closes ; les identités exactes (somme des d par cellule égale aux événements, clés et ordre de l'exposition, $1 \leq d \leq r$, survie au dernier saut égale à $1 - \hat{q}_x$) ; l'identité avec le binomial sur des cohortes d'âge exact simulées, qui sert aussi de test de débordement des entiers ; les refus, le message de etape_prealable_manquante, les types entiers, l'admission de age_atteint_annee ; l'invariance d'un plan sans kaplan_meier (objet identical() à celui du plan qui le retient, privé de ses seuls ajouts). Quatre tests portent sur les fenêtres transmises et l'appariement par clé (issue #38) : calculer_exposition() rend l'objet fenetres_exposition à côté de ses quatre éléments, l'étape brut donne des résultats identical() avec les fenêtres transmises et en appel direct sur les observations, et aucun objet de cette classe ne figure dans l'objet résultat ni dans la fiche ; trois permutations des lignes de la table d'exposition (l'inverse et deux tirages sous graine locale), par les deux voies, rendent la table kaplan_meier identical() à l'originale permutée, et la table des sauts et les diagnostics identical() ; chacun des cinq contrôles des fenêtres transmises arrête l'étape, un cas par contrôle, et des fenêtres inchangées ne l'arrêtent pas ; une table d'exposition dont une ligne est dupliquée arrête l'étape, en appel direct, par le message de la clé en double et non par celui de l'instant hors des cellules, et la table sans doublon ne l'arrête pas.

Contre-vérification par le package survival (ADR 0002 ; issue #7) : la production des valeurs par survfit se fait hors CI et hors batterie ; leur comparaison au moteur est un test ordinaire de la batterie unitaire, exécuté en CI. Le script tests/contre_verification/survfit_kaplan_meier.R, exécuté à la main sur un poste muni de survival et qui n'emploie pas tablexp, lit un jeu fixé (tests/contre_verification/jeu_kaplan_meier.csv : segments C, K et Z, chaque ligne en dates et en âges rationnels $x + j/L$) et appelle la fonction survfit de ce package, par cellule qui porte un événement, sur les intervalles $(a' - \delta, b']$ tronqués à la cellule, $\delta = 10^{-7} < 1/133\,590$, ce qui reproduit l'ensemble à risque fermé, intervalle sur l'échelle logit ; il écrit un fichier de valeurs dont l'en-tête porte la version de R, celle de survival, la date et sessionInfo(). Ce fichier, tests/contre_verification/valeurs_survfit_kaplan_meier.csv, est versionné ; il a été produit le 18/10/2026 sur le poste du mainteneur (Windows 11), hors CI, sous R 4.3.3 avec survival 3.5.8, versions consignées dans son en-tête, et compte onze sauts (cinq pour C, cinq pour K, un pour Z). Le test de test_brut_kaplan_meier.R compare ce fichier au moteur sur les cellules définies qui portent un événement, qui doivent être exactement celles que rend survfit : sept cellules (C 64, 65, 73 et 83, K 69 et 70, Z 70) et dix sauts ; la cellule C 38, rendue par survfit mais déclarée ensemble_a_risque_vide par le moteur, n'est pas comparée. Tolérances : âge exact à 10^{-13} près en absolu ; r et d exactement ; survie à chaque saut, $\hat{q}$ de la cellule

(comparé à $1 -$ surv au dernier saut) et écart-type de $\log \hat{P}$ à chaque saut à 10^{-13} près en relatif ; bornes de la cellule (comparées à `q_inf` et `q_sup` au dernier saut) à 10^{-12} près en relatif ; valeurs manquantes ou infinies (survie nulle, C 83 : écart-type infini, bornes manquantes) : identité exacte des positions et des valeurs. C'est un test ordinaire : un fichier absent, un en-tête incomplet (version de R, de `survival`, date ou `sessionInfo()`) ou un écart le fait échouer. Ses attendus sont des valeurs de *contre-vérification* (section 4.2) : produites par un package externe, écrites en dur, elles ne sont régénérées que sur ordre du mainteneur, avec un tableau avant / après (ADR 0002, point 4, et troisième annotation, A11). `survival` n'est chargé par aucune batterie.

`tests/test_reproductibilite.R` vérifie qu'une construction avec l'étape `brut` est `identical()` hors `execution` dans le même processus et dans un autre, que le rejeu de la fiche redonne l'objet, et l'invariance du plan sans `brut` sur deux plans de référence (R1 et R4, section 4.5) ; et de même pour le plan R6, qui retient `kaplan_meier`, avec l'invariance du plan sans `kaplan_meier`.

9.9 Constats de simulation : retrouver la loi vraie

Retrouver la loi vraie par les taux bruts est une batterie de constats de simulation (E2.19, E2.20), `tests/unitaires/test_constats_brut.R`, spécifiée par `actuary` le 17/10/2026 (issue #16, parties A.1 à A.8, et errata du même jour). Chaque test écrit, à chaque exécution, le nombre de têtes, le niveau α_k, la région d'acceptation, la fausse alarme et sa nature, la graine ; les valeurs observées ne sont pas reproduites ici (E3.4). Un constat de simulation n'est jamais un résultat exact sur le moteur : sous une graine fixée, la sortie du moteur tombe dans une région dont la probabilité de fausse alarme est écrite ; la graine étant fixe, le test est déterministe, et sa fausse alarme est la probabilité, sur les graines, qu'une implémentation juste échoue (comme pour le simulateur, section 16.4).

Cibles exactes sous le modèle simulé

Le simulateur tire des instants continus sous force constante par âge entier (section 16.1) ; le moteur ne voit que des jours. Dans une cellule de L jours, avec les forces μ_d (transition) et μ_r (sortie concurrente), $m = \mu_d + \mu_r$, une tête à risque au début d'un jour subit la transition avec la probabilité $h_d = (\mu_d/m)(1 - e^{-m/L})$, sort par la cause concurrente avec $h_r = (\mu_r/m)(1 - e^{-m/L})$ et survit au jour avec $e^{-m/L}$, probabilités constantes dans la cellule. D'où, en *résultats exacts* (preuve écrite de la spécification, A.1) :

— binomial et produit-limite sur une cellule observée entière, sans entrée ni censure et sans cause concurrente : $D \sim \mathcal{B}(n, q_x)$ exactement ; $E^i = n$ et le produit-limite égale le binomial (identité de la section 9.4, qui exige l'absence de censure dans la cellule). Avec une cause concurrente, $D \sim \mathcal{B}\big(n, (\mu_d/m)(1 - e^{-m})\big)$ reste exact, mais les sorties concurrentes censurent dans la cellule : $E^i < n$ dès qu'une sortie concurrente survient, le binomial ne vise plus q_x et le produit-limite vise q^* (point suivant) ;

— produit-limite sur la grille des jours : limite $q^* = 1 - (1 - h_d)^L$, égale à $1 - e^{-\mu_d}$ sans cause concurrente ;

— Poisson : $E[D - \mu^* E^c] = 0$, avec μ^* de la section 9.2 ;

— binomial, cohorte censurée par la période à la fraction c de la cellule : $D \sim \mathcal{B}(n, \pi_c)$, $\pi_c = 1 - e^{-\mu c}$, et les limites de la section 9.3, qui ne valent pas q.

Chaque test compare l'estimateur à sa cible exacte sous le modèle simulé, sauf en C2, où deux approximations de la cible sont faites et bornées (ci-dessous) ; l'écart de cette cible à la loi vraie est écrit à côté. Loi vraie des tests : celle du simulateur (Gompertz-Makeham $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, facteurs H 1 et F 0,6, plafond 1 ; rachat constant 0,05), convention `plus_fine`, délai nul, niveau 0,95.

Familles de tests

Hypothèse nulle, pour tous les tests sauf C3b : le moteur calcule correctement l'estimateur, si bien que sa sortie suit la loi déduite du modèle simulé (cibles ci-dessus) ; C3b a une hypothèse nulle composite, donnée avec lui. Pour chaque famille sont indiqués le jeu, la statistique et la région ; la fausse alarme et sa nature sont au tableau 19, les conditions de validité dans l'encadré qui suit.

— **C1, cohortes d'âge exact** : $n = 10^5$ têtes par cohorte, entrées le 2016-03-15 à l'âge exact x, période du 2016-03-15 au 2017-03-14, si bien que B est l'anniversaire $x+1$, que $L = 365$ et que la cellule x est observée entière (sauf C1.6, censurée au 2016-09-13, $c = 183/365$). *Statistiques* : binomial, $D \in [\text{qbinom}(\alpha_k/2, n, \pi), \text{qbinom}(1-\alpha_k/2, n, \pi)]$; Poisson, $U = D - \mu^* E^c \in [-t_{\text{lo}}, t_{\text{hi}}]$; produit-limite (C1.5), $z = (\text{logit}\,\hat{q} - \text{logit}\,q^*)\,\hat{q}(1-\hat{q})/\sqrt{\text{variance}}$, $|z| \leq \Phi^{-1}(1-\alpha_k/2)$. *Région de Poisson* : U est la somme de n variables indépendantes de même loi, de support fini (une valeur par jour de transition, de sortie concurrente, et la survie à la fin de la fenêtre) ; pour tout $\theta > 0$, $P(U \geq t) \leq \exp\bigl(-\theta t + n\Lambda(\theta)\bigr)$, Λ étant le logarithme de la fonction génératrice des moments d'un terme (inégalité $P(Y \geq a) \leq E[Y]/a$ pour $Y \geq 0$, appliquée à $Y = e^{\theta U}$) ; t_{hi} rend l'infimum en θ de cette borne égal à $\alpha_k/2$, et t_{lo} de même pour $-U$. La borne est exacte ; le calcul de t (`optimize()`, `uniroot()`) est une approximation numérique sans effet sur la validité : t est arrondi vers l'extérieur à 10^{-6} et la borne, recalculée en t, est vérifiée inférieure ou égale à $\alpha_k/2$. Le binomial n'est pas testé en C1.5, où le rachat concurrent censure dans la cellule : sa cible n'y est pas q.

— **C2, portefeuille tronqué et censuré, avec rachat** : mode uniforme, $n = 3 \cdot 10^5$, proportion d'hommes 0,5, âges 20 à 75, origine 2005-01-01, période 2015-01-01 au 2024-12-31, segmentation `sexe`, estimateurs `poisson` et `kaplan_meier`. Sélection *a priori*, sur l'exposition seule : cellules définies pour les deux estimateurs et $\lambda_x = \mu_x^* E_x^c \geq 30$; leur nombre $K = 83$ sous la graine déclarée est une constante du test, vérifiée à l'exécution ; une cellule sélectionnée où $\hat{q}$ vaut 0 ou 1 compte comme un rejet. Tests par cellule, à $a = 1{,}5 \cdot 10^{-4}/K$ chacun : Poisson, $D_x \in [\text{qpois}(a/2, \lambda_x), \text{qpois}(1 - a/2, \lambda_x)]$; produit-limite, $|z_x| \leq \Phi^{-1}(1-a/2)$, de cible $1 - e^{-\mu_{d,x}}$, et non la cible exacte q_x^* sur la grille des jours : l'écart relatif, de l'ordre de $-\mu_r/(2L)$, vaut environ $-7 \cdot 10^{-5}$ (rachat 0,05), négligeable devant l'erreur d'estimation par cellule. Cet écart de cible déplace les statistiques d'au plus des valeurs mesurées sous la graine déclarée, sur les $K = 83$ cellules : z_x d'au plus $1{,}4 \cdot 10^{-3}$ en valeur absolue par cellule, et la statistique globale $\sum_x z_x/\sqrt{K}$ de $-7{,}4 \cdot 10^{-3}$; la fausse alarme des tests produit-limite, par cellule et global, reste nominale asymptotique à ce déplacement près. Ce déplacement ne s'annule pas quand l'exposition croît : l'écart de cible est fixe et l'écart type de $\hat{q}$ décroît comme l'inverse de la racine de l'exposition, si bien que le déplacement de z_x croît comme $\sqrt{D_x}$. Tests globaux, à 10^{-4} chacun, $|z| \leq \Phi^{-1}(1 - 5 \cdot 10^{-5})$: Poisson signé, $\sum(D-\lambda)/\sqrt{\sum(1+\mu^*/365)\lambda}$, dont la variance est exacte à L fixé (scores de cellules disjointes non corrélés, preuve écrite de la spécification) ; produit-limite, $\sum_x z_x/\sqrt{K}$. *Approximation* : μ^*, donc λ_x, et la variance du test signé sont calculés à $L = 365$ pour toutes les cellules, y compris celles de 366 jours ; l'écart relatif de μ^* est alors au plus $6{,}7 \cdot 10^{-7}$ sur les 83 cellules retenues (mesuré), sans effet pratique sur les régions. Pas de test binomial : sa cible n'est pas q, et l'écart entre les deux définitions de l'exposition initiale y est indétectable à cet effectif.

— **C3, couverture des intervalles** : une simulation de $R \cdot m$ têtes indépendantes est découpée en R blocs de m têtes par un `segment` entier, $(\text{id_tete} - 1)$ div $m + 1$, ce qui donne R réplications indépendantes sous l'hypothèse usuelle sur le générateur ; un intervalle `NA` compte comme non couvrant. *C3a* : cohortes de femmes de 100 ans, $m = 50$, $R = 2\,000$; intervalles binomial exact, binomial logit, Poisson exact, Poisson log ; $N_{\text{couverts}} \sim \mathcal{B}(R, C)$ exactement, C étant la couverture exacte, calculée par le test par sommation sur la loi de D (et, pour Poisson, par convolution de la loi de l'exposition, $E^c = m - D + S/L$, calculée par transformée de Fourier rapide complétée de zéros jusqu'à une longueur donnée par `nextn()` : approximation numérique) ; région bilatérale exacte. *C3b* : portefeuille

d'hommes tronqué (âges 79 à 81 à l'entrée, origine et début de période le 2015-01-01, période jusqu'au 2016-12-31, rachat 0,05), $m = 600$, $R = 2\,000$, cellules 80 et 81 ; intervalles Poisson exact, Poisson log et produit-limite logit ; hypothèse nulle composite « couverture $\geq 0{,}95 - \eta$ », $\eta = 0{,}02$ déclaré (*choix de jugement*, sans source, appuyé sur les couvertures exactes calculées pour la spécification de #14) ; région $N \geq \text{qbinom}(\alpha_k, R, 0{,}93) = 1\,811$, de fausse alarme au plus $1{,}711 \cdot 10^{-5}$ pour toute couverture vraie $\geq 0{,}93$ (borne exacte, par monotonie). Chaque ligne écrit à part l'**erreur Monte-Carlo** de la couverture, $\sqrt{C(1-C)/R}$, fonction du seul nombre de réplications, et l'**erreur d'estimation**, demi-largeur moyenne des intervalles, fonction de l'exposition de chaque bloc. En C1 et C2, chaque test est une réplication unique : la largeur de la région y est l'erreur d'estimation, et il n'y a pas d'erreur Monte-Carlo de couverture.

Table 18 : Famille C1 : cibles, régions et fausses alarmes, valeurs calculées par le test (`tests/unitaires/test_constats_brut.R` ; $\alpha_k = 2 \cdot 10^{-5}$ par test ; graines 20261611 à 20261616).

Cohorte	π (binomial)	Région de D	FA exacte	Région de U	$\mu^*/\mu - 1$
C1.1 décès, 60 ans, H	0,008153179142	[697 ; 939]	$1{,}917 \cdot 10^{-5}$	[−133,169304 ; 140,755387]	$1{,}121 \cdot 10^{-5}$
C1.2 décès, 80 ans, F	0,03150085056	[2 917 ; 3 388]	$1{,}927 \cdot 10^{-5}$	[−265,646914 ; 272,957234]	$4{,}385 \cdot 10^{-5}$
C1.3 décès, 100 ans, F	0,182091641	[17 691 ; 18 731]	$1{,}998 \cdot 10^{-5}$	[−644,959663 ; 650,399965]	$2{,}754 \cdot 10^{-4}$
C1.4 rachat, 40 ans, $q^r = 0{,}05$, sans décès, unité contrat	0,05	[4 709 ; 5 296]	$1{,}989 \cdot 10^{-5}$	[−335,760302 ; 342,851378]	$7{,}027 \cdot 10^{-5}$
C1.5 décès, 60 ans, H, rachat concurrent $q^r = 0{,}30$	non testé	—	—	[−121,820313 ; 129,412807]	$5{,}000 \cdot 10^{-4}$
C1.6 décès, 100 ans, F, censure à $c = 183/365$	$\pi_c = 0{,}09586612604$	[9 192 ; 9 986]	$1{,}957 \cdot 10^{-5}$	[−466,670347 ; 473,207095]	$2{,}754 \cdot 10^{-4}$

En C1.5, le produit-limite est comparé à q^*, dont l'écart relatif à $1 - e^{-\mu_d}$ vaut $-4{,}864 \cdot 10^{-4}$; région $|z| \leq 4{,}2649$, fausse alarme nominale (asymptotique). En C1.6, la cible du binomial « cellule » est 0,1745649731, à $-4{,}13$ % de la loi vraie (section 9.3) ; le produit-limite y est à l'état `ensemble_a_risque_vide` : il refuse d'extrapoler là où le binomial s'appuie sur son hypothèse de répartition. *Identités* vérifiées à côté des constats, par des tests ordinaires : $E^i = n$ et $\hat{q}$ produit-limite $= \hat{q}$ binomial en C1.1 à C1.4 ; en C1.6, taux binomial $= D/((n-D)c+D)$ et E^c + `complement_initial_fenetre` $= nc$; en C3a, bornes du produit-limite égales aux bornes logit binomiales sur les 2 000 blocs. Couvertures exactes de C3a, écrites en dur et recalculées par le test : binomial exact 0,974129060872, binomial logit 0,958230588612, Poisson exact 0,968699258108, Poisson log 0,955857903542 ; régions [1 916 ; 1 975], [1 877 ; 1 951], [1 902 ; 1 967] et [1 871 ; 1 947], à $\alpha_k = 3 \cdot 10^{-5}$.

Fausse alarme familiale

Elle est bornée par l'inégalité de l'union pondérée, valable *sans hypothèse d'indépendance* ; les tests d'une même cohorte (binomial et Poisson) étant fortement dépendants, aucune valeur « sous indépendance » n'est donnée.

Table 19 : Allocation du niveau entre les familles (graines 20261611 à 20261616, 20261621, 20261631, 20261641, distinctes de celles du simulateur).

Famille	Tests	$\sum \alpha_k$	Nature de la fausse alarme
C1	12, à $2 \cdot 10^{-5}$	$2{,}4 \cdot 10^{-4}$	binomial exacte ; Poisson bornée exactement ; produit-limite (C1.5) nominale asymptotique
C2 par cellule	$2 \times K$, à $1{,}5 \cdot 10^{-4}/K$	$3{,}0 \cdot 10^{-4}$	nominale asymptotique (produit-limite : à un déplacement mesuré près ; Poisson : exacte si E^c était fixe, valeur calculée et écrite par le test)
C2 globaux	2, à 10^{-4}	$2{,}0 \cdot 10^{-4}$	nominale asymptotique (produit-limite : à un déplacement mesuré près)
C3a	4, à $3 \cdot 10^{-5}$	$1{,}2 \cdot 10^{-4}$	exacte
C3b	6, à $2 \cdot 10^{-5}$	$1{,}2 \cdot 10^{-4}$	bornée exactement
Total	190	$9{,}8 \cdot 10^{-4} \leq 10^{-3}$	dont exacte ou bornée exactement au plus $4{,}16 \cdot 10^{-4}$ (somme calculée par le test : $4{,}153 \cdot 10^{-4}$), nominale asymptotique $5{,}2 \cdot 10^{-4}$

La batterie unitaire porte ainsi deux familles de constats, celle du simulateur (au plus 10^{-3}, section 16.4) et celle-ci : sa fausse alarme est au plus $1{,}98 \cdot 10^{-3}$, nominale pour les parties asymptotiques.

Puissance et témoins

Deux témoins exécutés par le fichier montrent que les tests détectent une erreur plausible : en C1.6, l'exposition initiale « fenêtre » à la place de « cellule » donne un taux hors de la région binomiale (puissance exacte, calculée par sommation sur $\mathcal{B}(n, \pi_c)$, égale à 1 à 10^{-6} près) ; en C2, une cible décalée d'un an d'âge fait rejeter les deux tests globaux (constat sous la graine déclarée). Les tests par cellule de C2 n'ont pratiquement pas de puissance contre un tel décalage : ce sont les globaux qui la portent (puissances calculées par la spécification, A.4). La convention « prévisible » de l'ensemble à risque du produit-limite n'est pas détectable par ces constats (écart de l'ordre de 10^{-3} en relatif sur q, spécification, A.4) : sa garantie est le cas K (section 9.4).

Statuts et sources d'erreur (E2.20).

— *Résultats exacts* : les cibles sous le modèle simulé, la loi binomiale de D en C1 et de N_{couverts} en C3a, les couvertures exactes de C3a, la borne exponentielle de Poisson, la borne de C3b, l'inégalité de l'union, la puissance exacte du témoin C1.6.

— *Résultats asymptotiques* : les régions du produit-limite (C1.5, C2) et des tests de Poisson de C2 sous le modèle de survie.

— *Approximations numériques* : quantiles binomiaux et de Poisson, seuils t de la borne exponentielle (arrondis vers l'extérieur et vérifiés), convolution par transformée de Fourier.

— *Approximations de la cible* (C2), d'écart relatif écrit : le produit-limite comparé à $1 - e^{-\mu_d}$ plutôt qu'à q^* (environ $-7 \cdot 10^{-5}$), μ^* et la variance du test signé calculés à $L = 365$ pour toutes les cellules (au plus $6{,}7 \cdot 10^{-7}$ sur les 83 cellules retenues, mesuré). La première déplace z_x d'au plus $1{,}4 \cdot 10^{-3}$ par cellule et la statistique globale de $-7{,}4 \cdot 10^{-3}$ (mesurés sous la graine déclarée) ; la fausse alarme du produit-limite, test global compris, est nominale asymptotique à ce déplacement près, qui croît comme $\sqrt{D}$ et ne s'annule pas quand l'exposition croît.

— *Constat de simulation* : le passage des tests sous les graines déclarées.

— *Erreur Monte-Carlo* : en C3, la dispersion du nombre d'intervalles couvrants, fonction de R. *Erreur*

d'estimation : la largeur des régions de C1 et C2 et la demi-largeur des intervalles de C3, fonction de l'exposition.

— *Hypothèses* : indépendance des blocs et des cohortes sous le générateur (hypothèse usuelle, non démontrée) ; validité asymptotique des régions de C1.5 et de C2 ; $\eta = 0{,}02$, choix de jugement.

9.10 Fonctions du module

Table 20 : Définitions de premier niveau de R/brut.R.

Nom	Statut	Entrées	Sortie et rôle
ESTIMATEURS_BRUT	constante	—	estimateurs disponibles, dans l'ordre canonique des résultats
EXPOSITION_INITIALE_BRUT	constante	—	"cellule" : définition de l'exposition initiale du binomial, sans paramètre (section 9.3)
TOLERANCE_EFFECTIF	constante	—	10^{-9} : tolérance absolue sur $\lvert E^i - D \rvert$
ETATS_BRUT	constante	—	états possibles d'une cellule, par estimateur (section 9.6)
DENOMINATEUR_INSTANTS	constante	—	133 590 (365×366), entier : dénominateur commun des instants de kaplan_meier
ENSEMBLE_A_RISQUE_KM	constante	—	"ferme" : convention d'ensemble à risque (section 9.4)
HYPOTHESES_BRUT	constante	—	textes des hypothèses, par estimateur et pour la probabilité, inscrits dans les paramètres et dans la fiche
PORTEE_PROBABILITE	constante	—	portée de la probabilité de Poisson selon la cellule (année d'âge, année civile, âge × année)
STATUTS_BRUT	constante	—	statut, loi supposée et calcul de chaque grandeur (tableau 17)
controler_parametres_brut()	interne	estimateurs, niveau, par_annee	refus des valeurs (section 9.1) ; appelée par valider_plan() et, en appel direct (controler = TRUE), par la fonction d'étape (issue #22)
taux_poisson()	interne	D, E^c, niveau	taux, bornes exactes et asymptotiques, état (section 9.2)
taux_binomial()	interne	D, E^i, niveau, tolérance	taux, bornes, état, cellules où $D = E^i > 0$ (section 9.3)
taux_kaplan_meier()	interne	fenêtres transmises par l'étape exposition ou observations (appel direct), table d'exposition, segmentation, transition, période, délai, convention, niveau, paramètres de l'étape exposition	tables kaplan_meier et kaplan_meier_sauts, diagnostics (section 9.4)

Nom	Statut	Entrées	Sortie et rôle
calculer_taux_bruts()	interne	table d'exposition, segmentation, par_annee, convention, unité d'exposition, estimateurs, niveau ; pour kaplan_meier : observations (fenêtres transmises par l'étape exposition dans une construction, table des observations de l'étape donnees en appel direct ; issue #38), transition, période, délai, paramètres de l'étape exposition ; controler (TRUE par défaut)	fonction d'étape brut : paramètres effectifs, résultat, diagnostics, avertissements (section 9.7) ; avec controler = FALSE, réservé à construire_table(), ni la segmentation, ni les paramètres de l'étape, ni (pour kaplan_meier) la transition, la période et le délai ne sont recontrôlés (issue #22)

PARTIE VI

Module lissage

Lissage : R/lissage.R

Module prévu (E2.12) : lissages des taux bruts. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

10.1 Fonctions du module

TABLE 21 : Définitions de premier niveau de R/lissage.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/lissage.R absent			

PARTIE VII

Module positionnement

Positionnement sur une table de référence : R/positionnement.R

Module prévu (E2.12, E2.15) : positionnement d'une table d'expérience sur une table de référence. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

11.1 Fonctions du module

TABLE 22 : Définitions de premier niveau de R/positionnement.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/positionnement.R absent			

PARTIE VIII

Module prospective

Tables prospectives : R/prospective.R

Module prévu (E2.13, v2) : tables prospectives, toute table prospective étant une surface de mortalité.

12.1 Fonctions du module

TABLE 23 : Définitions de premier niveau de R/prospective.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/prospective.R absent			

PARTIE IX

Module fermeture

Fermeture aux âges extrêmes : R/fermeture.R

Module prévu (E2.12) : fermeture de la table aux âges extrêmes. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

13.1 Fonctions du module

TABLE 24 : Définitions de premier niveau de R/fermeture.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/fermeture.R absent			

PARTIE X

Module validation

Validation et suivi d'expérience : R/validation.R

Module prévu (E2.12, E2.18) : validation de la table construite et suivi d'expérience. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

14.1 Fonctions du module

TABLE 25 : Définitions de premier niveau de R/validation.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/validation.R absent			

Partie XI

Module rapport

Fiche de construction et empreintes : R/rapport.R

Le module rapport porte les quantités des rapports, et non l'affichage interactif (E4.1 et sa note de lecture du 24/10/2026, décision M108) : la fiche de construction, les empreintes des données reçues (section 15.2), du résultat et du code (section 15.3 ; issue #67), le texte du plan et de la fiche (section 15.4 ; ADR 0011, qui remplace le point 12 de l'ADR 0007) et le rapport figé, écrit en octets HTML sans calcul (section 15.5 ; ADR 0012, points 1 et 2). Le rejeu outillé, avec le contrôle de version_fiche, est dans R/construire_table.R, le module rapport ne pouvant pas appeler construire_table() (section 6.4 ; ADR 0012, point 3). L'écriture des tables CSV et xlsx à en-tête est reportée à la première branche qui produit une table finale (décision M93).

15.1 Fiche de construction

La *fiche de construction* est la description lisible par machine d'une construction, qui suffit à la rejouer à l'identique (E4.5 ; ADR 0005, point 6 ; ADR 0007, point 6.2, précisé par l'ADR 0011, point 5). Elle est assemblée par assembler_fiche() et rangée dans le champ fiche de l'objet. Elle ne contient **aucun horodatage** : deux constructions identiques ont des fiches identiques. Ses champs, dans cet ordre, en huit blocs :

— **identification** : version_fiche (VERSION_FICHE, 4) ; versions (tablexp, lue dans la description du package installé, et R.version.string) ; plateforme, dans l'ordre sysname, release et machine (système, version, machine, lus par Sys.info()), blas (BLAS, par extSoftVersion()), lapack (LAPACK, par La_library()) et bibliotheque_c (bibliothèque C, par extSoftVersion() ; ci-dessous, « Bibliothèque C ») ;

— **données** : roles, dans l'ordre du catalogue des rôles (CATALOGUE_ROLES) ; provenance, dans l'ordre fichier, empreinte, algorithme, et provenance_mention (« aucun fichier (données construites en mémoire) » si provenance est NULL) ; empreintes, dans l'ordre algorithme, representation, donnees, fichier, algorithme_fichier, representation_resultat, resultat, code, statut_resultat_code (sections 15.2 et 15.3) ;

— **plan** : plan, le **plan canonique** (ci-dessous) ;

— **paramètres retenus**, dans l'ordre du calcul : transition_code, code interne de la transition retenue ; fin_effective ; unite_observation ; recette ; convention (demandée, native, retenue, forcée) ; unite_exposition ; par_annee ;

— **exclusions** : exclusions (par motif, avec leur nombre ; une ligne exclue sous plusieurs motifs déclarés compte sous chacun) ; nombre_lignes_exclues (integer, nombre de lignes distinctes exclues, 0L sans exclusion ; ci-dessous) ;

— **hypothèses** : avertissement_unite, s'il y a lieu (message de l'avertissement unite_contrat) ; hypothese_censure ; hypotheses_brut, **seulement si l'étape brut est exécutée** : copie des paramètres effectifs de l'étape estimateurs, niveau, exposition_initiale, tolerance_effectif, hypotheses et statuts, plus ensemble_a_risque et denominateur_instants si l'estimateur kaplan_meier est retenu (section 9.7) ;

— **aléa** : graine et graine_mention (NA et « aucune étape aléatoire » dans cette branche) ; generateur (GENERATEUR) ;

— **avertissements** : avertissements, en dernier.

Ordre des champs (issue #64 ; avis d'actuary du 20/10/2026, consigné sous la décision M69, point 5). L'ordre suit la lecture d'un certificateur (E3.7) : ce qui identifie la construction, puis les données reçues, avec les rôles, la provenance et l'empreinte des données, que la procédure de rejeu fait vérifier avant toute exécution ; le plan déclaré ; les paramètres retenus, dans l'ordre où le calcul les fixe ; les exclusions ; les hypothèses, regroupées ; l'aléa ; les avertissements, en dernier. Auparavant, l'ordre de construction dispersait ces éléments : l'empreinte des données venait après l'aléa, la provenance était séparée des rôles, et hypothese_censure et hypotheses_brut étaient séparés par l'aléa, la provenance, les empreintes et les avertissements. L'ordre est fixé par une seule constante, ORDRE_CHAMPS_FICHE : assembler_fiche() construit la fiche puis la réordonne par ordonner_fiche(), qui garde les seuls champs présents ; un plan sans étape brut a donc le même ordre, sans hypotheses_brut. Un champ présent hors de cet ordre est une erreur de programmation, jamais un champ retiré en silence. Le corps de la fiche écrite (section 15.4) suit cet ordre, et les blocs de lecture en sont une partition (ci-dessous, « Blocs de lecture »). Seul l'ordre des noms de la fiche change : aucune valeur, aucune table ni aucun verdict.

Plan canonique (ADR 0011, points 5.1 et 5.2 ; issue #42). Deux plans de même sens doivent donner la même fiche et le même objet : avant l'issue #42, un plan aux champs permutés, à borne de période stockée en entier ou au délai 0L donnait des fiches différentes, et, pour la période, des paramètres d'étape différents (ADR 0011, « Contexte », mesure 3). construire_table() applique donc canoniser_plan() une seule fois, juste après valider_plan() : champs de premier niveau dans l'ordre de CHAMPS_PLAN ; transition dans l'ordre depart, arrivee ; periode dans l'ordre debut, fin, chaque borne de classe Date à stockage double ; delai_declaration en double ; graine : un integer sans NA est rendu tel quel, quelle que soit sa longueur, et un double unique, fini, à valeur entière et de valeur absolue au plus .Machine$integer.max est converti en integer, parce que set.seed() convertit son argument en entier et que deux graines de même valeur doivent donner la même fiche ; toute autre graine est une erreur de programmation (ADR 0011, annotation du 20/10/2026 du point 5.1 (g)). Cette conversion est **sans effet dans cette branche** : aucune étape ne tire d'aléa, si bien que valider_plan() refuse toute graine (graine_inattendue) avant la canonisation ; le contrat de validation de la graine sera fixé avec la première étape aléatoire (issue #61) ; paramètres de chaque étape dans l'ordre de PARAMETRES_ETAPES ; valeurs des trois champs qui forment un ensemble dans l'ordre de leur catalogue (ci-dessous, issue #63) : segmentation dans l'ordre de ROLES_SEGMENTATION, etapes$donnees$exclusions dans l'ordre du catalogue des motifs c(MOTIFS_LIGNES, MOTIFS_TETE) (tableau 12), etapes$brut$estimateurs dans l'ordre de ESTIMATEURS_BRUT ; toute autre valeur inchangée. Les conversions sont sans perte (tout entier de R est un double exact) et la canonisation est idempotente. Un stockage que la liste fermée de ces conversions ne prévoit pas, une étape absente du catalogue, ou un nom d'étape ou de paramètre en double, qui serait sinon réduit en silence à sa première occurrence, arrête le calcul par une erreur de programmation : il ne peut venir que d'un plan non validé, valider_plan() refusant ces cas en amont. Le plan canonique n'est pas un plan « résolu » : une convention déclarée plus_fine y reste plus_fine, la convention retenue étant dans fiche$convention, car

réécrire une déclaration par sa résolution serait une autre déclaration (aucun défaut implicite, ADR 0007, point 5.1). Le *plan effectif* d'E4.5 est ainsi la paire formée du plan canonique et des paramètres retenus (paramètres effectifs des étapes et, dans la fiche, `transition_code`, `fin_effective`, `unite_observation`, `recette`, `convention`, `unite_exposition`, `par_annee`, `hypothese_censure`, `hypotheses_brut`) (note de lecture d'E4.5, décision M64, Q6). L'ordre des rôles et celui des noms de la provenance n'ont d'effet ni sur les étapes ni sur l'empreinte des données ; la fiche les consigne dans un ordre fixe (décision M64, Q4). L'**asymétrie est voulue** : les données sont empreintées telles qu'elles sont reçues, parce que l'empreinte identifie l'entrée, tandis que le plan est canonisé selon son sens, parce que la fiche identifie la déclaration.

Ordre des valeurs des champs ensembles (issue #63 ; ADR 0011, quatrième annotation du 20/10/2026, qui révise le point 5.1 (h) ; décision M69, point 2). `segmentation`, `exclusions` et `estimateurs` sont des ensembles : `valider_plan()` y refuse les doublons, si bien que deux plans qui ne diffèrent que par l'ordre de ces valeurs ont le même sens et doivent, selon la règle ci-dessus, donner la même fiche. Tant que cet ordre était conservé tel qu'il était déclaré, la fiche se contredisait pour `estimateurs` : `fiche$plan` gardait l'ordre déclaré, alors que `fiche$hypotheses_brut$estimateurs` et les résultats de l'étape `brut` sont toujours dans l'ordre de `ESTIMATEURS_BRUT` (section 9) ; pour `exclusions` et `segmentation`, seul l'ordre de présentation changeait (lignes et colonnes des étapes, ordre des avertissements `exclusion`), aucune valeur. Les vecteurs d'ordre sont passés en argument à `canoniser_plan()` (`ordres`, liste nommée par le chemin du champ dans le plan : `segmentation`, `etapes/donnees/exclusions`, `etapes/brut/estimateurs`), comme `CHAMPS_PLAN` et `PARAMETRES_ETAPES` : `canoniser_plan()` ne lit ainsi aucune constante d'un autre module, le seul endroit qui sait quel champ suit quel catalogue est l'appel, et la fonction se teste avec des catalogues fabriqués ; ce choix de localité n'est pas imposé par le graphe des modules, qui autoriserait la lecture directe. La liste est construite au moment de l'appel par `ordres_canoniques()` (`R/construire_table.R`), et non par une constante de premier niveau : sans champ `Collate` dans `DESCRIPTION`, les fichiers de `R/` sont évalués par ordre alphabétique à l'installation, et `construire_table.R` l'est avant `donnees.R`, dont les constantes (`ROLES_SEGMENTATION`, `MOTIFS_LIGNES`, `MOTIFS_TETE`) ne sont pas encore définies ; `brut.R` (`ESTIMATEURS_BRUT`) l'est déjà, mais la fonction lit les trois vecteurs au même endroit. Aucune valeur n'est retirée en silence : avant de réordonner, une valeur absente du vecteur d'ordre, un doublon, un `NA`, un attribut ou un type autre que `character` arrêtent le calcul par une erreur de programmation (garde défensive, `valider_plan()` refusant ces cas en amont) ; une étape absente du plan (plan sans `brut`) n'a rien à canoniser. Un emboîtement de segments, s'il devient nécessaire avec plusieurs variables de segmentation, sera porté par un paramètre déclaré à part, jamais par l'ordre de `segmentation`. **Effet** : les plans R1 à R6, déjà dans l'ordre canonique, sont inchangés ; pour un plan non canonique sur l'un de ces champs, `fiche$plan` et l'ordre de présentation des étapes et des avertissements changent, aucune valeur numérique. La forme canonique des valeurs du plan fait partie de ce que numérote `version_fiche` : une fiche de version 3 ou ultérieure garantit cet ordre, une fiche de version 2 ne le garantit pas ; l'issue #63 tombe sous l'incrément de 2 à 3 porté par l'issue #57 (ADR 0011, point 10). Les tests de `tests/unitaires/test_construire_table.R` vérifient, pour chacun des trois champs et pour les trois ensemble, que l'objet d'un plan permuté est `identical()` hors `execution` à celui du plan canonique, fiche comprise, que la canonisation est idempotente et que son résultat reste valide, que le rejeu de la fiche d'un plan permuté redonne l'objet, et que la valeur inconnue, le doublon et l'attribut donnent l'erreur de programmation.

Copies retirées (issue #42, avec l'issue #22, point 6 ; décision M64, Q2). Jusqu'à la version 1 de la fiche, `transition`, `periode`, `delai_declaration` et `segmentation` figuraient à la fois dans `fiche$plan` et au premier niveau de la fiche, sans que celle-ci dise laquelle faisait foi, `periode` voisinant en outre `fin_effective`. Ces quatre copies sont retirées ; restent `transition_code` et `fin_effective`, qui sont des paramètres retenus et non des copies, ainsi que les reprises de l'objet nécessaires à une fiche autosuffisante (`convention`, `recette`, `exclusions`, `avertissements`, `hypotheses_brut` ; E3.7). Le changement

de l'ensemble des champs fait passer version_fiche de 1 à 2 (ADR 0011, point 10) ; il a été visé par actuary et consigné par la seconde régénération des références (section 4.5).

Nombre de lignes exclues (issue #57 ; décision M69, point 1). exclusions compte les lignes par motif : une ligne qui porte deux motifs déclarés y compte deux fois, si bien que la somme des décomptes surestime le volume écarté. nombre_lignes_exclues donne le nombre de lignes *distinctes* exclues, repris de l'étape donnees (resultat$nombre_lignes_exclues, section 7.3) : c'est le nombre d'éléments de l'union des numéros de lignes exclues sous les motifs déclarés. Les lignes conservées étant exactement les autres, le rapprochement « lignes reçues = lignes conservées + nombre_lignes_exclues » est une identité exacte, par construction, et non une approximation. Un plan sans exclusion, ou dont les motifs déclarés ne portent sur aucune ligne, donne 0L. Le champ suit exclusions dans la fiche et appartient au bloc de lecture « hypothèses » (ci-dessous, « Blocs de lecture »). Par décision du mainteneur (M69, point 1 ; issue #69), l'outil restitue des grandeurs objectives, le jugement de pertinence des exclusions et leur justification restant à l'entreprise et au certificateur, et aucune justification déclarée n'entre dans le plan. La même décision renvoie à l'issue #69 la question réglementaire des exclusions et les grandeurs qui mesureraient leur importance (décès et exposition retirés par les exclusions, SMR des lignes exclues). L'ajout du champ change l'ensemble des champs et fait passer version_fiche de 2 à 3 (ADR 0011, point 10 (c)) ; l'incrément est porté par le premier commit de la branche claude/fiche-rapport qui change la définition de la fiche, celui de l'issue #57, et il est le seul de la branche (ADR 0011, point 10 (a), et troisième et quatrième annotations du 20/10/2026). Le changement a été visé par actuary ; aucune table, aucun taux ni aucun verdict ne change.

Bibliothèque C (issue #77 ; ADR 0011, seconde annotation du 24/10/2026). fiche$plateforme$bibliotheque_c, dernier sous-champ de plateforme, est rendu par bibliotheque_c() : c'est l'élément iconv de extSoftVersion(), tel quel. R 4.3.3 construit cette valeur dans la fonction C do_eSoftVersion du fichier src/main/platform.c de ses sources (étiquette R-4-3-3 du dépôt Subversion de R, https://svn.r-project.org/R/tags/R-4-3-3/src/main/platform.c, l. 3592–3610, relu le 24/10/2026), où le préprocesseur retient la première de quatre formes, dans cet ordre :

1. « GNU libiconv X.Y » (« Apple or GNU libiconv X.Y » sous macOS) si R est lié à la bibliothèque GNU libiconv, **même sur un système glibc** ;
2. « win_iconv » sous Windows ;
3. « glibc X.Y » sous la glibc sans libiconv, la version étant celle que rend la fonction C gnu_get_libc_version **à l'exécution**, donc celle de la glibc chargée et non celle de la compilation ;
4. « unknown » sinon, gardé tel quel.

Le champ n'identifie la glibc, donc la bibliothèque mathématique (libm) dont viennent les écarts d'un ulp entre machines (section 15.3), **que si sa valeur commence par « glibc »** ; sinon il désigne l'implémentation d'iconv (libiconv, win_iconv) ou rien (« unknown »), jamais la libm, et la fiche n'en dit pas davantage. Rien n'est deviné : si l'élément est absent, NA, vide ou fait de blancs, ou n'est pas un texte, le champ vaut la mention « non relevée » (MENTION_BIBLIOTHEQUE_C_NON_RELEVEE), distincte de la valeur brute « unknown » de R. Sur un hôte hors de la plateforme de référence (Ubuntu 24.04.5, R 4.3.3 ; *constat de mesure* du 24/10/2026), la fonction rend « glibc 2.39 » ; sur la plateforme de référence, l'en-tête des références régénérées le 24/10/2026 porte « glibc 2.35 » (section 4.5). La forme Windows (« win_iconv », attendue d'après le cas 2) n'a pas encore été mesurée : elle reste *non vérifiée* par mesure (critère 2 de l'issue #77, à fournir par le mainteneur). Comme tout fiche$plateforme, le champ est **hors du périmètre** de l'empreinte du résultat (CHEMINS_HORS_RESULTAT, section 15.3) et des grandeurs des références, dont il n'occupe que l'en-tête. Son ajout change l'ensemble des champs et fait passer version_fiche de 3 à 4 (ADR 0011, point 10 (c)) : sans l'incrément, une fiche de version 3, qui n'a pas le

champ (« champ inexistant à cette version »), ne se distinguerait pas d'une fiche où la bibliothèque n'a pas été relevée (« relevé tenté, sans valeur »). C'est le seul incrément de la branche `claude/rapport-fige` ; `fiche$version_fiche` étant dans le périmètre de l'empreinte du résultat, celle-ci change pour les six plans de référence. Ces deux valeurs, avec l'empreinte du code, que change tout commit de `R/`, sont les seules des références qui aient changé (visa d'`actuary` du 24/10/2026, décision M83 ; section 4.5). Aucune table, aucun taux ni aucun verdict ne change.
Les champs qui n'ont pas d'objet (convention et unité d'exposition d'un plan sans étape `exposition`) valent `NULL` ou `NA`. **Rejeu** : `construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance)` redonne un objet `identical()` hors `execution` ; c'est un test de la batterie de reproductibilité, aussi à partir du texte de la fiche. Le rejeu outillé, `rejouer_fiche(donnees, fiche)`, fait cet appel après avoir contrôlé la fiche et les données, et refuse en particulier une fiche dont `version_fiche` n'est pas une version admise, la version courante seule (`VERSIONS_FICHE_REJEU` ; section 6.4). La fiche s'écrit et se relit en texte (section 15.4), sous les contraintes posées par l'ADR 0007, point 12.2 (texte lisible par un relecteur, aller-retour identique, nombres compris, aucune évaluation de code à l'import, R de base, version), maintenues par l'ADR 0011, qui remplace ce point.

15.2 Empreinte des données

Algorithme et portée

L'empreinte identifie l'entrée reçue par le moteur ; elle n'a pas de rôle de sécurité. Elle est un MD5, calculé par `tools::md5sum()` sur un fichier temporaire écrit en binaire (`empreinte_donnees()`), l'algorithme étant nommé dans la fiche (`empreintes$algorithme`, `"md5"`) pour qu'un changement ultérieur soit visible et daté (décision M45, P8 ; ADR 0007, point 9). R 4.3.3 n'offre ni SHA-256 ni empreinte directe d'un vecteur d'octets dans `tools` (ADR 0007, mesures), et le moteur n'admet aucun package hors priorité `base`. L'empreinte n'est pas calculée sur `serialize()` : l'en-tête de la sérialisation porte la version de R qui écrit, si bien qu'à données identiques l'empreinte changerait avec la version de R. Elle est calculée sur une *représentation textuelle canonique*, définie indépendamment de la manière de l'écrire, qu'un tiers peut recalculer avec n'importe quel outil MD5.

La fiche porte : `algorithme` ; `representation` (`ENTETE_CANONIQUE`, `"tablexp canonique 2"`) ; `donnees` ; `fichier` et `algorithme_fichier`, reçus par `provenance` (`NA` sans fichier) ; puis `representation_resultat`, `resultat`, `code` et `statut_resultat_code`, qui portent les empreintes du résultat et du code et leur statut (section 15.3). L'argument `provenance` est `NULL` ou une **liste nue**, dont le seul attribut admis est celui des noms, de **exactement trois éléments**, nommés `fichier`, `empreinte` et `algorithme` dans un ordre quelconque, chacun une chaîne (`character`) de longueur 1, sans attribut, ni `NA` ni vide ; un `data.frame`, un objet de classe, une liste munie d'un autre attribut, un élément de plus (même nommé `NA`) ou une chaîne munie de noms, d'une dimension ou d'une classe sont refusés : refus `provenance_invalide` (`controler_provenance()`, par `liste_aux_noms()` et `chaine_unique()`, qui s'appuient sur `forme_exacte()` ; issues #21 et #48).

Représentation canonique

`texte_canonique()` écrit les octets suivants (ADR 0007, point 9.3, et annotation du 17/10/2026, point f) :

— **périmètre** : les seules colonnes auxquelles un rôle est déclaré, **toutes les lignes reçues**, avant exclusions, dans l'ordre reçu ; les exclusions sont des paramètres du plan, non une propriété de l'entrée ; une base dont les lignes sont permutées a une autre empreinte, et ses résultats ne sont pas garantis identiques au bit près (E4.6 ne porte que sur des données identiques) : les lignes conservées

et les observations suivent l'ordre reçu ; les grandeurs entières ou à valeur entière (exposition en jours, événements, sorties concurrentes), les tables de Kaplan-Meier et les diagnostics sont restés identiques ; l'exposition centrale, les compléments initiaux, l'exposition initiale, les taux, les probabilités et leurs bornes (Poisson et binomial), sommés par cellule dans l'ordre des lignes (`somme_par_cellule()`), peuvent différer dans les derniers bits. Écart relatif maximal mesuré : $1{,}6 \cdot 10^{-14}$ sur l'entrée de référence (10^5 lignes, plans R1 à R6, six permutations, R 4.3.3, 19/10/2026), atteint par la borne supérieure exacte (`borne_sup_exacte` de l'estimateur binomial, plan R2) ; les taux et l'exposition centrale restent au plus à $5{,}73 \cdot 10^{-15}$. L'écart croît avec le nombre de termes sommés par cellule : sur l'exposition centrale (plan R1, une permutation tirée sous la graine 20261019), $3{,}97 \cdot 10^{-15}$ à 10^5 lignes, $1{,}37 \cdot 10^{-14}$ à 10^6 et $1{,}73 \cdot 10^{-14}$ à $3 \cdot 10^6$. C'est un **constat de mesure, non une borne démontrée** ; le test de permutation de la batterie de reproductibilité (plans R6, R4, R5, R3 et R2, R1 étant couvert par R6 ; deux permutations, un tirage sous la graine 20261019 et l'ordre inverse) le contrôle à la tolérance relative 10^{-12} (constante `TOLERANCE_PERMUTATION`, classe existante de `CLASSES_TOLERANCE` ; issue #39) ; il vérifie l'invariance à l'ordre des lignes, non la précision des sommes, que garde la comparaison aux références de non-régression dans l'ordre reçu (classe 10^{-14}) : un écart dépendant de l'ordre inférieur à 10^{-12} n'y est pas détecté ; les grandeurs qui ne sont pas des doubles, les doubles de classe exacte, les tables de Kaplan-Meier et les diagnostics de l'étape `brut` y sont comparés par `identical()`, les trois empreintes (données, lignes conservées, observations) devant au contraire différer ; tout double qui n'est dans aucune de ces deux classes (« double non classé ») fait échouer le test ;

— **ordre des colonnes** : par nom de rôle, en ordre d'octets ; le nom de colonne n'entre pas dans le texte (il est dans `fiche$roles`) : renommer une colonne ne change pas l'empreinte ;

— **structure** : une première ligne `tablexp canonique 2<TAB><nombre de lignes><TAB><nombre de colonnes>`, puis, pour chaque colonne, une ligne d'en-tête `#<rôle><TAB><type>` suivie d'une valeur par ligne ; lignes terminées par LF, aucun CR, fichier écrit octet par octet ;

— **valeurs** : `double` écrit en **octets IEEE 754** (représentation 2, issue #35 ; ADR 0011, point 2) : une valeur finie x s'écrit en seize chiffres hexadécimaux minuscules, les huit octets de $x + 0$ dans l'ordre petit-boutiste (`writeBin(x + 0, raw(), size = 8, endian = "little")` ; l'ajout de 0 normalise le zéro négatif), par exemple `666666666666ee3f` pour 0,95, `000000000000f03f` pour 1 et `0100000000000000` pour le plus petit sous-normal ; une valeur non finie s'écrit par un jeton, `NA`, `NaN`, `Inf` ou `-Inf`, **lu sur la valeur reçue**, jamais sur $x + 0$, et non par ses octets : ceux d'un `NA` ou d'un `NaN` ne sont pas stables d'une opération à l'autre (`NA_real_` et `NA_real_ + 1` n'ont pas les mêmes octets sous x86-64), et l'aide de R ne garantit pas qu'une opération préserve la distinction entre `NA` et `NaN` (ADR 0011, point 2.2, et seconde annotation du 19/10/2026) ; le jeton est lu par une règle écrite une seule fois, `jeton_non_fini()`, qui rend son rang dans `JETONS_NON_FINIS` (1 `NA` si `is.na()` et non `is.nan()`, 2 `NaN`, 3 `Inf`, 4 `-Inf`), commune aux formes « empreinte » et « texte » de l'écrivain (issue #118) ; le classement d'un `NaN` à charge utile en `NA` ou en `NaN` est celui de R (`is.na()`, `is.nan()`), non une règle de l'ADR 0011, et la batterie le fixe sur huit motifs par un test témoin ; un double muni d'un attribut de dimension s'écrit comme le vecteur nu ; `integer` en `%d` ; `logical` en `TRUE`, `FALSE`, `NA` ; `Date` en ISO 8601, l'année toujours sur quatre chiffres (`sprintf("%04d-%02d-%02d", ...)` sur les composantes de la date, `composantes_date()` ; `format.Date` écrivait `999-01-01` sans zéro initial), et une date manquante par un jeton lu sur la valeur reçue, `NA` ou `NaN`, sans les confondre, comme pour un double (ADR 0011, point 4, et seconde annotation du 19/10/2026, point (d) ; issue #19) ; l'écrivain refuse, en garde-fou des contrôles de l'étape `donnees`, une date hors du domaine (`date_hors_domaine`, ici pour la seule colonne écrite, son rôle et ses lignes dans `details`) et une date non entière (`date_non_entiere` : 100 et 100,5 jours s'écriraient de même) ; chaînes et facteurs (convertis en chaînes, les niveaux n'entrent pas) : octets UTF-8 entre guillemets droits, avec cinq

échappements (\\, \n, \r, \t, \"), NA sans guillemets pour une valeur manquante ;

— **codage** : les chaînes sont converties en UTF-8 puis traitées en octets ; le moteur refuse une chaîne au codage bytes (chaine_codage_bytes), une chaîne qui n'est pas de l'UTF-8 valide (chaine_non_utf8) et, dans une session dont la locale n'est ni UTF-8 ni latin-1, une chaîne non ASCII non marquée, dont le codage est indéterminé (chaine_codage_indetermine). La conversion est celle de la règle unique vers_utf8_locale() (module outils ; ADR 0007, point 9.3, annotation du 05/11/2026, (g) ; issue #113), appelée après les refus chaine_codage_bytes et chaine_codage_indetermine, qui restent dans le module rapport, et avant le refus chaine_non_utf8, qui porte sur la chaîne convertie : enc2utf8() sur toutes les chaînes sous une locale UTF-8 ou latin-1, sur les seules chaînes marquées latin-1 sinon. Elle équivaut, après ces deux refus, à enc2utf8() sur tout, en toute locale : sous une locale UTF-8 ou latin-1, les deux règles sont enc2utf8() par définition ; sous une autre locale, une chaîne restante est ASCII, marquée UTF-8 ou marquée latin-1, et enc2utf8() ne change que la dernière. L'ordre des segments (section 8.2) appelle la même fonction. Sous ces règles, l'empreinte ne dépend pas des paramètres régionaux.

Deux formes, un seul écrivain (ADR 0011, point 3 ; décision M64, Q1). ecrire_valeurs() écrit les valeurs d'une colonne sous deux formes, qui ne diffèrent que pour les doubles : la forme « empreinte » ci-dessus, seule employée par texte_canonique() et par le texte empreinté du résultat (section 15.3), et la forme « texte », pour les textes qu'un relecteur lit et qu'on relit (références de non-régression, section 4.5 ; texte du plan et de la fiche, section 15.4), où un double fini x s'écrit en dix-sept chiffres significatifs, forme %.17g, calculés par tablexp sans la bibliothèque C (octets_17g() ; issue #104, ci-dessous, « Écriture « texte » des doubles »), et un non fini par son jeton (NA, NaN, Inf, -Inf), le jeton étant lu, là aussi, sur la valeur reçue. Dix-sept chiffres significatifs suffisent à désigner un double sans ambiguïté, quinze non ; encore faut-il que l'écriture soit correctement arrondie et la relecture exacte. L'arrondi correct de l'écriture est établi par construction, par une preuve (hypothèse H), sur toute plateforme qui satisfait H, et *constaté* par deux juges (glibc et Python) (ci-dessous). La relecture exacte reste un *constat de mesure* sous la bibliothèque C de GNU (glibc) (ADR 0011, point 3.2 ; relecture exacte constatée sur 1 399 465 doubles), appuyé sur un argument d'ordre de grandeur, non une preuve. Sous Windows, avant l'issue #104, l'écriture par la bibliothèque C n'était pas toujours correctement arrondie, et la relecture n'était pas toujours exacte (constat sur le poste du mainteneur, ADR 0011, point 3.2). Avec l'écriture par tablexp, l'identité des octets écrits sous Windows (R 4.3.3 ucrt) à ceux de la plateforme de référence et la relecture exacte y sont désormais des *constats de mesure*, rapportés de l'étape (5) de la pull request #116, sur le poste du mainteneur (ADR 0011, annotation du 27/10/2026, (a) et (b)) : les empreintes MD5 des textes des témoins T2 et T3 y sont égales à celles de la CI, et 999 493 doubles finis, tirés uniformément sur les motifs de 64 bits, écrits puis relus, ne donnent aucune relecture fausse. Cet échantillon n'est pas celui que prévoyait le protocole (celui de l'étape (3a)), et il est dominé par des exposants hors des grandeurs d'un plan ou d'une fiche ; 0 faute sur 999 493 borne le taux de faute à $3{,}0 \cdot 10^{-6}$ par double (niveau 95 %, borne unilatérale de Clopper-Pearson), pour cette seule loi d'échantillonnage. H n'y est pas mesurée et reste une **hypothèse**, que ce constat ne contredit pas ; sous macOS, rien n'est mesuré (décision M61, issue #49). Les entiers, logiques, chaînes, facteurs et dates s'écrivent de même dans les deux formes. Une forme inconnue est refusée (forme_ecriture_inconnue).

Pourquoi des octets, et non %.17g, dans l'empreinte. L'écriture décimale par la bibliothèque C (sprintf) dépend du système : sous Windows, elle n'arrondit pas toujours correctement le dix-septième chiffre (constat sur le poste du mainteneur, ADR 0011, « Contexte », mesure 1), si bien qu'à données identiques au bit près l'empreinte d'une base portant une colonne double différerait d'un poste à l'autre. L'issue #104 a rouvert le calcul de l'écriture décimale par tablexp (option (v) de l'ADR 0011, point 2.6), **pour la seule forme « texte »** (ADR 0011, annotation du 26/10/2026, (a) 3) : l'empreinte n'a pas à être lisible, et ses octets sont exacts par construction sans aucun calcul de chiffres. Les octets ne dépendent

que de la norme IEC 60559 (IEEE 754), que R exige de toute plateforme (aide de double), et de l'ordre des octets, fixé. Deux autres écritures ont été écartées (ADR 0011, point 2.6) : %a, dont l'aide de sprintf dit le format non défini de façon unique, et le plus court texte décimal relu à l'identique, parce que la lecture décimale de R n'est pas correctement arrondie. **Statut** : écriture exacte par construction (norme IEC 60559 exigée par R, ordre des octets fixé, aucune mise en forme par la bibliothèque C), recoupée sous glibc par un juge indépendant (Python, struct.pack, section 15.6) ; sous Windows et macOS, **non mesurée** (tests sous Windows ajournés, décision M61, issue #49 ; ADR 0011, point 1). L'entrée de référence des tests ne porte aucune colonne double : ses colonnes s'écrivent à l'identique sous les représentations 1 et 2, et seul l'en-tête change son empreinte.

Le libellé de type de l'en-tête est pris dans une **liste fermée de six chaînes**, par des tests faits dans l'ordre du tableau 26 (ecrire_valeurs()) ; le premier qui réussit fixe le libellé. Un type hors de la liste est refusé (type_non_canonique), sans conversion. C'est une seconde barrière : le contrôle des types des rôles (type_incompatible, section 7.2) refuse déjà, avant le calcul de l'empreinte, toute classe hors du catalogue ; aucun rôle n'admet aujourd'hui une colonne logique, si bien que le libellé logical n'est pas produit par construire_table().

Table 26 : Libellés de type de la représentation canonique, dans l'ordre des tests.

Ordre	Libellé	Test
1	factor	facteur, ordonné ou non
2	Date	classe exactement Date (une classe Date augmentée échoue et tombe au test 3) ; valeur hors de [0001-01-01, 9999-12-31] ou non entière refusée (date_hors_domaine, date_non_entiere)
3	refus	toute autre classe (POSIXct, difftime, AsIs. . .)
4	character	chaîne
5	integer	entier
6	double	double
7	logical	logique
8	refus	tout autre type de base (liste, complexe, octets bruts. . .)

Pourquoi factor reste distinct de character. Les valeurs s'écrivent de la même façon ; seul le libellé distingue une colonne facteur d'une colonne de chaînes aux mêmes valeurs. L'empreinte identifiant l'entrée, deux data.frame dont une colonne n'a pas la même classe sont deux entrées différentes, que la fiche doit permettre de distinguer. L'empreinte ne dépend pas, en revanche, des niveaux d'un facteur ; c'est pourquoi aucun résultat ne doit en dépendre (clés de segmentation, section 8.2).

Règle durable. Changer la représentation canonique (libellé, ordre des tests, format d'une valeur) change l'empreinte des données de toutes les fiches, parce que la première ligne du texte empreinté porte la version : c'est un changement de résultat, qui appelle l'incrément de la version de la représentation (ENTETE_CANONIQUE, tablexp canonique N, portée par fiche$empreintes$representation), au plus une fois par branche, et le visa (ADR 0011, point 10). Une empreinte nomme ainsi sa définition. Cette version est distincte de version_fiche, qui numérote la structure de la fiche (ensemble des champs et types canoniques). La représentation est passée de 1 à 2 avec l'issue #35 (doubles en octets) : l'empreinte des données des six plans de référence a changé, l'entrée restant la même, ce qu'a prouvé la garde « entrée différente » (section 4.5, « Règles durables »).

Écriture « texte » des doubles

Règle (issue #104 ; décisions M148 et M161 ; ADR 0011, annotation du 26/10/2026). En forme « texte », `ecrire_valeurs()` écrit les doubles par `octets_17g()` : aucun double n'y est mis en forme par `sprintf()`, `format()`, `formatC()` ni `as.character()` ; les chiffres sont calculés par `tablexp` sur des entiers exacts, puis écrits en octets ASCII, une valeur par ligne, LF final. La ligne `doubles` de `CONVENTIONS_TEXTE`, en en-tête des plans, des fiches et des rapports figés, le dit au relecteur, mot pour mot : « le double relu fait foi ; 17 chiffres significatifs (forme %.17g de la norme C), arrondi au plus proche, égalité vers le chiffre pair, calculé par tablexp sans la bibliothèque C ; relecture au bit près constatée sous glibc et sous Windows (R 4.3.3 ucrt), non garantie ailleurs ; à l'import, toute écriture décimale est admise (0.95 comme 0.94999999999999996) ; refusés : décimal non fini, décimal non nul relu 0 ; jetons NA, NaN, Inf, -Inf » (depuis l'issue #120 ; auparavant « relecture au bit près constatée sous glibc, non garantie ailleurs »). « Non garantie ailleurs » n'implique aucune garantie sous glibc ni sous Windows : la relecture au bit près n'est démontrée sur aucune plateforme (`?NumericConstants`, R 4.3.3 : « one of the nearest representable numbers ») ; elle y est seulement constatée. « (R 4.3.3 ucrt) » désigne la seule compilation de R mesurée sous Windows (ADR 0011, annotation du 27/10/2026, (d) 3).

Raison. Avant l'issue #104, la forme « texte » était `sprintf("%.17g", x + 0)`, écrite par la bibliothèque C du système : sous Windows, 36 694 textes sur 551 008 y différaient de l'arrondi correct au dix-septième chiffre (constat sur le poste du mainteneur, ADR 0011, « Contexte », point 1), si bien que plan, fiche et rapport figé n'avaient pas les mêmes octets d'une plateforme à l'autre. Le calcul par `tablexp` retire cette dépendance. Son prix est la maintenance (le code ci-dessous, la preuve, les témoins de la batterie et un contrôle à deux juges) et un coût par double écrit d'un ordre de grandeur supérieur à celui de `sprintf` ; ce coût porte sur les seuls doubles écrits en forme « texte » (plan, fiche, rapport figé, références), non sur le calcul : `construire_table()` n'en écrit aucun (mesures de l'étape (3a) et seuils S0 à S7 : ADR 0011, annotation du 26/10/2026, (b)). Sous la glibc, aucun double écrit des plans, fiches, rapports figés et références de R1 à R6 n'a changé, et 0 écart sur 2 423 344 doubles (*constat de mesure* ; ADR 0011, annotation du 26/10/2026, (a) 2 et (c)). Pour un double quelconque, l'identité suppose en outre que la glibc arrondit correctement, ce que la présente documentation ne démontre pas.

Définition du texte. Un non fini s'écrit par son jeton, lu sur la valeur reçue par la même règle qu'en forme « empreinte » (`jeton_non_fini()` ; issue #118) : `NA` si `is.na()` et non `is.nan()`, `NaN`, `Inf` ou `-Inf`. Un fini x s'écrit depuis $y = x + 0$: le zéro négatif devient 0, écrit `0`. Pour $y \neq 0$, soit $a = |y|$, $D = d_1 d_2 \ldots d_{17}$ ($d_1 \neq 0$) et X les dix-sept chiffres et l'exposant décimal de l'arrondi de a à dix-sept chiffres significatifs, **au plus proche, égalité vers le chiffre pair** : $a \approx d_1{,}d_2 \ldots d_{17} \cdot 10^X$, X étant pris *après* l'arrondi. Si $-4 \leq X < 17$, notation fixe à $16 - X$ décimales ; sinon notation `d.ddde+XX`, exposant signé sur deux chiffres au moins (trois au-delà de 99) ; dans les deux cas, les zéros de fin de la partie décimale sont retirés, et le point s'il ne reste rien après lui ; le séparateur est toujours le point ; un `-` précède le texte si $y < 0$. Ainsi 0,95 s'écrit `0.94999999999999996`, 10^{-5} `1.0000000000000001e-05`, 10^{17} `1e+17` et 2^{-25}, dont le développement décimal exact $2{,}98023223876953125 \cdot 10^{-8}$ tombe exactement entre deux textes à dix-sept chiffres, `2.9802322387695312e-08` (dix-septième chiffre pair). C'est la forme de la conversion `g` de la norme C à la précision 17, sans le drapeau `#` : le texte de la norme n'a pas été relu pour le présent document, et la conformité de la forme, frontières de notation comprises (10^{16}, 10^{17}, 10^{-4}, 10^{-5}), est un *constat de mesure* (identité aux deux juges, ci-dessous).

Calcul (algorithme V1 de la spécification d'`actuary`, issue #104). Les grands entiers sont écrits en tranches de base $B = 10^7$ (`BASE_TRANCHES`), poids faible d'abord, stockées en doubles. `chiffres_17()` lit $a = m \cdot 2^e$ sur les octets, forme l'entier décimal exact Z comme le produit de m (trois tranches) par la ligne e de la **table des puissances** `PUISSANCES_TRANCHES`, puis arrondit sur les quatre tranches non nulles les plus hautes de Z. La table porte 5^{-e} si $e < 0$ et 2e si $e \geq 0$, pour e de -1074 à

1023 (2 098 lignes, 108 tranches au plus, pour 5^{1074}) ; c'est une constante calculée **à l'installation** du package par `table_puissances_tranches()`, par multiplications successives par 5 et par 2. Les retenues sont reportées par `reporter_retenues()`, qui divise chaque tranche par la base au moyen de `diviser_tranche()`. `positions_17g()` met les chiffres et l'exposant dans la forme ci-dessus, et `octets_17g()` assemble les octets et pose les jetons des non-finis. Les opérateurs `%%` et `%/%` ne portent que sur des entiers (type `integer`) ; `max.col()` reçoit toujours `ties.method` `"first"` ou `"last"`, si bien qu'aucun tirage n'est fait dans le générateur (la valeur par défaut, `"random"`, en ferait un ; `.Random.seed` inchangé, vérifié par la batterie). La mémoire est bornée : blocs de 2^{14} valeurs, matrices de tranches d'au plus 2^{18} éléments.

Preuve (invariants I1 à I5 de la spécification d'`actuary`). L'aide de `double` exige de toute plateforme de R des valeurs conformes à la norme IEC 60559 (binary64). La preuve suppose en outre (**hypothèse H**) que les opérations $+$, $-$, $\times$, $/$ de R y sont celles de la norme, correctement arrondies : un résultat exact représentable est alors rendu exactement. Sont représentables les entiers de valeur absolue au plus 2^{53} et les multiples de 2^7 inférieurs à 2^{60} ; `floor()` est exacte.

— **I1, décomposition exacte** : $a = m \cdot 2^e$ exactement, avec m impair, $1 \le m < 2^{53}$ et $-1074 \le e \le 1023$. Par définition du format binary64, un double fini positif de champ d'exposant E (11 bits) et de fraction f (52 bits) vaut $(2^{52} + f) \cdot 2^{E-1075}$ si $E > 0$, et $f \cdot 2^{-1074}$ si $E = 0$ (sous-normal). E et f sont lus sur les octets petit-boutistes (`writeBin()`) et recomposés par produits par 256 et sommes d'entiers inférieurs à 2^{53}, donc exactement. Tant que m est pair, $m \leftarrow m/2$ et $e \leftarrow e + 1$: la moitié d'un entier pair est exacte. Au plus grand champ, $E = 2046$, $e \le 2046 - 1075 + 52 = 1023$.

— **I2, entier décimal exact** : si $e \ge 0$, $Z = m \cdot 2^e$ et $s = 0$; si $e < 0$, $Z = m \cdot 5^{-e}$ et $s = -e$, puisque $2^e = 5^{-e} \cdot 10^e$. Dans les deux cas, $a = Z \cdot 10^{-s}$ exactement ; Z a au plus 767 chiffres (m en a au plus 16, 5^{1074} en a 751).

— **I3, exactitude des opérations** : les tranches, leurs produits et les sommes de ces produits sont des entiers de $[0, 2^{53})$, donc calculés exactement (H). Une tranche est inférieure à 10^7 ; $m = r_0 + Br_1 + B^2 q_2$ avec $r_0, r_1 < 10^7$ et $q_2 \le 90$ (car $m < 2^{53} < 91 \cdot 10^{14}$) ; chaque tranche du produit reçoit au plus trois produits, chacun inférieur à 10^{14}, soit moins de $3 \cdot 10^{14} < 2^{53} \approx 9{,}007 \cdot 10^{15}$; à la construction de la table, une tranche multipliée par 5 ou par 2 reste inférieure à $5 \cdot 10^7$. Les valeurs propres à la division par la base (qB et r) et au report des retenues sont traitées en I4.

— **I4, division par la base** : pour un entier $v \in [0, 2^{53})$, `diviser_tranche()` rend $q = \lfloor v/B \rfloor$ et $r = v - qB \in [0, B)$ exactement, quel que soit l'arrondi du quotient : le quotient calculé, arrondi au plus proche, diffère de v/B d'au plus $v/B \cdot 2^{-53} < 10^{-7}$, si bien que sa partie entière diffère de $\lfloor v/B \rfloor$ d'au plus une unité ; qB, multiple de 10^7 inférieur à $2^{53} + 10^7$, et $r = v - qB \in [-B, 2B)$ sont représentables, donc exacts (H), et la correction d'une unité ($r < 0$: $q - 1$ et $r + B$; $r \ge B$: $q + 1$ et $r - B$) rétablit $0 \le r < B$. Sous H, la partie entière est même exacte ($v/B = k + j \cdot 10^{-7}$, écart d'arrondi $< 10^{-7}$) : la correction du quotient ne joue jamais et reste un garde-fou (constat : 0 correction sur 4 003 004 valeurs, dont $kB - 1$, kB et $kB + 1$ jusqu'à 2^{53}). Chaque passe remplace v_j par $r_j + q_{j-1}$, ce qui conserve $\sum_j v_j B^j$ et la positivité. Avant report, $v_j < 3 \cdot 10^{14}$ (I3 ; $< 5 \cdot 10^7$ pour la table). Après la première passe, $v_j < 4 \cdot 10^7$; après la deuxième, $v_j \le B + 2$; après la troisième, $v_j \le B$. Dès lors, une tranche ne porte une retenue (égale à 1) que si elle vaut B, ce qui exige $r_j = B - 1$ et une retenue de la tranche immédiatement inférieure à la passe précédente. Les rangs porteurs d'une retenue se décalent donc d'au moins un rang par passe, et le report s'arrête en au plus $w + 3$ passes (w colonnes), sur l'écriture unique de Z en base B (constat : 2 passes au plus pour la table, 4 au plus sur 139 943 doubles). $m < B^3$ et la ligne de la table compte lg tranches, donc $Z < B^{lg+3}$, que contiennent les $lg + 3$ colonnes situées au-dessus des trois colonnes nulles. Comme $\sum_j v_j B^j = Z \cdot B^3 < B^w$ à chaque passe et $v_j \ge 0$, la dernière tranche reste $< B$: le garde-fou ne se déclenche pas sans erreur de programmation.

— **I5, arrondi correct** : soit $Z = \sum_{k=1}^{N} d_k 10^{N-k}$, $d_1 \neq 0$, d'où $a = d_1{,}d_2 \ldots d_N \cdot 10^X$ avec $X = N - 1 - s$. Si $N \leq 17$, D est $d_1 \ldots d_N$ complété de zéros, sans arrondi ; le code y lit $d_{18} = 0$ dans les tranches réservées et n'arrondit pas. Sinon, $H = d_1 \ldots d_{17}$ et le reste $\rho = \sum_{k \geq 18} d_k 10^{N-k} \in [0, 10^{N-17})$ donnent $a = (H + \rho/10^{N-17}) \cdot 10^{X-16}$. L'arrondi au plus proche, égalité vers le pair, monte à $H + 1$ si et seulement si $\rho/10^{N-17} > 1/2$, ou $= 1/2$ et H impair (c'est-à-dire d_{17} impair) ; or $\rho/10^{N-17} > 1/2$ si et seulement si $d_{18} > 5$, ou $d_{18} = 5$ et un d_k ($k \geq 19$) est non nul, et $\rho/10^{N-17} = 1/2$ si et seulement si $d_{18} = 5$ et tous les d_k ($k \geq 19$) sont nuls. C'est le test du code : $d_1 \ldots d_{18}$ sont lus dans les quatre tranches les plus hautes de Z (au moins 22 chiffres ; trois tranches nulles sont toujours réservées sous Z, pour qu'il y en ait quatre même quand Z en compte moins), et l'existence d'un d_k non nul ($k \geq 19$) sur les chiffres restants de ces tranches et sur l'existence d'une tranche non nulle plus basse. Si $H + 1 = 10^{17}$, D vaut 1 suivi de seize zéros et X augmente de 1. $D \cdot 10^{X-16}$ est donc l'arrondi correct de a à dix-sept chiffres significatifs.

Égalités exactes. Une égalité ($\rho/10^{N-17} = 1/2$) exige $Z = (10H + 5) \cdot 10^{N-18}$, avec $10H + 5$ impair, donc $N \geq 18$ et une valuation 2 de Z égale à $N - 18$. Si $e \geq 0$, cette valuation est e (m impair), et $10^{N-1} \leq Z < 2^{53+e} = 2^{N+35}$ exigerait $N < 17$: contradiction. Si $e < 0$, $Z = m \cdot 5^{-e}$ est impair, donc $N = 18$: $m \cdot 5^n \in [10^{17}, 10^{18})$ avec $n = -e$, possible seulement pour n de 2 à 25 : $m \geq 1$ impose $5^n < 10^{18}$, donc $n \leq 25$; $m < 2^{53}$ impose $5^n > 10^{17}/2^{53} \approx 11{,}1$, donc $n \geq 2$. Réciproquement, si $e < 0$ et $N = 18$, $Z = m \cdot 5^n$ est impair et multiple de 5, donc finit par 5 : c'est une égalité. Les égalités exactes sont exactement les doubles d'exposant $e < 0$ dont Z a dix-huit chiffres. Exemples : 2^{-25} ($m = 1$, $5^{25} = 298\,023\,223\,876\,953\,125$) et $2^{50} + 0{,}25$, écrit `1125899906842624.2`. Ces cas sont les seuls où la règle d'égalité vers le pair joue ; les témoins de la batterie et le contrôle à deux juges en couvrent chaque n de 2 à 25 (T2 : toutes les valeurs de m quand il y en a au plus 1 000, sinon 1 000 réparties et les bornes ; contrôle : 200 000 égalités tirées).

Statut (E2.20 ; ADR 0011, annotation du 26/10/2026, (c)).

— **Résultat exact** (preuve validée par `actuary`, pull request #116), sous l'hypothèse H : les dix-sept chiffres et l'exposant X sont l'arrondi correct de a et ne dépendent d'aucune plateforme qui satisfait H.

— La mise en forme (`positions_17g()` : notation, zéros de fin, signe) n'entre pas dans la preuve. Sa conformité à la définition et à la conversion `g` de la norme C est un *constat de mesure*. Elle n'emploie que des opérations sur des `integer` et `max.col()` à `ties.method` fixé.

— Sous Windows (R 4.3.3 ucrt), l'identité des octets à ceux de la plateforme de référence est un *constat de mesure* rapporté de l'étape (5) de la pull request #116 (empreintes MD5 des textes de T2 et T3 égales à celles de la CI ; ADR 0011, annotation du 27/10/2026, (b)) ; H n'y est pas mesurée et reste une **hypothèse**. Sous macOS : non mesurées (M61, #49).

— **Constat de mesure** : 0 écart à la glibc 2.39 et 0 écart à Python 3.13.17 sur 1 177 468 doubles finis, dont 68 886 égalités exactes, pour le prototype `tests/mesures/ecriture_17g/ecriture_17g.R`, dont le code du moteur est repris, hors plateforme de référence (rapport de mesure de l'étape (3a), pull request #116 ; prototype et scripts de mesure conservés sous `tests/mesures/ecriture_17g/`, hors batterie et sans seuil) ; puis, pour le moteur, 0 écart à la glibc 2.35 et 0 écart à Python 3.10.12 sur 2 423 344 doubles, sur la plateforme de référence (contrôle `tests/controles/ecriture_doubles_texte.R`, exécution 40308275084 de la CI, commit `3b42737`). Le constat vaut donc sous deux versions de la glibc et deux de Python ; que le juge Python soit indépendant de la glibc pour les chiffres est une **hypothèse** (CPython a sa propre conversion, dont la source n'a pas été relue).

— **Relecture** : inchangée par l'issue #104, *constat de mesure* sous glibc et sous Windows (R 4.3.3 ucrt), non une preuve (ci-dessus, « Deux formes, un seul écrivain »). Pour un texte écrit par `tablexp`, le contrôle « réécrire la valeur relue redonne le texte » déciderait l'aller-retour texte par texte sur toute plateforme, dix-sept chiffres correctement arrondis désignant un seul double ; ce contrôle n'est pas

implanté dans le lecteur des références (section 4.5), qui compare hors Windows le texte lu à `sprintf` de la bibliothèque C et n'en contrôle sous Windows que la forme.

Les témoins de la batterie unitaire et le contrôle à deux juges sont décrits à la section 15.6.

15.3 Empreintes du résultat et du code

La fiche porte, outre l'empreinte des données, une empreinte du résultat et une empreinte du code (E3.6 ; ADR 0005, point 6), calculées depuis l'issue #67 (ADR 0011, point 11, et cinquième à huitième annotations du 20/10/2026 ; décisions M70, M71 et M72). Comme celle des données, chacune est un MD5 (`ALGORITHME_EMPREINTE`) d'octets écrits en binaire dans un fichier temporaire, supprimé à la sortie (`md5_octets()`), et non de `serialize()`, dont l'en-tête porte la version de R.

Empreinte du résultat

— **Objet** : identifier conjointement l'entrée, dans ses colonnes de rôle (par `fiche$empreintes$donnees`, qui est dans le périmètre), la déclaration (`fiche$plan`, `fiche$roles`) et le résultat (`table`, `etapes`, `avertissements`, reste de la fiche).

— **Texte empreinté** : le texte de l'objet `construction` écrit par `texte_objet()` sous la grammaire de ligne du texte de la fiche (section 15.4 : `chemin<TAB>type<TAB>longueur<TAB>valeurs`, lignes `@noms`, `@longueur`, `@colonnes` et `@lignes`, segments `[i]`, profondeur au plus `PROFONDEUR_MAX_TEXTE`), les valeurs en forme « empreinte » (doubles en octets IEEE 754, jetons des non-finis lus sur la valeur reçue ; section 15.2), précédé de la première ligne `tablexp resultat 1` (`REPRESENTATION_RESULTAT`) et **sans ligne d'en-tête** `#`. Ce texte n'est jamais relu : il n'existe que pour être haché.

— **Version** : la première ligne numérote la définition du texte empreinté (grammaire de ligne, forme des valeurs, périmètre) ; elle est consignée dans `fiche$empreintes$representation_resultat`, avant `resultat`, distincte de `representation`, qui numérote la représentation des données, et de `version_fiche`, qui numérote la structure de la fiche (ADR 0011, point 10, et cinquième annotation du 20/10/2026, (a)). Changer cette définition change l'empreinte du résultat de toutes les fiches et suit la règle durable de la représentation canonique (section 15.2 : incrément de la version, au plus une fois par branche, et visa).

— **Périmètre** : l'objet entier **moins** les chemins de `CHEMINS_HORS_RESULTAT`, liste fermée déclarée dans le code : `execution`, qui change d'une exécution à l'autre ; `fiche$versions` et `fiche$plateforme`, qui décrivent l'environnement et non le calcul ; `fiche$empreintes$resultat`, l'empreinte elle-même ; `fiche$empreintes$code`, qui ne porte pas sur le résultat, et `fiche$empreintes$statut_resultat_code`, qui décrit les deux empreintes et change avec elles ; enfin `etapes$donnees$resultat$lignes_conservees` (décision M71, point 1). Cette dernière est la copie de l'entrée privée des lignes exclues, avec toutes ses colonnes, sa classe, ses attributs et ses noms de lignes : elle est entièrement déterminée par l'entrée, identifiée, dans ses colonnes de rôle, par l'empreinte des données (`fiche$empreintes$donnees`), et par `numeros_conserves`, qui reste dans le périmètre, et ce qu'elle porte en plus (colonnes hors rôle, noms de lignes, classe, attributs) n'est lu par aucun module, le moteur ne lisant l'entrée conservée que colonne de rôle par colonne de rôle. Ce surplus n'est couvert par aucune empreinte : celle des données ne porte que sur les colonnes de rôle (décision M71, point 1 ; ADR 0007, point 9.3). La garder dans le périmètre ferait échouer `construire_table()` sur des entrées valides que la grammaire ne sait pas écrire : noms de lignes non automatiques dès qu'une exclusion retire une ligne, colonne hors rôle `POSIXct`, classe `tbl_df`, attribut quelconque, dont l'attribut `simulation` d'un jeu simulé. Options écartées (décision M71) : la lecture stricte du périmètre, qui refuserait ces entrées ; une copie réduite aux colonnes de rôle, qui changerait le contrat « objet résultat » (ADR 0007, point 6.3) pour doubler une entrée déjà identifiée ;

une forme « empreinte » tolérante aux classes et aux attributs inconnus, qui les ignorerait en silence ; le retrait de la copie, changement de contrat hors de la branche. L'exclusion de l'attribut `simulation` par son nom, d'abord retenue (décision M70, point 2), est devenue sans objet, l'attribut ne se trouvant qu'à ce chemin.

— **Classes et attributs** : la classe `construction` de la racine est vérifiée et non écrite (la première ligne en tient lieu) ; dans le périmètre, tout attribut hors de ceux de la grammaire et toute valeur sans écriture dans le texte (par exemple un `POSIXct`) arrêtent le calcul par une **erreur de programmation**, qui nomme le chemin : le refus de l'écrivain est converti, rien n'est ignoré en silence. Aucune entrée valide connue n'y conduit : c'est un *constat* sur les témoins de la batterie unitaire (attributs de colonne, facteurs munis de `contrasts` ou de sous-classe, colonnes d'identifiant nommées, dans les deux unités), non une propriété démontrée. Il repose sur la normalisation de la table des observations par l'étape `donnees` (section 7.4, « Copie des colonnes dans la table des observations ») ; une colonne recopiée sans normalisation le contredirait, comme l'a montré la régression des noms de lignes corrigée par l'issue #67, survenue alors que cette normalisation existait déjà.

— **Facteurs** (décision M71, point 2) : un facteur est admis en forme « empreinte », écrit par ses libellés sous le type `factor`, comme dans l'empreinte des données ; il reste refusé en forme « texte » (grammaire 1, section 15.4). Seul est admis un facteur de classe exactement `factor` ou `c("ordered", "factor")` et dont les attributs sont parmi `names`, `levels` et `class` ; tout autre facteur, sous-classe comprise (par exemple `c("factor", "x")`), est refusé (`serialisation_impossible`, converti en erreur de programmation). Aucune construction valide n'est ainsi refusée, par construction : les facteurs de la table des observations sont reconstruits par `categorie`, qui ne produit que ces deux classes (section 7.4). Les niveaux ne sont pas écrits, et le libellé de type est `factor` pour un facteur ordonné ou non (tableau 26) : deux facteurs aux mêmes libellés et aux niveaux différents, ou un `factor` et un `ordered` aux mêmes libellés, ont donc la même empreinte. Ce choix vaut parce qu'aucun résultat ne dépend des niveaux (clés de cellule converties en chaînes, section 8.2) : sa **condition de validité** est le test d'invariance aux niveaux de `tests/unitaires/test_exposition.R` et le témoin de l'issue #67 ; s'ils tombaient, le choix serait à rouvrir.

— **Calcul** : `construire_table()` assemble l'objet (`fiche$empreintes$resultat` y vaut d'abord `NA`, posé par `empreintes_construction()`), calcule `empreinte_resultat()` sur cet objet, range la valeur dans `fiche$empreintes$resultat`, puis ajoute `execution` (section 6.2). Le coût supplémentaire, dû surtout à l'écriture de la table des observations, a été mesuré à 10^6 lignes et admis pour la branche (décision M71, point 5 ; E1.3 ; mesures consignées par l'ADR 0011, septième annotation du 20/10/2026, (f), et par M71) ; son optimisation, qui devait garder le même texte empreinté, est l'objet de l'issue #71. Celle-ci a réécrit l'écriture des entiers et des dates sans une chaîne par ligne, à texte empreinté inchangé octet pour octet, identité *constatée* sur les cas vérifiés, non démontrée (section 15.6, « Écriture des entiers et des dates »). Le gain est un constat de mesure à 10^6 lignes, hors plateforme de référence, consigné avec le commit `cf24e73` ; la cible de l'issue, un temps d'empreinte du résultat au plus égal à celui de l'empreinte des données (`empreinte_donnees()`), n'est pas atteinte, et le mainteneur a accepté ce gain, cible manquée consignée (21/10/2026).

— **Statut** : écriture exacte (octets fixés, MD5 d'octets), mais valeur **dépendante de la plateforme** : à entrée, plan et code identiques, un double du résultat peut différer au dernier bit d'une machine à l'autre, et l'empreinte change. *Constat de mesure* (20/10/2026) sur un hôte hors de la plateforme de référence (Ubuntu 24.04.5, glibc 2.39, processeur Intel, BLAS et LAPACK 3.12.0) : R1, R2, R3 et R6 y ont la même empreinte du résultat que sur la plateforme de référence, R4 et R5 une autre, parce qu'un élément de `probabilite_borne_sup_asymptotique` y diffère d'un ulp. Cette grandeur est calculée par `-expm1(-borne)` (`calculer_taux_bruts()`), dont l'argument est identique au bit près des deux côtés : l'écart naît dans `expm1()`, fonction de la bibliothèque mathématique du système

(libm de la glibc ; sur la plateforme de référence, sous Ubuntu 22.04.5, glibc 2.35 : *constat de mesure*, relevé par le script de plateforme dans le journal du job « Tests (R 4.3.3) » de la CI du 20/10/2026, sur l'image de référence, de même empreinte que celle de l'en-tête des références ; issue #72 ; les en-têtes des références la portent depuis leur régénération du 24/10/2026, sous « glibc (libm) » et sous fiche$plateforme/bibliotheque_c), et non dans LAPACK ; la mesure ne sépare pas la version de la glibc de la variante qu'elle choisit selon le processeur (ADR 0008, seconde annotation du 19/10/2026). De plus, la forme « empreinte » distingue NA de NaN, alors que R ne garantit pas lequel sort d'une opération sur NA (ADR 0011, point 2.2). L'empreinte du résultat est donc un **témoin d'identité sur la même machine seulement** (E3.6, E4.6 ; ADR 0011, point 11.2), **jamais un critère de rejeu chez un tiers**, dont le critère reste la comparaison à tolérance par classes (section 15.4, « Rejeu par un tiers »). La fiche le dit dans statut_resultat_code (ci-dessous).

Empreinte du code

— **Objet** : identifier le **texte source** du moteur effectivement installé, modifications non commitées comprises ; non son comportement numérique, qui dépend aussi de R et du BLAS, consignés à part (fiche$versions, fiche$plateforme), et de la libm, dont la fiche ne consigne la version que par fiche$plateforme$bibliotheque_c, et seulement quand sa valeur commence par « glibc » (section 15.1, « Bibliothèque C » ; issue #77) ; le script de plateforme la consigne aussi (issue #72).

— **Définition** (option (iv), décision M70, point 1) : le MD5 du *relevé des MD5* des fichiers R/*.R, et non celui de leur texte concaténé : une ligne <md5>, deux espaces, R/<fichier>, LF, par fichier, les fichiers triés par octets (ADR 0011, septième annotation du 20/10/2026, (d)). Ce relevé est la sortie de md5sum de GNU coreutils, si bien qu'un tiers recalcule l'empreinte depuis la racine des sources par LC_ALL=C md5sum -t R/*.R | md5sum, commande donnée aux tiers par la fiche elle-même (libellé ci-dessous ; issue #107, décision M151 ; ADR 0011, annotation du 26/10/2026, (e)), la locale C fixant l'ordre des fichiers. L'option -t (mode texte) est nécessaire sous Windows : le md5sum de Rtools (MSYS2) lit par défaut en mode binaire et préfixe les noms par « * », ce qui change le relevé, donc l'empreinte (issue #101). **Portée** : sous GNU/Linux, *constat de mesure* (coreutils 9.4) : avec ou sans -t, la commande rend la valeur de empreinte_sources(), et avec -b une autre valeur ; sous Windows avec Rtools 4.3 (MSYS2, coreutils 8.32), *constat de mesure sur le poste du mainteneur*, à une version antérieure du code (relevé du 25/10/2026 à 1c48475, issues #49 et #101) : sans -t, une autre valeur ; avec -t, celle de EMPREINTE_CODE ; remesuré sous Git Bash (coreutils 8.32) à l'état final de la dixième branche (aa1cf0b, dont les R/*.R sont ceux de 8bd29dd), même constat, rapporté (ADR 0011, annotation du 27/10/2026, (c)) ; sous macOS, **non mesurée** (décision M61) : le md5sum de GNU n'y est pas installé par défaut (hypothèse), et la commande ne vaut qu'avec lui, d'où la mention « GNU coreutils » dans le libellé.

— **Calcul** : EMPREINTE_CODE, constante de premier niveau de R/rapport.R, vaut empreinte_sources("."). Elle est évaluée une seule fois, pendant R CMD INSTALL, qui exécute le code de premier niveau depuis la racine des sources et range la valeur dans la base de chargement différé du package ; à l'exécution, empreintes_construction() lit cette chaîne et le moteur n'ouvre aucun fichier (E4.4 ; ADR 0011, point 8.1). empreinte_sources() n'emploie que des fonctions de base (read.dcf(), list.files(), tools::md5sum()) et aucune constante d'un autre module, les fichiers de R/ étant évalués par ordre alphabétique à l'installation (section 15.1).

— **Garde** : la racine doit être celle des sources de tablexp (DESCRIPTION lisible, Package: tablexp, R/rapport.R présent) ; sinon, ou si un fichier de R/ est illisible (tools::md5sum() rend alors NA), l'empreinte vaut NA, jamais le MD5 d'un relevé incomplet ou portant « NA » : rien n'est deviné (décision M72). L'installation n'échoue pas, et le statut le dit en toutes lettres (STATUT_CODE_INTROUVABLE, ci-dessous) : une installation sur une plateforme non mesurée reste utilisable et le signale. R CMD

INSTALL évalue aussi les fichiers .r, .S, .s et .q de R/, que le relevé ne couvre pas : un test vérifie que R/ ne contient que des fichiers *.R.

— **Statut** : objet portable (texte source, fins de ligne LF imposées par .gitattributes) et écriture exacte (MD5 d'octets). Le mécanisme, à savoir que le répertoire courant de R CMD INSTALL est la racine des sources, est un *constat de plateforme* sous Linux et R 4.3.3, non documenté par R, non mesuré sous Windows ni macOS (issue #49) ; il est surveillé par le test d'égalité au recalcul (section 15.6).

— **Options écartées** : (i), un manifeste sous inst/ écrit par un script et vérifié par la CI, donne la même valeur mais identifie le texte *déclaré* au dernier passage du script, non le texte installé : une installation locale d'un arbre modifié sans relancer le script aurait une empreinte périmée sans que rien le dise, et la CI ne le verrait qu'au push ; cette option demande en outre un contrôle de CI de plus. (i) reste le **repli documenté**, à rouvrir si le mécanisme de (iv) se révélait défaillant (garde ou test en échec, plateforme non conforme ; ADR 0011, sixième annotation du 20/10/2026). (ii), la version du package seule, est écartée parce qu'elle n'a pas changé depuis la création du package (0.0.0.9000) alors que R/ a été modifié par de nombreux commits ; (iii), le SHA du commit, parce qu'il est absent à l'exécution dans les jobs R sans git et qu'il ignore les modifications non commitées (décisions M69, point 3, et M70).

Statut des deux empreintes

fiche$empreintes$statut_resultat_code vaut statut_empreintes(EMPREINTE_CODE) : le libellé du résultat (STATUT_EMPREINTE_RESULTAT), le séparateur " ; ", puis celui du code calculé (STATUT_EMPREINTE_CODE) ou, si l'empreinte du code est NA, celui des sources introuvables (STATUT_CODE_INTROUVABLE) ; texte du moteur marqué UTF-8 (décision M71, point 3). Les trois libellés sont :

— « résultat : témoin d'identité sur la même machine seulement (E3.6, E4.6), jamais critère de rejeu chez un tiers » ;

— « code : MD5 du relevé des MD5 des fichiers R/*.R, fait à l'installation (recalcul depuis la racine des sources, avec le md5sum de GNU coreutils : LC_ALL=C md5sum -t R/*.R | md5sum) ; identifie le texte source, non le comportement numérique, qui dépend aussi de R, du BLAS et de la libm » ;

— « code : non calculée, sources introuvables à l'installation ».

La fiche ne dit donc jamais d'une empreinte plus que ce qu'elle établit. Dans les références de non-régression, les deux empreintes sont comparées par leur forme, et leurs valeurs écrites dans l'en-tête (section 4.5).

15.4 Texte du plan et de la fiche

Objet (ADR 0011, points 6, 8, 9 et 12, et troisième annotation du 19/10/2026 ; décision M66 ; issue #50). Le plan s'exporte et s'importe (E5.4) ; la fiche s'écrit, se relit et se rejoue (E3.7). Quatre fonctions exportées du module rapport convertissent un objet en texte et inversement : ecrire_plan(), lire_plan(), ecrire_fiche() et lire_fiche(). Elles rendent et reçoivent des **octets** (raw), jamais un fichier : l'appelant écrit par writeBin(octets, fichier) et lit par readBin(fichier, "raw", file.size(fichier)) (E4.4 ; ADR 0011, point 8). Des octets, et non une chaîne, parce que l'écriture en mode texte produirait des fins de ligne CRLF sous Windows (hypothèse non mesurée) et que la lecture ligne à ligne de R réencode selon la locale : les octets sont le contrat, et un fichier altéré, par exemple par un éditeur qui ajoute des CR, est refusé avec un message qui nomme l'altération plutôt que corrigé en silence. Ce format remplace le report du point 12 de l'ADR 0007, dont il garde les cinq

contraintes : texte lisible par un relecteur, aller-retour identique, nombres compris, aucune évaluation de code à l'import, R de base, version portée par le texte.

Grammaire du texte (grammaire 1)

— **forme** : UTF-8, lignes terminées par LF, LF final, ni CR ni marque d'ordre des octets ;

— **première ligne** : `tablexp plan 1` (`PREMIERE_LIGNE_PLAN`) ou `tablexp fiche 1` (`PREMIERE_LIGNE_FICHE`), qui nomme la nature et la version de la grammaire. La version est portée par le texte seul, jamais par un champ de l'objet, et le lecteur refuse toute autre première ligne, sans migration (ADR 0011, point 10) ;

— **en-tête** : des lignes `# <clé> : <valeur>`, **hors aller-retour**, que le lecteur ignore. Elles rappellent au relecteur les conventions (`CONVENTIONS_TEXTE` : grammaire, doubles, dates, chaînes, délais, codage). La fiche y ajoute `version_fiche`, la représentation canonique de l'empreinte des données, les blocs de lecture (`# lecture`) et la procédure de rejeu (`# rejeu`), décrits plus bas ;

— **corps** : une ligne par feuille, `chemin<TAB>type<TAB>longueur<TAB>valeurs`, dans l'ordre de l'objet (parcours en profondeur), jamais trié ; le chemin joint les segments par `/` ; un élément sans nom, ou dont le nom ne peut pas être un segment (vide, `NA`, en double, contenant `/`, une tabulation ou une fin de ligne, ou commençant par `@`, `#` ou `[`), a pour segment `[i]`, son rang, le nom restant porté par la ligne `@noms`, si bien que tout nom fait l'aller-retour ;

— **structure** : chaque conteneur déclare son effectif. Une liste nommée a une ligne `chemin/@noms`, puis ses éléments ; une liste sans noms a une ligne `chemin/@longueur` (`integer`, une valeur n), puis exactement ses éléments `chemin/[1]` à `chemin/[`n`]` ; un `data.frame` a ses lignes `chemin/@colonnes` et `chemin/@lignes`, puis ses colonnes (noms de lignes automatiques seulement) ; un vecteur atomique nommé, de longueur au moins 1, a sa ligne `chemin/@noms`, puis sa ligne de valeurs ; `NULL` s'écrit `chemin<TAB>NULL<TAB>0`. Conséquence : **tout texte écrit par `ecrire_plan()` ou `ecrire_fiche()` dont on retire au moins une ligne du corps, en le coupant à une frontière de ligne, est refusé à la lecture**, faute de quoi un fichier tronqué serait relu comme un objet plus court. Un vecteur atomique nommé de longueur 0 est pour cela refusé à l'écriture : sa ligne `@noms` seule est aussi l'écriture complète d'une liste nommée vide, si bien que son texte privé de la ligne de valeurs serait relu sans refus (ADR 0011, seconde annotation du 20/10/2026 ; aucun plan valide ni aucune fiche du package n'en contient). **Limite admise** : un texte édité à la main qui porte un tel vecteur est relu sans refus, si bien que le lecteur admet un objet que l'écrivain refuse ;

— **types** : `double`, `integer`, `logical`, `character`, `Date`, `NULL` (`TYPES_TEXTE`) ; valeurs écrites par `ecrire_valeurs()` en forme « texte » (section 15.2 : doubles en `%.17g`, dates ISO à quatre chiffres, chaînes entre guillemets avec cinq échappements, `NA` sans guillemets, distinct de `"NA"`). Un facteur est hors de la grammaire 1 : sans ses niveaux, il n'a pas d'aller-retour identique, et aucun plan valide n'en porte ; seul le texte empreinté du résultat, qui n'est jamais relu, l'admet par ses libellés (`texte_objet()` en forme `"empreinte"`, section 15.3). Une `Date` à stockage entier s'écrit comme la même date à stockage double et se relit en double ;

— **profondeur** : nombre de segments du chemin, sans le dernier s'il commence par `@` ; au plus `PROFONDEUR_MAX_TEXTE` (32), contrôlée à l'écriture et, à la lecture, avant toute reconstruction, si bien qu'aucun texte ne fait déborder la pile de R (débordement constaté entre 800 et 1 000 niveaux sur une pile de 8 Mo, profondeur des fiches de R1 à R6 égale à 4 : ADR 0011, troisième annotation, point (g)).

La forme de ligne et l'écrivain des valeurs sont ceux des références de non-régression (section 4.5) ; les **formats restent distincts**, et la grammaire 1 s'en écarte sur sept points (ADR 0011, troisième annotation,

point (f)) : liste sans noms (@longueur ici, @noms vide là) ; noms qui ne peuvent pas être des segments (écrits [i] ici, refusés là) ; facteur (exclu ici, admis là) ; lecture des doubles (lecteur de forme ici, stricte là) ; lignes propres aux références (@classe, @empreinte) ; en-tête # ; profondeur bornée.

Lecture sans évaluation

Le lecteur découpe les octets sur LF, puis chaque ligne sur la tabulation, et convertit par as.numeric(), as.integer() et as.Date() sur une forme contrôlée, plus le désechappement des chaînes ; il n'emploie **jamais** parse(), eval(), source() ni dget() (ADR 0007, point 2.3). Un texte qui contient du code R est relu comme une chaîne ou refusé, jamais évalué. Les types autres que double exigent leur écriture canonique ; une date relue doit, réécrite, redonner son texte. Pour les **doubles**, le lecteur est un *lecteur de forme* (décision M64, Q3) : tout décimal que as.numeric() lit en un double fini est admis, ainsi que les jetons NA, NaN, Inf et -Inf, et la valeur retenue est le double lu : un plan édité à la main peut écrire 0.95 au lieu de 0.94999999999999996. La valeur de référence pour un tiers est l'écriture %.17g que l'export réécrit. Un décimal non nul relu 0 (sous-dépassement, par exemple 1e-400) est **refusé** (décision M66) : l'erreur relative serait de 100 %, et un délai de déclaration écrit 1e-400 passerait la validation comme 0. **Statut** de l'aller-retour : exact pour tous les types sauf double ; pour un double, celui de la forme « texte » (section 15.2, « Écriture « texte » des doubles » : écriture correctement arrondie, établie par une preuve (hypothèse H) et constatée par deux juges ; relecture exacte constatée sous glibc et sous Windows (R 4.3.3 ucrt), non démontrée). Un texte écrit sous Windows par la bibliothèque C, avant l'issue #104, dont le dix-septième chiffre pouvait différer, se relit en le même double dans le cas testé : c'est un *constat de mesure*, que contredisent d'autres cas : sous Windows, sur 551 008 textes %.17g de la bibliothèque C, 5 ont été relus en un autre double, tous d'exposant supérieur à 266 (constat sur le poste du mainteneur, ADR 0011, point 3.2).

Table 27 : Refus de la lecture et de l'écriture du texte (module rapport ; à la lecture, numéro de la ligne fautive dans details$ligne (NA pour texte_non_octets et pour un texte vide), et en outre classe reçue dans details$classe (texte_non_octets), chemin lu et chemin attendu dans details$chemin et details$attendu (texte_chemin_inattendu ; chemin seul pour une ligne après la fin de l'objet, attendu seul en fin de texte), colonne dans details$colonne (texte_ligne_mal_formee et texte_longueur_incoherente, colonne non atomique ou d'une autre longueur que @lignes) ; à l'écriture, chemin fautif dans details$chemin).

Motif	Condition
texte_non_octets	texte reçu autrement qu'en octets (raw)
texte_version_inconnue	texte vide, ou première ligne autre que celle attendue (nature ou version de grammaire)
texte_marque_ordre_octets	marque d'ordre des octets en tête
texte_retour_chariot	octet CR (fins de ligne CRLF ou CR)
texte_octet_nul	octet nul
texte_sans_lf_final	dernière ligne sans LF
texte_utf8_invalide	ligne qui n'est pas de l'UTF-8 valide
texte_ligne_mal_formee	texte sans corps ; ligne hors de la forme (champ vide, longueur illisible, ligne d'en-tête dans le corps) ; ligne @longueur, @noms, @colonnes ou @lignes de type ou de valeur invalide ; NULL nommé ; colonne non atomique
texte_type_inconnu	type hors de TYPES_TEXTE (factor compris)
texte_longueur_incoherente	nombre de valeurs différent de la longueur déclarée ; NULL de longueur non nulle ; @longueur supérieur au nombre de lignes restantes (contrôlé avant toute allocation) ; noms et valeurs en nombres différents ; colonne d'une autre longueur que @lignes

Motif	Condition
texte_chemin_en_double	chemin présent deux fois
texte_profondeur_excessive	profondeur au-delà de PROFONDEUR_MAX_TEXTE
texte_chemin_inattendu	chemin autre que celui qu'attend la structure : élément manquant ou en trop, ligne après la fin de l'objet, corps qui ne commence ni par @noms ni par @longueur
texte_valeur_invalide	valeur hors de l'écriture de son type : chaîne sans guillemets droits ou à échappement inconnu ; double qui n'est ni un décimal lu en un double fini ni un jeton ; entier non canonique ou hors de l'étendue des entiers ; logique autre que TRUE, FALSE, NA ; date hors de la forme AAAA-MM-JJ, inexistante, hors du domaine ou non canonique
texte_double_sous_depassement	décimal non nul relu 0
serialisation_impossible	à l'écriture : racine qui n'est pas une liste ; facteur (en forme « empreinte » du texte empreinté du résultat, seulement un facteur de classe ou d'attribut non admis, section 15.3) ; classe ou attribut hors de la grammaire ; data.frame classé ou aux noms de lignes non automatiques ; colonne non atomique ; Date de stockage autre que double ou integer ; vecteur atomique nommé de longueur 0 ; profondeur au-delà de PROFONDEUR_MAX_TEXTE ; objet construction entier passé à ecrire_fiche() au lieu de sa fiche

L'écriture peut aussi lever les refus de l'écrivain des valeurs (codage d'une chaîne, date hors du domaine ; section 15.2) ; ecrire_fiche() refuse ainsi sous chaine_codage_bytes, rôle « @noms », une fiche dont un nom de premier niveau, à toute position, est d'encodage bytes, avant toute lecture par nom (message « Rôle « @noms » : chaîne au codage « bytes », non convertible en UTF-8 sans deviner. Nom de champ n° [position du premier] de la fiche : [nom cité]. » ; details : role, lignes, les positions, lu, les noms bruts ; section 2.1, « Noms d'encodage bytes » ; issue #114). Plus bas, l'écrivain refuse de même un nom d'encodage bytes d'une sous-liste (chaine_codage_bytes, rôle « *chemin*/@noms », par exemple « empreintes/@noms »), et une fiche ou une valeur de classe d'encodage bytes sous serialisation_impossible (classe citée sous le repère) ; les lectures par nom et par classe passent par lire_champ() et a_classe() (section 2.1, « Classes et noms d'encodage bytes sous le premier niveau » ; issue #128). Le message d'un refus de lecture (« Lecture du texte de [nature], ligne [n] : [...]. », sans numéro de ligne quand le refus n'en a pas) est marqué UTF-8 sous toute locale ; la première ligne lue (texte_version_inconnue), le type lu (texte_type_inconnu), le chemin en double (texte_chemin_en_double) et la valeur fautive (texte_valeur_invalide) y sont cités selon la règle unique des refus (section 2.1, « Citation d'une valeur dans un message » ; issue #94).

Plan importé, fiche relue, rejeu

ecrire_plan() écrit le plan tel qu'il est reçu, sans validation ni canonisation, et lire_plan() rend un plan **non validé** : le module rapport ne peut pas appeler R/construire_table.R, où vit valider_plan() (graphe des modules, section 1.3). La validation se fait chez l'appelant, par valider_plan(), ou à l'entrée de construire_table(), qui l'appelle toujours : un plan importé reçoit exactement les refus d'un plan construit en mémoire (ADR 0011, point 9.2). Pour un plan canonique, lire_plan(ecrire_plan(p)) est identical() à p ; pour un autre plan, au stockage des dates près. lire_fiche() ne contrôle pas version_fiche : ce contrôle appartient au rejeu, et le rejeu outillé rejouer_fiche() refuse une version non admise sous le motif version_fiche_non_admise (section 6.4 ; ADR 0011, troisième annotation du 19/10/2026, point (i), troisième annotation du 20/10/2026 et annotation du 24/10/2026 ; ADR 0012, point 3.5).

Rejeu par un tiers (E3.7 et sa note de lecture du 19/10/2026 ; ADR 0011, point 12), rappelé par la ligne `# rejeu` de la fiche écrite : le relecteur vérifie `version_fiche`, la représentation canonique et l'empreinte des données de la fiche ; il exécute `construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance)` sur le plan relu du texte ; il compare les grandeurs à tolérance par classes (section 4.5) sur une autre machine, au bit près sur la même (E4.6). L'empreinte du résultat ne sert de témoin que sur la même machine (section 15.3) : sur une autre machine, une empreinte du résultat différente n'est pas un échec du rejeu. Le rejeu outillé, `rejouer_fiche()`, fait les vérifications, l'exécution et la comparaison des témoins sur la même machine ; la comparaison à tolérance par classes sur une autre machine n'est pas outillée (section 6.4). Pour retrouver le code qui a produit la fiche, le relecteur recherche `fiche$empreintes$code` dans l'historique git par `tests/controles/commits_empreinte_code.R` (section 6.4, « Retrouver le code d'une fiche » ; issue #89).

Blocs de lecture. La ligne `# lecture` de la fiche écrite range ses champs en trois blocs (`BLOCS_LECTURE_FICHE`), qui suivent les trois questions d'E3.7 : ce qui a été *calculé*, avec quelles *hypothèses*, de quoi *rejouer*. Un test vérifie que les blocs partitionnent les champs de la fiche (aucun champ manquant, en double ni étranger), si bien qu'un champ ajouté sans bloc fait échouer la batterie. L'affectation, validée par `actuary` le 20/10/2026 (ADR 0011, point 6.5), est la suivante, les champs de chaque bloc étant dans l'ordre de la fiche (`ORDRE_CHAMPS_FICHE`, issue #64) : *calculé* : `plan`, `transition_code`, `fin_effective`, `unite_observation`, `recette`, `convention`, `unite_exposition`, `par_annee` ; *hypothèses* : `exclusions`, `nombre_lignes_exclues`, `avertissement_unite`, `hypothese_censure`, `hypotheses_brut`, `avertissements` ; *rejeu* : `version_fiche`, `versions`, `plateforme`, `roles`, `provenance`, `provenance_mention`, `empreintes`, `graine`, `graine_mention`, `generateur`. Le champ `empreintes` est rangé en « rejeu » à cause de l'empreinte des données, préalable du rejeu ; il porte aussi l'empreinte du résultat, simple témoin d'identité sur la même machine (`statut_resultat_code`, section 15.3), et celle du code. Un champ absent de la fiche (`hypotheses_brut` sans étape `brut`) est omis de la ligne.

15.5 Rapport figé

Le *rapport figé* est le document HTML autonome qui restitue une construction telle que le moteur l'a retournée, sans recalcul (E3.5, E3.6). Il est écrit par la fonction exportée `ecrire_rapport(x)` (issue #87 ; ADR 0012, point 2, précisé par sa seconde annotation du 24/10/2026), qui reçoit l'objet `construction` et rend un vecteur `raw` : les octets UTF-8 du document, lignes terminées par LF, LF final, sans CR ni marque d'ordre des octets, comme le texte de la fiche (section 15.4). L'appelant écrit le fichier (`writeBin(ecrire_rapport(x), "rapport.html")`) ; le package n'en ouvre aucun. La fonction n'a pas d'option : le rapport a une seule forme. Le rapport d'un rejeu s'écrit en lui passant l'élément `construction` du résultat de `rejouer_fiche()` (section 6.4) ; le bilan de rejeu, hors de l'objet, n'y figure pas.

Lieu : le package, module `rapport` (ADR 0012, point 1 ; note de lecture d'E4.1 du 24/10/2026, décision M108). Le « rendu » qu'E4.1 exclut du module `rapport` est l'affichage interactif, qui reste dans `app/affichage.R` ; le rapport figé est un livrable, produit en octets par le module `rapport`, dans une section délimitée en fin de `R/rapport.R`. Quatre raisons, par ordre de poids : un relecteur qui n'a que le package installé (`R CMD INSTALL` n'installe pas `app/`) doit pouvoir écrire le rapport de son propre rejeu et le rapprocher de celui qu'on lui remet (E3.7) ; le rapport écrit chaque valeur par l'écrivain unique `ecrire_valeurs()` et recalcule `empreinte_resultat()`, deux fonctions internes qu'un rendu hors du package aurait dû exporter ou dupliquer (ADR 0011, point 3.1) ; ses textes méthodologiques, texte du moteur, sont cités par le présent document et contrôlés par la batterie de concordance ; enfin ses contrôles ont un sens métier et vont dans le package (E4.3), et le document n'a aucune dépendance à Shiny. Aucun module nouveau n'est créé pour un seul rendu : un module dédié ne sera décidé qu'à l'arrivée d'un second (tables CSV et xlsx à en-tête, rapport du suivi d'expérience ; ADR 0012, point 1.4).

Aucun calcul (E3.6 ; ADR 0012, point 2.2). Chaque valeur affichée est lue dans l'objet, dans l'ordre de l'objet, et écrite par l'écrivain unique en forme « texte » (`ecrire_valeurs()` : doubles en `%.17g`, entiers en `%d`, dates ISO 8601, chaînes UTF-8 entre guillemets droits avec leurs échappements, `NA` sans guillemets ; section 15.4), puis échappée pour le HTML (`echapper_html()` : `&` d'abord, puis `<`, `>` et `"`, sur tout texte inséré, valeurs, chemins, noms, titres et textes fixes). Les valeurs du rapport sont donc, caractère pour caractère et à l'échappement HTML près, celles de la fiche écrite et des références de non-régression, sauf `fiche/empreintes/resultat` et `fiche/empreintes/code`, que les références remplacent par un jeton (section 4.5) : un tiers rapproche les trois sans conversion. Ni somme, ni ratio, ni tri, ni filtre ; `nrow()`, `length()` et `names()` ne servent qu'à lire la structure de l'objet. **Aucun arrondi** : 0,95 s'écrit `0.94999999999999996`, écriture à dix-sept chiffres significatifs du double le plus proche ; un certificateur ne recalcule un taux et ses bornes au bit près qu'à cette précision, et une vue arrondie, si elle est voulue, s'ajoutera sans remplacer celle-ci (question Q36 ouverte). La relecture au bit près de cette écriture est *constatée* sous glibc et sous Windows (R 4.3.3 ucrt), non garantie ailleurs, ni démontrée nulle part (ligne `doubles` de `CONVENTIONS_TEXTE`). Aucune autre mise en forme d'un nombre n'est admise dans la section du rapport : un test lit son code par `parse()` et refuse tout appel à `format`, `formatC`, `sprintf`, `as.character`, `cat`, à une fonction d'horloge ou d'environnement, et à une liste fermée d'autres fonctions d'écriture ; `paste()` et `paste0()`, que ce contrôle ne peut pas juger, n'y reçoivent que des littéraux, des sorties de l'écrivain ou de `echapper_html()` et du texte du moteur (revue ligne à ligne d'`audit` ; ADR 0012, seconde annotation, (b) 6).

Déterminisme (E4.6 ; ADR 0012, point 2.4). Le rapport ne porte **aucune date** : il ne lit rien dans `execution`, ni horloge, ni session, ni option. Deux constructions identiques hors `execution` donnent donc les mêmes octets, et un rapport se compare par son MD5. Les textes non ASCII assemblés sont marqués UTF-8 (`texte_moteur()` pour le texte du moteur, marque posée sur la sortie de l'écrivain), et l'échappement ne retire pas cette marque : les octets ne dépendent ni de la locale ni des options `OutDec`, `scipen` et `digits`. C'est un *constat* des batteries (tests unitaires sous plusieurs options et locales ; contrôle `tests/controles/locale_c.R`, rapports des six plans de référence identiques octet à octet sous la locale C et sous une locale UTF-8), non une propriété démontrée.

Contrôles avant l'écriture (`controler_rapport()` ; ADR 0012, point 2.5, et seconde annotation, (b) 3 et 4). Dans cet ordre, le premier refus arrête tout, et rien n'est écrit ; les refus sont levés sous le module `rapport` (tableau 28) :

1. **objet** : une liste de classe exactement `construction`, dont la fiche est une liste ;
2. **version** : `fiche$version_fiche identical()` à `VERSION_FICHE`, la version courante seule, parce que les chemins des sections dépendent de la version ; un objet d'une autre version ne peut venir que d'un objet sauvegardé par un code antérieur, dont le rapport s'écrit avec ce code ;
3. **intégrité** : l'empreinte du résultat, recalculée sur l'objet reçu par `empreinte_resultat()`, est égale à `fiche$empreintes$resultat` ; une erreur du recalcul (attribut inconnu, structure altérée) est elle aussi un objet modifié, de même qu'un nom d'encodage `bytes` au premier niveau de l'objet ou de sa fiche, à toute position, qui rend l'empreinte non recalculable et que le contrôle relève dès le contrôle 1, avant toute lecture par nom (`noms_bytes()` ; issue #114) ; une fiche de classe d'encodage `bytes`, ou une sous-liste de nom ou de classe `bytes` dans le périmètre de l'empreinte, la rend aussi non recalculable et est refusée ici, les champs étant lus par `lire_champ()` (issue #128) ; hors de ce périmètre (`versions`, `plateforme`), un nom `bytes` est refusé à l'écriture de la section par `chaine_codage_bytes`, une classe `bytes` par `type_non_canonique` ; une autre classe non canonique (par exemple « zzz ») sur `versions` ou `plateforme` y est rendue sans refus ni mention, et la classe n'apparaît pas dans le rapport (*constat* ; issue #141, en attente de décision) ;
4. **couverture** : aucun élément de l'objet hors des sections : tout élément de premier niveau autre que

table, etapes, avertissements, fiche et execution, toute étape ou tout champ de la fiche qu'aucune racine de SECTIONS_RAPPORT ne couvre, et toute racine « non produite » présente et non NULL sont refusés : **le rapport n'omet aucun élément en silence**, au sens de la couverture (un attribut comme la classe de versions, ci-dessus, n'est pas un élément).

Table 28 : Refus du rapport figé (module rapport).

Motif et contrôle	Message	details
rapport_objet_invalide, contrôle 1 (pas une liste)	« Rapport figé : objet de type [type] au lieu d'une liste de classe « construction » (objet rendu par construire_table(), ou élément construction du résultat de rejouer_fiche()). »	lu (classes), attendu « construction »
rapport_objet_invalide, contrôle 1 (autre classe)	« Rapport figé : objet de classe « [classes] » au lieu de « construction » (objet rendu par . . .) »	idem
objet_modifie, contrôle 1 (nom d'encodage bytes de l'objet, relevé avant la forme de la fiche ; puis de la fiche, une fois établi qu'elle est une liste)	« Rapport figé : objet modifié après sa construction : empreinte du résultat non recalculable (nom de champ d'encodage « bytes » : champ n° [position du premier] de [l'objet, ou la fiche], cité [nom cité]). Le rapport ne restitue un objet que tel que construire_table() l'a rendu. »	chemin (vide, ou « fiche »), lignes (positions), lu (noms bruts)
rapport_objet_invalide, contrôle 1 (fiche)	« Rapport figé : objet de classe « construction » dont la fiche est [valeur décrite, ou « absente »] au lieu d'une liste. »	chemin « fiche », lu (valeur brute), attendu « list »
version_fiche_non_admise, contrôle 2	« Rapport figé : version de fiche [valeur décrite, ou « absente »] au lieu de [VERSION_FICHE décrite] (seule version admise par ce code) ; objet construit par tablexp [version de la fiche], empreinte du code [empreinte du code de la fiche] ; code installé : empreinte [EMPREINTE_CODE]. Aucune conversion n'est faite : le rapport d'un objet d'une autre version s'écrit avec le code qui l'a construit. »	lu (valeur brute), attendu
objet_modifie, contrôle 3 (empreinte différente)	« Rapport figé : objet modifié après sa construction : empreinte du résultat recalculée [recalculée] au lieu de [consignée] (fiche/empreintes/resultat). Le rapport ne restitue un objet que tel que construire_table() l'a rendu. »	lu (consignée), attendu (recalculée)
objet_modifie, contrôle 3 (recalcul impossible)	« Rapport figé : objet modifié après sa construction : empreinte du résultat non recalculable ([message de l'erreur]) ; fiche/empreintes/resultat vaut [consignée]. »	lu (consignée), attendu NA
rapport_objet_invalide, contrôle 4	« Rapport figé : élément « [chemin] » de l'objet non prévu par les sections de ce rapport ; un objet de cette version de fiche n'en porte pas, et le rapport n'omet aucun élément en silence. »	chemin

Une valeur lue y est « décrite » par decrire_valeur() : un atomique de longueur 1 sans attribut, de type double, integer, logical ou character, s'écrit par l'écrivain suivi de son type (3 (integer), 4 (double)), toute autre valeur « objet de type [type], de classe [classes] » ; l'écriture et les classes sont citées selon la règle unique des refus (section 2.1, « Citation d'une valeur dans un message » : les échappements \\, \n, \r, \t et \" de l'écriture des chaînes restent tels quels, les autres caractères de l'ensemble neutralisé, VT, FF, NEL, U+2028. . ., s'écrivent <U+XXXX>). Les attributs sont lus avant la longueur, que length() calculerait en levant une erreur R sur une classe d'encodage bytes. Une valeur insérée telle quelle (version, empreinte) passe par valeur_message() : citée par la règle si c'est une chaîne unique que la règle cite telle quelle (citation_a_neutraliser() faux : ni caractère de l'ensemble neutralisé, ni octets hors UTF-8 valide, ni encodage bytes ; une chaîne latin-1 est convertie et le résultat marqué UTF-8), « absente » si elle manque, décrite sinon ; les classes de l'objet reçu (contrôle 1), le chemin d'un élément non prévu (contrôle 4) et le message d'une erreur du recalcul (contrôle 3) sont cités par la règle : un message tient sur une ligne. version_fiche_non_admise est le **même motif** que celui du rejeu outillé, levé ici sous le module rapport et là sous le module construire_table ; le texte du

message diffère.

Ce que garantit le contrôle d'intégrité. Le recalcul porte sur les bits de l'objet (forme « empreinte », section 15.3), non sur un calcul : il ne dépend pas de la machine qui le fait, par construction (écriture exacte des bits), ce qui n'est pas mesuré d'une machine à l'autre. Un objet sauvegardé puis rechargé (`saveRDS()`, `readRDS()`) passe et donne les mêmes octets (constat des tests, sur une même machine). Le contrôle est **interne** : sa référence est l'empreinte que porte l'objet lui-même. Il établit que l'objet restitué est, dans le périmètre de l'empreinte du résultat et à une collision du MD5 près, celui dont la fiche de cet objet porte l'empreinte ; une modification d'un ulp d'un double de ce périmètre est refusée. Pour rattacher le rapport à une fiche remise à part, le lecteur compare le `fiche/empreintes/resultat` affiché par le rapport à celui de cette fiche. Il ne protège **pas** contre une modification délibérée suivie du recalcul de l'empreinte : le MD5 est sans rôle de sécurité (section 15.2) ; un tel objet est restitué sans refus, sauf s'il porte un élément hors des sections (contrôle 4). Les chemins hors du périmètre (`CHEMINS_HORS_RESULTAT` sauf `execution` : `fiche$versions`, `fiche$plateforme`, les trois dernières empreintes et `etapes$donnees$resultat$lignes_conservees`) échappent au contrôle, à une exception près : `fiche$empreintes$resultat`, exclue du texte empreinté puisqu'elle en est l'empreinte, est la valeur à laquelle le contrôle compare le recalcul. Le rapport le dit dans la phrase qui suit le contrôle (ci-dessous), qui nomme cette valeur et n'énumère comme hors du contrôle que les cinq autres chemins (issue #94).

Sections (`SECTIONS_RAPPORT`, tableau 29). Le document a quatorze sections, dans un ordre fixe, chacune avec un identifiant (ancre HTML), un titre, une nature (`texte`, `objet`, `etape`, `renvoi`, `non_produite`), l'étape du catalogue pour la nature `etape`, et ses *racines*, chemins de l'objet qu'elle restitue dans l'ordre d'affichage. Chaque élément de premier niveau de l'objet hors `execution`, chaque étape et chaque champ de la fiche est couvert par une racine ; les racines `fiche/*` des sections 2 à 6 sont exactement les champs de `ORDRE_CHAMPS_FICHE` : la fiche est restituée en entier, y compris ce qu'elle recopie d'une étape.

Table 29 : Sections du rapport figé (`SECTIONS_RAPPORT`).

Identifiant	Titre	Nature	Racines et contenu
`lecture`	Lecture du rapport	texte	textes de lecture (ci-dessous) ; conventions d'écriture de `CONVENTIONS_TEXTE`, dans leur ordre ; empreinte du code qui écrit le rapport
`identification`	Identification : empreintes, versions et plateforme	objet	`fiche/version_fiche`, `fiche/empreintes`, `fiche/versions`, `fiche/plateforme` ; puis mentions des empreintes, phrase de plateforme et phrase du contrôle d'intégrité
`alea`	Graine et générateur	objet	`fiche/graine`, `fiche/graine_mention`, `fiche/generateur`
`donnees_recues`	Rôles, provenance, recette et unité d'observation	objet	`fiche/roles`, `fiche/provenance`, `fiche/provenance_mention`, `fiche/recette`, `fiche/unite_observation`, `fiche/avertissement_unite`
`plan`	Plan de construction (plan canonique), transition et fin effective de la période	objet	`fiche/plan`, `fiche/transition_code`, `fiche/fin_effective`

Identifiant	Titre	Nature	Racines et contenu
fiche_hypotheses	Paramètres retenus et hypothèses consignés dans la fiche	objet	fiche/convention, fiche/unite_exposition, fiche/par_annee, fiche/exclusions, fiche/nombre_lignes_exclues, fiche/hypothese_censure, fiche/hypotheses_brut, fiche/avertissements
etape_donnees	Étape donnees : exclusions, unité d'observation et relevés	etape	étape donnees : etapes/donnees, sous-arbre entier
etape_exposition	Étape exposition : convention d'âge, exposition et événements par cellule	etape	étape exposition : etapes/exposition
etape_brut	Étape brut : taux bruts et intervalles	etape	étape brut : etapes/brut ; note sur la colonne taux en tête, légendes après le tableau des statuts (ci-dessous)
suffisance	Indicateurs de suffisance	renvoi	aucune racine ; renvois à etapes/exposition/resultat/exposition et à etapes/brut/resultat, ou « non demandée par le plan » ; mention de la largeur des intervalles
avertissements	Avertissements	objet	avertissements
validations	Validations	non_produite	etapes/validation : absente, « non produites par cette version » ; présente, refus (contrôle 4)
table_finale	Table finale et comparaisons	non_produite	table : NULL, « non produites par cette version » ; autre valeur, refus (contrôle 4)
rejeu	Rejeu	texte	ligne # rejeu de la fiche (TEXTE_REJEU_FICHE) et textes du rejeu outillé (ci-dessous)

Une section de nature etape dont l'étape est absente de l'objet porte « Étape « [étape] » : non demandée par le plan (chemin etapes/[étape] absent de l'objet). » ; cette mention est lue dans l'objet. « Non produites par cette version » (validations, table finale, largeur des intervalles) ne peut pas l'être : elle vient de la liste des sections. Un test vérifie que les étapes des sections de nature etape sont exactement celles du catalogue ETAPES, dans son ordre, et qu'aucune racine « non produite » n'est sous une étape du catalogue : le jour où une étape validation existera, la batterie échouera tant que le rapport n'aura pas de section pour elle. Le module rapport ne lit pas ETAPES, que le graphe d'appels lui interdit : la concordance est un test, non une lecture.

Primitives de présentation. Deux suffisent. Une *table de lignes* chemin | type | longueur | valeurs restitue tout nœud qui n'est pas un data.frame, avec exactement les lignes du corps de la fiche écrite (section 15.4, grammaire 1) : une ligne chemin/@noms pour une liste ou un vecteur nommé, chemin/@longueur pour une liste sans noms, puis ses éléments, dans l'ordre de l'objet, aux mêmes segments de chemin ; NULL s'écrit chemin | NULL | 0 | (cellule vide) ; les valeurs d'une même ligne sont séparées par un retour à la ligne ; une racine absente porte « absent de l'objet ». Une *table de colonnes* restitue tout data.frame : une légende (<chemin> : data.frame ; lignes : N ; colonnes : M), un en-tête qui donne pour chaque colonne son nom et le type rendu par l'écrivain, puis une ligne par ligne de l'objet, dans son ordre, sans numéro de ligne ; la table de lignes en cours est fermée avant elle et rouverte après. Les tables dont le volume croît avec la base (CHEMINS_STRUCTURE_SEULE : lignes_conservees, numeros_conserves, lignes_exclues, observations et observation_des_lignes de etapes/donnees/resultat, kaplan_meier_sauts de etapes/brut/resultat) sont citées sur une seule ligne : chemin, type (data.frame, ou le type R), nombre de lignes ou longueur, « non repro-

duites (volume croissant avec la base) », suivi, pour un data.frame, des noms de ses colonnes, et, pour lignes_exclues, du renvoi « nombre de lignes par motif : etapes/donnees/resultat/exclusions ». Les numéros des lignes exclues ne sont donc pas reproduits : leur volume n'est pas borné, et une troncature serait à déclarer (question Q36).

Textes fixes (MENTIONS_RAPPORT, texte du moteur marqué UTF-8 ; spécification d'actuary du 24/10/2026). La section *lecture* dit ce qu'est le document (« il restitue, sans recalcul, l'objet rendu par construire_table() [. . .] chaque valeur est lue dans l'objet et écrite telle quelle, dans l'ordre de l'objet »), qu'il ne porte pas de date et que la construction est identifiée par ses empreintes, la règle des nombres (« Aucun arrondi : 0,95 s'écrit 0.94999999999999996 [. . .] »), les deux primitives, les quatre statuts des résultats (E2.20), avec renvoi au tableau etapes/brut/parametres/statuts, et l'empreinte du code **qui écrit le rapport** (EMPREINTE_CODE), distincte de fiche/empreintes/code, empreinte du code qui a construit l'objet. La section *identification* fait suivre le tableau de quatre mentions d'empreinte (algorithme MD5 sans rôle de sécurité ; empreinte des données portable, exacte sous glibc, non mesurée ailleurs ; empreinte du fichier calculée par la couche de lecture, non vérifiée par le moteur ; statut du résultat et du code consigné dans fiche/empreintes/statut_resultat_code) et de deux phrases :

— « L'empreinte du résultat dépend de la plateforme (R, BLAS, LAPACK, bibliothèque C) ; elle n'est un témoin d'identité que sur la même machine (E3.6, E4.6). » ;

— « Contrôle d'intégrité : avant l'écriture de ce rapport, l'empreinte du résultat a été recalculée sur l'objet et trouvée égale à fiche/empreintes/resultat, la valeur consignée à laquelle ce contrôle la compare (exclue du texte empreinté, puisqu'elle en est l'empreinte). Hors du périmètre de cette empreinte, et donc de ce contrôle : [chemins de CHEMINS_HORS_RESULTAT sauf execution et fiche/empreintes/resultat, joints par « , »] ; ce que le rapport en restitue (valeurs, ou structure seule pour une table dont le volume croît avec la base) est lu dans l'objet, sans vérification. »

La section *étape brut* s'ouvre sur la note « La colonne taux porte une force de transition pour l'estimateur poisson (D / Eˆc) et une probabilité pour binomial (D / Eˆi) et kaplan_meier ; un taux brut se désigne avec son estimateur. » Le tableau etapes/brut/parametres/statuts, qui précède les résultats dans l'ordre de l'objet, est restitué en entier, et suivi de la légende « Les mots « exact » et « asymptotique » des noms de colonnes désignent la formule de l'intervalle, non une couverture garantie : le statut de chaque grandeur et de chaque intervalle est celui du tableau ci-dessus (colonnes statut, loi_supposee et calcul), qui fait foi. », puis, si l'estimateur binomial est retenu, de « Intervalle binomial « exact » [. . .] : la formule par quantiles de lois bêta suppose D ~ Bin(Eˆi, q) avec Eˆi entier ; sous troncature à gauche et censure, l'exposition initiale Eˆi n'est en général pas entière, et la formule en est alors le prolongement continu, de couverture non établie. » (section 9.3). La section *suffisance* (E2.15) renvoie aux chemins qui portent exposition, événements et bornes, sans en calculer aucun, et dit « Largeur des intervalles : non produite par cette version. » : une largeur serait une quantité nouvelle, à calculer dans le package comme grandeur du résultat. La section *rejeu* reprend la ligne # rejeu de la fiche écrite, puis décrit le rejeu outillé et précise que le bilan d'un rejeu n'entre pas dans l'objet. La ligne # rejeu vient d'une seule constante, TEXTE_REJEU_FICHE, extraite du littéral d'ecrire_fiche() sans changer un octet de la fiche (MD5 des fiches des six plans de référence identiques avant et après, cités dans le message du commit 5135667).

Forme du document (ADR 0012, point 2.7). Un seul fichier HTML : <!DOCTYPE html>, <html lang="fr">, <meta charset="utf-8">, un titre constant, la feuille de style en ligne STYLE_RAPPORT (une ressource interne, non externe), une table des matières (<nav>) qui renvoie aux ancres des sections, puis une balise <section> par section, titre sans numéro. Ni <link>, ni <script>, ni <img>, ni police ou adresse externe : un test le vérifie, ainsi que la liste fermée des balises et des entités employées. Aucun tracé : R de base n'a pas d'encodeur base64, qui permettrait d'inclure une image dans le fichier.

Limites. Le rapport figé est le seul livrable qui porte les valeurs exactes : la fiche ne contient ni exposition

ni taux, et les tables CSV et xlsx à en-tête sont reportées à la première branche qui produit une table finale (décision M93). Il ne porte ni vue arrondie, ni numéros des lignes exclues, ni tracé, ni bilan de rejeu ; ses validations, sa table finale et la largeur des intervalles sont « non produites par cette version ». Sa taille et sa durée d'écriture sur les plans de référence sont des constats de mesure, consignés dans le message du commit `5135667`, sans seuil.

15.6 Vérifications

`tests/unitaires/test_construire_table.R` compare l'empreinte d'un petit `data.frame`, colonnes facteur, chaîne et double comprises, à une valeur calculée hors de R (texte écrit par `printf`, octets des doubles par `struct.pack` de Python, empreinte par `md5sum` de GNU coreutils). Il vérifie l'écrivain des valeurs dans ses deux formes : les vecteurs de test de l'ADR 0011 (octets calculés hors de R), sous-normaux, plus petit normal et maximum, jetons des non-finis lus sur la valeur reçue quels que soient leurs octets, zéro négatif, attribut de dimension, mélange de valeurs finies et de jetons, longueurs limites, types autres que double identiques dans les deux formes, refus ; les mêmes jetons dans les deux formes, par la règle unique `jeton_non_fini()`, sur `NA`, `NA + 1`, `NaN`, `Inf`, `-Inf` et huit motifs d'octets de `NaN`, dont ceux de `NaN` et de `NA_real_`, construits par leurs octets, seuls ou mêlés à des finis, le jeton attendu de chaque motif étant écrit en dur d'après `is.na()` et `is.nan()` de R 4.3.3 (témoin d'un changement de R, non règle de l'ADR 0011 ; issue #118) ; et un échantillon de 10^5 doubles tirés sous graine, dont le MD5 du texte doit être celui que produit le juge Python, hors de R. Le contrôle `tests/controles/ecriture_doubles.R`, **hors batterie et hors CI** (durée, dépendance à Python), étend ce recoupement à 10^6 doubles et aux cas limites, ligne à ligne, et sort avec un code non nul sur un écart ou si aucun interpréteur Python 3.8 ou plus n'est trouvé ; la batterie unitaire n'exerce du script lui-même que des chemins bon marché, juge absent et chemin à espace (section 5.1) ; il ne s'exécute pas sous Windows, où le texte `%.17g` qu'il transmet au juge n'est pas toujours correctement arrondi.

Écriture « texte » des doubles (`octets_17g()`, issue #104 ; section 15.2, « Écriture « texte » des doubles »). Dans `tests/unitaires/test_construire_table.R`, aucun test ne prend `sprintf()` pour juge ; les entrées sont construites par leurs octets (`readBin()`) ou par des opérations exactes (divisions par 2, produits d'entiers inférieurs à 2^{53}), sans lecture d'un décimal, et les attendus sont ceux de deux juges extérieurs à `tablexp`, la glibc (`sprintf("%.17g", x + 0)` de R) et Python (`format(x + 0.0, '.17g')`), identiques entre eux : *T1*, 29 doubles (sous-normaux, bornes du format, frontières de notation, égalités exactes et leurs voisins), textes attendus écrits en dur ; *T2*, les égalités exactes énumérées sans générateur (m impair, $m \cdot 5^n \in [10^{17}, 10^{18})$, chaque n de 2 à 25, $x = m \cdot 2^{-n}$; toutes les valeurs de m quand il y en a au plus 1 000, sinon 1 000 réparties et les bornes), leurs voisins à ± 1 ulp et leurs opposés, et *T3*, 10^5 doubles tirés sous graine par leurs octets, chacun comparé par le MD5 du texte attendu ; les jetons des non-finis (`NA`, `NA + 1`, `NaN`, `-NaN` donné par ses octets, `Inf`, `-Inf`), les longueurs 0 et 1 et le mélange de finis et de non-finis ; `.Random.seed` inchangé par l'écriture ; la table des puissances égale à son recalcul, entière, aux tranches dans $[0, 10^7)$. `tests/unitaires/test_serialisation.R` vérifie sans condition de plateforme l'écriture d'un niveau construit par ses octets (`4.8005694404694591e-06`, que la bibliothèque C de Windows écrit `4.800569440469459e-06`) et la relecture de ces deux textes en le même double (constat de mesure). Le contrôle `tests/controles/ecriture_doubles_texte.R` (tableau 7), **hors batterie** (durée, dépendance à Python) et exécuté par une étape de la CI, la batterie unitaire n'exerçant du script lui-même que des chemins d'échec bon marché, juge absent et vrai défaut sur une copie mutée (section 5.1), recoupe l'écriture du moteur sur plus de 10^6 doubles, compare chaque ligne aux deux juges, exige le nombre d'égalités exactes de chaque partie et recalcule les MD5 de T2 et de T3. **Statut** : les attendus sont des *constats* de deux juges, non des valeurs démontrées ; l'arrondi correct relève de la preuve. Sous Windows, *constat de mesure* sur le poste du mainteneur (Windows 11 Home, version 10.0.26300, R 4.3.3 ucrt, Python 3.14.7, 06/11/2026) : le contrôle complet passe, le juge glibc étant sauté ;

le juge Python relève 0 écart sur 2 423 344 doubles, les MD5 des textes de T2 et de T3 sont égaux aux constantes de la batterie unitaire et la table des puissances est sans écart. Ce constat ne garantit pas la portabilité : un seul poste, un seul juge, hors CI.

Les dates sont vérifiées sur quatre chiffres d'année (0001, 0019, 0999, 9999), sur tous les jours de 1930 à 2024 (identiques à l'écriture de `format.Date`, si bien que les références, dont les dates sont dans cet intervalle, n'ont pas changé), avec les jetons `NA` et `NaN` distincts (deux empreintes calculées hors de R) et les refus `date_hors_domaine` et `date_non_entiere` ; `construire_table()` traite une date `NaN` comme un manquant.

Écriture des chaînes (`octets_chaines()`, issue #22, point 1). Les chaînes sont écrites par blocs de lignes : les corps échappés, marqués `bytes` pour être copiés sans traduction en toute locale, sont collés par un `paste()`, puis convertis en octets par un seul `charToRaw()` par bloc, au lieu d'une liste de trois vecteurs d'octets par ligne. Une valeur manquante prend le corps « un guillemet » : sa ligne s'écrit de trois guillemets suivis d'un LF, puis est ramenée à `NA` et LF par un `gsub()`. Ce marqueur ne peut pas apparaître dans une autre ligne : les cinq échappements étant appliqués à la barre oblique inverse d'abord, puis à LF, CR, la tabulation et au guillemet, un corps échappé ne contient ni LF ni guillemet qui ne soit précédé d'une barre oblique inverse ; il ne commence donc jamais par un guillemet et n'en porte jamais deux de suite. Une autre ligne, guillemet ouvrant, corps, guillemet fermant et LF, porte ainsi au plus deux guillemets consécutifs (le dernier caractère du corps et le guillemet fermant, ou les deux guillemets d'un corps vide) devant sa fin de ligne, jamais trois, et le motif, qui se termine par LF, ne peut pas chevaucher deux lignes. Les blocs sont bornés par `bornes_blocs()` à 2^{30} octets de texte avant ce `gsub()`, sous la limite de $2^{31} - 1$ octets d'une chaîne R (aide `Memory-limits`) ; une ligne plus longue que la borne forme seule son bloc et s'écrit directement en octets. La règle d'écriture des chaînes (section 15.2) est inchangée. `tests/unitaires/test_construire_table.R` compare les octets produits à ceux de l'écriture antérieure, recopiée dans le test comme oracle, sur 17 cas adverses (`NA` et `"NA"`, chaîne vide, chaînes non ASCII en UTF-8 et en latin-1, cinq échappements, suites de guillemets, répétitions, longueurs 0 et 1, ligne de plus de 5 000 octets) pour 8 bornes de bloc, de 1 à 2^{30} octets, dans la session et dans un sous-processus lancé sous `LC_ALL=C` ; il vérifie les bornes rendues par `bornes_blocs()` sur 45 bornes et 3 jeux de corps (partition des lignes dans l'ordre, bloc de plusieurs lignes d'au plus la borne et maximal, ligne trop longue seule dans son bloc et seule marquée). Les statuts sont distincts : l'argument du marqueur *établit*, par raisonnement sur la construction, que la substitution ne porte que sur les lignes des valeurs manquantes ; l'identité des octets à ceux de l'écriture antérieure n'est que *constatée* sur ces cas. Le script `tests/mesures/octets_chaines.R`, hors batterie et sans seuil, mesure la durée et la mémoire de cette écriture et de l'empreinte à deux rôles caractère sur un jeu déterministe.

Écriture des entiers et des dates (`octets_entiers()`, `octets_lignes_indices()`, issue #71). La règle d'écriture (section 15.2) est inchangée : un `integer` s'écrit comme `sprintf("%d", x)`, une date en ISO 8601 ; seul l'assemblage des octets change, qui ne passe plus par une chaîne par ligne. `octets_entiers()` traite la colonne par blocs de 2^{20} valeurs consécutives (argument `bloc`, qui ne sert qu'aux tests du découpage ; issue #83), dont les octets sont concaténés dans l'ordre. Pour chaque bloc, elle forme une matrice d'octets d'une colonne par valeur et d'une ligne par position (signe, chiffres, LF), le nombre de chiffres étant celui de la plus grande valeur absolue du bloc, deux au moins pour le jeton `NA` ; un chiffre de rang $k > 0$ n'est écrit que si le quotient de la valeur absolue par 10^k est non nul, si bien qu'aucun zéro de tête n'apparaît ; les places inoccupées valent l'octet nul, qui n'apparaît dans aucune écriture et que la lecture de la matrice colonne par colonne retire. Chaque valeur s'écrit ainsi sans remplissage et sans dépendre de ses voisines : le découpage en blocs ne change aucun octet, ce qui découle de la construction (il ne fixe que la hauteur des matrices, dont les places inoccupées sont retirées). Il borne le travail transitoire, dont le plus gros objet est le masque logique des places occupées d'un bloc, de $4(n_c + 2)$ octets par valeur pour n_c chiffres ; la sortie et sa copie lors de la concaténation restent proportionnelles à la longueur

de la colonne, que ecrire_valeurs() rend entière. La baisse du pic mémoire est un constat de mesure, hors plateforme de référence, consigné avec le commit 2b4ebda. octets_lignes_indices() écrit une fois chaque date distincte, puis recopie ses octets, LF compris, aux lignes où elle figure, par sequence() ; les jetons NA et NaN d'une date manquante sont lus comme avant sur la valeur reçue. Dans les deux formes, ecrire_valeurs() emploie ces deux fonctions. tests/unitaires/test_construire_table.R compare les octets produits à ceux de l'écriture antérieure, recopiée dans le test comme oracle : pour les entiers, 14 cas (vecteur vide, NA, zéro, signes, de 1 à 10 chiffres, $\pm(2^{31}-1)$, noms, dimension, constante, 10^5 valeurs pseudo-aléatoires sans générateur), par octets_entiers() et par ecrire_valeurs() dans les deux formes, puis par octets_entiers() avec des blocs de 1, 2, 3, 7, 1 000 et 10^6 valeurs, plus une colonne de $2^{20}+3$ valeurs découpée en deux blocs de largeurs différentes (NA et entier négatif dans le second), et une sortie écrite à la main ; pour octets_lignes_indices(), 8 suites d'indices (vide, texte vide, répétitions, ordre inverse, 10^5 indices) ; pour les dates, NA, NaN, stockage integer, valeurs répétées et 10^5 jours du domaine, dans les deux formes. tests/test_reproductibilite.R vérifie en outre que l'empreinte du résultat de chacun des six plans de référence est égale à celle que donne un oracle, empreinte_resultat() réévaluée avec les écrivains antérieurs des entiers et des dates ; l'oracle remplace une valeur figée, l'empreinte du résultat ne valant que sur une même machine (section 15.3). Les plans de référence ne portent aucun entier négatif (les dates s'écrivent en ISO, sans signe) : le signe des entiers est gardé par les tests unitaires, non par cet oracle. Les statuts sont ceux de l'écriture des chaînes : l'identité des octets à ceux de l'écriture antérieure est *constatée* sur ces cas, non démontrée. Le script tests/mesures/temps_construire_table.R, hors batterie et sans seuil, mesure le temps des deux empreintes (médiane et étendue de mesures répétées).

tests/unitaires/test_serialisation.R vérifie le texte du plan et de la fiche (section 15.4) : forme des octets, première ligne et en-tête hors aller-retour ; aller-retour identical() sur des cas adverses (NA et "NA", NA et NaN, chaînes non ASCII, doubles extrêmes et sous-normaux, dates aux bornes du domaine, listes et vecteurs vides, NULL, noms qui ne peuvent pas être des segments) ; code R en valeur, en nom ou en chemin, jamais évalué (témoin non créé ; le code piégé, file.create() en R pur, chemin absolu et relatif, crée bien le témoin quand on l'évalue, contre-épreuve faite avant les tests puis témoin supprimé ; issue #110), et absence d'appel à une fonction d'évaluation dans les lecteurs ; chaque refus de la lecture et de l'écriture, avec son numéro de ligne ou son chemin ; lecteur de forme des doubles et refus du sous-dépassement ; plan importé soumis aux mêmes refus qu'un plan en mémoire ; liste sans noms, troncature à chaque frontière de ligne, profondeur 32 admise et 33 refusée ; partition des champs de la fiche par les blocs de lecture ; rejeu identical() hors execution. La batterie de reproductibilité refait l'aller-retour et le rejeu pour les six plans de référence et refuse chacun des préfixes stricts du corps de la fiche écrite de R1. tests/test_reproductibilite.R vérifie que l'empreinte est la même dans des processus R lancés sous LANG=C et sous LANG=C.UTF-8, égale à cette valeur, et que le rejeu de la fiche redonne l'objet. tests/unitaires/test_construire_table.R vérifie aussi chaque cas de refus de la provenance décrit à la section 15.2 (issue #21).

Empreintes du résultat et du code (issue #67 ; section 15.3). tests/unitaires/test_construire_table.R vérifie, pour le résultat : l'identité exacte sur deux constructions, au rejeu de la fiche et au recalcul sur l'objet ; l'insensibilité aux chemins hors du périmètre (execution, fiche$versions, fiche$plateforme, les trois derniers champs de fiche$empreintes, et lignes_conservees munie d'une colonne POSIXct et d'attributs) ; la sensibilité, sur des copies, à un ulp d'un double du périmètre, à la première ligne, à l'empreinte des données, au plan et aux rôles ; l'égalité à une valeur calculée hors de R, sur un petit objet (texte écrit et haché par Python, doubles par struct.pack), qui exerce les jetons, le zéro négatif, les dates, un facteur, les chaînes échappées, un data.frame, une liste sans noms et l'exclusion des chemins ; l'erreur de programmation sur un attribut inattendu (dont simulation hors de lignes_conservees), une classe de racine autre que construction

et une valeur sans écriture ; le facteur, admis en forme « empreinte » et refusé en forme « texte », une colonne de rôle en facteur aux niveaux permutés donnant la même empreinte ; la construction acceptée, avec la même empreinte que sans attribut, quand chaque colonne de rôle porte un attribut `label`, dans l'unité tête comme dans l'unité contrat, `id_tete` compris, et quand `sexe` ou `segment` est un facteur muni de `contrasts` ou de sous-classe, dans les deux unités (décisions M71, point 4, et M72). Pour le code : `EMPREINTE_CODE` n'est pas `NA` et égale le recalcul fait par la batterie sur `R/*.R` de la racine qu'elle a installée, en R et, si `md5sum` et `sh` sont présents, par `LC_ALL=C md5sum -t R/*.R | md5sum` (mode texte explicite, pour le `md5sum` de Rtools sous Windows ; issue #101) ; `R/` ne contient que des fichiers `*.R` ; `empreinte_sources()` rend `NA` sans `DESCRIPTION`, pour un autre paquet, sans `R/rapport.R` et pour un fichier illisible (lien cassé), et le recalcul sur une racine conforme ; le statut a le libellé exact, marqué UTF-8, et nomme les sources introuvables si et seulement si l'empreinte du code est `NA`. `tests/test_reproductibilite.R` recalcule l'empreinte du résultat dans ses tests d'invariance (plan sans `brut`, plan sans `kaplan_meier`) et exerce la comparaison par jeton des références (section 4.5).

Rapport figé (section 15.5 ; issue #87). `tests/unitaires/test_rapport.R`, sur un jeu réduit simulé sous la loi du jeu de référence, avec des plans de la forme de R1 à R6 et un plan réduit à l'étape `donnees`, vérifie : la forme des octets (UTF-8 valide, ni marque d'ordre ni CR, LF final, en-tête HTML) ; l'identité des octets pour deux écritures du même objet, pour deux constructions identiques hors `execution`, sous d'autres valeurs de `OutDec`, `scipen` et `digits`, sous la locale C en cours de session et après `saveRDS()` puis `readRDS()` ; l'objet inchangé par l'écriture ; chaque refus des quatre contrôles, avec son motif, son module et ses `details`, dont un double altéré d'un ulp, un attribut ajouté et une valeur de version ou d'empreinte à décrire ; la citation, selon la règle unique des refus, de classes, de valeurs de version (VT, FF, NEL, U+2028), du message d'une erreur du recalcul et d'un chemin non prévu, sur une ligne et marquée UTF-8 (issue #94) ; le refus nommé, au lieu d'une erreur R, d'un nom d'encodage `bytes` de l'objet ou de sa fiche, en première, quatrième et dernière positions, par `ecrire_fiche()` (`chaine_codage_bytes`) et par `ecrire_rapport()` (`objet_modifie` ; issue #114) ; pour l'issue #128, un nom `bytes` dans `empreintes`, `versions` ou `plateforme` (`chaine_codage_bytes`, rôle « *chemin*/@noms », ou `objet_modifie`) et une classe `bytes` sur la fiche ou ses sous-listes (`serialisation_impossible`, `objet_modifie` ou `type_non_canonique`), chaque fois au lieu d'une erreur R ; l'absence de toute ressource externe et la liste fermée des balises et entités ; l'échappement de modalités et de noms de colonne qui contiennent <, >, & et " ; l'ordre des sections et la concordance de leurs étapes avec `ETAPES` ; la couverture de l'objet par les racines ; les mentions « non demandée par le plan » ; l'égalité de chaque cellule décimale à l'écriture « texte » de sa valeur par l'écrivain du moteur (`ecrire_valeurs()`, et non plus `sprintf()` : ce test vérifie que les cellules passent par l'écrivain, dont l'exactitude relève des témoins T1 à T3 et du contrôle à deux juges, ci-dessus) ; la présence, dans le rapport, de chaque ligne du corps de la fiche écrite ; le contrôle statique des noms interdits, sur un témoin et sur `R/rapport.R` ; les textes fixes et leur place (légendes du tableau des statuts, légende binomiale selon les estimateurs retenus). Les tables de colonnes y sont comparées, cellule par cellule et en-têtes compris, à l'écriture échappée des colonnes de l'objet. `tests/test_reproductibilite.R` refait sur les six plans de référence l'ordre des sections, l'écriture des décimaux, les textes fixes et les légendes, et vérifie que la ligne `# rejeu` de la fiche écrite est `TEXTE_REJEU_FICHE`. `tests/controles/locale_c.R` compare les octets des six rapports sous les deux locales.

15.7 Fonctions du module

Table 30 : Définitions de premier niveau de R/rapport.R.

Nom	Statut	Entrées	Sortie et rôle
VERSION_FICHE	constante	—	version de la fiche (4)
VERSIONS_FICHE_REJEU	constante	—	versions de fiche admises au rejeu outillé : VERSION_FICHE, la version courante seule (section 6.4)
ENTETE_CANONIQUE	constante	—	"tablexp canonique 2", première ligne du texte empreinté et fiche$empreintes$representation (version de la représentation canonique)
ALGORITHME_EMPREINTE	constante	—	"md5"
REPRESENTATION_RESULTAT	constante	—	"tablexp resultat 1", première ligne du texte empreinté du résultat et fiche$empreintes$representation_resultat
STATUT_EMPREINTE_RESULTAT	constante	—	libellé du statut de l'empreinte du résultat (témoin d'identité sur la même machine seulement) ; littéral non ASCII lu par la sonde UTF-8 des batteries
STATUT_EMPREINTE_CODE	constante	—	libellé du statut de l'empreinte du code calculée (relevé des MD5 de R/*.R, commande de recalcul)
STATUT_CODE_INTROUVABLE	constante	—	libellé du statut de l'empreinte du code NA (sources introuvables à l'installation)
bornes_blocs()	interne	corps échappés (codage bytes), limite en octets	bornes debut et fin des blocs de collage d'octets_chaines() et indicateur seule : découpage glouton dans l'ordre des lignes, une ligne comptant les octets de son corps plus trois (deux guillemets et LF) ; un bloc de plusieurs lignes fait au plus limite octets et la ligne suivante n'y tiendrait pas ; une ligne plus longue que limite forme seule son bloc (seule)
octets_chaines()	interne	chaînes, rôle, limite (facultative, 2^{30} octets, réservée aux tests du découpage)	octets canoniques des chaînes ; refus de codage ; écriture par blocs de lignes (bornes_blocs()), un paste() et un charToRaw() par bloc, une ligne plus longue que limite écrite seule
octets_lignes()	interne	textes ASCII	octets, une valeur par ligne
octets_lignes_indices()	interne	textes ASCII, indices sans NA	octets des lignes textes[i], égaux à ceux d'octets_lignes() sur ces lignes ; chaque texte écrit une fois, ses octets recopiés par indice (sequence()) ; écriture des dates (issue #71)
octets_entiers()	interne	entiers (integer) ; bloc (défaut 2^{20}, tests seulement)	octets de sprintf("%d", x), une valeur par ligne, écrits en octets sans chaîne intermédiaire, par blocs de bloc valeurs ; écriture des entiers (issues #71 et #83)

Nom	Statut	Entrées	Sortie et rôle
texte_date()	interne	jours depuis 1970-01-01, entiers, finis, dans le domaine	écriture ISO 8601 AAAA-MM-JJ, année sur quatre chiffres ; partagée par ecrire_valeurs() et le contrôle de canonicité du lecteur (valeurs_texte())
BASE_TRANCHES	constante	—	base 10^7 des tranches des grands entiers de l'écriture « texte » des doubles (section 15.2, « Écriture « texte » des doubles »), poids faible d'abord ; produits et sommes de tranches restent des entiers inférieurs à 2^{53} (I3)
diviser_tranche()	interne	entiers de $[0, 2^{53})$ (vecteur ou matrice)	quotient et reste exacts par la base, quel que soit l'arrondi du quotient (correction d'une unité ; I4) ; ni %% ni %/% sur des doubles
reporter_retenues()	interne	matrice de tranches (une ligne par entier)	tranches dans $[0, 10^7)$, retenues reportées jusqu'à épuisement ; arrêt en erreur de programmation si la dernière tranche produit une retenue
table_puissances_tranches()	interne	—	table des puissances : 5^{-e} si $e < 0$, 2e sinon, e de -1074 à 1023, en tranches (matrice et longueur de chaque ligne), par multiplications successives par 5 et par 2
PUISSANCES_TRANCHES	constante	—	table des puissances, calculée à l'installation ; 2 098 lignes, 108 tranches au plus
chiffres_17()	interne	doubles finis > 0	dix-sept chiffres arrondis au plus proche, égalité vers le pair, et exposant décimal (I1 à I5) ; entier décimal exact calculé en tranches, groupé par classe de longueur de la table
positions_17g()	interne	doubles finis non nuls	octets ASCII de la forme %.17g (notation fixe ou à exposant, zéros de fin retirés, signe), une colonne par valeur
JETONS_NON_FINIS	constante	—	les quatre jetons des doubles non finis, NA, NaN, Inf, -Inf, dans l'ordre de leur rang ; seule liste, pour les deux formes de l'écrivain (ADR 0011, point 2.2 ; issue #118)
jeton_non_fini()	interne	doubles non finis	rang du jeton de chaque valeur dans JETONS_NON_FINIS, lu sur la valeur reçue sans opération préalable : 1 si is.na() et non is.nan(), 2 si is.nan(), 3 si positif, 4 sinon ; règle unique, appelée par octets_17g() et par l'écriture « empreinte » de ecrire_valeurs() (issue #118)

Nom	Statut	Entrées	Sortie et rôle
octets_17g()	interne	doubles	octets de l'écriture « texte », une valeur par ligne, LF final ; x + 0, jetons NA, NaN, Inf, -Inf lus sur la valeur reçue par jeton_non_fini() ; par blocs de 2^{14} valeurs ; appelée par ecrire_valeurs()
ecrire_valeurs()	interne	colonne, forme, rôle	libellé de type et octets ; forme "empreinte" (doubles en octets IEEE 754) ou "texte" (doubles en %.17g, calculé par octets_17g(), issue #104) ; refus forme_ecriture_inconnue, type_non_canonique (dont toute valeur de classe d'encodage bytes, facteur reconnu par a_classe() ; issue #128), et, en garde-fou, date_hors_domaine, date_non_entiere
texte_canonique()	interne	données, rôles, fichier	représentation canonique écrite en binaire ; colonnes lues par lire_champ() (issue #128)
empreinte_donnees()	interne	données, rôles	MD5 de la représentation canonique
controler_provenance()	interne	provenance	refus provenance_invalide
empreintes_construction()	interne	données, rôles, provenance	champ empreintes de la fiche, dans l'ordre de la section 15.1 ; resultat à NA, posé ensuite par construire_table() ; code égal à EMPREINTE_CODE ; contrôle d'abord, par octets_chaines(), le codage des valeurs des rôles et des champs de la provenance (refus de codage, issue #106)
statut_empreintes()	interne	empreinte du code	statut_resultat_code : libellé du résultat, " ; ", libellé du code calculé ou, si l'empreinte du code est NA, des sources introuvables ; marqué UTF-8
md5_octets()	interne	octets	MD5 des octets, écrits en binaire dans un fichier temporaire supprimé à la sortie
CHEMINS_HORS_RESULTAT	constante	—	liste fermée des sept chemins hors du périmètre de l'empreinte du résultat (section 15.3)

Nom	Statut	Entrées	Sortie et rôle
empreinte_resultat()	interne	objet construction	MD5 du texte de l'objet privé de CHEMINS_HORS_RESULTAT, en forme "empreinte", de première ligne REPRESENTATION_RESULTAT, sans en-tête ; classe de la racine vérifiée ; erreur de programmation sur un attribut ou une valeur sans écriture ; une liste de classe ou de nom d'encodage bytes n'est pas descendue au retrait des chemins exclus : l'écrivain la refuse, refus converti en cette erreur de programmation, que controler_rapport() cite sous objet_modifie (issue #128)
empreinte_sources()	interne	racine des sources	MD5 du relevé des MD5 de R/*.R (égal à LC_ALL=C md5sum -t R/*.R | md5sum) ; NA si la racine n'est pas celle de tablexp ou si un fichier est illisible
EMPREINTE_CODE	constante	—	empreinte du code, empreinte_sources(".") évaluée une fois par R CMD INSTALL depuis la racine des sources ; NA si elles sont introuvables
canoniser_plan()	interne	plan validé, champs, paramètres d'étape, ordres	plan canonique (ADR 0011, point 5.1) : champs dans l'ordre de CHAMPS_PLAN, transition en (depart, arrivee), periode en (debut, fin) à bornes Date stockées en double, délai en double, graine en integer, paramètres d'étape dans l'ordre de PARAMETRES_ETAPES, valeurs de segmentation, exclusions et estimateurs dans l'ordre des vecteurs de ordres (issue #63) ; idempotente ; erreur de programmation sur un stockage non prévu, un nom d'étape ou de paramètre en double, ou, dans un champ ensemble, une valeur hors du vecteur d'ordre, un doublon, un NA, un attribut ou un type autre que character ; appelée une fois par construire_table() après valider_plan()
MENTION_BIBLIOTHEQUE_C_NON_RELEVEE	constante	—	mention « non relevée » de fiche$plateforme$bibliotheque_c, quand la valeur lue est absente, NA, vide ou faite de blancs, ou n'est pas un texte (section 15.1)

Nom	Statut	Entrées	Sortie et rôle
bibliotheque_c()	interne	versions des bibliothèques (par défaut celles de R ; paramètre pour les tests)	élément iconv, tel quel (« unknown » compris), sinon la mention « non relevée » (constante ci-dessus) ; n'identifie la glibc que si la valeur commence par « glibc » (section 15.1)
assembler_fiche()	interne	rôles, plan, recette, étapes, avertissements, empreintes, provenance	fiche de construction : plan canonique, rôles dans l'ordre du catalogue, provenance dans l'ordre (fichier, empreinte, algorithme) ; champ hypotheses_brut si l'étape brut est exécutée ; champs réordonnés par ordonner_fiche() selon ORDRE_CHAMPS_FICHE
ordonner_fiche()	interne	fiche, ordre	fiche réordonnée selon l'ordre, réduit aux champs présents (hypotheses_brut absent sans étape brut) ; erreur de programmation sur un champ présent hors de l'ordre, jamais retiré en silence
ORDRE_CHAMPS_FICHE	constante	—	ordre des champs de la fiche en huit blocs (identification, données, plan, paramètres retenus, exclusions, hypothèses, aléa, avertissements) ; source unique de l'ordre du corps de la fiche écrite
informations_execution()	interne	instant de début	champ execution, hors comparaison
PREMIERE_LIGNE_PLAN	constante	—	"tablexp plan 1", première ligne du texte du plan (nature et version de la grammaire)
PREMIERE_LIGNE_FICHE	constante	—	"tablexp fiche 1", première ligne du texte de la fiche
TYPES_TEXTE	constante	—	types admis d'une ligne du texte : double, integer, logical, character, Date, NULL
PROFONDEUR_MAX_TEXTE	constante	—	profondeur maximale d'une ligne du texte (32), contrôlée à l'écriture et à la lecture
CONVENTIONS_TEXTE	constante	—	conventions d'écriture rappelées en lignes d'en-tête # <clé> : <valeur> du texte, ignorées à la lecture
BLOCS_LECTURE_FICHE	constante	—	partition des champs de la fiche en trois blocs (E3.7) : calculé, hypothèses, rejeu, champs de chaque bloc dans l'ordre de ORDRE_CHAMPS_FICHE ; rappelée par la ligne d'en-tête # lecture
joindre_chemin()	interne	chemin, segment	chemin d'un élément, segments séparés par /

Nom	Statut	Entrées	Sortie et rôle
segments_texte()	interne	noms, longueur	segment de chemin de chaque élément : son nom, ou [i] (rang) si le nom est absent, vide, NA, en double, contient /, une tabulation ou une fin de ligne, ou commence par @, # ou [
texte_objet()	interne	liste, première ligne, en-tête, forme	octets UTF-8 du texte : une ligne par feuille, dans l'ordre de l'objet ; valeurs par ecrire_valeurs() en forme "texte" (défaut : plan, fiche) ou "empreinte" (empreinte_resultat()) ; refus serialisation_impossible (facteur en forme "texte" ; en forme "empreinte", facteur de classe autre qu'exactement factor ou c("ordered", "factor") ou d'attribut autre que names, levels, class ; classe ou attribut non écrits, Date de stockage autre que double ou integer, vecteur atomique nommé de longueur 0, profondeur au-delà de PROFONDEUR_MAX_TEXTE)
valeurs_texte()	interne	type, textes d'une ligne, refus	valeurs relues sans évaluation ; refus texte_valeur_invalide, texte_double_sous_depassement (décimal non nul relu 0)
lire_texte()	interne	octets, première ligne, nature	objet relu sans évaluation ; refus nommés texte_..., numéro de la ligne fautive dans details$ligne
ecrire_plan()	exportée	plan	octets du texte du plan, écrit tel qu'il est reçu, sans validation ni canonisation
lire_plan()	exportée	octets	plan relu, **non validé** : à passer à valider_plan() ou à construire_table()
ecrire_fiche()	exportée	fiche	octets du texte de la fiche (x$fiche, refus serialisation_impossible pour l'objet construction entier ; chaine_codage_bytes pour un nom de premier niveau d'encodage bytes, et, par l'écrivain, pour un nom de cet encodage plus bas ; serialisation_impossible pour une classe de cet encodage ; issues #114 et #128) ; l'en-tête rappelle version_fiche, la représentation canonique, les blocs de lecture (# lecture) et la procédure de rejeu (# rejeu, TEXTE_REJEU_FICHE)

Nom	Statut	Entrées	Sortie et rôle
TEXTE_REJEU_FICHE	constante	—	ligne # rejeu de la fiche écrite (note de lecture d'E3.7 en une ligne), reprise par la section « Rejeu » du rapport figé ; texte du moteur marqué UTF-8
lire_fiche()	exportée	octets	fiche relue, dont le rejeu est construire_table(donnees, f$roles, f$plan, f$provenance), outillé par rejouer_fiche() ; ne contrôle pas version_fiche
SECTIONS_RAPPORT	constante	—	les quatorze sections du rapport figé, dans l'ordre : identifiant, titre, nature, étape, racines, renvois (tableau 29)
CHEMINS_STRUCTURE_SEULE	constante	—	tables dont le volume croît avec la base, citées par chemin, type et dimension, sans leurs valeurs
MENTIONS_RAPPORT	constante	—	textes fixes du rapport figé, aux noms ASCII, marqués UTF-8 (section 15.5)
STYLE_RAPPORT	constante	—	feuille de style en ligne de l'en-tête du document
echapper_html()	interne	textes	&, puis <, >, " remplacés par leurs entités ; marque UTF-8 conservée
texte_valeurs()	interne	valeurs, chemin	type et texte de l'écrivain unique en forme "texte", une valeur par ligne, marqué UTF-8
texte_entier()	interne	entier, chemin	dimension (nrow(), length()) écrite par l'écrivain
decrire_valeur()	interne	valeur	valeur décrite pour un refus : écriture et type d'un atomique de longueur 1 sans attribut, sinon type et classes ; NULL pour NULL
valeur_message()	interne	valeur	la chaîne elle-même si elle est unique et sans caractère de contrôle, « absente » si NULL, sinon decrire_valeur() : message sur une ligne
lire_racine()	interne	objet, chemin	élément de chemin donné, et indicateur de présence
controler_rapport()	interne	objet	les quatre contrôles avant l'écriture et leurs refus nommés (tableau 28) ; champs de la fiche lus par lire_champ(), si bien qu'une classe ou un nom d'encodage bytes donne objet_modifie au lieu d'une erreur R (issue #128)
table_colonnes()	interne	data.frame, chemin	table de colonnes HTML : légende, en-tête (nom, type), une ligne par ligne de l'objet

Nom	Statut	Entrées	Sortie et rôle
lignes_racines()	interne	objet, racines, insertions	tables de lignes et de colonnes des racines, dans l'ordre de l'objet ; structure seule pour CHEMINS_STRUCTURE_SEULE ; « absent de l'objet » ; valeur de classe d'encodage bytes passée à l'écrivain, qui la refuse (type_non_canonique ; issue #128)
paragraphe()	interne	textes	paragraphe HTML, texte échappé
element_liste()	interne	textes	élément de liste HTML, texte échappé
corps_section()	interne	objet, rang de la section	corps HTML d'une section de SECTIONS_RAPPORT ; erreur de programmation pour une section sans rendu
ecrire_rapport()	exportée	objet construction	octets UTF-8 du rapport figé HTML, après controler_rapport() ; aucun calcul, aucune date (section 15.5)

PARTIE XII

Module simulation

Simulateur : R/simulation.R

Le dépôt ne contient aucune donnée réelle : tous les jeux sont simulés sous une *loi vraie* connue et une graine (E1.5). Le simulateur est le modèle multi-états exécuté en avant (E2.19 ; ADR 0003, point 9) : il produit des lignes individuelles portant les rôles du catalogue, sur lesquelles le moteur est vérifié. Il reproduit le même modèle que le moteur, pas le même code : il n'appelle que `donnees` et `outils`, et aucun module du moteur ne l'appelle (section 1.3). Sa première version, dite *minimale* (issue #10), porte deux transitions concurrentes, vivant → décédé et en cours → racheté, une tête par contrat, troncature à gauche et censure par la période ; elle reste celle des modes `uniforme` et `age_exact`. Les modes multi-contrats `uniforme_multi` et `age_exact_multi` y ajoutent des têtes portant un nombre aléatoire de contrats, de loi déclarée (issue #92 ; section 16.2). Des *défauts paramétrés* peuvent être injectés dans le jeu propre (issue #93 ; section 16.3). Les autres formes de données (agrégats, stocks successifs) viendront avec une branche ultérieure (`docs/feuille-de-route.md`, § 2). Son point d'entrée est `simuler_portefeuille(loi, portefeuille, periode, graine, defauts = NULL)`, exporté (ADR 0007, point 3.2, et annotation du 25/10/2026, point (b)). Le contenu de fond vient de la spécification d'`actuary` (issue #11, partie A.2) et de son erratum du 17/10/2026, et, pour les têtes multi-contrats et les défauts paramétrés, de la spécification commune des issues #92 et #93 (`actuary`, 25/10/2026), dont le mainteneur a retenu les recommandations (décisions M131 à M136 ; ADR 0007, annotation du 25/10/2026).

16.1 Loi vraie et tables synthétiques

La loi vraie est une liste de deux lois, `deces` et `rachat` ; chacune associe une table de référence (classe `table_reference`, section 7.5) et un facteur par sexe, `c(H = , F = )`, réel fini ≥ 0. Les facteurs sont de *forme exacte* (`forme_exacte()`, section 2.3 ; issue #58) : un vecteur de type de base `double` ou `integer`, de longueur 2, dont le seul attribut admis est celui des noms, qui sont exactement `H` et `F`, dans un ordre quelconque, sans nom manquant ni doublé ; un vecteur sans noms, muni d'une classe, d'une dimension ou d'un autre attribut, de type logique, ou portant un autre nom, même non ASCII, est refusé (`loi_invalide`). Les noms sont comparés sans tri, si bien que le contrôle ne dépend pas de la locale et ne lève pas d'erreur R sur un nom non ASCII de codage indéterminé ou aux octets invalides ; un facteur de type `integer` et l'ordre `c(F = , H = )` donnent les mêmes lignes tirées. La loi vraie et chacune de ses deux lois sont des **listes nues**, dont le seul attribut admis est celui des noms, d'exactement autant d'éléments que de noms attendus (`deces` et `rachat` ; `table` et `facteurs`), dans un ordre quelconque : un `data.frame`, un objet de classe, une liste munie d'un autre attribut ou un élément de plus, même

nommé NA, sont refusés (loi_invalide ; liste_aux_noms(), section 2.3 ; issue #44). Pour un sexe s, la loi effective est

$$q_x^s = \min\bigl(f_s\, q_x,\ 1\bigr),$$

où q_x est lu dans la table du sexe si elle y figure, sinon dans la table ensemble (loi_effective()). Une table prospective (colonne annee) est refusée par le simulateur minimal. Seuls les sexes effectivement tirés (proportion d'hommes non nulle pour H, différente de 1 pour F) sont construits et contrôlés ; ils sont consignés dans sexes_controles.

Deux tables synthétiques, de statut synthetique et porteuses de leurs paramètres (attribut parametres), sont construites par le module :

— **Gompertz-Makeham** (table_gompertz_makeham()) : intensité $\mu(x) = A + B\,c^x$, avec $A \geq 0$, $B \geq 0$, $c > 1$ déclarés, sans valeur par défaut, sur des âges entiers déclarés ; la probabilité annuelle est l'intégrale exacte de μ sur $[x, x+1)$:

$$q_x = 1 - \exp\Bigl(-A - B\,c^x\,\frac{c-1}{\ln c}\Bigr)\ ;$$

la table est sans distinction de sexe (ensemble). Les tests emploient $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, âges 0 à 120 (*choix de conception* de la spécification, sans prétention de réalisme), qui donnent $q_{60} = 0{,}008153179142$ et $q_{100} = 0{,}303486068312$, et des facteurs $f_H = 1$, $f_F = 0{,}6$;

— **table constante** (table_constante()) : $q_x = q$ à tous les âges déclarés ; elle porte la loi de rachat ($q = 0{,}05$ dans les tests de portefeuille), ou une transition absente ($q = 0$). Une loi de rachat par ancienneté (axe secondaire) est hors du périmètre de la branche.

Hypothèse de répartition dans l'année

Le simulateur suppose une **force constante par âge entier**, $\mu_x = -\ln(1 - q_x)$, dans l'échelle des années d'âge (une année d'âge vaut 1, quelle que soit sa durée de 365 ou 366 jours ; section 8.1), et des **risques concurrents indépendants**. L'hypothèse est consignée dans le résultat (HYPOTHESE_REPARTITION). Elle est retenue pour deux raisons :

— sous cette hypothèse, $q_x = 1 - e^{-\mu_x}$ *exactement* ; le décompte des événements rapporté à l'exposition centrale est alors l'estimateur du maximum de vraisemblance de μ_x du modèle à force constante par âge entier, avec troncature à gauche et censure indépendante, pour une observation en temps continu (résultat exact), et la cible des estimateurs à venir est connue exactement. Sur des dates au jour (section 7.1), l'exposition est arrêtée au début du jour de toute sortie, d'où un écart relatif d'ordre $m_x/(2L_x)$, où m_x est la somme des forces de la transition et des sorties concurrentes, négligeable mais non nul (forme exacte : section 9.2) ;

— la force constante est stable par addition des forces : une force constante dans chaque décrément équivaut à une force constante dans la table à décréments multiples, alors que la répartition uniforme n'a pas cette propriété. Deux forces indépendantes s'additionnent, et la probabilité de décès en présence du rachat a une forme close :

$$q_x^{(d)} = \frac{\mu_x^d}{\mu_x^d + \mu_x^r}\Bigl(1 - e^{-(\mu_x^d + \mu_x^r)}\Bigr)\ ;$$

sous l'hypothèse de répartition uniforme dans chaque table à décrément simple, de taux q'^d et q'^r, la survie à double décrément $(1 - t\,q'^d)(1 - t\,q'^r)$ est quadratique en t dès que les deux taux sont strictement positifs, et la table à décréments multiples n'est plus uniforme : l'hypothèse ne serait plus la même pour les deux transitions (résultat exact, calcul direct).

Une probabilité $q_x = 1$ à un âge atteignable rend la force infinie : le simulateur la refuse (`qx_egal_a_un`).

16.2 Portefeuille simulé

Entrées

Le portefeuille est une liste dont tous les champs sont requis, sans valeur par défaut (`CHAMPS_PORTEFEUILLE`), selon quatre modes ; dans tous, `n` est le nombre de *têtes* :

— `uniforme` : `n` têtes d'un contrat chacune, `proportion_hommes`, `age_min`, `age_max`, `origine` (date au plus égale au début de la période) ; le jour d'entrée est uniforme discret sur les jours de [origine, fin], l'âge à l'entrée uniforme continu sur [age_min, age_max), et la naissance est l'entrée moins $\mathrm{round}(\text{âge} \times 365{,}25)$ jours ; l'âge exact est ensuite recalculé depuis les dates, selon la convention d'âge du moteur (section 7.1) ;

— `age_exact` : `n`, `proportion_hommes`, `age` (entier), `date_entree` ; toutes les têtes entrent à l'âge exact `age`, née le même jour et le même mois, `age` années avant l'entrée. Ce mode sert au constat de simulation (section 16.4). Une entrée le 29 février est refusée (`entree_29_fevrier`) : la naissance « `age` années avant » serait ambiguë ;

— `uniforme_multi` : les champs du mode `uniforme`, plus `loi_contrats` ; chaque tête porte K contrats, dont les jours d'entrée $e_1, \ldots, e_K$ sont tirés indépendamment, uniformes discrets sur les jours de [origine, fin] ; l'âge à la *première* entrée $e_{(1)} = \min_k e_k$ est uniforme continu sur [age_min, age_max), et la naissance est $e_{(1)}$ moins $\mathrm{round}(\text{âge} \times 365{,}25)$ jours ;

— `age_exact_multi` : les champs du mode `age_exact`, plus `loi_contrats` ; chaque tête porte K contrats, tous entrés le jour `date_entree`, à l'âge exact `age`.

Le *mode de base* d'un mode multi-contrats (`MODE_BASE`) : `uniforme` pour `uniforme_multi`, `age_exact` pour `age_exact_multi` ; ses champs communs, leurs contrôles et les bornes d'âge (ci-dessous) sont ceux du mode de base. `loi_contrats` donne les probabilités $p_1, \ldots, p_m$ du nombre K de contrats d'une tête, sur $\{1, \ldots, m\}$: loi discrète finie, déclarée, sans famille paramétrique ni valeur par défaut. Il est de forme exacte : type de base `double` ou `integer`, sans aucun attribut, de longueur $m \geq 1$, de valeurs finies et positives ou nulles, $p_m > 0$ (m est le plus grand nombre de contrats tirable) et $\left|\sum_k p_k - 1\right| \leq 10^{-12}$; sinon refus `portefeuille_invalide`, au message propre au champ. Le tirage par inversion (ci-dessous) emploie la loi $(p_1, \ldots, p_{m-1}, 1 - F_{m-1})$, $F_{m-1} = p_1 + \cdots + p_{m-1}$, qui s'écarte de la loi déclarée d'au plus 10^{-12} sur p_m (*approximation numérique*, bornée par le contrôle de la somme). K est tiré indépendamment de la mortalité et du rachat : le simulateur ne représente aucune antisélection, et une dépendance entre K et la mortalité est hors du périmètre. Le nombre de contrats tirés, au plus $n \times m$, doit être un entier R : si $n \times m$, calculé en `double`, dépasse `.Machine$integer.max`, le portefeuille est refusé (`portefeuille_invalide`) avant tout tirage. `loi_contrats` est requis dans un mode multi-contrats et refusé, comme champ inconnu, dans les modes `uniforme` et `age_exact`, dont rien ne change : leur attribut, leurs champs et leurs tirages sont ceux de la version minimale, et l'entrée de référence (section 4.5) est inchangée (ADR 0007, annotation du 25/10/2026, point (a) ; constat : l'empreinte des données des plans de référence, comparée par la batterie de reproductibilité, est la même, et les tests unitaires comparent les champs, l'ordre des tirages et les champs de l'attribut des modes existants à leurs littéraux). Avec `loi_contrats = 1`, un mode multi-contrats tire exactement les mêmes nombres que son mode de base et rend les mêmes lignes, hors attribut (constat des tests pour les deux paires de modes).
Le champ `mode` est une chaîne unique sans attribut, ni `NA` ni vide, au sens de `chaine_unique()` (une chaîne nommée ou munie d'une classe est refusée), prise parmi les quatre modes ; le portefeuille porte exactement les champs de son mode, sans champ manquant, inconnu ni en double, et ses champs sont lus par leur nom exact, sans appariement partiel (un champ `mode_x` ne tient pas lieu de `mode`) ; sinon refus

portefeuille_invalide (issue #60). Les noms sont comparés par ecart_noms(), règle unique des noms des arguments du simulateur, partagée avec les défauts (issue #113) : un nom manquant (NA) ou en double donne le message « Portefeuille (mode [mode]) : noms manquants ou en double. », et un champ absent ou inconnu « Portefeuille (mode [mode]) : champs attendus [champs entre guillemets] ; manquants : [noms entre guillemets, ou « aucun »] ; inconnus : [idem]. », mode et noms cités par la règle unique des refus (un crochet désigne une valeur insérée) ; les details sont mode, attendus et noms (noms reçus). Le portefeuille lui-même est de *forme exacte* (forme_exacte() ; issue #78) : une **liste nue**, dont le seul attribut admis est celui des noms ; un data.frame, une liste munie d'une classe ou d'un autre attribut sont refusés (portefeuille_invalide), et les noms des champs sont comparés sans tri. Ses champs numériques (n, proportion_hommes, age_min, age_max, age) sont de type de base double ou integer, de longueur 1, finis et **sans aucun attribut** (ni noms, ni dimension, ni classe) ; origine et date_entree sont des dates de la forme des bornes de la période ci-dessous (classe Date seule, sans noms ni autre attribut ; prédicat date_unique(), écrit une seule fois dans R/outils.R ; issue #113) ; sinon refus portefeuille_invalide, au message propre au champ. Un champ n de type integer plutôt que double, ou des champs dans un autre ordre, donnent les mêmes lignes tirées ; l'attribut simulation consigne le portefeuille tel que reçu, donc toujours une liste nue de cette forme. Pour garantir une naissance strictement antérieure à l'entrée, il faut age_min $\times$ 365,25 $\geq$ 1 (la fonction round() de R arrondit au pair : $\mathrm{round}(0{,}5) = 0$) dans les modes uniforme et uniforme_multi, et un âge entier au moins égal à 1 dans les modes age_exact et age_exact_multi ; ces conditions sont contrôlées (portefeuille_invalide). La période d'observation est une liste nue, de champs exactement debut et fin (même contrôle liste_aux_noms() que la loi vraie ; refus periode_invalide), de deux dates uniques, non manquantes, finies et entières, de la forme exacte des bornes de la période du plan (section 6.1) : stockage double ou integer, seul attribut la classe Date exacte, sans noms ni autre attribut ni classe ajoutée (issue #58) ; la censure a lieu à l'instant $B = \text{fin} + 1$ (le simulateur n'a pas de délai de déclaration).

Algorithme

1. Pour chaque transition et chaque tête, un **temps latent** est tiré par inversion du risque cumulé H, linéaire par morceaux sur l'âge (force constante par âge entier) : l'âge de l'événement est celui où H atteint $H(a_0) + E$, avec a_0 l'âge exact à l'entrée (pour le décès d'une tête multi-contrats, à sa première entrée ; pour le rachat d'un de ses contrats, à l'entrée de ce contrat) et $E \sim \mathrm{Exp}(1)$, cherché par findInterval() (temps_latent()).
2. La sortie est le **minimum** des deux temps latents et de la censure à B, comparés en temps continu ; une égalité, de probabilité nulle, est attribuée au décès.
3. L'instant de l'événement, dans l'année d'âge k à la fraction φ, est converti en jour : $\mathrm{anniv}(k) + \lfloor \varphi\, L_k \rfloor$, avec L_k la durée de l'année d'âge, borné à $[\text{entrée}, \mathrm{anniv}(k+1) - 1]$. Le jour reste dans l'année d'âge : **la probabilité d'un événement dans l'année k est conservée exactement** ; la discrétisation n'affecte que la répartition à l'intérieur de l'année, et la borne ne corrige que l'arrondi de la virgule flottante.

Têtes multi-contrats

Dans les modes uniforme_multi et age_exact_multi (issue #92 ; décisions M131 et M135), le modèle distingue la tête et ses contrats :

— **par tête** : le sexe, la naissance (ci-dessus) et le **décès**, temps latent tiré à partir de la première entrée $e_{(1)}$, à l'âge exact de la tête à cette date ;

— **par contrat** : l'entrée e_k et le **rachat**, temps latent tiré à partir de e_k, à l'âge de la tête à cette date ; décès et rachats sont des risques concurrents indépendants ;

— **contrat non souscrit** : soit t_d l'instant (continu) de décès latent de la tête ; un contrat n'est écrit que si $e_k < t_d$. Le premier contrat l'est toujours, t_d suivant strictement $e_{(1)}$ (le tirage exponentiel est strictement positif). Le code écrit la règle « $e_k \leq e_{(1)}$ ou $e_k < t_d$ » : la première clause est une garde contre un arrondi en virgule flottante qui rendrait t_d égal à $e_{(1)}$, sans effet en arithmétique exacte, où $e_k \leq e_{(1)}$ entraîne $e_k = e_{(1)} < t_d$. Un contrat non souscrit n'est pas écrit et il est compté dans l'attribut (`contrats_non_souscrits`, ci-dessous). Tirer le décès à partir de la *dernière* entrée, c'est-à-dire faire survivre la tête jusqu'à elle, créerait un *biais de temps immortel* : entre sa première et sa dernière entrée, la tête serait exposée sans pouvoir décéder, et la mortalité observée serait sous-estimée (variante écartée, décision M135 : le test B1 ci-dessous l'a détectée, avec $Z = -16{,}3$, sur le prototype de la spécification commune de l'issue #92 ; constat de simulation sur le prototype, que la batterie ne rejoue pas, la variante n'étant pas codée dans le package) ;

— **sortie** : avec $B = \text{fin} + 1$ et $t_{r,k}$ l'instant de rachat latent du contrat k, décès si $t_d \leq t_{r,k}$ et $t_d < B$, rachat si $t_{r,k} < t_d$ et $t_{r,k} < B$, en cours sinon ; à égalité, le décès l'emporte. Le décès est commun à la tête : tous ses contrats en cours à cet instant sortent par `deces`, à la même date. Une assertion vérifie que la date de décès n'est antérieure à l'entrée d'aucun contrat écrit : un écart serait une erreur de programmation, jamais un refus ;

— **bornes d'âge** : celles du mode de base restent valables, la naissance étant fixée à partir de la première entrée et les entrées suivantes restant dans [origine, fin] (ou égales à `date_entree`) ; l'assertion sur les âges parcourus porte sur chaque tête.

Ordre des tirages et graine

Les tirages ont lieu sous `sous_graine()`, avec la graine reçue en argument, sans défaut, sous `GENERATEUR` (section 3.3), dans un ordre fixé, qui fait partie du contrat de reproductibilité et est consigné (`ORDRE_TIRAGES`) : (1) sexe, `runif(n)` comparé à la proportion d'hommes ; (2) jour d'entrée, `sample.int()` avec remise ; (3) âge à l'entrée, `runif(n)` ; (4) décès, `rexp(n)` ; (5) rachat, `rexp(n)`. En mode âge exact, (2) et (3) ne sont pas tirés. Les modes multi-contrats ont leur propre ordre, consigné de même (`ORDRE_TIRAGES_MULTI`), sous un seul appel de `sous_graine()`, $N = \sum K$ désignant le nombre de contrats tirés : en `uniforme_multi`, (1) sexe, `runif(n)` ; (2) nombre K de contrats par tête, par inversion explicite, `1 + findInterval(runif(n), cumsum(loi_contrats)[-m])` ; (3) jour d'entrée de chaque contrat tiré, têtes puis contrats, `sample.int()` avec remise, N tirages ; (4) âge à la première entrée, `runif(n)` ; (5) décès par tête, `rexp(n)` ; (6) rachat de chaque contrat tiré, souscrit ou non, `rexp(N)` ; en `age_exact_multi`, (1), (2), puis décès `rexp(n)` et rachat `rexp(N)`. Quand `loi_contrats` n'a qu'une probabilité non nulle, K n'est pas tiré (il vaut m) : d'où l'identité des tirages avec le mode de base pour `loi_contrats = 1`. L'inversion explicite, plutôt que `sample()` avec probabilités, fixe dans le contrat l'ordre et le nombre des tirages, que l'implémentation de `sample()` déterminerait sinon. Les lignes sont rangées par tête, puis par date d'entrée, puis par rang de tirage (tri stable). Changer l'un de ces ordres change tout jeu simulé : c'est un changement de résultat.

Couverture de la table : aucune extrapolation

Le simulateur refuse (`table_trop_courte`) une loi qui ne couvre pas tous les âges atteignables, *avant tout tirage*, si bien que le refus ne dépend pas de la graine (`controler_portefeuille()`, `controler_couverture()`). Les âges atteignables sont bornés a priori par les seuls paramètres :

— **borne basse** : l'âge révolu à l'entrée est au moins

$$y_{\min} = \max\{y : 365\,y + \lceil y/4 \rceil \leq \text{round}(\text{age_min} \times 365{,}25)\}$$

(age_revolu_minimal()), parce que y années comptent au plus $\lceil y/4 \rceil$ jours de 29 février ; la borne est prudente, les années séculaires non bissextiles ne faisant que retirer des jours. Elle donne $y_{\min} = 20$ pour age_min $= 20$ comme pour 21 : la borne $\lfloor$age_min$\rfloor$ serait fausse, une naissance arrondie au jour pouvant donner l'âge révolu 20 à une tête entrée à 21 ans (erratum A.2, point 4). En mode âge exact, la borne basse est l'âge déclaré ;

— **borne haute** : age_max $+$ $(B -$ origine$)/365{,}25 + 1$ en mode uniforme, et age $+$ $(B -$ date_entree$)/365{,}25 + 1$ en mode âge exact. *Preuve.* Soit $\mathrm{span}(y)$ le nombre de jours de la naissance à l'anniversaire y. Pour $y \leq 130$ (âges des tables des tests), $\mathrm{span}(y) \geq 365{,}25\,y - 2{,}75$ et $\mathrm{span}(y) \leq 365{,}25\,y + 0{,}75$; une tête d'âge révolu y à B vérifie $\mathrm{span}(y) \leq B - n$, et, en mode uniforme, $B - n \leq (B -$ origine$) +$ age_max $\times\, 365{,}25 + 0{,}5$; donc $y \leq$ age_max $+ (B -$ origine$)/365{,}25 + 3{,}25/365{,}25$, strictement inférieur à la borne. En mode âge exact, $B - n = (B -$ date_entree$) + \mathrm{span}($age$)$ et le même raisonnement donne une marge de $3{,}5/365{,}25$. *Statut des deux encadrements de* span : ils sont établis par calcul exhaustif, toutes naissances du 1er janvier 1700 au 31 décembre 2150 et $1 \leq y \leq 130$ (minimum $-2{,}75$, maximum $+0{,}75$, remesurés sur R 4.3.3 avec les fonctions de dates du module) ; le calendrier grégorien étant périodique de 400 ans, cette plage contient un cycle complet de dates de naissance, et les encadrements valent pour toute naissance (résultat exact). Au-delà de 130 ans, l'encadrement bas mesuré jusqu'à 260 ans est $-3{,}75$ (atteint pour $y = 207$, naissance au 2096-03-01), l'encadrement haut restant $+0{,}75$; la même preuve donne alors une marge de $4{,}25/365{,}25$ en mode uniforme ($4{,}5/365{,}25$ en mode âge exact), inférieure à un an pour $y \leq 260$.

Après les tirages, une assertion interne vérifie que les âges parcourus restent dans ces bornes : un écart serait une erreur de programmation, jamais un refus dépendant de la graine. Avec le portefeuille des tests (age_max $= 75$, origine 2005-01-01, fin 2024-12-31), la borne haute vaut 96 (erratum A.2, point 1), couverte par la table de 0 à 120 ans.

Sortie

`simuler_portefeuille()` rend un `data.frame` de colonnes `id_contrat`, `id_tete` (égal à `id_contrat` dans les modes `uniforme` et `age_exact` ; de 1 à `n` dans les modes multi-contrats, où `id_contrat` numérote de 1 au nombre de lignes les contrats écrits, dans l'ordre des lignes), `sexe`, `date_naissance`, `date_entree`, `date_sortie` (manquante si le contrat est en cours à la fin) et `cause_sortie` (`deces`, `rachat` ou `en_cours`), qui portent les noms des rôles du catalogue. Toutes les lignes sont conservées, y compris celles sorties avant le début de la période, qui exercent la troncature à gauche ; seuls manquent les contrats non souscrits des modes multi-contrats. Les lignes sont cohérentes au sens exact, naissance $<$ entrée $\leq$ sortie $\leq$ fin, et passent les contrôles du module `donnees` sans exclusion, pour les deux transitions (constat des tests).

L'attribut `simulation` consigne : la loi vraie (tables, facteurs, loi effective par transition et par sexe avec q_x et μ_x, transitions, hypothèse de répartition, `sexes_controles`) ; le portefeuille et la période reçus ; la graine, `GENERATEUR` et l'ordre des tirages ; la version de R ; des décomptes (lignes, décès, rachats, en cours, sorties et entrées antérieures au début de la période). Dans les modes `uniforme` et `age_exact`, il n'a pas d'autre champ. Dans les modes multi-contrats, il a les mêmes champs, dans le même ordre, puis des ajouts *en fin seulement* : l'ordre des tirages est celui du mode ; `decomptes` garde ses six champs, `lignes` comptant les contrats écrits, suivis de `tetes` (n), `contrats_tires` (N), `contrats_non_souscrits` (N moins le nombre de lignes), `tetes_decedees_observees` (têtes qui ont au moins une ligne à décès) et `tetes_decedees_sans_contrat` (têtes dont le décès latent précède B sans qu'aucun de leurs contrats écrits ne soit en cours à cet instant, tous étant déjà rachetés : leur décès n'apparaît sur aucune ligne) ; un champ final `contrats` porte la loi déclarée (`loi`, en `double`) et le nombre de têtes par nombre de contrats tirés (`tires_par_tete`). Tous ces décomptes se recomptent sur les lignes et sur `tires_par_tete`, sauf `tetes_decedees_sans_contrat`, qui porte sur des décès absents des lignes : les tests n'en vérifient que

la positivité.

Refus du simulateur : loi_invalide, portefeuille_invalide, periode_invalide, entree_29_fevrier, table_trop_courte et qx_egal_a_un ; et, par sous_graine(), graine_invalide. Les modes multi-contrats n'ajoutent aucun code : loi_contrats et la garde de débordement sont refusés sous portefeuille_invalide. Les défauts paramétrés en ajoutent deux, defauts_invalides et defauts_ineligibles (section 16.3).

16.3 Défauts paramétrés

Un *défaut paramétré* est une altération déterministe d'une ligne ou d'une tête du *jeu propre*, le jeu que simuler_portefeuille() tire sous la loi vraie et la graine, avant toute injection (CONTEXT.md ; issue #93 ; décisions M132 à M134 et M136 ; ADR 0007, annotation du 25/10/2026, points 6 à 9). Il sert à vérifier les contrôles du module donnees : chaque défaut est consigné avec le motif d'exclusion ou le code de refus que ce module doit produire.

Argument defauts

La signature est simuler_portefeuille(loi, portefeuille, periode, graine, defauts = NULL). La valeur NULL signifie « aucun défaut injecté » : le jeu rendu est le jeu propre, sans attribut defauts, identique à celui de l'appel à quatre arguments. Ce n'est pas une graine par défaut : graine reste requise, sans valeur par défaut (section 3.3) ; la forme est celle de l'argument provenance = NULL de construire_table(), où NULL désigne l'absence de l'objet. Une fonction séparée, qui altérerait un jeu reçu, est écartée : rien ne garantirait que ce jeu soit propre ni tiré par le simulateur, et l'attribut ne pourrait plus attester la disjonction ni l'ordre des tirages (décision M132).

Sinon, defauts est une liste nue (seul attribut les noms) de champs exactement graine et declarations :

— graine : la graine des défauts, déclarée, sans défaut (M133), de la forme exigée par sous_graine() (nombre entier unique, fini, sans attribut, de valeur absolue au plus .Machine$integer.max ; même prédicat, forme_graine(), écrit une seule fois ; issue #113), et **différente de la graine du jeu**. sous_graine() réinitialise le générateur à chaque appel : sous la même graine, la sélection des défauts reprendrait les premiers nombres du jeu propre, ceux qui ont tiré le sexe : en mode uniforme, pour une première déclaration, exprimée en taux, dont le type et la cible admettent les trois causes de sortie (toutes les lignes sont alors éligibles, dans l'ordre du tirage), une sélection $u_i <$ taux avec un taux au plus égal à la proportion d'hommes ne retiendrait que des hommes ($u_i <$ taux entraîne $u_i <$ proportion_hommes) : une exclusion informative non déclarée (mesure consignée par l'ADR 0007, annotation du 25/10/2026, point (f) ; réponse d'actuary du 25/10/2026). Le refus porte sur l'égalité seule : deux graines voisines sont admises ;

— declarations : une liste sans aucun attribut, vide admise (le jeu est alors le jeu propre, avec un attribut defauts sans ligne), de déclarations traitées dans l'ordre reçu. Chacune est une liste nue de champs type, cible et exactement l'un de taux (réel de $[0, 1]$, tirage de Bernoulli) et nombre (entier positif ou nul, tirage sans remise), de type double ou integer sans attribut. type est un type du catalogue (tableau 31) ou une combinaison admise. cible, sous-ensemble non vide, sans doublon, de deces, rachat et en_cours (causes de sortie du jeu propre), est requise pour un type de ligne et interdite pour un type de tête.

Les noms des listes sont comparés sans tri (issue #100), par la règle unique ecart_noms(), partagée avec le portefeuille (issue #113). **Cible structurellement vide** (ADR 0007, annotation du 05/11/2026, seconde, (c) et (d) ; décisions du mainteneur M206, M207 et M208, du 05/11/2026, sur l'avis d'actuary ;

issues #112 et #135). Une déclaration dont aucune unité ne peut être éligible, et cela se sait *avant tout tirage*, par le seul portefeuille déjà contrôlé, la période et le catalogue, quelle que soit la graine, est refusée (`defauts_invalides`), en taux comme en nombre, zéro compris. Deux cas :

— une cible disjointe, par construction, des causes que le catalogue admet pour le type (pour une combinaison, pour tous ses types) : aucune ligne n'y serait éligible, quel que soit le jeu ;

— un type dont une condition d'éligibilité, lue dans la colonne `condition` du catalogue, est vide pour ce portefeuille : `conditions_vides()` calcule une fois l'ensemble de ces conditions, et la déclaration est refusée si une condition de son type (ou d'un type de sa combinaison) en fait partie ; un type ajouté au catalogue avec l'une de ces conditions hérite ainsi du refus. Les types de tête (`tete_naissance_incoherente`, `tete_sexe_incoherent`, `tete_observee_apres_deces`) exigent une tête d'au moins deux lignes : leurs conditions sont vides quand le portefeuille n'a qu'un contrat par tête, dans les modes `uniforme` et `age_exact`, et dans les modes `uniforme_multi` et `age_exact_multi` dont `loi_contrats` est de longueur 1. Sur la *loi déclarée*, sous la condition $p_m > 0$ imposée à `loi_contrats` (m étant sa longueur, section 16.2), le nombre de contrats vaut 1 pour toute tête si et seulement si $m = 1$ (*résultat exact*) : `loi_contrats = c(0, 1)`, qui donne deux contrats à chaque tête, n'est pas un cas de refus (issue #112). La *loi tirée*, $(p_1, \ldots, p_{m-1}, 1 - F_{m-1})$, s'écarte de la loi déclarée d'au plus 10^{-12} sur p_m (section 16.2) : une loi déclarée dont p_m est de cet ordre peut donner une loi tirée de p_m nul. Ainsi `loi_contrats = c(1, 1e-13)` passe le contrôle du portefeuille ($F_1 = 1$, d'où un p_2 tiré nul), donne un seul contrat à chaque tête quelle que soit la graine (résultat exact par lecture : `cumsum(c(1, 1e-13))[-2]` vaut 1 en double et `runif()` ne rend jamais 1 ; conséquences sur les défauts : *constat*, deux graines ; issue proposée au mainteneur), et n'est pas refusée avant tout tirage : une déclaration d'un type de tête y est rendue avec zéro éligible en taux, et refusée par `defauts_ineligibles`, après le tirage, en nombre positif. La condition `entree_avant_fin` du type `en_cours_avant_fin` (entrée antérieure à la fin de la période) est vide quand aucune entrée ne précède la fin : `date_entree` égale à la fin dans les modes `age_exact` et `age_exact_multi`, où toute entrée vaut `date_entree` ; `origine` égale à la fin dans les modes `uniforme` et `uniforme_multi`, où les entrées sont tirées entre `origine` et la fin. Le contrôle du portefeuille imposant déjà `date_entree` au plus égale à la fin, et `origine` au plus égale au début, lui-même au plus égal à la fin, ces égalités sont exactement les cas vides (issue #135).

Le message nomme le type et la raison, formée à partir des valeurs reçues et citée (« Défauts : déclaration [k] : type [type] : aucune unité éligible par construction ([raison]), quel que soit le tirage. », la raison étant « un seul contrat par tête : mode « [mode] » », suivie en mode multi-contrats de « , loi_contrats de longueur 1 », ou « aucune entrée avant la fin de la période : date_entree « [date] » égale à la fin « [fin] » », `origine` au lieu de `date_entree` dans les modes `uniforme` et `uniforme_multi`) ; les `details` sont `parametre` (« defauts$declarations ») et `declaration` (le rang). Raisons du refus plutôt que d'un jeu rendu sans unité retenue : c'est la règle déjà appliquée à la cible disjointe ; le refus ne dépend pas de la graine ; un jeu rendu avec zéro éligible laisserait passer un test vide, fausse assurance dans un dossier de certification ; et le simulateur n'a pas d'accumulateur d'avertissements, si bien qu'un signal changerait la structure de l'attribut `defauts`. Le contrôle, dans `controler_defauts()`, suit pour chaque déclaration ceux du champ `cible` et de la cible disjointe, et précède celui de `taux` et de `nombre` : une déclaration de taux ou de nombre mal formé sur une cible vide reçoit le refus de la cible vide. Un ensemble d'éligibles vide qui *dépend du jeu tiré* (cause absente du jeu propre, y compris une transition de loi nulle, vide connu de la loi, hors de la règle (*constat* ; issue proposée au mainteneur) ; aucune tête à deux lignes dans un mode multi-contrats à plusieurs probabilités, sauf l'exception ci-dessus d'une loi tirée de p_m nul ; aucune entrée tirée avant la fin quand `origine` précède la fin) n'est pas refusé à ce stade : en taux, il donne zéro unité retenue, tracé dans les décomptes ; en nombre, il relève du refus `defauts_ineligibles` ci-dessous.

Catalogue

Le catalogue (CATALOGUE_DEFAUTS) compte quinze types de ligne, dont quatorze correspondent chacun à un motif de ligne du module donnees et le quinzième, date_hors_domaine, au refus du même nom, et trois types de tête, chacun lié à un motif de tête. Les altérations sont déterministes ; sans exclusion déclarée, chaque motif attendu conduit au refus ligne_invalide (section 7.3). Les éligibles d'un type de ligne sont les lignes dont la cause de sortie (dans le jeu propre) est à la fois admise par le catalogue et dans la cible.

Table 31 : Catalogue des défauts paramétrés (CATALOGUE_DEFAUTS ; spécification d'actuary du 25/10/2026). Le motif attendu porte le nom du type, sauf pour date_hors_domaine (refus).

Type	Unité	Éligibles	Altération
contrat_duplique	ligne	toutes	copie de la ligne ajoutée en fin de jeu ; motif attendu sur l'original et sur la copie
id_contrat_manquant	ligne	toutes	id_contrat manquant
id_tete_manquant	ligne	toutes	id_tete manquant
naissance_manquante	ligne	toutes	date de naissance manquante
entree_manquante	ligne	toutes	date d'entrée manquante
sortie_manquante	ligne	décès, rachat	date de sortie manquante
cause_manquante	ligne	toutes	cause de sortie manquante
cause_hors_modalites	ligne	décès, rachat	cause de sortie en majuscules (toupper())
sexe_manquant	ligne	toutes	sexe manquant
sexe_hors_modalites	ligne	toutes	sexe en minuscule (tolower())
naissance_apres_entree	ligne	toutes	naissance = entrée +1 jour
sortie_avant_entree	ligne	décès, rachat	sortie = entrée −1 jour
en_cours_avant_fin	ligne	en cours, entrée antérieure à la fin	sortie = entrée
age_aberrant	ligne	toutes	naissance au 1er janvier de l'année d'entrée moins 150
date_hors_domaine	ligne	toutes	naissance au 31 décembre de l'an 0 ; refus date_hors_domaine, aucun motif
tete_naissance_incoherente	tête	au moins deux lignes	naissance de la dernière ligne de la tête moins un jour
tete_sexe_incoherent	tête	au moins deux lignes	sexe de la dernière ligne de la tête inversé
tete_observee_apres_deces	tête	au moins deux lignes, dont une à décès	la dernière ligne de la tête autre que sa première ligne à décès devient en cours, sortie manquante

Les éligibilités de sortie_manquante et de sortie_avant_entree excluent les lignes en cours, qui n'ont pas de date de sortie. Celle de cause_hors_modalites les exclut aussi, pour que le défaut ne lève pas un autre motif : sur une ligne en cours, il lèverait aussi sortie_manquante (interférence mesurée sur le prototype de la spécification). Une **combinaison** de types sur une même ligne n'est admise que si elle figure dans la liste fermée COMBINAISONS_DEFAUTS, qui ne contient que cause_manquante et sexe_manquant (issue #57) ; chaque ligne retenue reçoit toutes ses altérations, et les types sont consignés dans l'ordre du catalogue, quel que soit l'ordre de saisie. Toute combinaison nouvelle demande l'avis d'actuary. Trois motifs du module donnees sont **hors du périmètre**, faute de colonne dans le jeu simulé : annee_naissance_incoherente, segment_manquant et tete_segment_incoherent (M136) ; un test de clôture vérifie que les motifs attendus du catalogue, réunis à ces trois, sont exactement les motifs du module donnees, si bien qu'un motif ajouté sans décision sur le simulateur fait échouer la batterie.

Non-circularité. Les colonnes motif_attendu et refus_attendu de l'attribut sont *lues dans le catalogue*, écrit en dur d'après la spécification ; le simulateur n'appelle aucun contrôle du module donnees pour les établir. Ce n'est pas le graphe des modules, qui admettrait l'appel (simulation peut appeler donnees), mais cette règle qui l'interdit : une attente calculée par le code qu'elle sert à vérifier ne vérifierait rien.

Ordre des tirages, disjonction et graine

Les défauts s'injectent **après le tirage complet du jeu propre**, qui n'en est pas changé : ses lignes non altérées et son attribut `simulation` sont identiques dans le jeu altéré. Un seul appel de `sous_graine()`, sous la graine des défauts, traite les déclarations dans l'ordre reçu (`ORDRE_TIRAGES_DEFAUTS`) : pour chacune, les éligibles sont les unités (lignes du jeu propre, ou têtes) **non encore altérées**, en ordre croissant ; avec `taux`, une unité est retenue si `runif(nombre d'éligibles) < taux` ; avec `nombre`, les unités retenues sont `sort(eligibles[sample.int(nombre d'eligibles, nombre)])`. **Disjonction** : une ligne est altérée au plus une fois ; une tête retenue rend toutes ses lignes inéligibles ensuite ; une tête n'est éligible à un défaut de tête que si aucune de ses lignes n'est déjà altérée. Les copies de `contrat_duplique` prennent les numéros de ligne $N+1, N+2, \ldots$, N étant le nombre de lignes du jeu propre, dans l'ordre des déclarations puis des lignes, avec les valeurs de l'original.

Attribut `defauts`

Le jeu altéré porte un attribut `defauts`, **séparé** de l'attribut `simulation`, qui reste celui du jeu propre (M134) ; il est absent si `defauts = NULL`. Ses champs, dans cet ordre : `parametre` (l'argument reçu, tel quel), `graine`, `generateur`, `ordre_tirages` (`ORDRE_TIRAGES_DEFAUTS`), `validite`, `lignes` et `decomptes`.

— `lignes` : un `data.frame` d'une ligne par couple (ligne, motif), de colonnes `ligne` (numéro dans le jeu altéré), `declaration`, `defaut`, `unite` (le défaut porte sur une `ligne` ou une `tete`), `role` (ce qu'est la ligne pour l'altération : `alteree`, ligne modifiée ; `copie`, doublon ajouté ; `original`, ligne dupliquée, non modifiée ; `tete`, autre ligne d'une tête en défaut, non modifiée), `motif_attendu`, `refus_attendu` et `premier_deces` (pour `tete_observee_apres_deces`, la date du premier décès de la tête). Pour un défaut de tête, toutes les lignes de la tête sont déclarées, le module `donnees` levant le motif sur chacune ;

— `decomptes` : lignes du jeu propre, copies, lignes du jeu altéré ; `lignes_en_defaut`, lignes qui portent un motif ou un refus attendu, originaux et lignes `tete` compris ; `lignes_alterees`, lignes modifiées ou ajoutées (rôles `alteree` et `copie`) ; par déclaration, le nombre d'éligibles et d'unités retenues ;

— `validite` : les **conditions de validité** des attentes, consignées avec leur texte : les sept rôles déclarés, chacun sur la colonne du même nom ; un plan dont la période est celle de la simulation ; un délai de déclaration nul. Pour `tete_observee_apres_deces`, le motif n'est attendu que si le premier décès de la tête précède fin $-$ délai $+ 1$, ce qui est toujours le cas sous un délai nul. `date_hors_domaine` mène au refus du même nom quelles que soient les exclusions déclarées. Si aucune ligne n'a été retenue pour `date_hors_domaine` : avec toutes les exclusions des motifs attendus déclarées, les lignes exclues par le module `donnees` sont celles de l'attribut, motif par motif ; sans exclusion, le module refuse (`ligne_invalide`).

L'attribut survit à l'exclusion dans la table des lignes conservées de l'étape `donnees` ; `construire_table()` et `ecrire_rapport()` acceptent le jeu altéré. Il n'entre ni dans l'empreinte des données, qui ne lit que les colonnes à rôle, ni dans l'empreinte du résultat, la table des lignes conservées en étant exclue (section 15.3) : à colonnes égales, un jeu avec ou sans cet attribut a les mêmes empreintes (constat, R 4.3.3).

Refus

— `defauts_invalides` (module `simulation`) : forme ou valeur de `defauts` ou d'une déclaration, type inconnu, combinaison non admise, `cible` manquante, interdite, mal formée ou disjointe par construction, type dont une condition est structurellement vide pour le portefeuille (ci-dessus ; issue #112), `taux` et `nombre` tous deux présents ou tous deux absents, graine des défauts mal formée ou égale à celle

du jeu. Il est levé **avant tout tirage**, après les contrôles de la loi, du portefeuille et de la couverture, et avant le contrôle de la graine du jeu par `sous_graine()` : si la graine du jeu et les défauts sont tous deux invalides, le refus est `defauts_invalides` ; il ne dépend d'aucun tirage.

— `defauts_ineligibles` (module `simulation`) : un `nombre` dépasse le nombre d'unités éligibles. Il est levé **après le tirage du jeu propre**, dont il dépend : à la différence du précédent, il dépend de la graine du jeu. Il ne couvre plus que les vides qui dépendent du tirage, à deux exceptions près, vides connus avant tout tirage mais hors de la règle de la cible structurellement vide : une transition de loi nulle, et un mode multi-contrats dont la loi tirée de `loi_contrats` a un p_m nul, comme `c(1, 1e-13)` (ci-dessus ; *constat* ; issue proposée au mainteneur). Un type de tête dans un portefeuille à un seul contrat par tête, et `en_cours_avant_fin` quand aucune entrée ne précède la fin, auparavant refusés ici quand `nombre` était positif, et rendus avec zéro unité retenue quand il était nul ou exprimé en taux, sont désormais refusés par `defauts_invalides`, avant tout tirage, même sous une graine du jeu invalide (issues #112 et #135).

L'état du générateur de l'appelant est restauré dans tous les cas, comme pour tout tirage sous `sous_graine()`.

16.4 Constats de simulation

Retrouver la loi vraie est une batterie de tests, chacun un *constat de simulation* qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (E2.19, E2.20). Le constat du simulateur minimal porte sur cinq cohortes de $n = 10^6$ têtes, entrées le 2016-03-15 à l'âge exact x (mode `age_exact`), période du 2016-03-15 au 2017-12-31 : l'année d'âge x est observée entière, sans censure par la période. On compte les événements D datés avant l'anniversaire $x + 1$. Sous le modèle simulé, D suit exactement la loi binomiale $\mathcal{B}(n, p)$, p étant la probabilité de l'événement dans l'année.

— **Région d'acceptation** : bilatérale exacte, $[\text{qbinom}(\alpha_k/2, n, p), \text{qbinom}(1 - \alpha_k/2, n, p)]$.

— **Correction multiple** : partage égal du niveau, justifié par l'inégalité de l'union, $\alpha = 10^{-3}$ global, $K = 5$, $\alpha_k = 2 \cdot 10^{-4}$.

— **Fausse alarme exacte** de chaque test : $FA_k = P(D < \text{lo}) + P(D > \text{hi})$, calculée et écrite par le test.

— **Fausse alarme familiale** : elle est bornée par $\sum_k FA_k = 9{,}857 \cdot 10^{-4} \leq 10^{-3}$ (inégalité de l'union, valable *sans hypothèse d'indépendance* ; c'est le critère vérifié). Sous indépendance des cinq cohortes, tirées sous des graines distinctes (20261016 à 20261020), elle vaut $1 - \prod_k (1 - FA_k) = 9{,}853 \cdot 10^{-4}$; cette indépendance est l'hypothèse usuelle sur le générateur, non démontrée, et la valeur n'est donnée qu'à titre d'information.

Table 32 : Constat de simulation du simulateur minimal : probabilité exacte p sous le modèle, région d'acceptation et fausse alarme exacte (`tests/unitaires/test_simulation.R`).

Cohorte	p	Région	FA_k
1. décès, $x = 60$, H, sans rachat	0,008153179142	[7 821 ; 8 490]	$1{,}951 \cdot 10^{-4}$
2. décès, $x = 80$, F (facteur 0,6), sans rachat	0,03150085056	[30 853 ; 32 152]	$1{,}981 \cdot 10^{-4}$
3. décès, $x = 100$, F, sans rachat	0,182091641	[180 658 ; 183 528]	$1{,}995 \cdot 10^{-4}$
4. rachat, $x = 40$, $q^r = 0{,}05$, sans décès	0,05	[49 191 ; 50 812]	$1{,}983 \cdot 10^{-4}$
5. décès, $x = 60$, H, rachat concurrent $q^r = 0{,}30$	0,006859320306	[6 554 ; 7 168]	$1{,}947 \cdot 10^{-4}$

La probabilité du test 5 est la forme close $q^{(d)}$ ci-dessus ; le rachat concurrent y est porté à 0,30 pour que le test soit sensible à son oubli (choix de la spécification). La graine étant fixe, chaque test est déterministe :

sa fausse alarme est la probabilité, sur les graines, qu'une implémentation juste échoue. Les valeurs observées de D sont écrites par la batterie à chaque exécution ; elles ne sont pas reproduites ici (E3.4).

Statuts et sources d'erreur (E2.20).

— *Résultats exacts* : la formule de q_x de Gompertz-Makeham (intégrale de μ), la forme close $q^{(d)}$, la loi binomiale de D sous le modèle, les régions d'acceptation et les FA_k (quantiles et probabilités de la loi binomiale), l'inégalité de l'union, la fausse alarme familiale $1 - \prod_k (1 - FA_k)$, sous l'hypothèse d'indépendance des cohortes (hypothèse sur le générateur, non démontrée), la conservation de la probabilité annuelle par la conversion en jours, les bornes de couverture.

— *Résultat asymptotique* : aucun dans ce constat.

— *Approximations numériques* : l'évaluation en virgule flottante de q_x, de μ_x et des temps latents, des quantiles binomiaux et des FA_k ; le critère $\sum_k FA_k \leq 10^{-3}$ est vérifié sur les valeurs évaluées.

— *Constat de simulation* : le passage des cinq tests.

— *Erreur Monte-Carlo* : la dispersion binomiale de D autour de np, due au nombre fini de têtes simulées ; c'est elle que mesure la largeur de la région. *Erreur d'estimation* : elle n'intervient pas ici, D étant comparé à la probabilité exacte du modèle, sans taux estimé sur une exposition ; pour les taux bruts, elle est écrite par les constats de la section 9.9.

Têtes multi-contrats : familles A et B

Les modes multi-contrats (issue #92) sont vérifiés par deux familles de constats de simulation, qui se distinguent par le statut de leur fausse alarme : *exacte* pour la famille A, *nominale asymptotique* pour la famille B (décision M131 ; ADR 0007, annotation du 25/10/2026, point (d)). Les attentes viennent de formules fermées, de décomptes faits directement sur les lignes et de l'attribut ; aucune n'est établie par une fonction du module `donnees` (l'âge révolu est recalculé dans le test), dont les sorties, par `construire_table()`, sont ce qui est vérifié (non-circularité).

Famille A (fausse alarme exacte), mode `age_exact_multi`. $n = 3 \cdot 10^5$ têtes, toutes des hommes, d'âge exact $x = 60$ à l'entrée, le 2016-03-15, période du 2016-03-15 au 2017-03-14 : l'année d'âge 60 est observée entière ($L = 365$ jours), sans censure avant $B = \text{anniv}(61)$; décès de la table de Gompertz-Makeham des tests, rachat constant $q^r = 0{,}30$, loi_contrats $= (0{,}5\,;0{,}3\,;0{,}2)$, graine 20261025. Toutes les entrées ayant lieu le même jour, aucun contrat n'est non souscrit. Avec $\mu = -\ln(1 - q_{60}) = 0{,}00818659807815$ et $\rho = -\ln 0{,}7 = 0{,}356674943939$ (forces sur l'année d'âge), soit N_c le nombre de têtes qui décèdent dans l'année avec exactement c contrats en cours. Les têtes étant indépendantes (hypothèse sur le générateur, conditions de validité ci-dessous), N_c suit *exactement* la loi binomiale $\mathcal{B}(n, \pi_c)$, avec $\pi_c = \sum_K p_K \, \pi_c(K)$ et

$$\begin{aligned}\pi_c(K) &= \int_0^1 \mu e^{-\mu t} \binom{K}{c} e^{-c\rho t} \left(1 - e^{-\rho t}\right)^{K-c} \mathrm{d}t \\ &= \binom{K}{c} \sum_{j=0}^{K-c} \binom{K-c}{j} (-1)^j \frac{\mu}{\mu + (c+j)\rho} \left(1 - e^{-(\mu + (c+j)\rho)}\right)\end{aligned}$$

(développement binomial de $(1 - e^{-\rho t})^{K-c}$; résultat exact, recoupé par `stats::integrate()` à 10^{-12} relatif) ; le nombre de têtes à k lignes suit $\mathcal{B}(n, p_k)$, et le nombre de décès à 60 ans compté par le moteur en unité tête (estimateur `poisson`) est $\sum_{c \geq 1} N_c$, de loi $\mathcal{B}\left(n, \sum_{c \geq 1} \pi_c\right)$. Sept tests binomiaux bilatéraux exacts, de région $[\text{qbinom}(a_k/2, n, p), \text{qbinom}(1 - a_k/2, n, p)]$, avec un partage égal du niveau $\alpha = 10^{-3}$ (inégalité de l'union, $a_k = \alpha/7 \approx 1{,}429 \cdot 10^{-4}$) ; fausse alarme familiale bornée par $\sum_k FA_k = 9{,}566 \cdot 10^{-4} \leq \alpha$, sans hypothèse d'indépendance.

— **Hypothèse nulle** : les lignes sont tirées sous le modèle déclaré (décès de la table de Gompertz-

Makeham des tests, rachat constant, loi_contrats, décès et rachats en risques concurrents indépendants) et le moteur compte exactement les décès en unité tête.

— **Statistique** : les sept décomptes du tableau 33, faits sur les lignes (A1 à A6) ou lus dans la sortie du moteur (A7).

— **Loi de référence, région et fausse alarme** : ci-dessus et tableau 33 ; fausse alarme exacte par test, bornée exactement pour la famille.

— **Conditions de validité** : l'indépendance des têtes, qui rend les lois binomiales exactes, est celle de tirages successifs d'un même flux du générateur : hypothèse sur le générateur, non démontrée ; la loi tirée de K coïncide avec la loi déclarée à 10^{-12} près (approximation numérique, encadré ci-dessous) ; aucune condition d'effectif, les lois étant exactes pour tout n.

Table 33 : Famille A : probabilité exacte sous le modèle, région d'acceptation et fausse alarme exacte (tests/unitaires/test_simulation.R).

Test	p	Région	FA_k
A1. têtes à 1 ligne	0,5	[148 958 ; 151 042]	$1{,}408 \cdot 10^{-4}$
A2. têtes à 2 lignes	0,3	[89 046 ; 90 955]	$1{,}419 \cdot 10^{-4}$
A3. têtes à 3 lignes	0,2	[59 168 ; 60 835]	$1{,}408 \cdot 10^{-4}$
A4. N_1	0,00416937354957	[1 119 ; 1 387]	$1{,}380 \cdot 10^{-4}$
A5. N_2	0,00224283291693	[577 ; 774]	$1{,}329 \cdot 10^{-4}$
A6. N_3	0,00100193504543	[237 ; 369]	$1{,}231 \cdot 10^{-4}$
A7. décès à 60 ans, moteur, unité tête	0,00741414151193	[2 048 ; 2 405]	$1{,}390 \cdot 10^{-4}$

Les *identités exactes* de la famille A, déterministes, sans fausse alarme : décès en unité tête $= \sum_c N_c$ et en unité contrat $= \sum_c c\,N_c$ (moteur) ; exposition en jours, en unité tête, $= \sum_{\text{têtes}} (\max_k f_k - e)$ et, en unité contrat, $= \sum (f_k - e)$, f_k étant la fin de la fenêtre du contrat k et e l'entrée commune ; chevauchements et contiguïtés = lignes − têtes, aucun intervalle disjoint, aucun des vingt motifs. Les rapports entre unité contrat et unité tête ont, sous le modèle, les valeurs exactes $E[D_c]/E[D_t] = 1{,}572784$, $E[E_c]/E[E_t] = 1{,}572448$, d'où un rapport des taux attendus de 1,000214, et $\varphi = E[C^2]/E[C] = 1{,}900216$, C étant le nombre de contrats en cours au décès d'une tête décédée. L'écart du rapport des taux à 1 vient seulement du décompte de l'exposition en jours entiers (l'exposition s'arrête au début du jour de sortie, section 16.1) : en temps continu, le taux attendu vaut μ dans les deux unités et le rapport vaut exactement 1. En unité contrat, le taux est donc sans biais notable ; mais un même décès y est compté C fois, une fois par contrat en cours, de sorte que les décès comptés ne sont pas des événements indépendants : en temps continu, l'écart entre les décès comptés et les décès attendus sur l'exposition (processus de comptage compensé) a une variance égale à φ fois le nombre attendu de décès comptés, alors que l'intervalle nominal la suppose égale à ce nombre (loi de Poisson) ; il sous-estime donc l'erreur d'estimation du taux en unité contrat et il est trop étroit d'un facteur $\sqrt{\varphi} \approx 1{,}38$ (résultat asymptotique, grande exposition). Les quatre valeurs sont vérifiées par le test à $5 \cdot 10^{-7}$ près ; les valeurs observées sont imprimées sans verdict.

Famille B (fausse alarme nominale asymptotique), mode uniforme_multi. 10^5 têtes, loi vraie, portefeuille et période de l'entrée de référence (section 4.5), graine 20261016, loi_contrats $= (0{,}6\,;0{,}3\,;0{,}1)$; moteur sans délai de déclaration, segmenté par sexe, estimateur poisson. Deux tests : B1, décès en unité tête ; B2, rachat en unité contrat.

— **Hypothèse nulle** : les lignes sont tirées sous la loi vraie et le moteur compte exactement les événements et l'exposition.

— **Statistique** : $Z = \left(\sum_x D_x - \sum_x \mu_x E_x^c\right) \Big/ \sqrt{\sum_x \mu_x E_x^c}$, sommes sur les cellules (sexe, âge) du moteur, μ_x force vraie ($-\ln(1 - f_s q_x)$), E_x^c exposition centrale en années d'âge.

— **Loi de référence et région** : sous l'hypothèse nulle, le nombre d'événements moins l'intégrale de la force sur l'exposition est une martingale du processus de comptage, et Z est asymptotiquement normal centré réduit (P. K. Andersen, Ø. Borgan, R. D. Gill, N. Keiding, *Statistical Models Based on Counting Processes*, Springer, 1993, sections II.5, « Limit Theory », et V.1, « One-Sample Tests » : intitulés lus dans la table des matières, contenu non vérifié) ; région $|Z| \leq 3{,}4808$, quantile d'ordre $1 - 10^{-3}/4$ de la loi normale à 10^{-4} près.

— **Fausse alarme** : nominale, $5 \cdot 10^{-4}$ par test et 10^{-3} pour les deux (inégalité de l'union) ; *non exacte* (statut asymptotique, accepté comme tel par le mainteneur, M131).

— **Conditions de validité** : des processus de comptage à sauts unitaires, ce qui fixe le choix des unités : le décès est testé en unité tête (B1), car en unité contrat un décès compte une fois par contrat en cours et le processus saute de C ; le rachat est testé en unité contrat (B2), chaque contrat ayant son propre instant de rachat latent, et le rachat d'un contrat étant un événement unique ; une censure (rachat concurrent, fin de période) et une troncature (entrée en observation) indépendantes du risque testé, ce que donne le modèle par construction (risques concurrents indépendants, temps latents tirés à partir de l'entrée du contrat (rachat) ou de la première entrée de la tête (décès), la force ne dépendant que de l'âge atteint) ; des têtes indépendantes (hypothèse sur le générateur, non démontrée) ; des nombres d'événements attendus grands, ce que donnent 10^5 têtes ; l'arrêt de l'exposition au début du jour de sortie (section 16.1) décalerait la moyenne de Z d'environ $0{,}02$ écart-type selon la spécification : *hypothèse non mesurée*. Borne de robustesse, calcul exact sur la loi normale : un décalage δ de la moyenne de Z porte la fausse alarme de chaque test à $\Phi(-3{,}4808 - \delta) + \Phi(-3{,}4808 + \delta)$, soit $4{,}999 \cdot 10^{-4}$ pour $\delta = 0$, $5{,}012 \cdot 10^{-4}$ pour $\delta = 0{,}02$ et $5{,}326 \cdot 10^{-4}$ pour $\delta = 0{,}1$.

Identités exactes de la famille B : aucun motif ni aucune exclusion dans les deux transitions ; décès en unité tête = têtes distinctes ayant un décès daté dans la période ; rachats en unité contrat = lignes à rachat datées dans la période ; exposition en jours, en unité tête, = réunion des fenêtres de la tête et, en unité contrat, = somme des fenêtres des lignes ; décomptes de l'attribut = décomptes refaits sur les lignes. Valeurs informatives, imprimées sans verdict : X^2 par degré de liberté sur les cellules, $\hat{\varphi}$, rapports des décès et des expositions entre unité contrat et unité tête.

Statuts et sources d'erreur des familles A et B (E2.20).

— *Résultats exacts* : les lois binomiales de la famille A (sous l'hypothèse d'indépendance des têtes, hypothèse sur le générateur, non démontrée), la forme close de π_c, les régions et les FA_k, l'inégalité de l'union, les valeurs des rapports entre unité contrat et unité tête, les identités des deux familles, la borne de robustesse (calcul sur la loi normale).

— *Résultats asymptotiques* : la loi normale de Z (famille B), donc sa fausse alarme nominale ; le facteur $\sqrt{\varphi}$ d'élargissement de l'intervalle en unité contrat (famille A).

— *Approximations numériques* : l'écart d'au plus 10^{-12} entre la loi déclarée et la loi tirée de K ; l'évaluation en virgule flottante des probabilités, quantiles et FA_k.

— *Hypothèse non mesurée* : le décalage de la moyenne de Z dû à l'arrondi au jour.

— *Constat de simulation* : le passage des neuf tests.

— *Erreur Monte-Carlo* : la dispersion due au nombre fini de têtes simulées, que mesurent les régions ; *erreur d'estimation* : elle n'intervient pas, les décomptes étant comparés aux probabilités exactes (famille A) ou à la force vraie (famille B), sans taux estimé ; le facteur $\sqrt{\varphi}$ porte sur l'erreur d'estimation d'un taux en unité contrat, hors des tests.

Défauts paramétrés : jeu à exclusion informative

Un jeu à défauts (section 16.3) est vérifié par des identités exactes (lignes non altérées et attribut `simulation` identiques au jeu propre ; lignes modifiées, établies par comparaison au jeu propre, égales aux lignes de rôle `alteree` ; tirages rejoués sous la graine des défauts ; lignes exclues par le module

donnees égales à l'attribut, motif par motif) et par un constat de simulation sur un jeu à *exclusion informative*, préparé pour l'issue #69 : mode uniforme, 10^5 contrats, loi vraie et portefeuille de l'entrée de référence, graine 20261016, période du 2015-01-01 au 2019-12-31 ; défauts sous la graine 20261025, sexe_manquant sur les lignes à décès au taux 0,5 et sur les lignes à rachat ou en cours au taux 0,02. Exclure le motif sexe_manquant retire alors la moitié des décès mais une faible part de l'exposition : le taux brut est biaisé vers le bas, ce qu'un contrôle de l'exclusion doit révéler. Niveau $\alpha = 10^{-3}$, deux tests, $a = \alpha/2$ chacun.

— **Décès exclus** (fausse alarme *exacte*). Soit D_w le nombre de lignes à décès datées dans la période du jeu propre et K_D le nombre de celles dont le sexe est retiré. Conditionnellement au jeu propre, chaque ligne à décès étant retenue par un tirage uniforme au taux 0,5, ces tirages étant indépendants entre eux et du jeu propre, K_D suit *exactement* $\mathcal{B}(D_w, 0{,}5)$; région bilatérale exacte $[\text{qbinom}(a/2, D_w, 0{,}5), \text{qbinom}(1-a/2, D_w, 0{,}5)]$, fausse alarme exacte calculée et écrite par le test (elle dépend de D_w, qui dépend de la graine du jeu).

— **Exposition exclue** (fausse alarme *bornée exactement*). Soit e_ℓ le nombre de jours d'exposition dans la période de la ligne ℓ du jeu propre, p_ℓ son taux de sélection (0,5 ou 0,02) et $E_e = \sum_\ell B_\ell\, e_\ell$ l'exposition exclue, les B_ℓ étant des variables de Bernoulli indépendantes de paramètre p_ℓ, conditionnellement au jeu propre (même hypothèse d'indépendance que ci-dessus). Les termes étant bornés par 0 et e_ℓ, l'inégalité de Hoeffding donne $P\big(|E_e - \sum_\ell p_\ell e_\ell| \geq s\big) \leq 2\exp\big(-2s^2/\sum_\ell e_\ell^2\big) = a$ pour $s = \sqrt{\sum_\ell e_\ell^2 \ln(2/a)/2}$: la fausse alarme est au plus a, sans approximation (W. Hoeffding, *Probability inequalities for sums of bounded random variables*, *Journal of the American Statistical Association*, 58, 1963, p. 13–30 ; numéro du théorème non vérifié sur l'original). Ce test est un **garde-fou de faible puissance** (avis d'actuary du 25/10/2026) : la borne, valable pour toute loi bornée, est large ; la vérification forte de l'exposition exclue est portée par les identités et le rejeu des tirages.

— **Identités exactes** entre le jeu propre et le jeu altéré, construits par le moteur, l'exclusion sexe_manquant étant déclarée pour le second : écart des décès $= K_D$; écart des jours d'exposition $= E_e$; lignes exclues = lignes altérées. Rejeu : les lignes retenues sont celles que redonnent, sous la graine des défauts, runif() sur les lignes à décès puis sur les autres.

— **Hypothèse nulle** des deux tests : conditionnellement au jeu propre, chaque ligne éligible est altérée par un tirage uniforme indépendant, au taux déclaré pour sa cause de sortie (l'exclusion exacte des lignes altérées par le moteur relève des identités exactes ci-dessus, non de ces deux tests).

— **Conditions de validité** : l'indépendance des tirages des défauts entre eux et à l'égard du jeu propre, qui rend exacte la loi de K_D et légitime la borne de Hoeffding, est une hypothèse sur le générateur, non démontrée, pour deux flux tirés sous deux graines distinctes (20261016 pour le jeu, 20261025 pour les défauts) ; elle est fausse à coup sûr pour deux graines égales, les défauts reprenant alors les premiers nombres du jeu propre, d'où le refus defauts_invalides d'une graine des défauts égale à celle du jeu (section 16.3) ; les deux tests sont conditionnels au jeu propre, sans condition d'effectif (loi exacte, borne valable pour tout nombre de lignes).

Statuts (E2.20) : loi de K_D, région et fausse alarme : résultats exacts, sous l'hypothèse d'indépendance des tirages des défauts (hypothèse sur le générateur, non démontrée) ; borne de Hoeffding : borne exacte de la fausse alarme, non sa valeur ; identités et rejeu : exacts, déterministes ; passage des deux tests : constat de simulation. L'erreur Monte-Carlo est celle de la sélection des défauts, mesurée par les régions ; aucune erreur d'estimation n'intervient, les décomptes étant comparés à leur loi exacte ou à une borne. Les valeurs observées (proportions de lignes altérées, de décès et d'exposition exclus, taux bruts avant et après exclusion) sont imprimées par la batterie, sans verdict, et ne sont pas reproduites ici (E3.4).

16.5 Vérifications

tests/unitaires/test_simulation.R vérifie : les valeurs des tables synthétiques (valeurs de la spécification, et recoupement par stats::integrate(), que le simulateur n'emploie pas) ; les dates d'anniversaire, 29 février compris (recoupement par as.Date() ; comparaison à age_revolu() du module donnees, écrite indépendamment ; l'accord des jours civils et des anniversaires du simulateur avec les arithmétiques des modules donnees et exposition est vérifié en outre par tests/unitaires/test_exposition.R, section 8.4) ; la cohérence exacte des lignes et l'âge révolu à l'entrée ; le passage du module donnees sans exclusion, pour les deux transitions ; l'identité des jeux à graine égale et la restauration de l'état du générateur de l'appelant ; chaque refus, dont table_trop_courte indépendant de la graine ; le constat ci-dessus. Pour les modes multi-contrats (issue #92), il vérifie en outre : les champs, l'ordre des tirages et l'attribut des modes existants comparés à leurs littéraux ; ORDRE_TIRAGES_MULTI comparée à son littéral et les deux premiers tirages rejoués sous la graine ; l'identité, hors attribut, d'un mode multi-contrats de loi_contrats = 1 et de son mode de base, pour les deux paires ; les colonnes, les types, l'ordre des lignes et les identifiants ; la cohérence des lignes, la naissance et le sexe constants par tête, le décès commun ; les décomptes de l'attribut refaits sur les lignes ; les familles A et B et leurs identités ; chaque refus de loi_contrats et la garde de débordement, sans tirage, pour n en double comme en integer. Pour les défauts paramétrés (issue #93) : le catalogue, les combinaisons et l'ordre des tirages comparés aux littéraux de la spécification, recopiés dans le test et non lus dans le package ; le test de clôture des motifs ; defauts = NULL identique à l'appel sans l'argument ; l'identité des lignes non altérées et de l'attribut simulation ; les champs et colonnes de l'attribut defauts ; le rejeu des tirages, la disjonction, les copies, chaque altération, valeur par valeur ; sur un jeu uniforme_multi portant chaque type, les lignes exclues par preparer_donnees() égales à l'attribut, motif par motif, le refus ligne_invalide sans exclusion et le refus date_hors_domaine sur un jeu à part ; construire_table() et ecrire_rapport() sur le jeu altéré ; chaque refus, leur ordre et la restauration de l'aléa de l'appelant ; pour la cible structurellement vide (issue #112), chaque type de tête dans chaque portefeuille à un seul contrat par tête (modes uniforme et age_exact, modes multi-contrats de loi_contrats de longueur 1), en taux et en nombre, zéro compris, sous deux graines du jeu dont l'une invalide : refus defauts_invalides, ses details et le mode cité, avant le refus graine_invalide et sans changer l'aléa de l'appelant ; les modes multi-contrats de loi_contrats $(0{,}5\ ;0{,}5)$ et $(0\ ;1)$ admis ; un nom vide dans les conditions vides sans effet ; defauts_ineligibles d'un type de tête repris dans un mode multi-contrats avec un nombre supérieur aux éligibles ; pour en_cours_avant_fin (issue #135), dans les quatre modes, une date d'entrée ou une origine égale à la fin : refus avant tout tirage, en taux et en nombre, zéro compris, sous une graine du jeu valide ou manquante, avec ses details, la raison et les dates citées, et l'aléa de l'appelant inchangé, et, dans les quatre modes, le cas voisin d'une entrée la veille de la fin admis ; conditions_vides() ; le constat ci-dessus. tests/test_reproductibilite.R vérifie qu'un jeu simulé est identique, attribut compris, dans un autre processus R.

16.6 Fonctions du module

Table 34 : Définitions de premier niveau de R/simulation.R.

Nom	Statut	Entrées	Sortie et rôle
CAUSES_SIMULEES	constante	—	causes produites et transitions associées
HYPOTHESE_REPARTITION	constante	—	hypothèse consignée : force constante par âge entier, risques indépendants

Nom	Statut	Entrées	Sortie et rôle
ORDRE_TIRAGES	constante	—	ordre des tirages des modes `uniforme` et `age_exact`, consigné
ORDRE_TIRAGES_MULTI	constante	—	ordre des tirages de chaque mode multi-contrats, consigné
CHAMPS_PORTEFEUILLE	constante	—	champs requis du portefeuille, par mode
MODE_BASE	constante	—	mode de base de chaque mode (mêmes champs communs, contrôles et bornes d'âge)
CATALOGUE_DEFAUTS	constante	—	catalogue des 18 types de défauts paramétrés : unité, causes éligibles, condition, altération, motif et refus attendus, écrits en dur (tableau 31)
COMBINAISONS_DEFAUTS	constante	—	combinaisons de types admises sur une même ligne (liste fermée)
ORDRE_TIRAGES_DEFAUTS	constante	—	ordre des tirages des défauts, consigné dans l'attribut `defauts`
table_gompertz_makeham()	interne	A, B, c, âges	table synthétique de Gompertz-Makeham
table_constante()	interne	q, âges, nom	table synthétique à q_x constant
loi_effective()	interne	loi d'une transition, cause, sexes	q_x^s et μ_x par sexe ; refus `loi_invalide`, dont la forme exacte des facteurs
jours_civils()	interne	année, mois, jour	jours depuis 1970-01-01 (arithmétique entière)
jour_anniversaire()	interne	composantes de naissance, âge	jour de l'anniversaire (29 février : 1er mars des années non bissextiles)
composantes_distinctes()	interne	jours	composantes de date, calculées sur les valeurs distinctes
age_anniversaire()	interne	composantes, année, jour	âge au dernier anniversaire
en_date()	interne	jours (numériques)	dates de classe `Date`, stockage `double` ; seule définition, pour les colonnes de dates du jeu simulé et du jeu à défauts (issue #113)
ecart_noms()	interne	liste, noms requis, noms admis	`""` si les noms reçus sont complets et admis, sinon le texte de l'écart (noms manquants ou en double, ou champs attendus, manquants et inconnus cités par `liste_guillemets()`), comparé sans tri ; règle unique des noms du portefeuille, de `defauts` et de chaque déclaration (section 16.3 ; issue #113)
controler_portefeuille()	interne	portefeuille, période	refus des entrées, dont `loi_contrats` et la garde de débordement des modes multi-contrats ; noms par `ecart_noms()`, dates par `date_unique()` (issue #113) ; bornes d'âge a priori
age_revolu_minimal()	interne	écart minimal en jours	borne basse $y_{\min}$
controler_couverture()	interne	loi effective, bornes, sexes	refus `table_trop_courte`, `qx_egal_a_un`

Nom	Statut	Entrées	Sortie et rôle
temps_latent()	interne	loi, composantes, âge et fraction à l'entrée, entrée, tirages E	instant et jour de l'événement latent
controler_defauts()	interne	défauts, graine du jeu, vides (conditions structurellement vides, vecteur nommé, aucune par défaut)	refus defauts_invalides, avant tout tirage, dont une déclaration d'un type à condition vide (issue #112) ; vides est rendu par conditions_vides() ; un nom vide ne fait jamais refuser un type sans condition ; déclarations normalisées (types rangés dans l'ordre du catalogue, cible, taux ou nombre)
conditions_vides()	interne	mode, portefeuille contrôlé, période	conditions de la colonne condition de CATALOGUE_DEFAUTS structurellement vides pour ce portefeuille, vecteur nommé (noms : conditions, jamais ""; valeurs : raison formée par formuler() à partir des valeurs reçues, citée au refus), vide si aucune : conditions des types de tête pour un seul contrat par tête (issue #112), entree_avant_fin quand date_entree ou origine égale la fin de la période (issue #135 ; section 16.3)
injecter_defauts()	interne	jeu propre, défauts contrôlés, défauts reçus, période	sélection sous la graine des défauts, refus defauts_ineligibles, altérations déterministes, copies en fin de jeu ; jeu altéré et attribut defauts
simuler_portefeuille()	exportée	loi, portefeuille, période, graine, défauts (NULL par défaut)	jeu simulé (une ligne par contrat écrit) et attribut simulation ; jeu altéré et attribut defauts si des défauts sont déclarés

Partie XIII

Index et traçabilité

Index et traçabilité

17.1 Index des fonctions exportées

Le tableau 35 recense les fonctions exportées de `tablexp` (directives `export` de `NAMESPACE` ; ADR 0007, point 3.4) : les points d'entrée et les fonctions que l'application, les tests ou un tiers appellent. Les autres fonctions sont internes ; elles figurent dans le tableau des fonctions de leur module.

Table 35 : Fonctions exportées de `tablexp`.

Fonction	Module	Rôle	Section
`catalogue_roles()`	`donnees`	catalogue fermé des rôles, lu par l'application pour la déclaration des rôles	7.2
`table_reference()`	`donnees`	constructeur contrôlé d'une table de référence, appelé par l'application pour l'import de tables	7.5
`simuler_portefeuille()`	`simulation`	point d'entrée du simulateur : jeu simulé sous une loi vraie et une graine	16
`valider_plan()`	`construire_table`	validateur du plan, contrôle de saisie que l'application appellera (E4.3)	6.1
`construire_table()`	`construire_table`	point d'entrée unique : exécute un plan de construction et retourne l'objet `construction`	6
`rejouer_fiche()`	`construire_table`	rejeu outillé d'une fiche (E3.7) : contrôles, un appel de `construire_table()`, bilan de rejeu	6.4
`ecrire_plan()`	`rapport`	texte du plan en octets UTF-8, pour l'export du plan (E5.4) ; l'appelant écrit le fichier	15.4
`lire_plan()`	`rapport`	plan relu d'octets, sans évaluation, pour l'import du plan (E5.4) ; non validé	15.4
`ecrire_fiche()`	`rapport`	texte de la fiche de construction en octets UTF-8 ; l'appelant écrit le fichier	15.4
`lire_fiche()`	`rapport`	fiche relue d'octets, sans évaluation, pour le rejeu (E3.7)	15.4

Fonction	Module	Rôle	Section
`ecrire_rapport()`	`rapport`	rapport figé HTML en octets UTF-8, sans calcul (E3.5, E3.6) ; l'appelant écrit le fichier	15.5

17.2 Traçabilité des exigences

Le tableau 36 relie les exigences du cahier des charges (`docs/exigences.md`) aux sections qui les décrivent. Une exigence qui n'y figure pas n'est pas encore mise en œuvre (méthodes de lissage, de positionnement, de fermeture et de validation, interface, tables de référence embarquées) ; une exigence mise en œuvre en partie renvoie aux sections de la partie réalisée ; le tableau est complété avec chaque partie rédigée.

Table 36 : Exigences et sections qui les décrivent.

Exigence	Objet	Section
E1.1	construction d'une table d'expérience, estimation d'une transition	Objet du document ; 6
E1.3	rigueur, traçabilité, reproductibilité	3, 4, 15.1
E1.5	aucune donnée réelle : jeux simulés sous une loi vraie et une graine	16
E2.2	catalogue fermé des rôles, lecture d'une colonne par son rôle	7.2
E2.3	recette la plus fine, nommée, refus si aucune ; convention d'âge, forçage plus grossier ; conventions de temps communes	7.1, 8.1
E2.4	unité d'observation, fusion par tête, incohérences entre contrats	7.4
E2.5	période déclarée, délai de déclaration	7.1, 8.2
E2.6	troncature à gauche, censure à droite, censure non informative nommée	8.2
E2.7	refus par défaut, motifs en liste fermée, exclusions déclarées, données inchangées	7.3
E2.8	pondération par les montants : rôle refusé tant que ses intervalles ne sont pas spécifiés	7.2
E2.9	segmentation déclarée, vide explicite admis (le test d'homogénéité viendra avec la validation)	6.1, 8.2
E2.11	noyau multi-états : une table estime une transition ; seconde transition couverte par les tests	6.1, 16
E2.12	taux bruts : estimateurs de Poisson, binomial et produit-limite avec troncature à gauche, hypothèses de répartition dans la cellule (en partie : les autres méthodes de la v1 viendront avec leurs modules)	9
E2.14	intervalles exacts et asymptotiques des taux bruts, avec leur statut (le bootstrap par têtes viendra avec les tables lissées et positionnées)	9.2, 9.3, 9.4, 9.5
E2.19	simulateur : loi vraie, entrées, têtes multi-contrats, sorties concurrentes, censure, défauts paramétrés, constat de simulation ; retrouver la loi vraie par les taux bruts	16, 16.3, 9.9
E2.20	quatre statuts, erreur Monte-Carlo et erreur d'estimation	3.1, 8.1, 16.4, 9.5, 9.9
E3.1	document LaTeX unique, une partie par module, charte d'`outil_USP`, tableaux de fonctions	ensemble du document
E3.2	fiche de chaque méthode : objet, hypothèses, estimateur, fonction, statut, références ; fiche de chaque vérification	9.2, 9.3, 9.4, 9.9
E3.3	test de concordance entre la documentation et le code	4.1
E3.4	aucun chiffre issu d'une exécution	Objet du document

Exigence	Objet	Section
E3.5	rapport figé HTML autonome et fiche de construction (en partie : les tables CSV et xlsx à en-tête viendront avec la première table finale)	15.1, 15.5
E3.6	restitution sans recalcul par le rapport figé ; empreintes des données reçues, du fichier, du résultat et du code ; dépendance de l'empreinte du résultat à la plateforme (en partie : validations, table finale et largeur des intervalles « non produites par cette version »)	15.2, 15.3, 15.5
E3.7	relecture sans le code, rejeu à partir de la fiche, rejeu outillé	15.1, 15.4, 6.4
E4.1	R 4.3.3, package `tablexp`, point d'entrée unique, modules ; rapport figé écrit par le module `rapport` (note de lecture du 24/10/2026)	1.2, 6, 15.5
E4.2	packages de priorité `base` ; aucune dépendance à Java	5.1, 5.2
E4.3	frontière calcul / affichage ; contrôles de saisie dans le package	1.1, 6.1
E4.4	le moteur reçoit un `data.frame` et des rôles, refuse un type incompatible	1.1, 7.2
E4.5	plan exécuté d'un bloc, objet complet, contenu de la fiche	6, 15.1
E4.6	reproductibilité, graine locale, générateur fixé, aucun horodatage comparé	3, 6.3
E4.7	références de non-régression sur la plateforme de référence	4.3, 4.5
E4.9	un module par famille, index des fonctions contrôlé par la concordance	1, 17.1
E5.4	export et import du plan, écriture de la fiche (fonctions du moteur ; l'interface viendra avec l'application)	15.4
E5.5	messages qui disent ce qui manque, sans rien corriger	2.1, 7.3

17.3 Références

— P. K. Andersen, Ø. Borgan, R. D. Gill, N. Keiding, *Statistical Models Based on Counting Processes*, Springer, 1993 (sections II.5 et V.1 : intitulés lus dans la table des matières, contenu non vérifié) ; section 16.4, famille B.

— W. Hoeffding, *Probability inequalities for sums of bounded random variables*, *Journal of the American Statistical Association*, 58, 1963, p. 13–30 (numéro du théorème non vérifié sur l'original) ; section 16.4, défauts paramétrés.

— M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`) ; section 3.2.

— R Core Team, *R : A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienne, 2024 (`https://www.R-project.org/` ; référence rendue par `citation()` sous R 4.3.3).